

AI Agent / Memory

长期记忆、MCP、coding agent、Agent framework 与个人 AI 工具链。

69 篇 · 2026-10-04

目录

1. Agent Memory 项目地图
2. AI Agent Frameworks 项目地图
3. Agent 长期记忆
4. Claude Code
5. Agent Framework 编程模型对比地图
6. Agent Memory 细分选型矩阵
7. Agent Runtime / Sandbox 项目地图
8. Agent Skills / Plugin System 对比地图
9. Coding Agent / Personal Agent 选型地图
10. MCP Server / Tool Gateway 对比地图
11. Agent Delegation
12. agent-recall
13. agent-recall 架构与设计思路分析
14. AgentCube
15. AgentCube 架构与设计思路分析
16. agentgateway
17. agentgateway 架构与设计思路分析
18. agentmemory
19. agentmemory 架构与设计思路分析
20. AgentScope
21. AgentScope Runtime
22. AgentScope Runtime 架构与设计思路分析
23. AgentScope 架构与设计思路分析
24. AI Agent Frameworks Star 项目清单整理
25. AI Agent 外挂的 9 条设计原则
26. AI Gateway
27. AI 作为压缩器
28. Claude Agent SDK

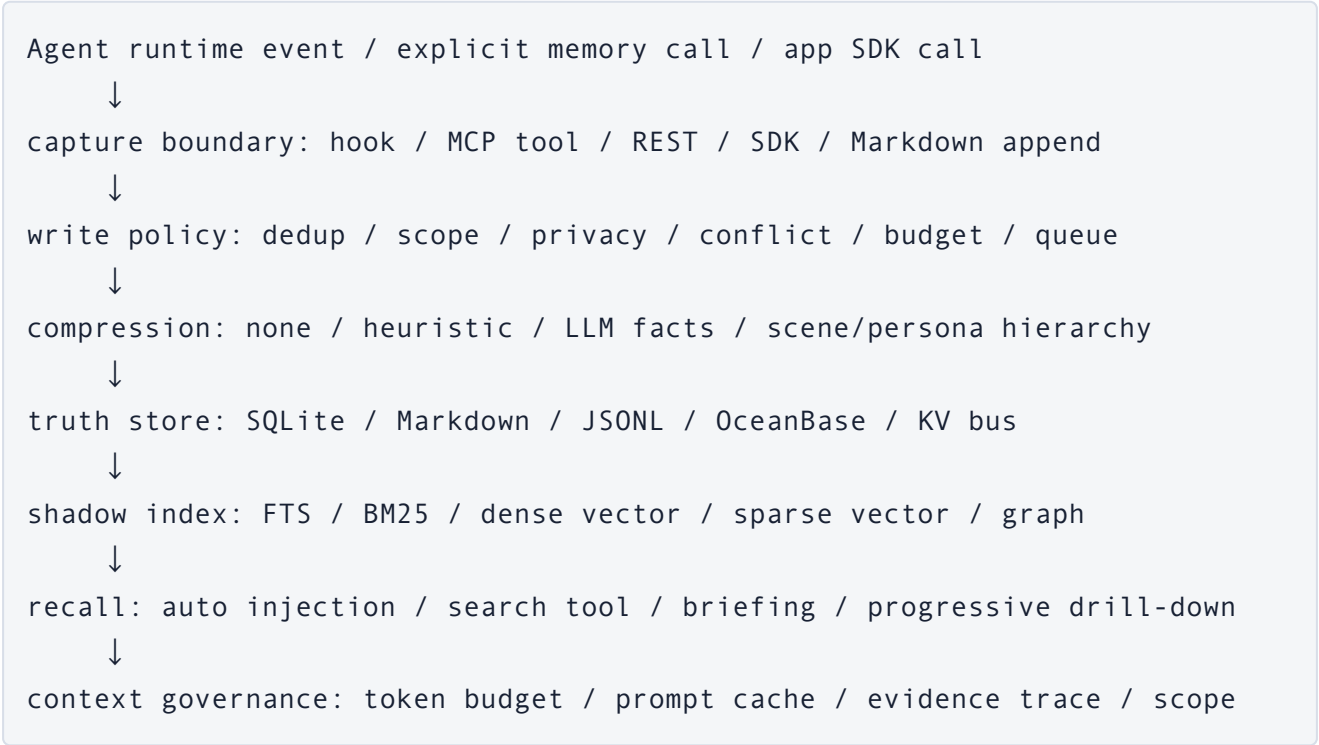
- 29. Claude Code 插件机制
- 30. Claude Context
- 31. Claude Context 架构深度分析
- 32. claude-mem
- 33. claude-mem 架构与设计思路分析
- 34. Code Graph
- 35. code-review-graph
- 36. code-review-graph 架构与设计思路分析
- 37. Codex Plugin for Claude Code
- 38. Codex Plugin for Claude Code 架构与设计思路分析
- 39. Ebbinghaus 遗忘曲线
- 40. Envoy AI Gateway 架构与设计思路分析
- 41. k8m
- 42. k8m 架构与设计思路分析
- 43. kagent
- 44. kagent 架构与设计思路分析
- 45. kubectl-ai
- 46. kubectl-ai 架构与设计思路分析
- 47. MCP Lifecycle Operator
- 48. MCP Lifecycle Operator 架构与设计思路分析
- 49. mem0
- 50. mem0 架构与设计思路分析
- 51. memsearch
- 52. memsearch 架构与设计思路分析
- 53. Model Context Protocol (MCP)
- 54. nanobot
- 55. nanobot 架构与设计思路分析
- 56. NemoClaw
- 57. NemoClaw 架构与设计思路分析
- 58. Open Cowork 架构与设计思路分析

- 59. OpenAI Codex CLI
- 60. OpenKruise Agents
- 61. OpenKruise Agents 架构与设计思路分析
- 62. OpenShell
- 63. OpenShell 架构与设计思路分析
- 64. ReMe
- 65. ReMe 架构与设计思路分析
- 66. TencentDB-Agent-Memory
- 67. TencentDB-Agent-Memory 架构与设计思路分析
- 68. 三层搜索协议
- 69. 事件驱动的记忆管道

Agent Memory 项目地图

这页把当前已摄入的 memory 相关项目横向整理。核心结论：这些项目不是在做同一件事，而是在同一个问题空间里选择了不同的事实源、宿主入口、压缩成本、检索治理边界和上下文注入策略。

从工程架构看，Agent Memory 不是一个“向量库 + top-k 搜索”的问题，而是一条完整的数据管线：



真正的架构分歧主要发生在三个地方：

- 写入边界：谁决定“这值得记住”。
- 真相层：系统崩溃、索引损坏、模型更换后，应该从哪里恢复事实。
- 注入边界：什么内容可以自动进入 prompt，什么内容必须让 Agent 主动搜索下钻。

一句话分层

项目	一句话定位	最适合的问题
claude-mem	Claude Code 专用长期记忆插件，hook 采集 + worker 异步压缩 + SQLite/Chroma 双索引	给 Claude Code 单机用户加跨会话记忆
agent-recall	本地优先的 MCP-native 结构化记忆库，SQLite + scope hierarchy + AI briefing	多项目/多客户场景下保存实体、关系、决策和 scoped facts
agentmemory	跨 Agent 本地记忆服务，REST/MCP/hooks + iii-engine + BM25/vector/graph 三流检索	多个 coding agent 共用一个本地 memory worker 和 viewer

项目	一句话定位	最适合的问题
powermem	应用级记忆中间件，SDK/CLI/API/MCP/Dashboard 多入口 + OceanBase/SeekDB 后端	把 LLM app / agent framework 的记忆做成可部署服务
memsearch	跨平台 coding agent 语义记忆，Markdown source-of-truth + Milvus shadow index	让 Codex/Claude/OpenCode/OpenClaw 从历史会话 Markdown 里渐进式回溯
tencentdb-agent-memory	OpenClaw/Hermes 记忆插件，L0→L3 分层长期记忆 + context offload	同时处理长期 persona/scene memory 和短期工具日志压缩

横向对比

维度	claude-mem	agent-recall	agentmemory	powermem	memsearch	tencentdb-agent-memory
主宿主	Claude Code	MCP clients / CLI / Claude hooks	Claude Code / Codex / Cursor / MCP clients	SDK / CLI / API / MCP / IDE 插件	Claude Code / Codex / OpenCode / OpenClaw	OpenClaw / Hermes / Gateway
主形态	宿主插件 + 本地 worker	MCP-native 结构化记忆库	本地常驻服务 + MCP/REST/viewer	应用中间件 / 服务化 memory layer	CLI/plugin + Markdown 记忆库	插件/sidecar + 分层 pipeline
写入口	lifecycle hooks	MCP memory tools / CLI	hooks / REST / MCP	Memory.add() / HTTP / MCP / plugin hook	平台 hook append Markdown	before/after hook + tool/offload
写入语义	捕获 tool event，再由后台判断	Agent 主动声明 entity/fact/relation	观察事件 + remember/save API	应用显式提交 memory item	追加可审计会话摘要	捕获 L0，再抽 L1/L2/L3
事实源	SQLite observations	SQLite entities/slots/relations	iii-engine SQLite KV	OceanBase / SeekDB / pgvector / SQLite	.memsearch/ *.md Markdown	JSONL records + scene/persona files

维 度	claude-mem	agent-recall	agentmemory	powermem	memsearch	tencentdb-agent-memory
索引层	SQLite FTS5 + Chroma	SQLite FTS5/ LIKE	in-process BM25 + vector + graph	vector + FTS + sparse + graph	Milvus dense + BM25 sparse	SQLite FTS/ vector 或 TCVDB hybrid
压缩策略	后台 LLM 压成 structured observations	raw graph → AI briefing cache	默认零 LLM 启发式，可选 LLM	LLM fact extraction + importance eval	daily/PROJECT/ USER maintenance	L0→L1→L2 →L3 多级 LLM 管线
权限/隔离	profile/env/path 隔离为主	MCPBridge scope enforcement	API/MCP 暴露面 + privacy filter	多租户 scope/ permission/ privacy controllers	项目目录 + collection 隔离	host config + store backend + pipeline scope
注入方式	SessionStart auto context + mem-search skill	SessionStart AI briefing	context injection 可配置 + MCP tools	API/search 返回给应用或插件注入	启动提示 + \$memory-recall 下钻	prepend L1 + append L2/L3 + tools
恢复模型	outbox / SQLite / Chroma watermark	SQLite truth + cache 可重建	KV truth + index persistence/ rebuild	后端 DB truth + optimizer metadata	Markdown truth, Milvus 可重建	JSONL/files truth, store 可重建
主要强项	Claude Code 集成自然，模式清晰	scope hierarchy 和 bitemporal facts	多入口、多工具、viewer、零 LLM 默认	后端/provider/部署形态最完整	Markdown 可审计，可重建索引	分层记忆 + context offload 合一
主要代价	生态偏 Claude Code	向量/语义能力较轻	iii-engine 依赖和工具面较大	部署/配置复杂度更高	依赖 Milvus 路径和索引维护	管线复杂、宿主偏 OpenClaw/ Hermes

架构交叉矩阵

这些项目经常“看起来不同，底层同构”。真正可复用的是工程模式，而不是某个具体后端。

交叉模式	采用项目	工程含义
Hook → worker → store	claude-mem , agentmemory , memsearch , tencentdb-agent-memory , powermem Claude 插件	Hook 只做捕获和转发, AI 压缩、embedding、索引重建都放到后台; 关键是失败不能阻塞 Agent 主流程
MCP memory tools	agent-recall , agentmemory , powermem, tencentdb-agent-memory	把“记住/查询/删除/展开”变成 Agent 可调用协议; 写入质量取决于 tool instructions 和模型是否愿意主动保存
SQLite 本地事实层	claude-mem , agent-recall , agentmemory , tencentdb-agent-memory SQLite 模式	单机部署简单, 事务/WAL/FTS 成熟; 难点是多进程锁、索引重建和 scope enforcement
Markdown / JSONL 可审计真相	memsearch , tencentdb-agent-memory	人类可读、可 diff、可备份, 索引可重建; 代价是需要额外处理 chunk ID、行号 anchor、文件一致性
Hybrid + RRF	agentmemory , powermem, memsearch , tencentdb-agent-memory	BM25/FTS 抓字面, vector 抓语义, graph/scope/time 做约束; RRF 比单一相似度更稳
三层/渐进式召回	claude-mem , memsearch , tencentdb-agent-memory , agent-recall	先给摘要/片段, 再按需展开原文, 避免 top-k 原文直接撑爆 prompt
AI 压缩层与真相层分离	claude-mem , agent-recall , powermem, tencentdb-agent-memory	LLM 输出是压缩视图, 不应该成为唯一证据; 否则 hallucinated memory 会永久化
显式成本控制	agentmemory , agent-recall , memsearch , tencentdb-agent-memory	默认不自动塞大量上下文, 或者只注入轻量 hint/briefing; 用户需要时再 search/expand

交叉 1: Hook 插件和后台服务的边界

[claude-mem](#)、[agentmemory](#)、[tencentdb-agent-memory](#) 都把 hook 设计得很薄:

```

hook receives stdin JSON
  ↓
normalize minimal payload
  ↓
HTTP/process handoff with timeout
  ↓
swallow errors or return lightweight context

```

这条边界的关键不是“怎么调用 HTTP”，而是 hook 不能把 Agent 运行时变成 memory 系统的人质。工具执行后的 hook 如果同步做 LLM 压缩或 embedding，一次 provider 慢请求就会把用户的 coding agent 卡住。所以这些项目都倾向于：

- PostToolUse / agent_end 捕获事件后异步处理。
- SessionStart / before_prompt_build 只读 cache 或做轻量检索。
- worker 崩溃时降级为无记忆，而不是阻断宿主。

差异在于后台服务形态：[claude-mem](#) 内置 worker + BullMQ/SQLite/Chroma；[agentmemory](#) 外置 iii-engine 作为函数总线；[tencentdb-agent-memory](#) 把核心封装成 TdaiCore，在 OpenClaw in-process 和 Hermes sidecar 之间复用。

交叉 2：MCP 是工具入口，不等于存储模型

[agent-recall](#)、[agentmemory](#)、powermem 都支持 MCP，但 MCP 在三个项目里的角色不同：

- [agent-recall](#)：MCP 是主入口。它用 server instructions 要求 Agent 主动保存 people、decisions、facts、context，并在 MCPBridge 做 scope 读写过滤。
- [agentmemory](#)：MCP 是完整能力面之一。真实业务函数在 worker/iii-engine 里，MCP tools 只是和 REST/hook 并列的触发层。
- powermem：MCP 是中间件接入形态之一。核心仍是 Memory/AsyncMemory SDK 和存储适配层。

所以“是否支持 MCP”不是关键问题；关键问题是 MCP tool call 是否承载写入语义、权限边界和行为引导。[agent-recall](#) 的工程亮点就在这里：它把主动记忆策略嵌进 MCP server instructions，并把权限 enforcement 放在 bridge，而不是底层 store。

交叉 3：Source-of-truth 和 shadow index 的分离

Agent Memory 系统迟早会遇到索引损坏、embedding model 更换、chunk 切分规则变化、vector dimension mismatch、LLM 压缩失败等问题。因此成熟设计都会区分：

- 事实真相层：SQLite rows、Markdown、JSONL、scene blocks、persona file、OceanBase records。
- 可重建索引层：FTS table、Chroma collection、Milvus collection、in-memory BM25/vector、TCVDB hybrid collection。

[memsearch](#) 把这个边界做得最清楚：Markdown 是 truth，Milvus 是 shadow index；chunk ID 绑定 source/line/content/model，模型切换不会污染旧索引。[tencentdb-agent-memory](#) 也类似：JSONL/scene/persona 是恢复材料，SQLite/TCVDB 是检索引擎。[agentmemory](#) 的难点在 index persistence：BM25/vector 会周期性落回 KV，维度不匹配时拒绝启动，防止静默错误。

交叉 4：混合检索不是“加一个向量库”

这六个项目里，纯向量 top-k 基本不是最终形态。原因很实际：

- coding agent 的记忆常包含文件名、命令、函数名、issue id、路径、专有名词；这些更适合 BM25/FTS。
- 用户常问“上次那个设计”“之前某个客户约束”；这些更适合 dense vector。
- 多项目/多客户/多人场景需要 scope、time、entity graph；这些不是相似度能自然解决的。

powermem 是最企业化的检索栈：vector + FTS + sparse + graph，再做 RRF、衰减加权、rerank。

[agentmemory](#) 是本地轻量版本：in-process BM25 + Float32Array vector + graph BFS。[memsearch](#) 把 dense 和 BM25 都放进 Milvus。[tencentdb-agent-memory](#) 在 SQLite 路径用 FTS/vector/RRF，在 TCVDDB 路径用 native hybridSearch。

Memory 写入方式的区别

写入方式决定了 memory 的质量上限。检索再强，也救不了错误写入、噪声写入和越权写入。

写入类型	代表项目	写入动作	优点	主要风险
被动事件捕获	claude-mem , agentmemory , tencentdb-agent-memory	hook 捕获 prompt/tool/session event	用户无感，覆盖率高	噪声多、需要强 dedup/压缩
主动工具写入	agent-recall , agentmemory	Agent 调 MCP tools 保存实体/事实/关系	语义清楚，结构化强	依赖模型自觉，容易漏记
应用 SDK 写入	powermem	app 调 <code>Memory.add()</code>	产品可控，能结合业务 schema	对 coding agent 无法完全无侵入
Markdown append	memsearch	hook/skill 追加 daily/PROJECT/USER Markdown	可审计、可手改、可重建	需要维护 chunk/anchor/index 一致性
分层管线写入	tencentdb-agent-memory	L0 捕获 → L1 atom → L2 scenario → L3 persona	长短期结构清楚，注入层次自然	pipeline 调度、冲突、半写入复杂
Briefing/cache 写入	agent-recall	raw context → AI briefing cache	启动快，scope-aware	cache 不是 truth，staleness 要治理
Context offload 写入	tencentdb-agent-memory	工具日志写 refs，MMD 节点进上下文	解决短期上下文爆炸	node_id、原文、摘要必须能互相追踪

1. 被动事件捕获：覆盖率高，治理成本也高

[claude-mem](#) 的写路径是典型模式：

```
PostToolUse / UserPromptSubmit / Stop
  ↓
bun-runner lightweight handoff
  ↓
worker outbox / queue
  ↓
LLM observation generation
  ↓
SQLite transaction + Chroma sync
```

这类系统的优势是用户不用记得“保存记忆”。coding agent 的重要事实往往来自工具执行：读了哪个文件、改了哪个模块、发现了什么失败路径、最后怎么修复。被动捕获能把这些都送进候选集。

代价是治理困难：

- 工具结果常有大量临时输出，直接入库会污染检索。
- 同一个事实会被多次触发，需要 content hash、时间窗或语义 dedup。
- Hook 必须短超时、吞错、降级，不能影响用户主流程。
- LLM 压缩如果异步失败，需要重试和 outbox，否则事件会丢。

[agentmemory](#) 对这个问题的取舍更激进：默认不调用 LLM，而是 buildSyntheticCompression() 做启发式压缩，保护用户 API token；LLM 自动压缩要显式打开。它牺牲一部分语义质量，换来默认成本可控。

2. 主动工具写入：结构清楚，但依赖 Agent 行为

[agent-recall](#) 的主路径不是“抓所有工具日志”，而是让 Agent 主动调用 MCP memory tools：

```
create_entities / add_observations / add_relations
  ↓
MCPBridge input limits + scope filtering
  ↓
MemoryStore SQLite entities / slots / observations / relations
  ↓
context_gen generates scoped AI briefing
```

这个模式适合 people、organizations、project decisions、customer-specific facts 这类结构化长期知识。它的关键设计是把语义边界前移：写入时就知道这是 entity、slot、relation 还是 observation。

但它有两个工程难点：

- Agent 可能忘记写，或者把临时判断写成长期事实。
- 权限不能靠 store 自觉。[agent-recall](#) 明确让 MemoryStore 不做 scope enforcement，而把 MCP 可见边界集中在 MCPBridge，这是更清晰的信任模型。

3. 应用 SDK 写入：适合产品，不一定适合 terminal agent

powermem 的写路径更像一个 LLM app memory middleware：

```
Memory.add(prompt, user_id, agent_id, run_id)
  ↓
LLM fact extraction
  ↓
ImportanceEvaluator + Ebbinghaus metadata
  ↓
embedding / sparse embedding
  ↓
StorageAdapter route to OceanBase / SeekDB / pgvector / SQLite
```

它的优势是应用拥有调用点，可以在业务语义最清楚的地方写入。比如客服、教育、CRM、个人助手可以把 user_id、agent_id、run_id、scope、permission 都作为一等参数传入。

它不一定适合纯 coding agent 插件的原因是：terminal agent 的事实来源经常分散在工具调用和文件读写里，应用层没有一个干净的 add() 边界。因此 PowerMem 也提供 Claude Code plugin / MCP / IDE 扩展，但它的架构核心仍是服务化 memory layer。

4. Markdown append：可审计性优先

[memsearch](#) 的写入不是直接进 DB，而是先写 Markdown：

```
platform hook parses transcript
  ↓
append .memsearch/memory/YYYY-MM-DD.md
  ↓
scanner/chunker computes stable chunk_id
  ↓
Milvus dense + BM25 sparse index
```

这个模式的工程价值很大：memory 可以被人类 review、diff、迁移、手改；Milvus 坏了可以重建；embedding model 切换可以重新生成 collection。

代价也很明确：

- Markdown 结构必须足够稳定，否则 heading/line anchor 会漂移。
- chunk ID 要绑定 content/model/source，否则增量索引会脏。
- hook 生成的摘要质量会直接影响后续检索。
- 需要 search → expand → transcript 的渐进式协议，否则单个 chunk 太短容易断章取义。

5. 分层写入：把“记忆”拆成证据、事实、场景、画像

[tencentdb-agent-memory](#) 是这组里写入管线最复杂的：

```
L0 Conversation JSONL
  ↓
L1 Atom records: persona / episodic / instruction memory
  ↓
L2 Scenario scene_blocks/*.md
  ↓
L3 Persona persona.md + scene navigation
```

这不是单纯“多摘要几次”，而是在解决不同注入层的稳定性问题：

- L0 是证据，不适合直接注入。
- L1 是可检索事实，适合 prepend 动态召回。
- L2 是场景导航，适合 append 到 system context。
- L3 是用户画像，变化慢，适合长期稳定注入。

这条路的难点是 pipeline consistency：L1 有 threshold/idle/flush 三种触发，L2 用 downward-only timer，L3 要全局串行避免并发改 persona。LLM 还会被允许改写 scene_blocks/，因此必须有沙箱、备份和恢复逻辑，防止半写入破坏状态。

项目工程剖面

[claude-mem](#)：最标准的事件驱动闭环

[claude-mem](#) 的价值在于把 Claude Code 插件 memory 的最小闭环打穿：

```
Lifecycle hooks
→ worker queue
→ LLM observation generation
→ SQLite/FTS + Chroma
→ SessionStart injection + mem-search skill
```

它的工程核心是 边缘轻、后台重。Hook 负责接力，worker 负责 AI 压缩、解析、事务写入和 Chroma 同步。MemoryItem schema 把自然语言 narrative、facts、concepts、filesRead、filesModified 拆开，说明它不是把 transcript 原文塞进向量库，而是先变成可索引观察。

适合借鉴的点：

- PostToolUse 不直接做 LLM。
- outbox / hash / retry 处理 AI 非确定输出和失败。
- search → timeline → get_observations 防止一次塞太多上下文。

主要风险：

- 宿主强绑定 Claude Code。
- 如果 observation 压缩 prompt 质量不稳，会把噪声永久化。
- Chroma 与 SQLite 的双写一致性需要 watermark/同步治理。

agent-recall：结构化事实和 scope governance 优先

agent-recall 更像“给 Agent 的本地知识图谱”，而不是“会话日志搜索”。它把 people、decisions、facts、context 作为主动写入对象，底层 SQLite 维护 entities、slots、observations、relations。

它最重要的工程边界是：

```
MCP client
→ MCPBridge: limits + scope read/write filter
→ MemoryStore: trusted SQLite operations
→ context/briefing: scoped context assembly
```

MemoryStore 不做 scope enforcement 不是缺陷，而是明确把底层 store 定义成可信 API；面向多 Agent 的 MCP 入口才是权限边界。这个设计比“每层都检查一点权限”更容易审计。

适合借鉴的点：

- scope hierarchy + local-over-parent inheritance。
- bitemporal slots 表达事实更新，而不是覆盖丢历史。
- AI briefing 是 cache，不是 truth。

主要风险：

- 主动写入依赖模型行为，漏记问题不可避免。
- 语义检索能力较轻，复杂回忆可能需要结合外部 vector layer。
- CLI 和 MCP 的信任模型不同，需要文档非常清楚。

agentmemory: 跨 Agent worker 和成本可见性

agentmemory 的架构很宽: hooks、REST、MCP、viewer、iii-engine、50+ functions、32+ KV scopes。它的核心取舍不是“最简单”，而是让多个 Agent 共用一个本地记忆服务。

写路径和检索路径都体现了成本控制:

- Hook standalone, 不导入 iii-sdk, 只 HTTP POST。
- 默认零 LLM 压缩, 避免每个 tool call 都烧 token。
- Context injection 默认关, 避免自动注入 4000 字记忆击穿用户预算。
- Embedding provider 缺失时退化 BM25-only。

它的检索是本地轻量混合检索:

```
BM25 inverted index
+ VectorIndex Float32Array cosine
+ GraphRetrieval BFS
→ RRF + optional rerank
```

适合借鉴的点:

- 默认安全/低成本, 重功能显式开启。
- 向量维度不匹配直接失败, 避免 silent bad search。
- fire-and-forget rebuild, 让服务先可用, 索引质量逐步收敛。

主要风险:

- iii-engine 是强依赖, 部署和调试复杂度更高。
- MCP tool 面太大, 默认只暴露 8 个核心工具是必要收敛。
- 业务函数数量多, 接口同步和回归测试成本高。

powermem: 把 memory 做成中间件

powermem 的工程目标和 coding-agent 插件不同。它不是只服务某个 terminal agent, 而是把 memory layer 做成 SDK/CLI/API/MCP/Dashboard/IDE plugin 共用的基础设施。

核心路径是:

```
Memory.add()
→ LLM fact extraction
→ importance / retention metadata
→ dense + sparse embedding
```

```
→ StorageAdapter
→ OceanBase / SeekDB / pgvector / SQLite
```

它的关键抽象是 `StorageAdapter` 和 provider factories。Core 不直接绑 OceanBase SDK，LLM/embedder/reranker 也通过 factory 切换。这让 PowerMem 可以走“本地 SQLite/SeekDB”到“企业 OceanBase”的连续部署路径。

适合借鉴的点：

- `memory_type / importance_score / retention_strength / next_review` 都是 metadata，Intelligence 不直接改库。
- hybrid retrieval 做 adaptive weight normalization，避免多路命中文档天然占优。
- `scope/permission/privacy/collaboration` 作为组件，而不是后补字段。

主要风险：

- LLM fact extraction 在写入链路前台发生，provider 延迟和失败会影响 add 的体验。
- 配置矩阵大，用户需要理解 provider/storage/rerank/sparse 组合。
- 企业级能力带来更高部署成本。

[memsearch](#)：Markdown truth + Milvus shadow index

[memsearch](#) 的架构重点是可审计和可重建。它不把 Milvus 当 truth，而是把 `.memsearch/*.md` 作为事实源。

```
hook summary
→ Markdown append
→ scanner/chunker
→ chunk_id(source,line,content,model)
→ Milvus dense + BM25
→ search / expand / transcript
```

这特别适合 coding agent 记忆，因为很多记忆需要人类能看见和修正：之前的设计讨论、踩坑记录、项目约束、用户偏好。Markdown 让 memory 变成 repo-local artifact，而不是黑盒数据库。

适合借鉴的点：

- HTML comments 从 embedding 内容剥离，但原文保留，避免 metadata 污染语义。
- `search → expand → transcript` 保持证据链。
- chunk ID 绑定 embedding model，模型切换不污染旧索引。

主要风险：

- Markdown 结构漂移会影响 anchor 和 expand。

- Milvus/Milvus Lite 仍是额外运行时依赖。
- 自动摘要如果写得太粗，后续检索只能找到粗粒度结论。

tencentdb-agent-memory：长期记忆和短期 offload 合并

[tencentdb-agent-memory](#) 最有辨识度的是同时处理两类“记忆”：

- 长期语义记忆：L0/L1/L2/L3。
- 短期任务上下文：tool result offload + Mermaid MMD。

它把召回分成不同注入位置：

```
before_prompt_build
→ L1 dynamic recall as prependContext
→ L2 scene navigation / L3 persona as appendSystemContext
→ memory tools for active drill-down
```

这个设计很工程化：动态 L1 经常变，如果放进 system 前缀容易击穿 prompt cache；L2/L3 更稳定，适合系统上下文。Context offload 则把大工具日志从 prompt 里挪到 refs/*.md，只注入 MMD 节点和摘要，让 Agent 需要时按 node_id 下钻。

适合借鉴的点：

- TdaiCore host-neutral，OpenClaw 插件和 Hermes sidecar 复用同一管线。
- L0 证据、L1 atom、L2 scene、L3 persona 分层明确。
- pipeline manager 用 threshold/idle/flush/downward-only/global-serial 控制节奏。

主要风险：

- LLM 参与文件改写，必须处理沙箱、备份、半写入恢复。
- L1/L2/L3 多层一致性和 stale propagation 很复杂。
- 宿主生态偏 OpenClaw/Hermes，迁移到其他 agent runtime 需要适配 hook/context 协议。

核心难点

1. 写入质量：不是所有上下文都值得记住

Agent 的原始上下文充满噪声：命令输出、临时错误、重复尝试、token 统计、日志片段、无效搜索结果。如果系统把这些都写入长期 memory，检索层会被污染。

工程上常见解法：

- [claude-mem](#)：用 LLM observation schema 压缩，只保留 facts/concepts/files。
- [agentmemory](#)：默认启发式压缩 + dedup + privacy stripping。
- powermem：fact extraction + importance evaluation。

- [tencentdb-agent-memory](#): L1 batch dedup/update/merge/skip。
- [memsearch](#): 把会话摘要写 Markdown, 再通过 maintenance 维护 PROJECT/USER 摘要。

核心判断: memory write 应该是一个 filtering boundary, 不是日志落盘。

2. 延迟隔离: 记忆系统不能阻塞 Agent

所有成熟设计都在保护 Agent 主路径:

- Hook 只做轻量接力。
- LLM 压缩、embedding、索引重建放后台。
- search/injection 失败时降级为空记忆。
- rebuild 不阻塞服务启动。
- cache stale 时可以先返回旧 briefing 或 raw context。

如果 memory 系统把每次 tool call 都变成同步 LLM 请求, 用户会把它关掉。这也是 [agentmemory](#) 默认零 LLM、context injection 默认关的根本原因。

3. 真相与索引一致性: 索引必须可丢弃

Memory 系统经常要面对:

- embedding provider 切换导致维度变化。
- chunker 规则变化导致 chunk id 变化。
- LLM 压缩失败或输出格式漂移。
- vector DB 写入成功但 SQLite 写入失败, 或反过来。
- watcher/indexer 中途退出。

比较稳的设计是: truth store 明确、index 可重建。[memsearch](#) 的 Markdown truth、[tencentdb-agent-memory](#) 的 JSONL/files truth、[agent-recall](#) 的 SQLite truth 都是这个方向。[claude-mem](#) 的双索引则需要 watermark 和同步状态治理。

4. 冲突和时间: 长期事实会变

Agent memory 里最危险的是“曾经正确的事实”。例如客户偏好、项目架构、API 版本、负责人、部署路径都会变化。

不同项目的处理方式:

- [agent-recall](#): bitemporal slots, 用 valid_from / valid_to 保存事实历史。
- powermem: importance/retention/next_review + optimizer 做衰减和晋升。
- [agentmemory](#): strength decay、auto-forget、consolidation。
- [tencentdb-agent-memory](#): L1 batch conflict detection + L2/L3 增量改写。

纯向量库很难表达“旧事实已被新事实覆盖”。这就是结构化 metadata、时间字段、scope 和 conflict handling 必须进入核心模型的原因。

5. Scope 和权限：多 Agent 不是多一个 user_id

多项目、多客户、多 Agent 的 memory 不能只靠 user_id 分库。真实问题包括：

- 一个 entity 在不同 scope 下有不同 facts。
- parent scope 的默认事实可以被 local scope override。
- orchestrator 能看全局，普通 topic agent 只能看局部。
- CLI 管理员和 MCP client 不是同一信任级别。

[agent-recall](#) 的 MCPBridge 是最清晰的参考：底层 store 不做权限，MCP 边界做读写过滤。powermem 则把 scope/permission/privacy/collaboration 做成组件级控制器，更适合企业服务。

6. 上下文注入：召回成功不等于应该自动塞进 prompt

自动注入最容易引发三个问题：

- token 成本不可见。
- prompt cache 被动态上下文击穿。
- 旧记忆抢占当前任务注意力。

项目里的几种策略：

- [agentmemory](#)：context injection 默认关。
- [memsearch](#)：启动只提示使用 \$memory-recall，查询后再 expand。
- [agent-recall](#)：SessionStart 注入 scoped AI briefing，而不是所有 raw facts。
- [tencentdb-agent-memory](#)：L1 动态 prepend，L2/L3 稳定 append，conversation search 主动下钻。
- [claude-mem](#)：auto context + mem-search skill 双轨。

工程判断：默认注入应该短、稳定、可解释；深层证据应该工具化。

7. 检索融合：RRF 权重和过滤比模型更难调

Hybrid search 的难点不是“同时跑两路”，而是结果融合后的质量治理：

- BM25 和 vector 分数不可直接比较。
- 有些 query 只有字面命中，有些 query 只有语义命中。
- graph/scope/time 命中应该是过滤、加权还是 rerank。
- reranker 成本和延迟是否值得。
- top-k 长尾是否有足够绝对相似度。

powermem 的 adaptive weight normalization 是值得注意的细节：按实际命中路径重新归一化权重，避免多路都有结果的文档天然占优。[agentmemory](#) 的 BM25/vector/graph RRF 更轻量，适合本地规模。[memsearch](#) 把 dense/BM25 放到 Milvus hybrid_search，适合把检索复杂度外包给向量数据库。

8. 可观测和可恢复：memory 系统需要自己的运维面

一旦 memory 常驻后台，就会出现普通插件没有的问题：

- worker 是否启动。
- queue 是否积压。
- index 是否 stale。
- embedding 维度是否匹配。
- cache 是否过期。
- LLM 压缩失败率如何。
- 哪条记忆被注入了 prompt。
- 某条错误记忆从哪里来。

[agentmemory](#) 的 viewer、metrics、index persistence，powermem 的 Dashboard/audit/telemetry，[claude-mem](#) 的 worker service/viewer，都是在补这个运维面。Agent Memory 如果没有可观测性，很快会变成“模型偶尔想起一些不知道哪里来的东西”。

设计分型

分型	代表项目	架构重心	适合场景
宿主插件型	claude-mem , tencentdb-agent-memory OpenClaw 插件	hook 生命周期、自动注入、宿主协议	给单个 agent runtime 加强记忆
MCP 结构化记忆型	agent-recall	tool semantics、scope、entity graph	多项目/多客户知识保存
本地常驻 worker 型	agentmemory	多入口、统一服务、viewer、低成本默认	多个 coding agent 共享记忆
应用中间件型	powermem	SDK/API、多后端、多租户、provider matrix	产品化 LLM app / 企业 memory service
Markdown truth 型	memsearch	可审计 source-of-truth、可重建索引	coding agent 历史会话回溯
分层语义金字塔型	tencentdb-agent-memory	L0 证据 → L1 事实 → L2 场景 → L3 画像	长期 persona + 短期 offload 合一

这些分型可以组合。例如一个理想的 coding agent memory 可能采用：

- [memsearch](#) 的 Markdown truth 和 progressive recall。
- [agent-recall](#) 的 scope/permission 模型。
- [claude-mem](#) 的 hook-worker 异步闭环。
- [agentmemory](#) 的成本默认和本地 viewer。

- [tencentdb-agent-memory](#) 的 context offload。
- powermem 的 provider/storage adapter 思路。

但不要一开始就把所有能力塞进一个系统。memory 架构最怕的是“写入太宽、检索太黑、注入太猛”。更稳的落地顺序是：

1. 明确 truth store。
2. 做轻量写入和可审计日志。
3. 加 BM25/FTS 搜索。
4. 再加 vector/hybrid。
5. 最后做自动注入和 LLM 压缩。

选型建议

你的目标	优先看	工程关注点
只想给 Claude Code 加个人长期记忆	claude-mem	hook-worker 边界、SQLite/Chroma 同步、SessionStart 注入
想要最清晰的本地 SQLite + MCP 结构化记忆	agent-recall	scope hierarchy、MCPBridge、bitemporal slots
想让多个 Agent 共用本地服务，并有 REST/MCP/viewer	agentmemory	iii-engine 总线、默认零 LLM、三流 RRF、index persistence
想把 memory 做成应用中间件或企业服务	powermem	StorageAdapter、provider matrix、多租户 controller、hybrid/RRF
想保留 Markdown 事实源，并用 Milvus 做可重建索引	memsearch	chunk ID、Markdown anchor、Milvus dense/BM25、expand
想研究 OpenClaw/Hermes 的分层记忆和 context offload	tencentdb-agent-memory	L0-L3 pipeline、offload refs/MMD、prompt cache 分层注入

当前知识库缺口

- 还缺少一篇“[agent-memory](#) 2026 项目分型”概念扩展，把以上项目抽象成 taxonomy。
- 还缺少 openclaw / Hermes / OpenCode 的实体页，否则 [tencentdb-agent-memory](#) 和 [memsearch](#) 的宿主生态关系还不完整。
- 可以补一篇“memory system evaluation”分析页，对比 LOCOMO、LongMemEval、Coding-Life 等评测口径。
- 可以补一篇“memory 安全与隐私”分析页，专门整理 scope enforcement、PII stripping、API auth、prompt injection 边界。

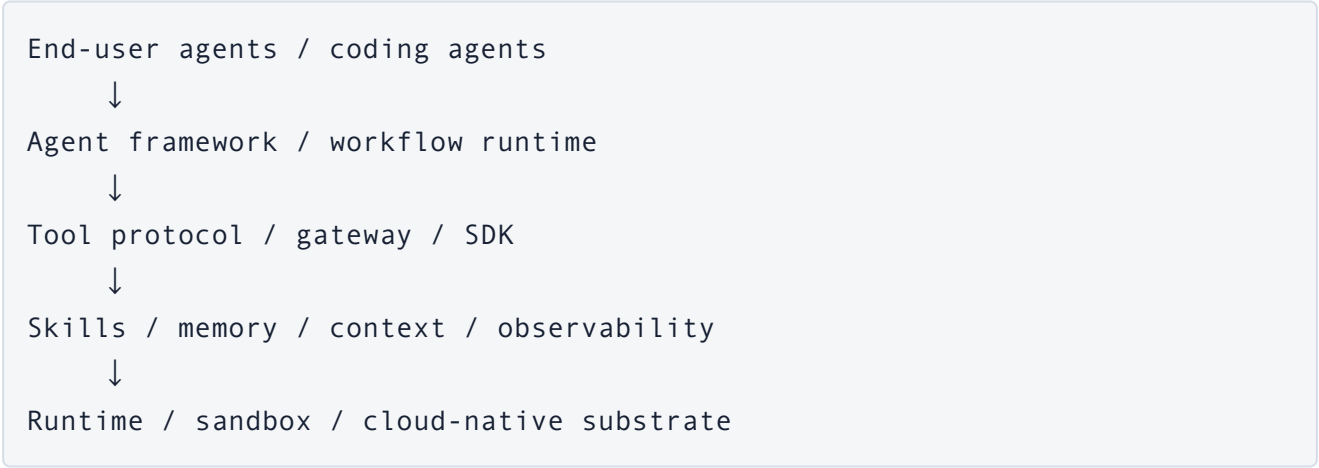
- 可以补一篇“memory write semantics”概念页，专门展开自动捕获、主动写入、SDK 写入、Markdown append、分层写入的设计边界。

相关页面

- [agent-memory](#)
- [event-driven-memory-pipeline](#)
- [three-tier-search-protocol](#)
- [ai-as-compressor](#)
- [hybrid-search-rrf](#)
- [claude-mem](#)
- [agent-recall](#)
- [agentmemory](#)
- [powermem](#)
- [memsearch](#)
- [tencentdb-agent-memory](#)

AI Agent Frameworks 项目地图

这页把当前知识库 AI Agent framework / coding agent / Agent OS / MCP / skills / memory / runtime 相关材料整理成一张工程地图。核心结论：Agent 生态已经明显分层，不再是“哪个框架能调工具”这么简单。



如果说 [agent-memory-project-map](#) 关注“Agent 如何记住”，[agent-runtime-sandbox-project-map](#) 关注“Agent 如何安全运行”，这页关注的是“Agent 工程生态如何分层组合”。

分层地图

层	代表项目 / 页面	解决的问题
End-user agents / coding agents	claude-code , OpenClaw, Hermes, OpenCode, OpenClaude, NemoClaw, nanobot	用户直接使用的 Agent、terminal coding agent、个人 assistant
Agent framework / workflow runtime	AgentScope , LangGraph, LangChain, Dify, Eino, ADK, Dapr, YoMo	构建 Agent 应用、workflow、状态图、多 Agent 协作
Tool protocol / gateway / SDK	mcp , FastMCP, GitHub MCP, Playwright MCP, agentgateway , Claude Agent SDK	工具/资源接入、协议统一、流量治理
Skills / prompt pack	Anthropic Skills, agent-skills, code review skills, architecture diagram skills	把能力做成 Markdown + scripts + workflow 的可迁移包
Memory / context / observability / eval	agent-memory-project-map , claude-context , Langfuse, AgentOps, OpenJudge	长期记忆、代码语义上下文、trace、cost、eval
Runtime / sandbox / cloud-native	agent-runtime-sandbox-project-map , agent-sandbox, agentcube , OpenShell, SkyPilot	隔离、会话编排、K8s、多云算力

横向对比

维度	End-user Agent	Framework / Workflow	MCP / Gateway	Skills	Memory / Context	Runtime / Sandbox
面向用户	终端用户 / 开发者	应用开发者	平台/工具开发者	Agent 使用者和作者	Agent / 平台	平台/运维
核心对象	Agent session / task	Agent / graph / event loop	tool/resource/API route	skill directory / prompt pack	memory item / context chunk	sandbox / session / pod
主要接口	CLI / chat / IDE	Python/Go SDK / HTTP service	MCP / Gateway API / xDS	Markdown + scripts	search / recall / inject	CRD / CLI / SDK
状态模型	会话历史和任务状态	AgentState / workflow state	route/session/policy state	文件系统	DB/Markdown/vector index	K8s/Gateway/session store
工程难点	体验、恢复、工具安全	可组合、并发、权限、人类确认	鉴权、RBAC、federation、observability	可迁移性、边界、依赖	写入质量、检索、注入预算	隔离、凭据、网络、恢复

生态信号

来自 [src-ai-agent-frameworks-stars](#) 的 109 个项目可以归纳出几条趋势：

- Claude Code 形成事实生态：router、templates、skills、memory、trace viewer、token tracker、IM bridge、Codex plugin、code graph 都围绕 terminal coding agent 扩展。
- OpenClaw / Hermes / OpenClaude / Claw 系信号强：个人 Agent + 多平台入口 + sandbox + memory + messaging 正在产品化。
- MCP 从协议变成基础设施层：FastMCP、GitHub MCP、Playwright MCP、Kubernetes MCP、[agentgateway](#) / Plano 都在 tool/resource 接入面竞争。
- Skills 成为可迁移能力包：Agent 能力从硬编码插件迁移到 Markdown + scripts + workflow。
- Memory/context/observability 正在产品化：[claude-mem](#)、mem0、[agentmemory](#)、powermem、[claude-context](#)、Langfuse、AgentOps 分别占不同基础设施位置。
- cloud-native runtime 进入 Agent 主线：agent-sandbox、[agentcube](#)、OpenShell、Docker Agent、SkyPilot、KAgent、kubewizard 都说明 Agent 需要隔离、调度和可观测。

关键项目剖面

[AgentScope](#)：生产 Agent 应用框架

[AgentScope](#) 的核心不是固定 DSL，而是事件流：

```
User Msg
  ↓
Agent.reply_stream()
  ↓
reasoning event / tool call event / confirm event / tool result event
  ↓
AgentState + storage + SSE
```

它把模型 provider、toolkit、MCP、skills、workspace、权限、人类确认、长任务 offload 和 FastAPI service 拆成可组合层。它适合做生产 Agent app，而不是底层安全 sandbox。

值得借鉴：

- AgentEvent / Msg 统一 SDK、SSE、存储和 UI。
- 人类确认和外部执行是 continuation event。
- 工具执行拆成 permission/context lifecycle 与 raw I/O。
- Workspace 承接 tools/MCP/skills/context offload。

[nanobot](#)：小内核个人 Agent

[nanobot](#) 的价值是“小内核 + 可插拔层”：

- 17 个 channel 通过 pkgutil + entry_points 自动发现。
- 两条 asyncio.Queue 解耦 channel 和 agent。
- 8 态 AgentLoop 状态机处理 RESTORE/COMPACT/COMMAND/BUILD/RUN/SAVE/RESPOND。
- Provider 级 fallback 对 Agent 透明。
- Skills/Memory/Dream/Heartbeat 都是上下文层，而不是硬编码编排层。

它适合研究个人 Agent 如何保持可读、可改、可恢复。与 AgentScope 相比，nanobot 更轻、更端侧、更偏个人运行；AgentScope 更服务化、更面向生产多租户。

HiClaw：Agent 平台化和声明式运维

HiClaw 代表“Agent 是 K8s 资源”的路线。它用 Worker/Team/Human/Manager CRD 表达多 Agent 系统，用 Matrix 作为协作平面，用 Higress 托管真实凭据。

它和 Python framework 的差异很大：

维度	HiClaw	普通 Agent framework
Agent 表达	CRD	Python class / graph node
部署单元	容器 / Pod	进程内对象
协作面	Matrix room	in-process bus / state graph
人在回路	默认房间成员	需要自建 UI
凭据	网关托管	常见是 Agent 持 key
运维	reconcile / RBAC / Helm	应用自己补

它说明企业级 Agent 平台不会停留在 SDK 层，最终会进入 controller、网关、IM、对象存储和权限系统。

[agentgateway](#): Agent 流量基础设施

[agentgateway](#) 说明 Agent 生态需要独立的数据面。它统一 LLM API、MCP tools 和 A2A 通信，复用 Gateway API / xDS / CEL / HBONE 等云原生治理模式。

在 frameworks 地图里，它的位置是 tool/gateway layer:

- Agent framework 通过 MCP/HTTP 访问工具。
- Gateway 负责路由、认证、RBAC、federation、observability。
- 上游可以是 LLM provider、MCP server、OpenAPI endpoint 或 A2A agent。

这和 [agent-runtime-sandbox-project-map](#) 里的安全 runtime 形成互补：sandbox 控制 Agent 进程，gateway 控制 Agent 流量。

agent-sandbox / [agentcube](#): Agent 执行基座

framework 能构建 Agent，但安全运行需要 substrate:

- agent-sandbox 把单个有状态 Agent 容器建模成 Sandbox CRD。
- [agentcube](#) 把 Sandbox 包成 HTTP invocation session。

这两者说明 Agent framework 生态正在从“库”扩展到“运行时平台”。LangGraph/AgentScope/nanobot 这类框架可以把 sandbox/session layer 作为 backend，而不是自己处理 Pod、PVC、NetworkPolicy、WarmPool。

Memory / context layer: 独立产品层

[agent-memory-project-map](#) 已经显示：Agent memory 不是框架内部一个 list，而是完整管线。代码语义上下文也类似，[claude-context](#) / milvus / hybrid-search-rrf / code-semantic-search 组成独立层。

Agent framework 越轻，越需要外部 memory/context；Agent product 越重，越倾向把 memory/context 内置成用户体验的一部分。

设计轴

1. Library、runtime、product 是三种不同形态

Agent 生态常被混称为“框架”，但至少有三种形态：

- **Library/framework**: AgentScope、LangGraph、Eino。重点是编程模型。
- **Runtime/platform**: HiClaw、agent-sandbox、AgentCube、OpenShell。重点是运行、隔离、会话和运维。
- **Product/agent**: Claude Code、OpenClaw、Hermes、nanobot。重点是用户可直接完成任务。

三者可以组合，但不要混为一谈。一个库不应该承担全部 sandbox 安全；一个 sandbox 原语也不应该决定 Agent 的 reasoning loop。

2. MCP 是协议层，不是 Agent 框架

[mcp](#) 解决 tool/resource 接入，但它不定义：

- Agent 如何循环推理。
- 工具结果如何写入上下文。
- 人类确认如何暂停/恢复。
- 多 Agent 如何协作。
- sandbox 如何隔离。
- memory 如何写入和检索。

所以 MCP 更像 TCP/HTTP 之于 Web 应用：它是必要基础，但不是完整应用框架。

3. Skills 正在替代一部分插件代码

Skills 的关键价值是把能力封装成：

```
description
  ↓
instructions / workflow
  ↓
scripts / assets / templates
  ↓
LLM decides when to invoke
```

这比硬编码插件更容易迁移到 Claude Code、Codex、OpenCode、OpenClaw 等不同 Agent。缺点是权限、依赖、版本和测试要单独治理。

4. Observability 和 eval 会成为框架边界

Agent 框架如果只返回最终文本，就很难调试。生产 Agent 需要：

- event stream。
- tool call trace。
- token/cost。
- model/provider fallback。
- memory recall evidence。
- eval/benchmark。
- human approval logs。

[AgentScope](#) 的事件流、[nanobot](#) 的 checkpoint、Langfuse/AgentOps/OpenJudge 这类项目说明观测和评测会从周边工具变成基础层。

5. Cloud-native runtime 会吞掉“本地 demo”边界

Agent 一旦进入生产，就会需要：

- sandbox。
- credential isolation。
- network policy。
- storage/session。
- queue/offload。
- autoscaling。
- GPU/resource scheduling。

这解释了为什么 agent-sandbox、[agentcube](#)、OpenShell、HiClaw、SkyPilot 都出现在 Agent frameworks 的生态列表里。它们不是偏题，而是 Agent 工程从 demo 到 production 的必经层。

核心难点

1. Agent loop 和工具权限必须解耦

模型是否要调用工具是一回事，工具是否允许执行是另一回事。[AgentScope](#) 把 tool execution 拆成验证/权限/上下文写入与 raw I/O，[agentgateway](#) 用 CEL 做工具/流量 RBAC，[OpenShell](#) 在 sandbox proxy 做网络裁决。这个边界越清楚，系统越安全。

2. 人类确认要进入状态机

Agent workflows 经常需要审批、外部执行、人工介入。把它做成 blocking input 会破坏服务化。更稳的是 AgentScope continuation event、HiClaw Matrix room、OpenShell policy proposal 这种状态机/协作平面设计。

3. 多 Agent 协作不能只靠 in-process bus

Python 框架里的多 Agent 通常是进程内对象互调。企业场景需要身份、房间、审计、凭据隔离、文件共享和跨进程恢复。HiClaw 用 Matrix + MinIO + Higress + CRD 给出了平台化答案。

4. 跨 Agent 可迁移能力需要文件协议

Skills、memory Markdown、AGENTS.md、MCP server config 都说明：Agent 能力包更适合用文件/目录作为接口，而不是只靠 Python import。文件协议利于审计、迁移、版本控制和 LLM 读取。

5. 上下文层会越来越独立

Agent framework 内置短期 context 不够。代码语义检索、长期记忆、tool result offload、workspace artifacts 都会成为独立层。框架需要定义清楚 context provider 接口，而不是把所有上下文都塞进一个 prompt builder。

6. 产品入口和工程底座经常错位

用户看到的是 Claude Code/OpenClaw/nanobot；工程底座却是 MCP、skills、memory、sandbox、gateway、observability。项目地图的价值就是把这些层分开，避免拿“最终产品”和“底层原语”直接比较。

设计分型

分型	代表	架构重心
Terminal coding agent	claude-code , OpenCode, OpenClaw	CLI/IDE 入口、tool use、workspace、hooks
Personal always-on agent	nanobot , Hermes, OpenClaude	channels、long-running loop、memory、scheduler
Agent app framework	AgentScope , LangGraph, LangChain, Eino	event loop、workflow graph、toolkit、service
Agent platform / OS	HiClaw, OpenShell/NemoClaw	runtime、identity、policy、collaboration
Tool protocol layer	mcp , FastMCP, GitHub MCP, Playwright MCP	tool/resource 接入和 schema
Agent gateway layer	agentgateway , Plano	LLM/MCP/A2A traffic、policy、telemetry
Capability package layer	Anthropic Skills, agent-skills	Markdown + scripts + reusable workflow
		recall、semantic search、context control

分型	代表	架构重心
Memory/context layer	agent-memory-project-map , claude-context	
Runtime substrate	agent-runtime-sandbox-project-map	sandbox、session、credential、network

选型建议

目标	优先看	工程关注点
做生产多 Agent 应用框架	AgentScope	AgentEvent、tool permission、workspace/offload、FastAPI
做轻量个人 Agent 内核	nanobot	channel bus、8 态状态机、provider fallback、skills/context
做企业多 Agent 平台	HiClaw	CRD、Matrix、Higress、credential isolation
做 tool/resource 协议层	mcp / FastMCP	tool schema、transport、client identity
做 Agent 流量网关	agentgateway	Gateway API、CEL RBAC、LLM/MCP/A2A federation
做 coding agent 生态扩展	claude-code 周边	hooks、skills、memory、trace、templates
做运行时安全与会话编排	agent-runtime-sandbox-project-map	sandbox、session、policy、provider credentials
做长期记忆和上下文	agent-memory-project-map , claude-context	source-of-truth、hybrid search、context injection

当前知识库缺口

- 还缺少 OpenClaw、OpenCode、Hermes、LangGraph、Dify、Eino、ADK、FastMCP、Anthropic Skills 的深入 source 页。
- 还缺少 MCP server 生态地图，可把 FastMCP/GitHub MCP/Playwright MCP/Kubernetes MCP/agentgateway 放在同一页。
- 还缺少 Skills 项目地图，专门比较 Anthropic Skills、Codex skills、agent-skills、code-review/architecture/finance skills。
- 还缺少 Agent observability/eval 地图，整理 Langfuse、AgentOps、OpenJudge、trace viewer、token tracker。

相关页面

- [claude-code](#)
- [mcp](#)
- [agent-memory](#)
- [agent-memory-project-map](#)
- [agent-runtime-sandbox-project-map](#)
- [ai-agent-plugin-patterns](#)
- agent-sandbox
- [agentcube](#)
- [agentgateway](#)
- HiClaw
- [nanobot](#)

Agent 长期记忆

让 AI Agent 跨会话保留上下文的设计领域。核心矛盾：LLM 上下文窗口有限 + 对话历史天然噪声大 + 用户期望"上次说过的事这次还记得"。

基础形态对比

方案	存储	检索	缺点
全量对话历史	原始 message 数组	按时间截取	噪声大、token 浪费、超窗即丢
摘要压缩	LLM 生成的会话摘要	加载完整摘要	摘要损失细节、跨会话不易关联
结构化观察 + 双索引	facts / concepts / files 等可索引字段 + 向量	三层搜索协议	实现成本高，但精度和 token 效率最好

[claude-mem](#) 采用第三种。

关键设计要点

1. 记忆生命周期 ≠ 会话生命周期

claude-mem 用 session_id 双轨制：

- contentSessionId — 当前对话会话 ID（短期）
- memorySessionId — 跨 session 累积的记忆线 ID（长期）

必须分两个 ID 才能跨 session 累积同一项工作的上下文。

2. 压缩比检索更关键

LLM 的真正用途不是"问答"而是"信息浓缩"。详见 [ai-as-compressor](#)。

3. 双索引：全文 + 向量

- 全文（FTS5）——精确关键词命中
- 向量（Chroma）——语义/概念相似
- 两者结果合并排序

4. 反向调用入口 (Skill)

给 Agent 注册一个"查记忆"工具（如 mem-search Skill），提示词写清触发时机（"用户提到过去/上次时调用"），模型自己学会触发。比手动每次塞历史更优雅。

5. 边缘剥离敏感数据

敏感数据零信任处理——`<private>...</private>` 标签在 hook 层（边缘）就剥离，永不进入压缩管道。代码在 `src/utils/tag-stripping.ts`。

完整管道

详见 [event-driven-memory-pipeline](#)。

适用场景

- 长周期项目助手（代码 Agent、研究 Agent）
- 多轮调试场景（用户问"这个 bug 上次怎么改的"）
- 需要审计的 AI 操作（医疗、合规）

参考

- [src-claude-mem-architecture](#)

Claude Code

Anthropic 出品的命令行 AI Agent，运行在终端中，通过工具调用（Read / Edit / Bash / Grep 等）协助软件工程任务。

与 claude-mem 的关系

Claude Code 作为宿主 runtime，向插件暴露 6 个 Lifecycle Hook：

Hook	触发时机
Setup	插件初始化
SessionStart	启动 / <code>/clear</code> / <code>/compact</code>
UserPromptSubmit	用户提交 prompt
PreToolUse	工具调用前
PostToolUse	工具调用后
Stop	会话结束

[claude-mem](#) 通过注册这些 hook 实现无侵入的事件采集。Hook 必须返回 `exit 0`（否则 Windows Terminal 会累积标签页）。

关键限制

- 本身没有跨会话记忆——每次开新会话都是白板
- 这正是 [claude-mem](#) 试图解决的问题

生态位

- 宿主：Claude Code CLI
- SDK： [claude-agent-sdk](#)（用于在脚本/插件内调用 Claude 做编程任务）
- 插件机制： `plugin/hooks/hooks.json` 注册生命周期钩子（如 [claude-mem](#)）
- Skill 机制：通过 `SKILL.md` 注册可被 Agent 自主调用的工具（如 `claude-mem` 的 `mem-search`）
- [MCP](#) 客户端：通过 `claude mcp add` 加载第三方工具 server（如 [claude-context](#)）

参考

- [src-claude-mem-architecture](#)

Agent Framework 编程模型对比地图

这页比较“写 Agent 应用”的框架，而不是直接可用的 coding agent 产品。核心问题是：你希望用 graph、workflow、event loop、role-play multi-agent，还是 Go 代码优先框架。

GitHub 当前核验

截至 2026-06-11 通过 GitHub API 重新核验：

项目	仓库	最近 push	stars	主语言	定位
LangGraph	https://github.com/langchain-ai/langgraph	2026-06-11	34k	Python	resilient agents graph runtime
LangChain	https://github.com/langchain-ai/langchain	2026-06-11	138k	Python	agent engineering platform
Dify	https://github.com/langgenius/dify	2026-06-11	144k	TypeScript	production-ready agentic workflow platform
AgentScope	https://github.com/agentscope-ai/agentscope	2026-06-09	26k	Python	see/understand/trust agents
Eino	https://github.com/cloudwego/eino	2026-06-11	11k	Go	Go LLM/AI app framework
ADK Go	https://github.com/google/adk-go	2026-06-10	8k	Go	code-first Go toolkit for AI agents
AutoGen	https://github.com/microsoft/autogen	2026-04-15	58k	Python	programming framework for agentic AI
CrewAI	https://github.com/crewAIInc/crewAI	2026-06-11	53k	Python	role-playing autonomous agent orchestration

选型

需求	推荐
状态图、可恢复 agent workflow	LangGraph
Python LLM app 生态和组件库	LangChain

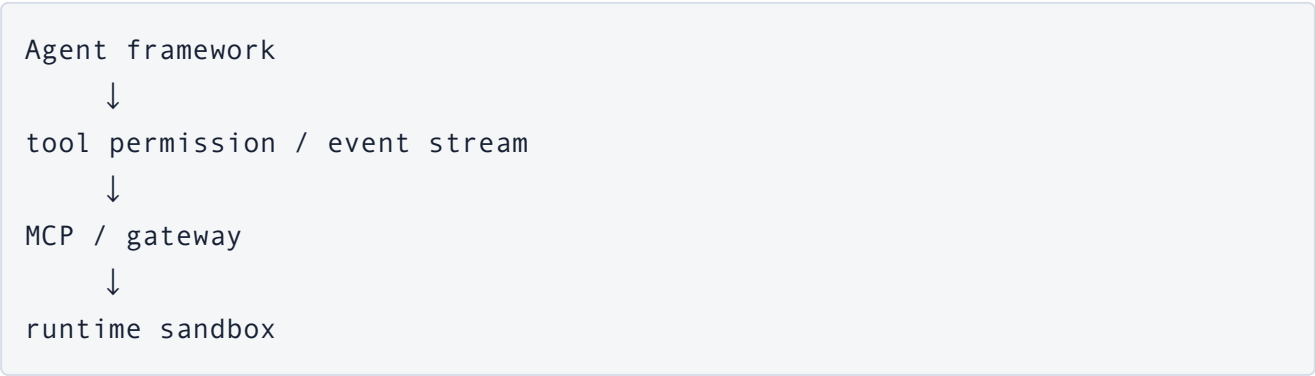
需求	推荐
可视化/产品化 workflow 平台	Dify
事件流、tool/MCP/skill、服务化 Agent app	AgentScope
Go 代码优先 LLM app	Eino / ADK Go
多 Agent 对话/协作研究	AutoGen / CrewAI

编程模型差异

维度	LangGraph	Dify	AgentScope	Eino / ADK Go	AutoGen / CrewAI
主抽象	graph/state	workflow app	AgentEvent / Msg / toolkit / workspace	typed Go components	agents / roles / conversations
人在回路	graph interrupt/checkpoint	workflow 节点/UI	continuation event / confirm event	代码自建	conversation callback
Tool 接入	LangChain tools / MCP bridge	platform tools	Toolkit / MCP / Skill	Go tool interfaces	function/tool calls
服务化	LangGraph platform / app	内置平台	FastAPI service	自建服务	自建或框架服务
最适合	可恢复复杂流程	低代码产品平台	可观测 Agent 应用	Go infra 团队	多 Agent 角色编排

和 runtime/sandbox 的边界

这些框架通常不应该自己承担强 sandbox。更稳的组合是：



例如 [AgentScope](#) 可以编排 tool 和 workspace，但如果要运行不可信代码，仍应接 [agent-runtime-sandbox-project-map](#) 中的 sandbox 或外部 workspace backend。

避坑条件

- “多 Agent”不是自动带来更好结果；先确认任务是否真的需要角色分工。
- 图式框架适合可恢复流程，但简单 coding task 可能过重。
- 低代码平台适合快速产品化，但深度 runtime/sandbox 改造成本高。
- Go 框架适合基础设施团队，但 Python 生态的模型/provider/tool 覆盖通常更快。

Agent Memory 细分选型矩阵

已有 [agent-memory-project-map](#) 做了完整横向地图。这页进一步按“选型问题”拆：你要给 coding agent、产品应用、企业平台还是可审计知识库加记忆。

GitHub 当前核验

截至 2026-06-11 通过 GitHub API 重新核验：

项目	仓库	最近 push	stars	主语言	当前信号
claude-mem	https://github.com/thedotmack/claude-mem	2026-06-11	81k	JavaScript	从 Claude Code 扩展到多 Agent 的 persistent context
agent-recall	https://github.com/mnardit/agent-recall	2026-04-03	12	Python	小而专注，SQLite + MCP scope memory
agentmemory	https://github.com/rohitg00/agentmemory	2026-06-10	22k	TypeScript	跨 coding agent，本地 worker + MCP/REST
powermem	https://github.com/oceanbase/powermem	2026-06-10	703	Python	应用级 memory middleware，OceanBase/SeekDB 路线
memsearch	https://github.com/zilliztech/memsearch	2026-06-01	1.9k	Python	Markdown truth + Milvus shadow index
tencentdb-agent-memory	https://github.com/TencentCloud/TencentDB-Agent-Memory	2026-06-04	5.2k	TypeScript	OpenClaw/Hermes 本地分层记忆

按需求选

需求	推荐	为什么
Claude Code 用户马上要跨会话记忆	claude-mem	hook 接入自然，自动采集和注入路径成熟
多个 coding agent 共用本地记忆	agentmemory 或 memsearch	agentmemory 偏服务/工具面，memsearch 偏 Markdown 可审计
多项目/多客户 scoped facts	agent-recall	scope hierarchy、bitemporal slots、MCP instructions 更明确

需求	推荐	为什么
产品应用内用户记忆	powermem	SDK/API/MCP/Dashboard、provider 和存储适配完整
OpenClaw/Hermes 长任务和 persona	tencentdb-agent-memory	L0→L3 语义金字塔 + context offload
人类要能 review/修正记忆	memsearch	Markdown source-of-truth，索引可重建

架构差异

维度	claude-mem	agent-recall	agentmemory	powermem	memsearch	tencentdb-agent-memory
写入边界	hook 被动采集	Agent 主动 MCP 写入	hooks/REST/MCP 多入口	SDK/API 显式写入	hook 追加 Markdown	hook 捕获 + 分层 pipeline
真相层	SQLite observations	SQLite graph/slots	iii-engine SQLite KV	OceanBase/SeekDB/PG/SQLite	Markdown files	JSONL/files + SQLite/TCVDB
索引层	FTS5 + Chroma	FTS5/LIKE	BM25 + vector + graph	vector + FTS + sparse + graph	Milvus dense + sparse	SQLite/TCVDB hybrid
LLM 成本	压缩核心	briefing 可选/cache	默认零 LLM	抽取/评估核心	维护摘要/可选	L0→L3 管线核心
注入策略	SessionStart 自动上下文	scoped briefing	默认可关闭注入	应用决定	search/expand 渐进	before_prompt_build + offload

关键取舍

- 自动记忆 vs 主动记忆：[claude-mem](#)、[agentmemory](#)、[tencentdb-agent-memory](#) 覆盖率高但需要去噪；[agent-recall](#) 结构更清楚但依赖 Agent 主动保存。
- 数据库 truth vs Markdown truth：数据库查询快、结构强；Markdown 可 diff、可审计、可手改。[memsearch](#) 是 Markdown truth 的代表。
- 零 LLM 默认 vs AI 压缩：[agentmemory](#) 默认保护 token 成本；powermem 和 [tencentdb-agent-memory](#) 更依赖 LLM 抽取换取语义质量。
- 个人插件 vs 服务化层：[claude-mem](#) 贴近宿主；powermem 更像应用基础设施；[agentmemory](#) 在中间。

避坑条件

- 记忆系统必须区分 truth store 和 shadow index；不要把 embedding collection 当唯一事实。
- 自动注入默认要保守，否则会消耗 coding agent 上下文窗口。
- 多 Agent 共享记忆必须有 scope / tenant / project 边界。
- “支持 MCP”不等于写入策略成熟；要看 tool instructions、权限 enforcement 和恢复模型。

Agent Runtime / Sandbox 项目地图

这页把当前知识库里和 Agent runtime、sandbox、会话编排、安全控制面相关的项目放到同一张工程地图里。核心结论：这些项目都在解决“Agent 怎么安全、可恢复、可观测地长期运行”，但抽象层不一样。

```
Developer / user API
  ↓
Agent app framework / CLI / SDK
  ↓
session orchestration / agent control plane
  ↓
sandbox lifecycle primitive
  ↓
runtime enforcement: container / supervisor / proxy / policy
  ↓
network, credentials, storage, inference, tool access
```

通用 Agent Session 生命周期

```
Create / Claim
  ↓
Provision or WarmPool adoption
  ↓
Ready + route + identity
  ↓
Running session
  ├── tool/model/network request → policy → external service
  ├── idle timeout → suspend / hibernate
  ├── checkpoint → snapshot / PVC / workspace
  ├── resume → restore route + identity
  └── expiry / delete → revoke → cleanup / retain data
```

这个方向不能只看“能不能启动一个容器”。Agent runtime 的特殊性在于：它是长寿命、有状态、会执行不可信输入、会持有工具能力、会访问模型和外部网络，并且经常需要人类中途介入。真正的系统边界包括生命周期、身份、凭据、网络、文件、推理路由、工具权限和恢复。

一句话分层

项目	一句话定位	抽象层
agent-sandbox	K8s SIG Apps 的 Sandbox CRD，把单个有状态 Agent 容器建模成 K8s 一等资源	Sandbox 基础设施原语
openkruise-agents	OpenKruise 的 Agent sandbox lifecycle platform，把 SandboxSet/Claim、E2B API、Envoy 路由和 runtime/CSI/identity 扩展组合起来	K8s sandbox 平台层
agentcube	基于 agent-sandbox 的 serverless session orchestration layer，给 Code Interpreter / AgentRuntime 提供 HTTP/SDK 调用入口	会话编排层
OpenShell	NVIDIA 的安全私有 Agent runtime，Gateway 控制面 + sandbox Supervisor + OPA/Z3 policy proxy	安全 runtime / enforcement
NemoClaw	OpenShell sandbox 内 always-on Agent 的 host-side CLI 控制面，编排 onboarding、provider、policy 和 agent setup	Host-side 编排层
HiClaw	K8s operator + Matrix IM + Higress 网关的多 Agent 协作平台	多 Agent 平台 / 协作层
AgentScope	Python 多 Agent 应用框架，事件流 ReAct loop + toolkit/MCP/skill + workspace/offload + FastAPI service	应用框架 / 服务层
agentgateway	面向 LLM / MCP / A2A 的 Rust L7 数据面 + Gateway API 控制面	Agent 网络 / 工具网关

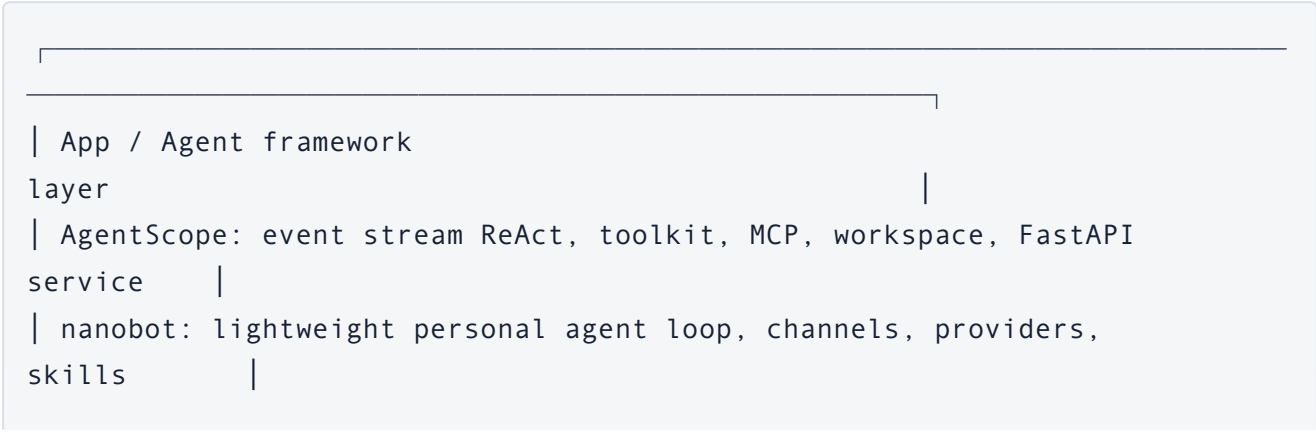
横向对比

维度	agent-sandbox	openkruise-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
主问题	安全运行一个有状态 Agent 容器	K8s 上运营 E2B-like sandbox 服务	把 Sandbox 变成可调用 session	sandbox 内 runtime enforcement	OpenShell onboarding + agent setup	多 Agent 运维与协作	Agent app 构建与服务化	AI 协议流量治理
用户入口	K8s CRD / Go/Python SDK	E2B API + K8s CRD	HTTP invocation / Python SDK / Dify /	CLI / SDK / TUI	nemoclave / nemohermes CLI	Matrix / CLI / CRD	Python SDK / FastAPI	Gateway API / xDS / UI

维度	agent-sandbox	openkruise-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
			LangChain					
控制面	K8s controller-runtime	sandbox-manager + agent-sandbox-controller	Router + WorkloadManager + Redis	Gateway object store + compute runtime	Host-side FSM + OpenShell adapters	K8s operator + embedded/in-cluster mode	FastAPI service + storage / session manager	Go controller + ADS server
数据面	Pod + headless Service + PVC	Envoy ext_proc / Go filter + Sandbox Pod	Sandbox Pod / PicoD / reverse proxy	Supervisor + restricted child + policy proxy	OpenShell sandbox + OpenClaw/ Hermes	Worker/ Manager containers + Matrix rooms	Agent event loop + workspace	Rust proxy for LLM/ MCP/ A2A
生命周期	Sandbox 0/1 replicas, shutdownTime, WarmPool	SandboxSet pool、Claim、pause/resume、clone/checkpoint	session create/route/GC, WarmPool claim	create/delete/watch/reconcile/resume	onboard resume/fresh, registry lock	CR reconcile worker/team/human/manager	session run serialization, continuation events	listener / route/ backend config hot update
隔离机制	委托 K8s runtimeClass/ NetworkPolicy/ securityContext	K8s runtime + agent-runtime/ CSI/ traffic-proxy 扩展	继承 agent-sandbox + Router JWT	Supervisor/ proxy/seccomp/netns/TLS/OPA	继承 OpenShell policy/shields	容器隔离 + Higress 凭据隔离	Workspace Local/ Docker /E2B	CEL RBAC, mTLS/ HBONE, guardrails
凭据	不直接托管凭据	E2B API key	Router → PicoD	Gateway 托管		Higress 托管真	Credential/	backend

维度	agent-sandbox	openkruis e-agents	agentcube	OpenShell	NemoClaw	HiClaw	AgentScope	agentgateway
模型		storage + identity token propagation	JWT, WorkloadManager auth	provider credentials	Gateway 是凭据 system-of-record	实凭据, Worker 只持 consumer key	session / workspace service	credentials + policy/filter
状态	K8s API + PVC	K8s API + route registry + key storage + memberlist	Redis/ValKey session registry + K8s	SQLite / PostgreSQL object store	~/ .nemoclaw JSON + Gateway state	etcd/kine + Matrix + MinIO + gateway	storage / session / workspace	xDS stores + control plane
典型强项	K8s-native、简单、WarmPool	CRD + E2B API + Envoy route + runtime 扩展一体化	产品化 invocation API	安全 enforcement 边界最细	Onboarding 和 provider/policy 编排	IM-first 协作和凭据零暴露	Agent app 编程体验	LLM/MCP/A2A 统一网关
典型代价	只提供底层原语	系统面宽, API/roadmap 仍在快速演进	依赖 agent-sandbox API 演进	系统复杂度高	host-side 状态机复杂	学习曲线和容器开销	不是 sandbox 原语	不直接运行 Agent

分层图谱



↓

| Multi-agent / session product
layer |
| HiClaw: Worker/Team/Human/Manager CRD + Matrix +
Higress |
| OpenKruise Agents: SandboxSet/Claim + E2B API + Envoy
route |
| AgentCube: AgentRuntime/CodeInterpreter + Router +
WorkloadManager |
| NemoClaw: OpenShell onboarding + OpenClaw/Hermes
setup |

↓

| Sandbox / runtime primitive
layer |
| agent-sandbox: Sandbox/Claim/Template/WarmPool → Pod/PVC/
Service |
| OpenShell: Gateway desired state → driver → Supervisor/proxy
enforcement |

↓

| Network / credential / tool gateway
layer |
| agentgateway: LLM/MCP/A2A proxy, Gateway API, CEL, HBONE,
xDS |
| Higress/OpenShell gateway: provider credentials and inference
routing |

关键关系：

- [agentcube](#) 明确站在 agent-sandbox 之上：前者做 session API，后者做 Sandbox 生命周期。
- [openkruise-agents](#) 站在 agent-sandbox 相邻但更平台化的位置：它把 SandboxSet/Claim、E2B API、Envoy 路由、agent-runtime、CSI 和 identity token 放到同一套 K8s 发行版里。
- [NemoClaw](#) 明确站在 [OpenShell](#) 之上：前者做 onboarding 和 agent setup，后者做 sandbox runtime 和 enforcement。
- HiClaw 和 agent-sandbox 是互补关系：HiClaw 管多 Agent 协作，agent-sandbox 可以成为底层隔离原语。
- [AgentScope](#) 和 agent-sandbox 处在相邻问题域：AgentScope 的 Workspace 能接 Local/Docker/E2B，agent-sandbox 是更云原生的执行环境原语。
- [agentgateway](#) 不运行 Agent，但控制 Agent 访问模型、MCP、A2A 的网络边界。

工程剖面

agent-sandbox：Sandbox 作为 K8s 原语

agent-sandbox 的核心贡献是把 Agent runtime 从普通 Deployment/StatefulSet 里抽出来。Agent sandbox 的语义是：

- 单实例，replicas 只能 0/1。
- 有稳定身份，自动创建 headless Service。
- 有持久存储，PVC 生命周期不随 Pod 自动删除。
- 可以暂停/恢复，scale to 0/1。
- 可以过期，shutdownTime 是声明式生命周期。
- 可以 WarmPool 预热，然后通过 Claim 领养。

它避免把安全机制做成私有实现，而是把隔离委托给 K8s 原语：runtimeClassName、gVisor/Kata、NetworkPolicy、securityContext、serviceAccount。这个设计的工程价值是可组合：任何上层平台都可以继承 K8s 生态能力。

风险也清楚：它只提供底层原语，不负责多 Agent 协作、工具协议、凭据网关、HTTP invocation 或 Agent app framework。这些必须由上层补齐。

[openkruise-agents](#)：把 Sandbox 原语扩展成 E2B-like 平台

[openkruise-agents](#) 的关键不是再造一个最小 Sandbox CRD，而是把 Agent sandbox 生命周期的产品面补齐：

```
E2B API / K8s CRD
  ↓
sandbox-manager
  ↓
SandboxClaim / SandboxSet / Sandbox
```

```
↓
agent-sandbox-controller
↓
Envoy route registry → Sandbox Pod IP:port
```

它把 warm pool 明确建模成 `SandboxSet`，把用户请求建模成 `SandboxClaim`，把 API 入口做成 E2B-compatible /sandboxes / /snapshots，再通过 Envoy ext_proc 或 Go filter 把请求路由到具体 sandbox。这个组合适合做自托管 E2B-like workspace 服务。

风险在于系统面比 agent-sandbox 大很多：key storage、route registry、memberlist peer sync、dynamic CSI mount、identity token propagation、checkpoint/hibernation 和 runtime sidecar 都需要分别验证。选型时应把它看成平台工程项目，而不是单一 controller。

[agentcube](#)：把 Sandbox 包成可调用会话

[agentcube](#) 的定位比 agent-sandbox 高一层。它关心开发者如何通过 HTTP/SDK 调用一个 Code Interpreter 或 AgentRuntime，而不是让用户直接写 Sandbox CR。

关键路径：

```
Client invocation
↓
Router checks x-agentcube-session-id
↓
WorkloadManager creates Sandbox or SandboxClaim
↓
agent-sandbox starts/adopts Pod
↓
Router reverse-proxies to PodIP/entrypoint
↓
Redis/ValKey tracks session TTL and activity
```

它最重要的架构判断是不重写 `Sandbox controller`。WorkloadManager 只把 AgentCube 的 AgentRuntime / CodeInterpreter 翻译成 agent-sandbox CRD，底层 lifecycle、WarmPool adoption、Pod/PVC/Service 都交给 agent-sandbox。

安全上，PicoD 用 Router-signed JWT 替代 SSH；这简化 SDK，但 Router 变成强信任边界，后续需要 auth、mTLS、RBAC、审计一起兜住。

[OpenShell](#)：Gateway desired state, Supervisor runtime enforcement

[OpenShell](#) 的分层边界最安全工程化：

```
Gateway owns desired state
↓
compute driver provisions sandbox
↓
Supervisor owns runtime enforcement
↓
policy proxy controls process/network/inference egress
```

Gateway 保存 sandbox、provider、policy、settings、inference、sessions 等状态；sandbox 内 Supervisor 才真正执行网络、进程、文件、TLS、provider credential 注入和 proxy enforcement。这个边界成立，因为只有 sandbox 内能看见进程身份、socket、binary hash、TLS 流和运行时凭据。

OpenShell 的关键不是“有 sandbox”，而是把 Agent egress 变成可审计、可裁决、可提案的策略流。普通网络访问走 OPA；inference.local 是特殊本地能力，代理到 provider，但 Agent 不直接持有真实 key。

NemoClaw: OpenShell 上的 host-side Agent 编排

NemoClaw 是 OpenShell 的产品化编排层，不是另一个推理或 sandbox 引擎。它把用户真正要完成的步骤串起来：

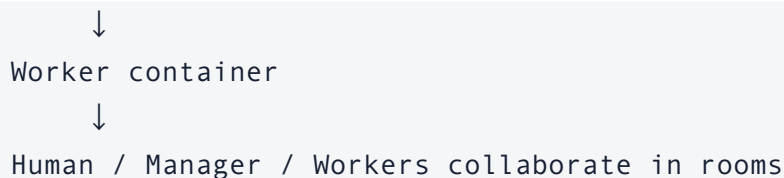
- preflight 检查 Docker/OpenShell/GPU/DNS。
- 启动或复用 named OpenShell gateway。
- 选择 provider/model/credential route。
- 创建或复用 sandbox。
- 安装/同步 OpenClaw 或 Hermes runtime config。
- 应用 baseline policy 和 presets。
- 编译 messaging channel manifest。

它的核心是可恢复 onboard FSM：host-side JSON registry、session state、全局 lock、resume/fresh/non-interactive。这个模式适合复杂本地开发工具，但也带来状态恢复、部分写入和版本迁移成本。

HiClaw: Agent 是 CR, 也是 IM 用户

HiClaw 的设计很鲜明：把 Agent 运维做成 K8s operator，同时把协作面放到 Matrix IM。

```
Worker CR
↓
controller reconcile
↓
Matrix user + room
↓
Higress consumer key
```



这解决两个常被 Agent framework 忽略的问题：

- 人类如何自然介入：直接进 Matrix 房间。
- Agent 凭据如何隔离：Worker 只拿 Higress consumer key，真实 provider/API key 在网关侧。

它的代价是平台复杂度高。每个 Worker 是容器、Matrix 用户、gateway consumer 三位一体；Provisioner 同时编排 Matrix/Higress/Credentials/Manager bootstrap，横向拆分压力已经出现。

AgentScope：应用框架里的 runtime 边界

AgentScope 不提供强 sandbox 原语，但它定义了生产 Agent 应用的运行时边界：

- AgentEvent / Msg 是 SDK、SSE、存储和 UI 的统一协议。
- ReAct loop 把人类确认和外部执行建模成 continuation event。
- Tool execution 被拆成权限/上下文写入和 raw I/O。
- Workspace 承接 tools、MCP、skills 和 context/tool-result offload。
- FastAPI service 每轮从 storage/session/workspace 重新组装 Agent。

这类框架更适合应用服务，而不是底层隔离。它可以把 Docker/E2B/未来 agent-sandbox 作为 Workspace backend，把自身保持在 Agent 编排与事件流层。

agentgateway：Agent 访问外界的网络控制点

agentgateway 解决的是 Agent 流量，而不是 Agent 生命周期。它把 LLM API、MCP tools、A2A 通信统一到一个 Rust L7 数据面，控制面复用 Gateway API/xDS。

对 runtime/sandbox 体系来说，它的价值是：

- LLM provider 统一协议和 guardrails。
- MCP tools federation + CEL RBAC。
- A2A Agent Card URL rewrite 和代理。
- HBONE/mTLS/telemetry/access log。
- 用声明式 CRD 管路由、策略和后端。

如果 agent-sandbox 是“Agent 跑在哪里”，agentgateway 就是“Agent 能访问什么、怎么访问、怎么审计”。

核心难点

1. 生命周期不是 Pod 生命周期

Agent sandbox 的生命周期包含：创建、恢复、暂停、续期、过期、热领养、GC、startup resume、长任务 offload。这比普通 Pod 的 Running/Terminated 更复杂。

- agent-sandbox 用 shutdownTime 和 0/1 replicas 表达暂停/过期。
- [openkruise-agents](#) 用 SandboxSet / SandboxClaim 表达 pool 与领取，用 E2B API 管 pause/resume/clone/checkpoint。
- [agentcube](#) 用 Redis sorted set 管 session TTL 和 last activity。
- [OpenShell](#) 用 Gateway store + driver watch/reconcile/resume。
- [NemoClaw](#) 用 host-side FSM 和 registry 做 onboarding resume。

结论：Agent runtime 必须显式建模“会话”和“工作负载”的差异。

2. 凭据不能进 Agent 进程

Agent 会执行模型生成的工具调用，也会读取用户输入。真实 API key、GitHub PAT、云 AK 如果进入 Agent 进程，就会被 prompt injection 放大。

成熟方向都是网关托管：

- HiClaw：真实 key 在 Higress，Worker 只持 consumer key。
- [OpenShell](#) / [NemoClaw](#)：Gateway 是 provider credential system-of-record，sandbox 通过 inference.local 使用模型。
- [agentgateway](#)：后端 credentials 与 policy/filter 在网关侧。

这和 agent-credential-isolation 是同一条设计线。

3. 网络策略需要进程级语义

K8s NetworkPolicy 只能管 Pod 维度，Agent sandbox 内可能有多个进程：agent、tool、shell、browser、package manager。[OpenShell](#) 通过 proxy + /proc/net/tcp + binary identity 把网络裁决细化到进程/二进制身份，这是更细粒度的 runtime enforcement。

对于大多数平台，第一阶段可以用 NetworkPolicy + egress proxy；高风险场景才需要 OpenShell 这种 Supervisor/proxy 模型。

4. WarmPool 是性能优化，也是所有权协议

预热不是“提前启动 Pod”这么简单。关键难点是 ready sandbox 被 claim 领养时，ownerReference、labels、status、Pod exclusivity 必须一致。

agent-sandbox 的 SandboxClaim adoption 协议是底层实现；[agentcube](#) 只在 CodeInterpreter warmPoolSize 上暴露能力。这是合理分工：底层保证所有权不变量，上层只表达“我需要低冷启动”。[openkruise-agents](#) 则把 warm pool 升级成平台级 SandboxSet + SandboxClaim 协议，并在 claim 时叠加 inplace update、dynamic CSI mount、runtime init 和 identity token。

5. 人在回路不是 UI 附加项

多 Agent 系统需要人类在任务中途介入、批准、观察和接管。

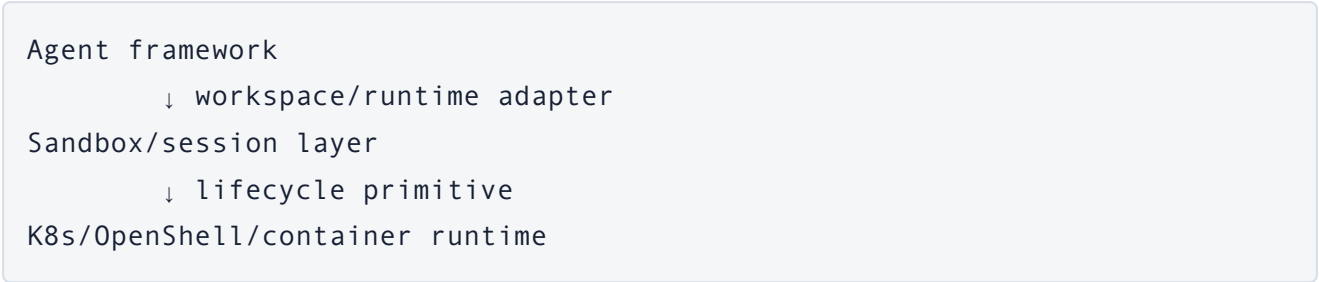
- HiClaw 用 Matrix 房间把人类和 Agent 放在同一个协作平面。
- [AgentScope](#) 用 RequireUserConfirmEvent 和 continuation event 把人类确认放进事件流。
- [OpenShell](#) 的 policy proposal 默认人工审批。

人在回路应该是协议和状态机能力，不应该只是一个前端按钮。

6. App framework 和 sandbox runtime 要分层

AgentScope、nanobot 这类应用框架解决“如何思考、调用工具、持久化上下文”；agent-sandbox/OpenShell 解决“如何安全运行”。把两层混在一起会导致框架既要懂 ReAct，又要懂容器、网络、凭据、恢复，复杂度失控。

比较稳的组合是：



设计分型

分型	代表项目	设计重心	适合场景
Sandbox 原语型	agent-sandbox	CRD、Pod/PVC/Service、WarmPool、K8s 原语	云原生 Agent 执行环境底座
Session 编排型	agentcube	HTTP invocation、session registry、Router/WorkloadManager	Code interpreter / serverless agent runtime
安全 runtime 型	OpenShell	Gateway desired state、Supervisor、policy proxy	高风险 autonomous agent、凭据隔离、网络审计
Host-side 编排型	NemoClaw	onboarding FSM、provider/policy/agent setup	把安全 runtime 包成可用产品

分型	代表项目	设计重心	适合场景
多 Agent 平台型	HiClaw	CRD + IM + 网关凭据 + Worker/Team	企业多 Agent 协作和运维
应用框架型	AgentScope , nanobot	ReAct loop、event stream、tools、workspace	构建 Agent 应用或个人 Agent
网络网关型	agentgateway	LLM/MCP/A2A 流量、policy、telemetry	Agent 外部访问治理

选型建议

目标	优先看	工程关注点
想把 Agent 容器建模成 K8s 原语	agent-sandbox	Sandbox/Claim/WarmPool、PVC 生命周期、NetworkPolicy
想做 Code Interpreter / AgentRuntime 会话服务	agentcube	Router/WorkloadManager、session registry、WarmPool adoption
想研究最细的安全 sandbox runtime	OpenShell	Gateway/Supervisor 分界、policy proxy、inference.local
想把 OpenShell 包成用户可用 CLI 工作流	NemoClaw	onboard FSM、provider/policy/messaging manifest
想做多 Agent 协作平台	HiClaw	CRD、Matrix、Higress、credential isolation
想做 Agent 应用服务	AgentScope	AgentEvent、tool permission、workspace/offload、FastAPI
想治理 Agent 的 LLM/MCP/A2A 流量	agentgateway	Gateway API、xDS、CEL、MCP federation、guardrails

相关页面

- [agent-sandbox](#)
- [agentcube](#)
- [HiClaw](#)
- [agentgateway](#)
- [declarative-agent-management](#)
- [agent-credential-isolation](#)
- [cloud-native-security](#)
- [ai-agent-plugin-patterns](#)

- [mcp](#)

Agent Skills / Plugin System 对比地图

Skills 和 plugins 在 Agent 生态里解决不同问题：plugin 更像宿主扩展点，skill 更像可迁移能力包，MCP tool 更像可调用外部能力。本页按工程边界对比。

GitHub 当前核验

截至 2026-06-11 通过 GitHub API 重新核验：

项目	仓库	最近 push	stars	主语言	定位
Anthropic Skills	https://github.com/anthropics/skills	2026-06-09	149k	Python	Agent Skills 官方公共仓库
agent-skills	https://github.com/addyosmani/agent-skills	2026-06-11	52k	Shell	面向 AI coding agents 的工程 skills
Matt Pocock skills	https://github.com/mattpocock/skills	2026-06-10	124k	Shell	个人工程 skills 集合
awesome-agent-skills	https://github.com/libukai/awesome-agent-skills	2026-03-26	4.6k	Python	skills 指南与精选资源

三种扩展形态

形态	例子	适合	不适合
Host plugin	claude-code-plugin , OpenCode plugin	生命周期 hook、UI/CLI 集成、宿主事件采集	跨宿主迁移
Skill pack	Anthropic Skills, agent-skills, Codex skills	操作流程、领域方法、脚本模板、可迁移经验	需要长期 daemon 或强权限 server
MCP tool	GitHub MCP, Playwright MCP	标准 tool/resource 访问	复杂 workflow 说明和人工判断

选型判断

- 需要监听 UserPromptSubmit / PostToolUse / Stop：写 plugin。
- 需要教 Agent “怎么做代码审查/架构图/调研”：写 skill。
- 需要访问 GitHub、浏览器、K8s、数据库：写 MCP server 或接现成 server。

- 需要跨 Claude Code / Codex / OpenCode 复用：优先 skill + scripts，宿主适配层尽量薄。

架构差异

维度	Plugin	Skill	MCP server
触发	宿主生命周期或命令	LLM 根据描述选择	LLM tool call
状态	可持有宿主状态	文件/脚本/模板为主	server 进程/外部系统状态
权限	常接近宿主权限	取决于脚本执行权限	取决于 server 凭据
可迁移	低到中	高	中
测试方式	宿主集成测试	prompt + script 测试	tool schema + server 测试

避坑条件

- skill 描述要短而可判别，否则 Agent 不会在正确时机调用。
- plugin 不要把重任务放在 hook 同步路径，参考 [event-driven-memory-pipeline](#) 的薄 hook 模式。
- MCP server 不要直接暴露全权限 token，优先通过 gateway/secret 管理。
- 同一能力不要同时做成 plugin、skill、MCP，除非边界清楚：plugin 负责捕获，skill 负责流程，MCP 负责外部动作。

Coding Agent / Personal Agent 选型地图

这页聚焦“用户直接使用的 Agent”：终端 coding agent、个人长跑 agent、Agent OS 和带 sandbox 的桌面/运行时封装。它和 [agent-framework-programming-model-map](#) 的区别是：这里先看产品入口和日常开发体验，不是先看框架 API。

GitHub 当前核验

截至 2026-06-11 通过 GitHub API 重新核验：

项目	仓库	主语言	最近 push	GitHub 信号	一句话
claude-code	https://github.com/anthropics/claude-code	Python	2026-06-10	131k stars	官方 terminal agentic coding tool
OpenCode	https://github.com/anomalyco/opencode	TypeScript	2026-06-11	172k stars	开源 coding agent 主线
OpenClaude	https://github.com/Gitlawb/openclaude	TypeScript	2026-06-11	28k stars	“runs anywhere, uses anything” 的 OpenClaude 路线
NemoClaw	https://github.com/NVIDIA/NemoClaw	TypeScript	2026-06-11	21k stars	在 OpenShell 内安全运行 Hermes / OpenClaw
nanobot	https://github.com/HKUDS/nanobot	Python	2026-06-10	44k stars	多渠道个人长跑 Agent

这些仓库都还在活跃变化，旧版 source summary 只能作为架构底稿，选型时应以当前 README、release、examples 和目录结构为准。

选型结论

场景	首选	原因	避免条件
终端内软件工程主力	claude-code	官方产品、hook/MCP/skills 生态完整，适合代码读写和 git workflow	需要完全开源内核或自定义运行时
想要开源 coding agent 主线	OpenCode	TypeScript 生态、活跃、适合研究 terminal coding agent 的开放实现	依赖商业 Claude Code 生态能力
多平台个人 Agent / Agent OS 实验	OpenClaude / OpenClaw 系	强调不同环境、工具和宿主适配	需要稳定企业部署 SLA

场景	首选	原因	避免条件
安全 sandbox 内运行 Agent	NemoClaw + OpenShell	明确处理 provider routing、policy、sandbox、credential 边界	只想要轻量本地 CLI
IM / 多渠道个人 assistant	nanobot	17 channel、provider fallback、MCP、skills/memory 都在小内核里	主要需求是大型代码库自动修改

架构差异

维度	claude-code	OpenCode	OpenClaude / OpenClaw 系	NemoClaw	nanobot
主入口	CLI / IDE-like terminal	CLI	Agent runtime / product shell	nemoclaw / nemohermes CLI	CLI / IM / WebSocket / API
扩展方式	hooks / slash commands / skills / mcp	plugin / config / tools	runtime adapter / tools / memory	blueprint + onboard FSM + OpenShell adapter	pkgutil entry_points + skills markdown
状态模型	session transcript + CLAUDE.md + hooks	repo/session state	runtime/session state	host registry + OpenShell gateway state	8 态 loop + checkpoint
安全边界	依赖本地权限、hooks、MCP server 配置	依赖 agent 实现和 shell 权限	取决于 runtime	OpenShell supervisor / policy proxy / inference.local	轻量本地进程，非强 sandbox
记忆接入	claude-mem , memsearch , agentmemory	memsearch , agentmemory	tencentdb-agent-memory , memsearch	memory provider + sandbox runtime	内置 Markdown memory / dream

关键判断：Claude Code / OpenCode 是“coding loop 产品”，NemoClaw / OpenShell 是“安全运行底座”，nanobot 是“个人长跑 Agent 内核”。它们不是互斥层，可以组合。

快速采用路径

- 个人开发效率：先用 [claude-code](#)，再加 [claude-context](#) 做代码语义检索、加 [agent-memory-project-map](#) 中的 memory 插件。
- 开源实现研究：用 OpenCode 对照 Claude Code 的工具循环、会话恢复、插件格式。
- 安全执行研究：从 [src-openshell-architecture](#) + [src-nemoclaw-architecture](#) 读起，看 gateway desired state、sandbox supervisor、provider credential routing。
- 多渠道 agent：用 [nanobot](#) 观察 channel/provider/skill 如何保持小内核。

后续核验清单

- OpenCode / OpenClaude 的插件和工具协议是否已稳定。
- Claude Code 官方 repo 是否只承载安装包/文档，核心实现是否可审计。
- NemoClaw 对 OpenShell / OpenClaw / Hermes 版本 pin 是否频繁变动。
- nanobot 的 channel/provider 插件 API 是否已承诺兼容。

MCP Server / Tool Gateway 对比地图

MCP 生态已经从“给 Agent 接几个工具”扩展成三层：SDK/framework、具体工具 server、gateway/proxy/data plane。本页把 FastMCP、官方/垂直 MCP server 和 agentgateway/Plano 放在一起看。

GitHub 当前核验

截至 2026-06-11 通过 GitHub API 重新核验：

项目	仓库	最近 push	stars	主语言	定位
FastMCP	https://github.com/PrefectHQ/fastmcp	2026-06-06	25k	Python	Pythonic MCP servers/clients
GitHub MCP	https://github.com/github/github-mcp-server	2026-06-10	30k	Go	GitHub 官方 MCP server
Playwright MCP	https://github.com/microsoft/playwright-mcp	2026-06-10	33k	TypeScript	浏览器自动化 MCP server
kubectl MCP	https://github.com/rohitg00/kubectl-mcp-server	2026-04-08	908	Python	Kubernetes MCP server
agentgateway	https://github.com/agentgateway/agentgateway	2026-06-10	3.2k	Rust	LLM/MCP/A2A agentic proxy
Plano	https://github.com/katanemo/plano	2026-06-09	6.5k	Rust	AI-native proxy and data plane

分层

层	代表	解决的问题
MCP SDK/framework	FastMCP, MCP Python SDK	快速写 server/client，处理 schema、transport、lifecycle
Tool server	GitHub MCP, Playwright MCP, kubectl MCP	把 GitHub、浏览器、K8s 等具体能力暴露给 Agent
Gateway/proxy	agentgateway , Plano	federation、鉴权、RBAC、observability、routing、policy

选型

需求	选择
快速给内部 API 写 MCP server	FastMCP
让 Agent 操作 GitHub issue/PR/repo	GitHub MCP
让 Agent 控浏览器做验证/抓取	Playwright MCP
让 Agent 读 K8s 集群状态	kubectl MCP，但要强 RBAC 和只读优先
多 MCP server federation + policy	agentgateway
Agentic app data plane / smart routing	Plano

架构差异

维度	FastMCP	GitHub MCP	Playwright MCP	kubectl MCP	agentgateway / Plano
抽象	SDK/ framework	domain server	browser server	K8s server	gateway/proxy
主要风险	schema/鉴权 自建	token 权限过宽	浏览器可执行外部 站点动作	集群权限过 宽	策略和路由复杂
运维边界	app 进程	server + GitHub token	browser runtime	kubeconfig / RBAC	数据面 + 控制面
适合规模	单 server 到 中型	team / repo workflow	自动化测试/浏览器 任务	集群诊断	多 server / 多 Agent / 多租户

避坑条件

- MCP server 默认不是安全边界；凭据、RBAC、allow-list 要外置治理。
- Browser/K8s/GitHub 这类高权限工具必须最小权限启动。
- 多 server 组合时，不要让 Agent 直接持有所有 token；应通过 agent-credential-isolation 和 gateway 管控。
- “支持 MCP”不等于可观测；生产环境要记录 tool call、参数、结果、token 和审批。

Agent Delegation

Agent delegation 是让一个 Agent/工作流把审查、修复、取证或后台任务委托给另一个 Agent 的模式。它不同于普通工具调用：被委托方通常有自己的 session、状态、模型和输出格式。

典型实现

[codex-plugin-cc](#) 是当前 wiki 里的代表：Claude Code slash command 通过 companion/broker/job state 调用 [codex](#) 做 review、adversarial review 或 rescue task。

设计要点

- 边界清晰：委托方不应把被委托方当成普通函数调用；需要 job id、status、result、cancel。
- 输出可审计：review/rescue 输出应保留原文，避免中间层过度改写。
- 生命周期清理：session end 时要清理后台 job、broker 和残留进程。
- 权限最小化：review-only 与 write-capable task 要分开。

和插件/技能的关系

[agent-skills-plugin-system-map](#) 讨论 plugin / skill / MCP tool 三种扩展形态；agent delegation 是更高一层的协作模式，常由 plugin 或 slash command 承载。

agent-recall

agent-recall 是 Max Nardit 开源的本地优先 [agent-memory](#) 系统（仓库 [mnardit/agent-recall](#)，MIT）。它面向 [claude-code](#)、Cursor、Windsurf、Cline 等 MCP-compatible coding agents，通过 [mcp](#) tools 保存实体、关系、观察和日志，再在 session start 时注入 AI briefing。

解决的问题

coding agent 跨 session 会忘记人名、项目决策、偏好、上下文和未完成事项。agent-recall 用一份本地 SQLite 数据库保存这些事实，并用 scope hierarchy 让同一个人或项目在不同客户、团队、topic 下有不同上下文。

核心特征

- MCP-native memory tools: `create_entities`、`add_observations`、`search_nodes`、`open_nodes` 等 9 个工具，server instructions 要求 Agent 主动保存重要上下文。
- Scope hierarchy: `memory.yaml` 定义 hierarchy、tiers、agent types；MCPBridge 按 scope chain 做读写过滤。
- 本地 SQLite: 默认 `~/.agent-recall/frames.db`，FTS5 可用时做全文搜索，无向量库或云服务默认依赖。
- Bitemporal slots: 结构化 key-value 旧值归档，不直接覆盖历史。
- AI briefings: session start 时优先读取 per-agent cache；cache miss 时从 store 组装 raw context，也可调用 Claude CLI / Anthropic API 生成压缩 briefing。
- Claude hooks: `agent-recall-session-start` 自动注入上下文，`agent-recall-post-tool-use` 在记忆写入后做 adaptive cache invalidation。

架构骨架

- 接口层: `mcp_server.py`、`cli.py`、`hooks.py`
- 权限层: `mcp_bridge.py`，多 Agent 读写 scope enforcement 边界
- 存储层: `store.py` + `migrations.py`，单 SQLite 文件和 FTS5 fallback
- 上下文层: `context.py` + `context_gen/*`，raw context assembly、模板、LLM caller、cache

完整分析见 [src-agent-recall-architecture](#)。

与同类关系

维度	agent-recall	claude-mem	agentmemory	powermem
部署	Python package + SQLite	Claude Code plugin	Node/iii-engine worker	数据库/服务层 memory middleware

维度	agent-recall	claude-mem	agentmemory	powermem
客户端	MCP clients + CLI + hooks	Claude Code 为主	多 Agent hooks + MCP + REST	SDK / API / MCP / Dashboard
隔离	scope hierarchy	profile/ workspace	shared worker scopes	应用侧租户边界
AI 用法	briefing 压缩, 可 cache	LLM 压缩核心	默认零 LLM, 可选压缩	LLM 抽取/优化更深

设计上, agent-recall 比 [claude-mem](#) 更强调多 scope 组织结构, 比 [agentmemory](#) 更轻量, 比 powermem 更贴近个人 coding agent 本地 workflow。

相关页面

- [src-agent-recall-architecture](#)
- [agent-memory](#)
- [mcp](#)
- [claude-code](#)
- [claude-mem](#)
- [agentmemory](#)
- powermem
- [ai-as-compressor](#)

agent-recall 架构与设计思路分析

原文: raw/agent-recall-architecture-analysis.md · 仓库: [mnardit/agent-recall](#) · 分析版本 main HEAD dcf21b5 (2026-04-03)

一句话定位

[agent-recall](#) 是一个给 AI coding agents 使用的本地持久化记忆系统：它通过 [mcp](#) tools 让 Agent 主动保存 people / decisions / facts / context，通过 SQLite 保存结构化知识图谱，并在 session start 时把记忆压缩成 AI briefing 注入上下文。它和 [claude-mem](#)、[agentmemory](#)、powermem 同属 [agent-memory](#) 方向，但重点放在 scope hierarchy、MCP-native 写入和本地 SQLite。

核心架构图



| normalized operations



| Policy / orchestration

layer

| MCPBridge: input limits, protected types, scope read/write

filtering

| MemoryConfig: hierarchy, tiers, agent types, briefing and path

resolution

| ScopedView: local-over-parent slot inheritance for context

assembly

| trusted store calls



| State

layer

| MemoryStore: SQLite entities / slots / observations / relations /

logs

| FTS5 when available, LIKE fallback, WAL mode, schema

migrations

| raw scoped graph



| Briefing

layer

| context.py: budget-aware raw context

assembly

| context_gen/: orchestrator/topic assembly, prompt templates, LLM

caller

| cache.py: per-agent markdown cache, stale markers, generation

logs

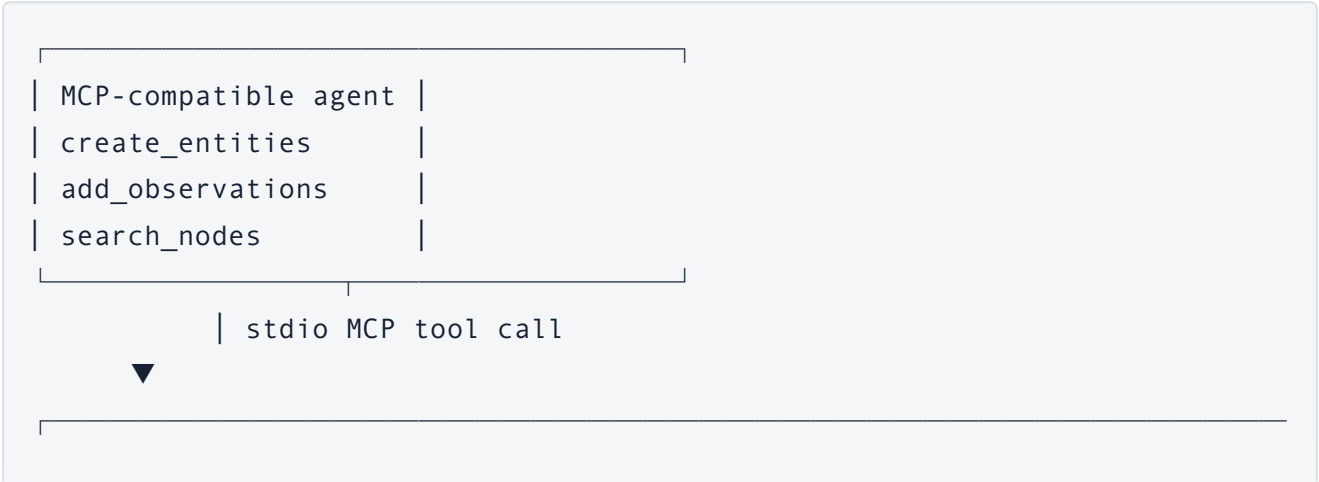
模块分层

层 / 模块	职责
外部接口层	暴露 mcp tools、命令行和 claude-code hooks；把外部协议输入转成统一 store / bridge 操作。
配置与 scope 层	解析 <code>memory.yaml</code> 、环境变量和默认路径；推导 agent scope chain、tier、agent type；提供 scope 继承视图。
MCP 策略层	兼容 MCP memory protocol，同时做输入上限、写权限、读过滤和 protected type 防删。
持久化层	单 SQLite 文件管理 entities、slots、observations、relations、logs、documents；负责 FTS、WAL、迁移和 bitemporal slot。
原始上下文组装	从 store 读取 scope-filtered graph，按优先级和 budget 拼成原始 markdown context。
AI briefing 生成	选择 agent 模板，拼 prompt，调用 Claude CLI 或 Anthropic API，缓存 briefing，维护 stale marker 和 generation log。
辅助工具	实体去重候选、Obsidian vault 页面生成和可选 git auto-commit。

这个分层的关键约束是：MemoryStore 明确不做 scope enforcement，保持简单、可测试、可复用；所有 MCP 客户端可见的权限边界集中在 MCPBridge。CLI 直接访问 store，因此更像本机管理工具；MCP server 则是多 Agent 运行时的安全入口。

关键数据流

MCP 写入 / 查询路径



```

| mcp_server.py |
| 1. FastMCP exposes 9 memory tools |
| 2. _bridge() loads memory.yaml |
| 3. slug = AGENT_RECALL_SLUG or cwd name |
| 4. config.get_agent(slug) derives scope chain and agent type |

```

| JSON-like dict/list payload



```

| MCPBridge |
| 1. cap item count, text length, query length, relation count |
| 2. compute allowed scopes = chain + local children |
| 3. writes: _entity_writable() checks entity non-global scopes |
| 4. reads: _read_scope_set() and _entity_visible() filter results |

```

| trusted operations



```

| MemoryStore |
| resolve_entity / add_observation / add_relation / search |
| SQLite tables + FTS5 triggers + WAL + migrations |

```

| result dict



```

| JSON string to agent |

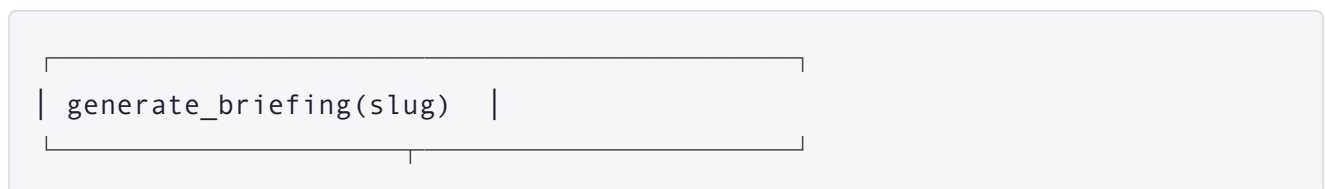
```

错误与回退路径: MCPBridge 对单次调用做截断和 blocked list, 而不是直接抛出所有错误; FTS5 初始化失败时 store 记录 `_has_fts=False` 并回退到 LIKE 搜索; MCP server 如果缺少 mcp extra 会在导入阶段抛出明确安装提示。

SessionStart briefing 路径



AI briefing generation path





设计决策与哲学

- 本地优先，数据库极简：默认一份 ~/.agent-recall/frames.db，目录和 DB 权限收紧；运行时核心依赖只有 pyyaml + click，[mcp](#) 与 Anthropic SDK 都是 optional。

- **scope enforcement** 放在 MCPBridge，不放在 Store：store 是可信底层 API，MCPBridge 才是多 Agent 读写权限边界；这让本机 CLI 管理和 MCP 客户端访问有不同信任模型。
- **MCP-native 行为引导**：server instructions 明确要求 Agent 主动保存 people、decisions、facts、context，并先 search 再 create，记忆策略嵌进协议入口。
- **bitemporal slots + observations 分离**：结构化事实用 slots，旧值归档；自由文本用 observations，单独 FTS；适合同时表达稳定属性和会话记录。
- **AI briefing 是压缩层，不是存储层**：真相仍在 SQLite，LLM 只把 raw scoped context 压缩成启动 briefing；这和 [ai-as-compressor](#) 思路相近，但更强调 scope hierarchy。
- **Hook 只做轻量边界工作**：SessionStart 优先读 cache，PostToolUse 只做 memory write 后的 stale marker / vault regen，避免每次启动或写入都强制跑 LLM。

核心组件

MemoryStore

MemoryStore 是单 SQLite 文件的 state layer。它创建 entities、slots、observations、relations、log_entries、documents，启用 WAL、foreign keys 和可选 FTS5。slots 通过 valid_from / valid_to 表达 bitemporal 旧值归档；observations 和 relations 通过 archived_at / status 软删除。它不关心调用者是谁，也不判断是否越权。

MCPBridge

MCPBridge 是 agent-recall 的安全和协议语义中心。它把 MCP memory tools 映射到底层 store，同时限制输入规模。写路径通过实体已有的非 global scopes 判断是否可写；读路径通过 scope set 过滤 search_nodes、open_nodes 和 read_graph。orchestrator 可以关闭读过滤，普通 scoped agent 则只能看到自己 scope chain 内的数据。

Context / Briefing

context.py 负责不调用 LLM 的 raw markdown 组装，context_gen 负责把 raw context 变成适合注入 session 的 AI briefing。生成器会根据 agent type 选择 orchestrator、topic 或默认 scoped context，再合并 extra context、context files 和自动发现的 CLAUDE.md / README.md，最后用模板和 LLM caller 写入 per-agent cache。

相关页面

- [agent-recall](#)
- [agent-memory](#)
- [mcp](#)
- [claude-code](#)
- [claude-mem](#)
- [agentmemory](#)

- powermem
- [ai-as-compressor](#)

AgentCube

AgentCube 是 Volcano 社区面向 AI Agent / Code Interpreter 的 Kubernetes 原生会话编排层。它不是替代 agent-sandbox，而是在 agent-sandbox 的 Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool 之上，补上应用级会话、HTTP 入口、代码执行 API、SDK 入口和生命周期回收。

详细架构见 [src-agentcube-architecture](#)。

关键事实

- 仓库: [volcano-sh/agentcube](#)
- 当前核验: GitHub API 显示 main 最近 push 2026-06-12, 已有 tag v0.1.0
- 本 wiki 架构分析版本: HEAD 208da32 (2026-06-01)
- 阶段: 已从 Proposal 进入早期 release, 但 API 仍需按上游 agent-sandbox 演进持续核验
- 主要语言: Go 1.24.4, 模块 [github.com/volcano-sh/agentcube](#)
- 核心依赖: [sigs.k8s.io/agent-sandbox v0.1.1](#) (当前 main 的 go.mod 仍如此)
- 2 个 AgentCube CRD: AgentRuntime 与 CodeInterpreter
- 主要组件: Router、WorkloadManager、PicoD、Redis / ValKey session registry

架构定位

AgentCube 把 agent-sandbox 的底层运行时原语包装成更贴近 AI 应用的会话模型:

- AgentRuntime 面向长期运行的 Agent 服务, 声明 targetPort、PodTemplate 和 session timeout。
- CodeInterpreter 面向代码执行工作负载, 声明执行镜像、端口、session timeout、最大会话时长、warm pool 规模和鉴权模式。
- Router 是数据平面, 根据 x-agentcube-session-id 查询 Redis / ValKey, 把请求反向代理到已存在的 Sandbox, 必要时调用 WorkloadManager 创建新 session。
- WorkloadManager 是控制平面, 负责创建 Sandbox 或 SandboxClaim, 等待 Ready、做 endpoint TCP probe、写入 session 映射并执行 GC。
- PicoD 是轻量 HTTP daemon, 提供 /api/execute 与 /api/files, 用 Router 签发的 JWT 做访问控制, 替代传统 SSH 进入容器。

与 agent-sandbox 的结合

AgentCube 复用了 agent-sandbox 的隔离、Pod 生命周期和 WarmPool 能力, 但把它们隐藏在更高层 API 后面:

- 冷启动路径: AgentRuntime / CodeInterpreter -> WorkloadManager -> Sandbox -> Pod。
- 预热路径: CodeInterpreter.warmPoolSize -> SandboxTemplate + SandboxWarmPool -> 首次请求创建 SandboxClaim -> 领取预热 Sandbox。

- 会话路径：Router 从 session registry 解析 sessionId -> endpoint，后续请求不再走 K8s reconcile，而是直接代理到 PicoD 或 Agent 服务。

这个分层让 agent-sandbox 继续专注"安全、有状态、可回收的单实例容器"，AgentCube 则负责"用户请求如何变成一个可复用的 AI / Code session"。

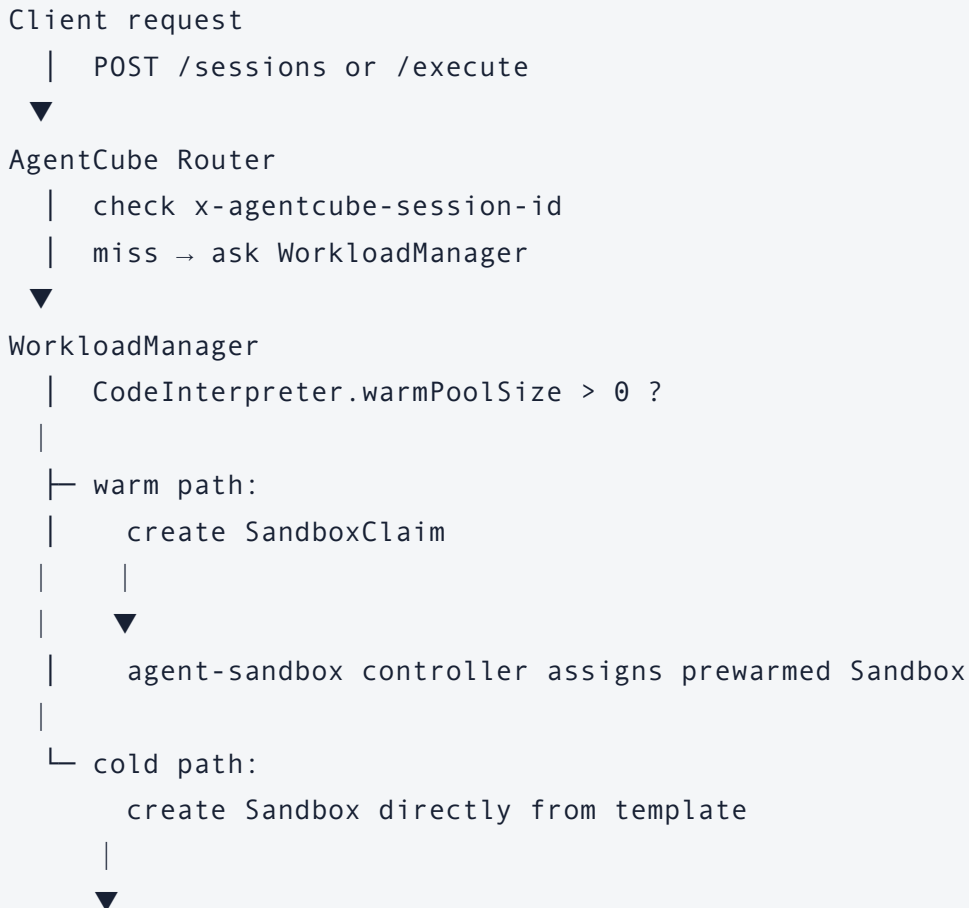
agent-sandbox API 兼容性

AgentCube 当前已发布 v0.1.0，但仍依赖 sigs.k8s.io/agent-sandbox v0.1.1。这意味着它和当前 agent-sandbox v0.4.6 / v0.5.0rc1 之间可能存在 API 追赶成本：

- SandboxTemplate / SandboxWarmPool 字段可能需要适配新版本 rolling update / backend 解耦；
- claim / warm pool 的 status 和 owner reference 语义可能变化；
- router/session registry 依赖 Sandbox Ready、endpoint、port 等状态字段，升级时要重点回归；
- AgentCube 的 CodeInterpreter.warmPoolSize 是对 agent-sandbox WarmPool 的上层封装，上游 API 变动会直接影响这一功能。

选型上，AgentCube 当前适合作为“agent-sandbox 上层应用 API”的设计参考；生产采用前应先验证它是否已跟上目标 agent-sandbox 版本。

CodeInterpreter warm pool 请求时序



```
wait Sandbox Ready + endpoint TCP probe
|
▼
write sessionId → endpoint into Redis / ValKey
|
▼
Router reverse proxy → PicoD `/api/execute` / `/api/files`
```

这条路径把高频请求和低频 K8s reconcile 分开：首次 miss 需要控制面介入，后续同一 session 直接由 Router 查 registry 并代理到 PicoD。

AgentCube 架构与设计思路分析

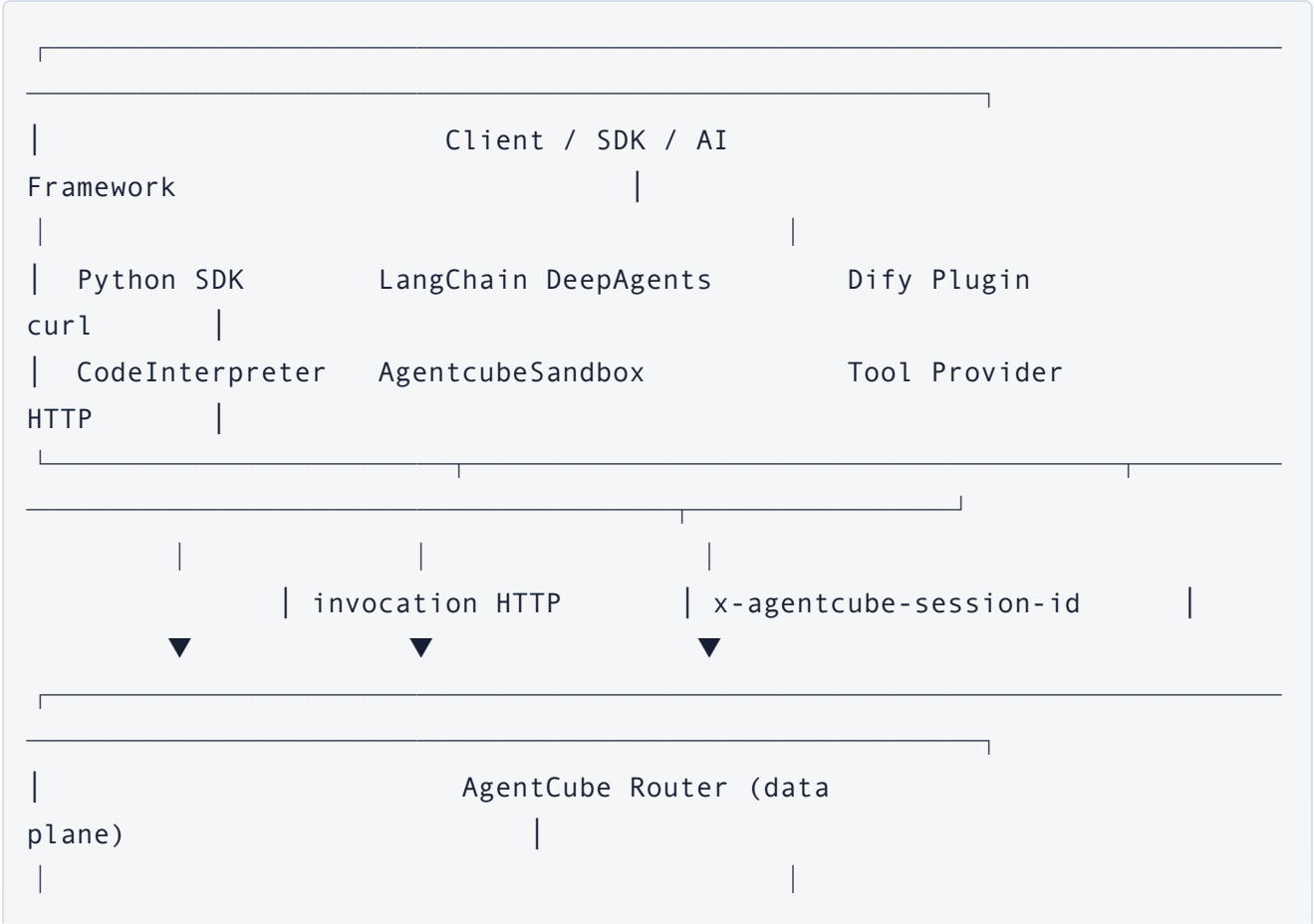
原文: raw/agentcube-architecture-analysis.md · 仓库: <https://github.com/volcano-sh/agentcube> · 分析版本 HEAD 208da32 (2026-06-01)

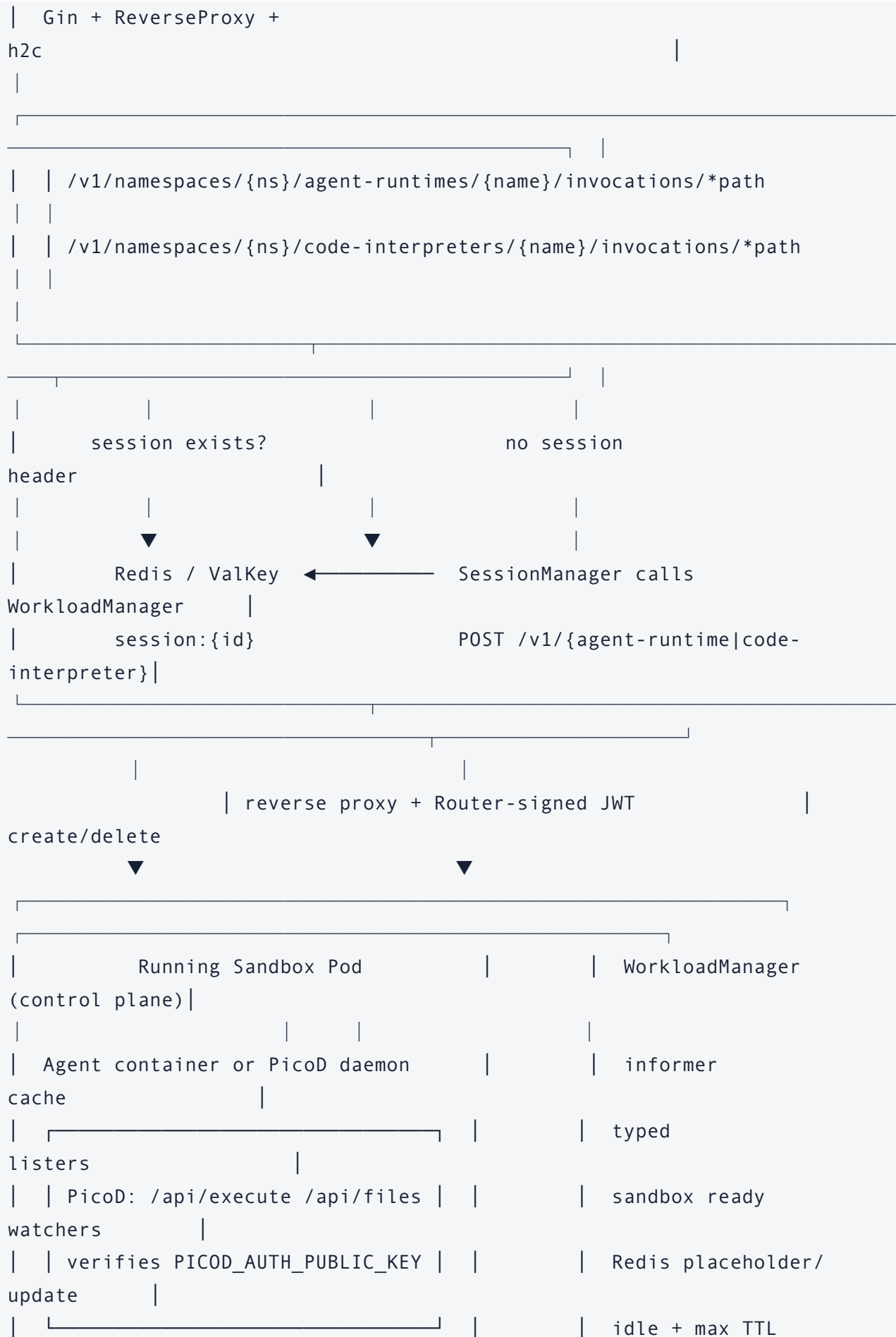
一句话定位

volcano-sh/agentcube 是 Volcano 社区面向 AI Agent / Code Interpreter 工作负载的 **serverless session orchestration layer**: 上层用 AgentRuntime / CodeInterpreter 两个 CRD 表达"可被调用的 agent/runtime 模板", 中层用 Router + WorkloadManager + Redis/ValKey 管 session、路由、GC 和预热, 下层直接复用 agent-sandbox 的 Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool 来真正创建隔离执行环境。

它和 agent-sandbox 不是竞争关系, 而是更明确的上下层关系: agent-sandbox 提供"一个稳定、有状态、可预热/可领取的 Sandbox 原语", AgentCube 把这个原语包装成"开发者能通过 HTTP/SDK 调用的 AgentRuntime / CodeInterpreter 会话服务"。

核心架构图







模块分层

层 / 模块	职责
AgentCube API 层	定义 AgentRuntime / CodeInterpreter 两个面向用户的 CRD。前者保留完整 PodSpec，后者收敛成 image/resources/runtimeClass/authMode/warmPoolSize。
数据面 Router	暴露 invocation API，按 x-agentcube-session-id 做 session affinity；没 session 时调用 WorkloadManager 创建；有 session 时查 Redis/ValKey；最后反向代理到 sandbox endpoint。

层 / 模块	职责
控制面 WorkloadManager	读 AgentCube CRD，构造 agent-sandbox CR，等待 Sandbox Ready，探测 entrypoint，写 session store，并周期性 GC。
Sandbox 内 runtime	PicoD 用轻量 HTTP API 替换 SSH：执行命令、上传/下载/列文件、workspace path jail、JWT 验证、32MB body limit。
状态存储	Redis / ValKey 抽象；session:{id} 保存 SandboxInfo，两个 sorted set 分别索引 ExpiresAt 和 LastActivityAt。
安全与身份	Router→PicoD 用 RS256 JWT；Router→WorkloadManager 可用 SA token 或 SPIFFE/mTLS；mTLS 文件来源可来自 SPIRE / cert-manager / 静态 Secret。
SDK / 集成	Python SDK、LangChain Deep Agents backend、Dify plugin、kubectl 风格 CLI，把底层 session API 包成开发者接口。
部署与 E2E	Helm 安装 Router/WorkloadManager/CRDs/SPIRE；E2E 覆盖 CodeInterpreter、WarmPool、agent-sandbox 安装和 session 生命周期。

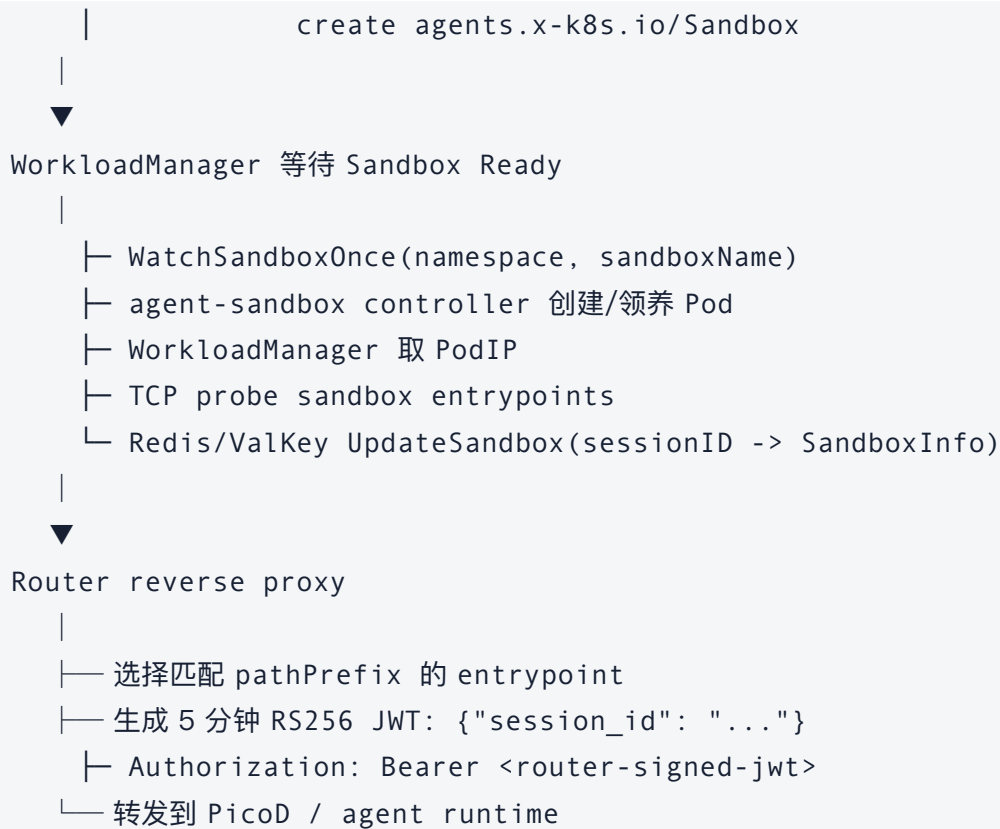
关键数据流

无 session 的首次调用

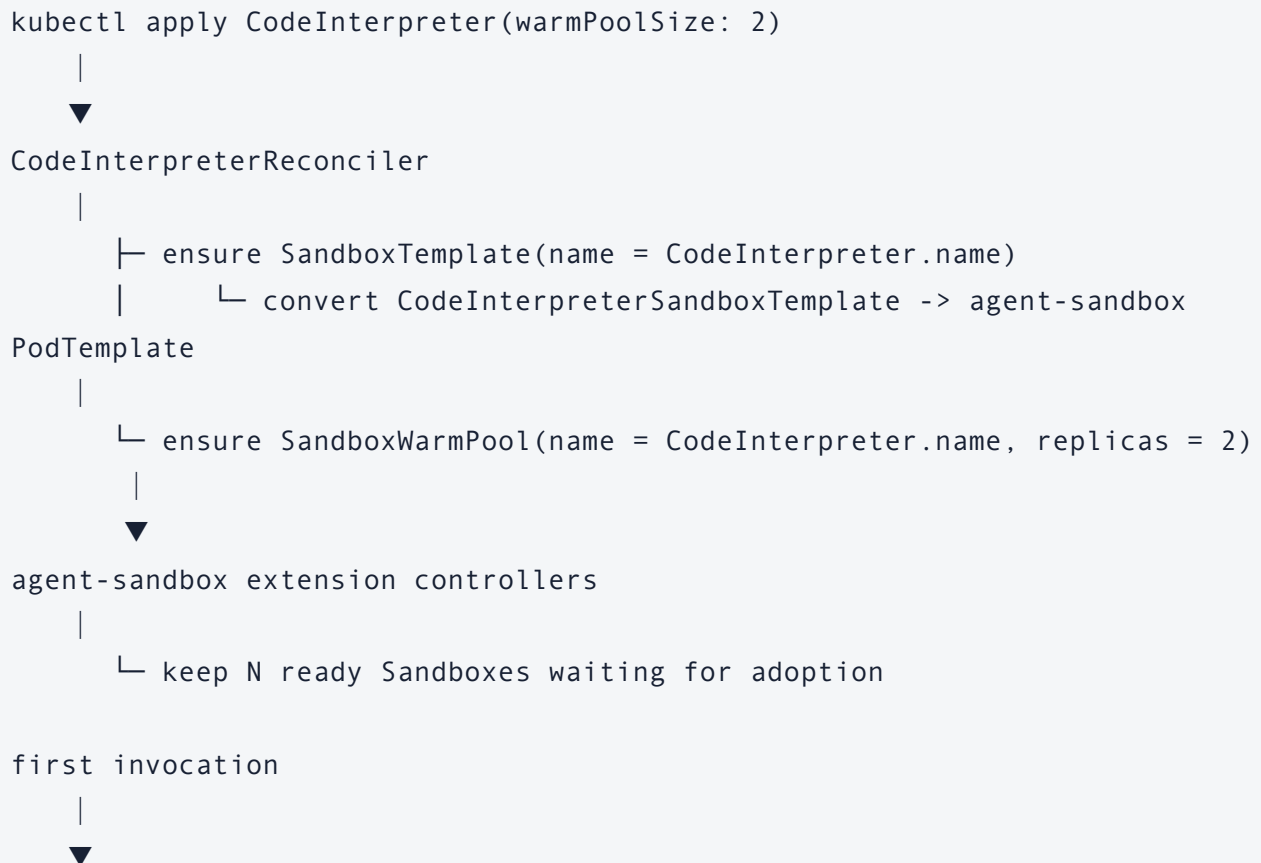
```

Client / SDK
|
|   | POST /v1/namespaces/default/code-interpreters/my-interpreter/
invocations/api/execute
|   | (no x-agentcube-session-id)
▼
AgentCube Router
|
|   | SessionManager.GetSandboxBySession(sessionID="")
|   |
|   |   ▼
|   |   | WorkloadManager POST /v1/code-interpreter
|   |   |
|   |   |   | informer lister 读取 CodeInterpreter CR
|   |   |   |   | 生成 sessionID + sandboxName
|   |   |   |   | 如果 warmPoolSize > 0:
|   |   |   |   |   | create extensions.agents.x-k8s.io/SandboxClaim
|   |   |   |   |   | 否则:

```



CodeInterpreter WarmPool 路径



```
WorkloadManager buildSandboxByCodeInterpreter
```

```
|  
├─ create SandboxClaim(templateRef = CodeInterpreter.name)  
├─ entry.Kind = SandboxClaim  
└─ store session placeholder  
|  
▼
```

```
agent-sandbox SandboxClaimReconciler
```

```
|  
├─ pick ready Sandbox from WarmPool  
├─ transfer ownerReference to Claim  
└─ annotate SandboxPodName  
|  
▼
```

```
WorkloadManager sees Sandbox Ready -> PodIP -> entypoints -> session store
```

设计决策与哲学

- 把 agent-sandbox 当 substrate，不重新实现 Sandbox controller: go.mod 直接依赖 sigs.k8s.io/agent-sandbox v0.1.1, WorkloadManager 构造 agents.x-k8s.io/Sandbox 和 extensions.agents.x-k8s.io/SandboxClaim。AgentCube 自己碰 Pod reconcile 细节，专注 session API、路由、GC、SDK 和集成。
- 两个 CRD 对应两类 AI workload: AgentRuntime 允许完整 PodSpec，适合长会话 agent、volume、credential、sidecar; CodeInterpreter 收敛成 image/resources/runtimeClass/authMode/warmPoolSize，适合执行不可信代码。也因此只有 CodeInterpreter 一等支持 warmPoolSize，因为 code interpreter 更需要低冷启动延迟。
- Router / WorkloadManager 分平面: Router 是高频数据面，负责 session header、反向代理、路径匹配和并发限制; WorkloadManager 是低频控制面，负责 kubernetes API、informer、Sandbox 创建和 GC。这样 Router 可以水平扩展，状态放进 Redis/ValKey，而不是把请求路径绑死在 controller 进程内。
- 创建先写 placeholder，再等 Ready，再 update store: WorkloadManager 在真正创建 K8s 资源前先 StoreSandbox placeholder，失败时 rollback K8s 资源和 store。成功路径必须等 Ready、拿到 PodIP、并 TCP probe endpoint 后才把 store 更新为可路由信息。
- PicoD 用 HTTP + JWT 替代 SSH: PicoD 是 sandbox 内最薄的执行面，POST /api/execute 执行命令，/api/files 做文件操作。当前代码已经从早期"客户端私钥签名"演进到"Router 持有 RSA 私钥、PicoD 校验 Router 公钥"的模型: Router 创建 picod-router-identity Secret, WorkloadManager 从 Secret 缓存 public key，再注入 PICOD_AUTH_PUBLIC_KEY。这降低 SDK 复杂度，但也让 Router 成为更强的信任边界。

- mTLS 是可选增强，不阻断低延迟路径：pkg/mtls 支持文件热加载证书和 SPIFFE ID 校验，Helm 可以通过 SPIRE sidecar 给 Router / WorkloadManager 注入 SVID。Router→WorkloadManager 可 mTLS，Router→PicoD 当前仍主要走 JWT，以避免每个短会话 sandbox 的 TLS 握手成本。

与 agent-sandbox 的结合方式

维度	agent-sandbox	AgentCube
抽象层	基础设施原语：Sandbox / SandboxClaim / SandboxTemplate / SandboxWarmPool	产品化会话层：AgentRuntime / CodeInterpreter + Router + SDK
用户入口	kubectl apply Sandbox/ SandboxClaim 或 SDK 直接管 Sandbox	HTTP invocation API、Python SDK、LangChain/Dify/CLI
生命周期语义	1 个 Sandbox = 1 个有状态隔离 Pod，可暂停/恢复/过期	1 个 session = 1 个 Sandbox，附加 session header、activity、TTL、GC
Warm Pool	提供 WarmPool 和 Claim adoption 机制	CodeInterpreter.spec.warmPoolSize 驱动创建 Template/WarmPool，首次调用时创建 Claim
路由	headless Service / Pod 访问由上层决定	Router 根据 SandboxInfo.EntryPoints 反向代理到 PodIP:port
状态存储	K8s API / Status	Redis/ValKey session registry，跨 Router 副本共享
隔离策略	委托 K8s runtimeClassName / NetworkPolicy / securityContext	在 CRD 模板中暴露 runtimeClassName / resources，默认 image 可跑 PicoD
安全边界	运行时隔离与 K8s 原语	Router JWT、可选 WorkloadManager auth、可选 SPIFFE/mTLS

结合后的层次可以概括成：



▼
agent-sandbox CRDs own lifecycle mechanics

|

▼
Kubernetes Pod / RuntimeClass / NetworkPolicy / Service

最重要的设计收益是：AgentCube 没有把"AI Agent 平台"做成另一个私有调度器，而是把 AI Agent 的会话和工具调用语义加在 agent-sandbox 上。底层依然能继承 agent-sandbox 的 gVisor/Kata/NetworkPolicy/WarmPool 能力，上层则补齐了 HTTP/API/SDK/Redis session registry 这些 agent 应用真正需要的东西。

关键组件深入解读

WorkloadManager：从 AgentCube CRD 到 agent-sandbox CRD

核心转换在 pkg/workloadmanager/workload_builder.go。buildSandboxByAgentRuntime 读取 AgentRuntime，复制 spec.podTemplate.spec，归一化空 runtimeClassName，生成 sessionID 和随机后缀 sandboxName，然后构造 agent-sandbox 的 Sandbox。这条路径完全保留用户的 PodSpec，适合通用 agent runtime。

buildSandboxByCodeInterpreter 更专门：先检查 authMode=picod 时 public key 是否已缓存，然后为没有 ports 的 CodeInterpreter 默认补 / + 8080/HTTP。如果 warmPoolSize > 0，它不直接创建完整 Sandbox，而是创建 SandboxClaim(templateRef = codeInterpreterName)；如果没有 WarmPool，则构造带 PicoD image/env/resources 的直接 Sandbox。

风险与演进信号

- 项目仍在 alpha / proposal 到 release 的过渡期：README 还写 Early Design，release 文档写 v0.1.0。字段命名也有历史痕迹，比如 AgentRuntime.spec.targetPort vs CodeInterpreter.spec.ports，podTemplate vs template。
- Router 是强信任边界：当前 Router 持有 PicoD 私钥并能为任何 session 签执行请求。相比"客户端私钥不出本机"，这更容易用，但要靠 Router auth、mTLS、RBAC、审计来补强。
- WorkloadManager 目前承担两种职责：既是 HTTP control plane，又跑 controller-runtime manager 和 GC。规模变大后可能会拆成 API server / reconciler / GC 三个部署单元。
- agent-sandbox 版本偏早：依赖 v0.1.1，而本 wiki 之前分析的是 v0.4.5+11。AgentCube 当前锁定的 API 可能需要跟随 agent-sandbox 后续 v1alpha1 演进。

相关页面

- agent-sandbox
- kubernetes
- declarative-agent-management

- agent-credential-isolation

agentgateway

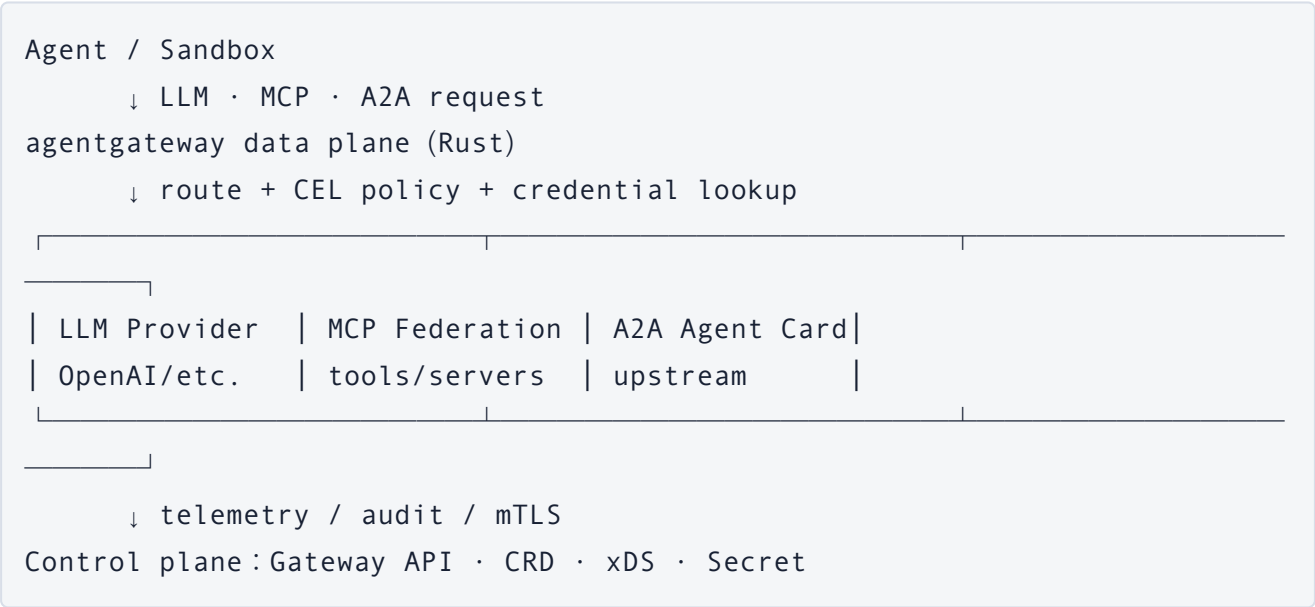
Solo.io / Istio 系出品的 AI-native L7 网关（Apache 2.0）。把三种 AI 流量统一在一个 Rust 数据面里：

- **LLM Gateway**: OpenAI / Anthropic / Gemini / Bedrock / Vertex / Azure / Copilot 七家 provider 统一为 OpenAI 兼容 API + guardrails
- **MCP Gateway**: stdio / HTTP Streamable / SSE 三种 transport + OpenAPI 自动转 MCP 工具 + 多 MCP server federation
- **A2A Gateway**: Google Agent-to-Agent 协议代理，通过重写 Agent Card 让所有调用走向 gateway

架构骨架

- **控制面 (Go)** : Gateway API + 自家 CRD (AgentgatewayPolicy / AgentgatewayBackend / AgentgatewayParameters) , KRT 反应式表, xDS Delta ADS 分发
- **数据面 (Rust)** : tokio + hyper + rustls, proxy/gateway → http/route → llm|mcp|a2a → upstream, CEL 做策略 IR, HBONE 跟 Istio mesh 互操作
- **UI (TypeScript)** : Next.js 管理界面

Agent 出口请求流程



核心设计哲学

1. 沿用 Istio 基建做 AI Gateway: Gateway API + KRT + xDS + HBONE 全套复用，只把数据面换成专为 AI 协议优化的 Rust 代理
2. 三协议共享同一条 pipeline: Route / Policy / Backend / CEL 求值统一，差别只在 Backend.kind 决定走哪个 provider 适配器

- 3. 凭据托管：LLM provider 真凭据放在 backend secret，Agent 只面对 gateway 自身认证，参见 agent-credential-isolation
- 4. 跟 agent-sandbox 互补：前者出口流量治理，后者运行时隔离，并集 = AI 工作负载完整治理面

详见 [src-agentgateway-architecture](#)。

LLM provider 适配器对照

Provider	请求侧职责	响应侧职责	典型差异
OpenAI / OpenAI-compatible	作为内部标准格式，基本透传	返回 OpenAI-compatible SSE / JSON	其他 provider 多向它收敛
Anthropic	OpenAI messages → Anthropic messages / tools	Anthropic event stream → OpenAI chunk	thinking/tool schema/ stream event 名称不同
Gemini / Vertex	OpenAI chat → generateContent / Vertex endpoint	Gemini candidates → OpenAI choices	project/location/auth 和多模态字段不同
Bedrock	OpenAI chat → Bedrock invoke，叠加 AWS SigV4	Bedrock body/stream → OpenAI-compatible	AWS auth、model id、区域化 endpoint
Azure OpenAI	OpenAI schema + Azure deployment endpoint	Azure OpenAI response → OpenAI-compatible	deployment 名称、api-version、endpoint path
Copilot	适配 GitHub Copilot provider 入口	统一成 OpenAI-compatible	认证和可用模型集合不同

agentgateway 的关键不是“支持更多 provider”本身，而是把 provider 差异压到 AIProvider / Provider trait 后面。上层 HTTPRoute、Policy、Backend、CEL、telemetry 不需要知道具体 provider 细节。

MCP federation 与 CEL RBAC

MCP Gateway 把多个 MCP server 聚合到一个入口。设计重点是命名空间和授权：

- transport 层支持 stdio / HTTP Streamable / SSE，上游协议差异由 mcp/upstream 吸收；
- federation 时要避免工具名冲突，通常需要按 server 或 route 分命名空间；
- 权限不应散落在工具实现里，而应在 gateway route / policy 层用 cel 表达式做统一判定；
- agent 只拿 gateway 侧身份，后端 MCP server 的真实凭据仍由 gateway 管。

这和 HiClaw 的 worker consumer key 模式一致：Agent 能调用哪些工具，由网关侧 consumer、route 和 policy 决定，而不是由 prompt 里的自觉性决定。

和同类 AI Gateway 的边界

项目	主要定位	agentgateway 的差异
LiteLLM	Python LLM proxy / provider adapter	agentgateway 更偏 Rust 数据面 + Gateway API/xDS/mesh 集成
OpenRouter	托管模型路由服务	agentgateway 是可自托管 infra 组件，控制面和策略在用户集群里
Portkey	托管/企业 LLM gateway	agentgateway 更云原生，强调 CRD、mesh、MCP/A2A
Envoy AI Gateway	Envoy Gateway 上的 GenAI gateway	agentgateway 不复用 Envoy 数据面，而是自写 Rust 数据面并同时覆盖 LLM/MCP/A2A

部署 walkthrough 速记

1. 安装 Gateway API CRD 与 agentgateway 自家 CRD。
2. 部署 controller，它 watch Gateway / HTTPRoute / AgentgatewayPolicy / AgentgatewayBackend。
3. 创建 GatewayClass 与 Gateway，由 AgentgatewayParameters 控制 dataplane 部署参数。
4. 创建 AgentgatewayBackend，把 AI/MCP/A2A 后端和 secret 挂进去。
5. 用 HTTPRoute.backendRef 指向 backend，并用 AgentgatewayPolicy 加鉴权、限流、转换或 CEL RBAC。
6. dataplane 通过 xDS 增量接收配置，开始处理 LLM / MCP / A2A 流量。

如果需要运行时隔离，把 Agent 容器放在 agent-sandbox / [agentcube](#) 一类 substrate 上；如果需要 LLM serving endpoint picking，把入口和 gateway-api-inference-extension / llm-d 对齐。

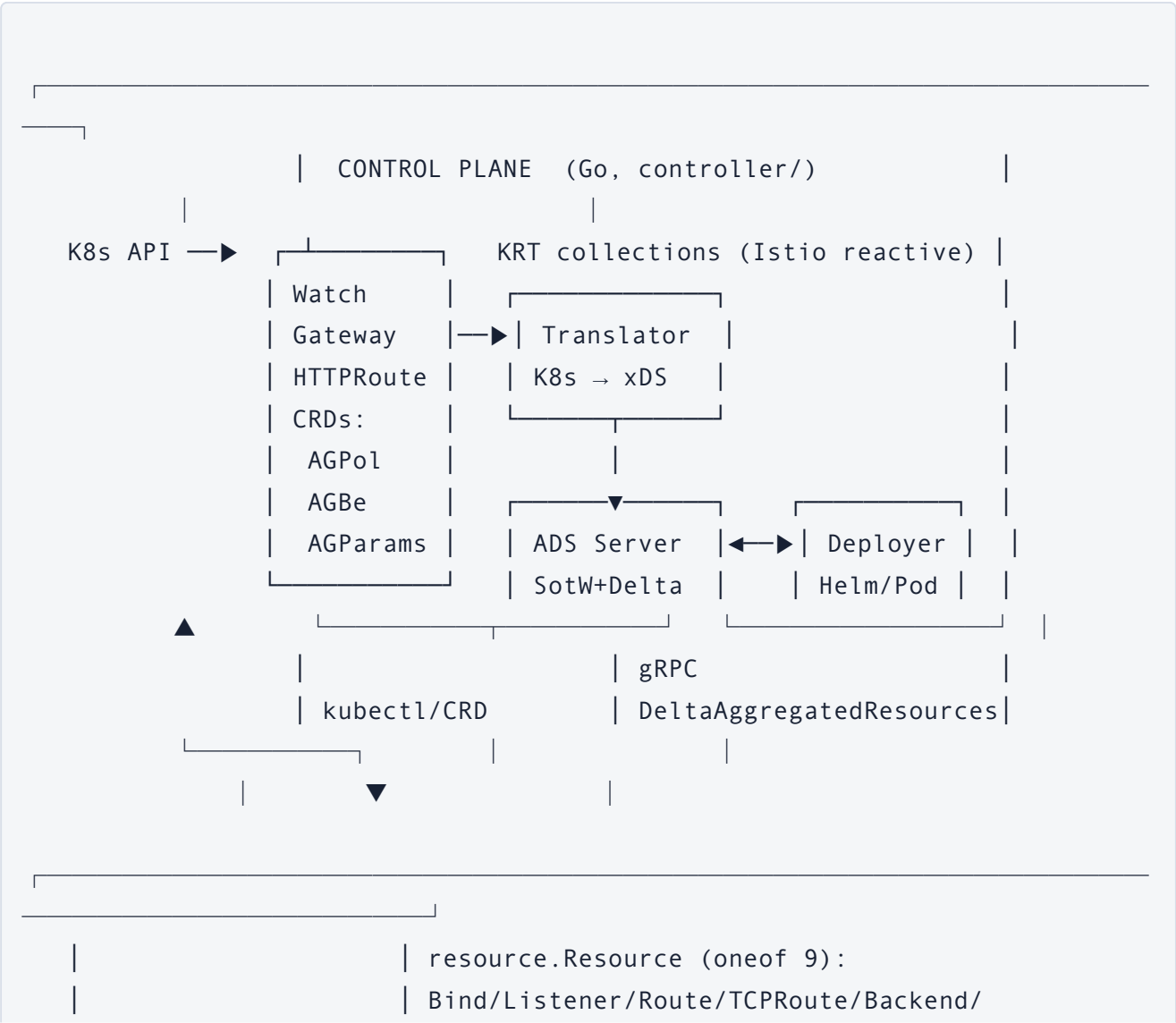
agentgateway 架构与设计思路分析

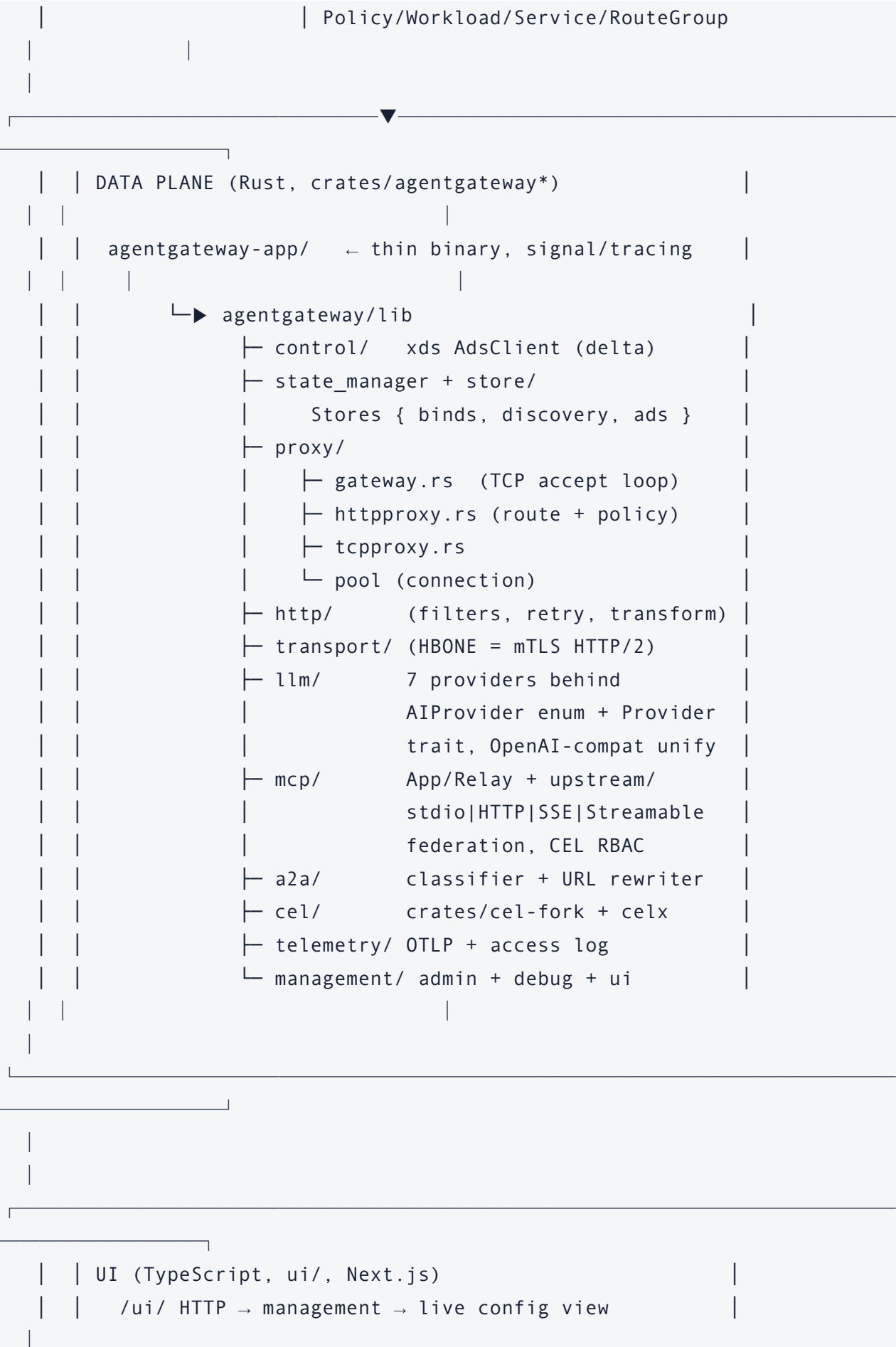
原文: raw/agentgateway-architecture-analysis.md · 仓库: [agentgateway/agentgateway](#) · 分析版本 v1.2.0-alpha.2+24 (HEAD 9ca3e04)

一句话定位

[agentgateway](#) 是面向 AI 流量的 L7 代理：把 LLM API (OpenAI / Anthropic / Gemini / Bedrock / Vertex / Azure / Copilot)、MCP 工具调用、A2A Agent 间通信 统一在一个 Rust 数据面里，用 Gateway API CRD + xDS 做声明式控制面分发。本质上是「Istio 的控制面骨架 + 为 AI 协议特化的 Rust 数据面」的组合——把 mesh 经过验证的反应式配置分发、KRT 反应式表、HBONE mTLS 隧道全套基建搬来做 AI Gateway。

核心架构图





模块分层

Workspace 顶层

模块	语言	角色
crates/agentgateway-app/	Rust	薄壳二进制（信号 / 日志 / panic hook）
crates/agentgateway/	Rust	数据面主库，三协议网关都在此
crates/xds/	Rust	xDS 客户端 + proto 绑定
crates/hbone/	Rust	Istio HBONE（mTLS over HTTP/2 CONNECT）传输
crates/celx/ + crates/cel-fork/	Rust	CEL 策略求值（fork 加 HTTP 集成）
controller/	Go	controller-runtime + ADS server + Helm deployer
api/	Go	resource.proto 的 Go 绑定
ui/	TypeScript	Next.js 管理 UI

数据面分层（crates/agentgateway/src/）

层	模块	职责
L1 配置	control/ config.rs	本地 yaml + xDS client + outbound client
L2 状态	state_manager.rs store/	反应式 Store 聚合 binds/discovery/ads
L3 代理	proxy/gateway.rs proxy/httpproxy.rs	TCP accept → TLS → route → upstream
L4 HTTP	http/	filter chain / retry / timeout / transform
L5 协议	llm/ mcp/ a2a/	三个 AI 协议网关
L6 横切	cel/ telemetry/ transport/	策略求值 / 可观测 / HBONE 隧道
L7 管理	management/ ui.rs	管理 HTTP / 健康 / UI 静态资源

自家 CRD

CRD	短名	用途
AgentgatewayPolicy	agpol	CEL 策略容器，挂在 Gateway / Listener / Route 上
AgentgatewayBackend	agbe	后端配置：AI / MCP / A2A / Static
AgentgatewayParameters	—	GatewayClass 级 Pod 模板参数

上游 Gateway API 直接复用：Gateway HTTPRoute GRPCRoute TCPRoute TLSRoute GatewayClass ListenerSet InferencePool。

关键数据流

一次 LLM 请求的端到端数据流





▼ 响应 (流式 SSE / chunked)

```
| provider.transform_response() |
| - 反向 schema 转换 → 统一 OpenAI 格式 |
| - guardrails (post-flight) |
| - telemetry: tokens_in / tokens_out / cost / latency |
| - access log + OTLP span |
```

▼

Client receives OpenAI-compatible SSE stream

控制面 → 数据面 配置分发

```
kubectl apply Gateway/HTTPRoute/agpol/agbe.yaml
```

▼

controller-runtime watch → KRT collection mutate

▼

Translator: K8s objs → resource.Resource oneof

```
| Bind          (TCP listen addr)
| Listener      (TLS / SNI / protocol)
| Route         (HTTP route + filters)
| TCPRoute
| Backend       (AI / MCP / A2A / Service / Static)
| Policy        (CEL IR + transform + retry + ratelimit)
| RouteGroup
| Workload/Service (Istio Address 兼容)
```

▼

ADS Server (gRPC, controller/pkg/xds/server.go)

DeltaAggregatedDiscoveryService

▼ Delta xDS (增量)

agentgateway/control/AdsClient

```

    ▼ apply
state_manager + Stores { binds, discovery }
|
    ▼ 触发
proxy/gateway 重建 Listener / Route 索引

```

xDS 资源类型只有两种：

- `istio.workload.Address` —— 兼容 Istio ambient mesh 的 workload/service 发现
- `agentgateway.dev.resource.Resource` —— 主资源 oneof (9 个变体覆盖整个数据面状态)

只用 Delta ADS，单连接复用，不每种资源一个 stream。

一次 MCP 工具调用

```

MCP Client (Cursor / Claude Desktop / OpenAI Responses API)
| JSON-RPC over stdio | HTTP Streamable | SSE
▼
mcp/router.rs — 路由层根据 session id 找到对应的上游
|
▼
mcp/handler.rs — 工具列表 federation (merge_tools)
|   |
|   └─ 多个 upstream MCP server 的 tools/list 合并
|       + CEL RBAC 过滤 (mcp/rbac.rs)
|
▼ tools/call
mcp/upstream/{stdio.rs | streamablehttp.rs | sse.rs | openapi.rs}
|
▼
Upstream MCP server / OpenAPI endpoint

```

下游 transport: HTTP Streamable / SSE / stdio。上游 transport: 同三种 + `openapi.rs` 把任何 OpenAPI spec 自动桥接成 MCP tools (无需上游侧改造)。federation 通过 `handler.rs::merge_tools()` 合并多个 MCP server 的工具集，命名空间用 `_` 前缀避冲突，CEL RBAC 按 client identity 过滤可见性。

A2A (Agent-to-Agent)

```

Agent Client
| HTTP GET /.well-known/agent-card.json
▼
a2a/mod.rs — 拦截 Agent Card 发现请求

```

- | 一 透传上游，但 *重写响应里的 URL*
- | 把 upstream 真实地址换成 gateway 自身地址
- | 一 之后所有 Agent 调用都走向 gateway，自然拿到 policy/observability

A2A 是三协议里最轻的——核心就 URL 重写。Google A2A 协议把入口放在 Agent Card well-known endpoint，gateway 控制了发现，就控制了所有后续调用。

设计决策与哲学

- 沿用 Istio 的基建做 AI Gateway: Gateway API + KRT collections + xDS + HBONE 全套复用，控制面是 Istio 的影子结构。开发团队 (Solo.io / Istio 系) 选择不重新发明轮子，把 mesh 经过验证的反应式配置分发 + 反应式表建模 + mTLS 隧道搬来，只把数据面替换成「专为 AI 协议优化的 Rust 代理」。Day 1 即获得完整 mesh 兼容性 + 控制面/数据面可独立替换的解耦能力。
- 三协议共享同一条 pipeline: LLM / MCP / A2A 在数据面共享 Route / Policy / Backend / CEL 求值的同一条管道，差别只在 Backend.kind 决定走哪个 provider 适配器。带来单一策略语言 + 统一观测 + 统一 mTLS。
- CEL 作为策略 IR: 所有授权 / 转换 / 速率限制都被编译成 CEL 表达式存到 Policy.spec.expression，运行时由 Rust 数据面解释执行。controller 不预编译，避免 Go/Rust CEL 实现差异。用 cel-fork + celx 是因为原生 cel-rust 缺与 HTTP 请求对象的深度集成。
- 凭据托管 prompt injection: 与 HiClaw 的 Higress 凭据托管哲学一致——LLM provider 真凭据 (OpenAI key / AWS IAM / GCP service account) 放在 backend secret，gateway 在请求时注入，Agent / 业务端只面对 gateway 自己的认证 (JWT / API key / mTLS)。Agent 即使被注入也偷不到真凭据。详见 agent-credential-isolation。
- 控制面 / 数据面通过 proto 解耦: Go controller + Rust 数据面只用 resource.Resource proto 通信，可独立演进。未来能换掉任一端 (Crossplane 直产 xDS / envoy 实现 agentgateway proto 扩展)。
- 两个 tokio runtime 拆分热路径: main runtime 跑控制面 (admin HTTP / xDS / reload)，worker runtime 跑数据面 accept，避免控制面任务抢占 worker 影响 P99。
- OpenAI 兼容 API 做统一前门: 客户端代码零改造接 7 个 LLM provider; 非 OpenAI 特性 (Gemini 多模态、Bedrock 私有模型) 通过扩展字段暴露。

关键组件深入: LLM Provider 适配器

crates/agentgateway/src/llm/ 是数据面最厚的协议适配层。统一抽象骨架:

```
enum AIProvider {
    OpenAI, Anthropic, Gemini, Bedrock, Vertex, Azure, Copilot,
}

trait Provider {
    async fn transform_request(&self, req: OpenAIRequest) ->
    ProviderRequest;
```

```

    async fn transform_response(&self, resp: ProviderResponse) ->
OpenAIResponse;
    async fn stream_chunk(&self, chunk: Bytes) -> OpenAIChunk;
}

```

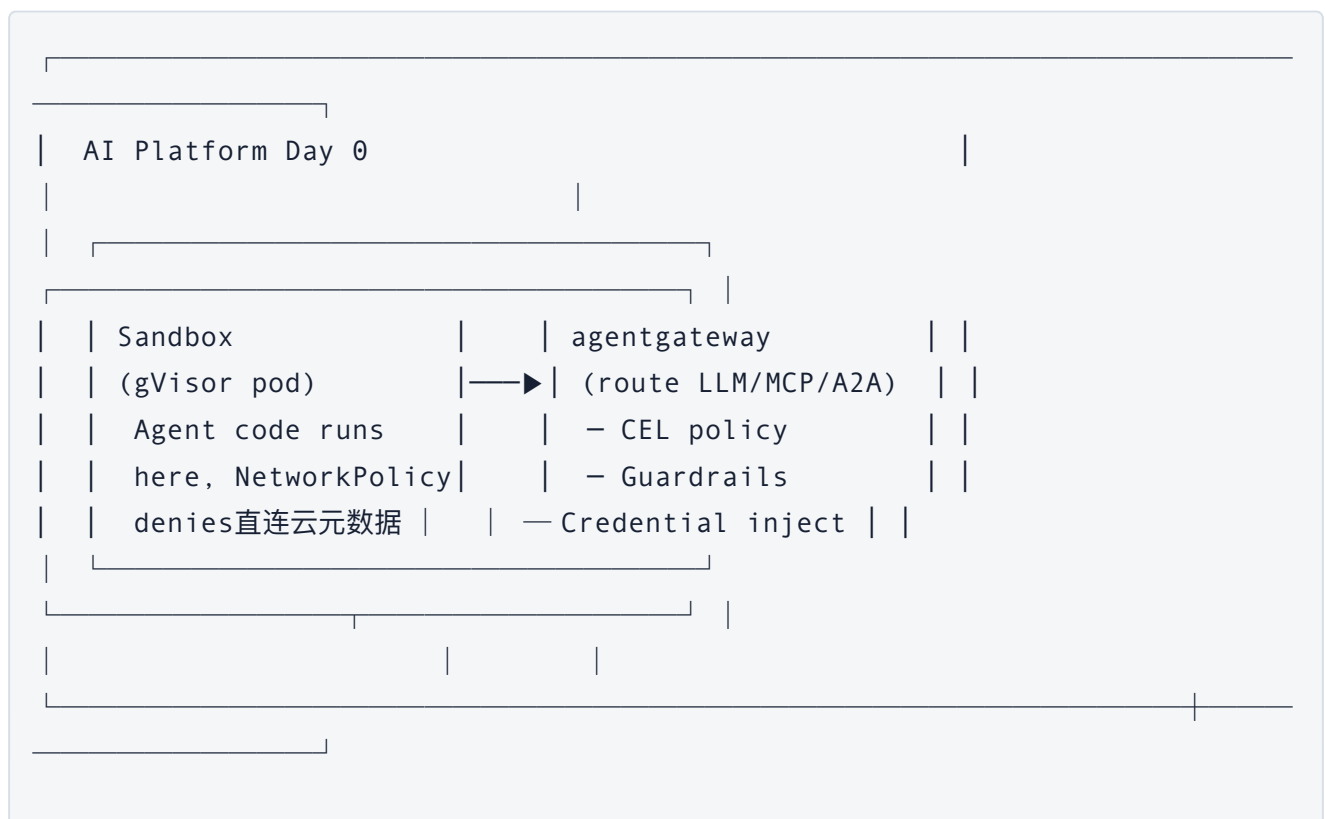
外部统一暴露 OpenAI 兼容 API (/v1/chat/completions / /v1/embeddings)，内部 dispatch 到 provider 双向翻译。

Guardrails —— LLM 流量特有的内容审查，pre-flight / post-flight 双向，支持 regex / OpenAI Moderation / AWS Bedrock Guardrails / Google Model Armor / Azure Content Safety / 自定义 webhook，每 backend 按声明顺序串成 chain。

负载均衡 —— 跨副本走 **two-random-choice** (tail-latency 比 round-robin 稳)；跨 model alias 走 weighted random。

与 HiClaw / agent-sandbox 的关系

- 跟 HiClaw 是不同层次互补：HiClaw 在应用层做 Agent 协作 (Matrix IM + Worker CRD)，agentgateway 在基础设施层做 AI 流量代理。HiClaw 的 Worker 跑用户代码访问 LLM / MCP 时，理论上可以让 agentgateway 当出口代理，由 agentgateway 接管凭据 / 审计 / guardrails。
- 跟 agent-sandbox 拼成 AI 平台底盘：agent-sandbox 做 Agent 运行时隔离 (Pod + gVisor / Kata + NetworkPolicy)，agentgateway 做 Agent 出口流量治理 (LLM / MCP / A2A)。两者并集 = AI 工作负载完整的"运行 + 通信"治理面。NetworkPolicy 限制 Sandbox Pod 出口只能走 agentgateway，强制所有 AI 流量过 policy + guardrails + 审计。



相关页面

- [agentgateway](#) —— 项目主页
- [mcp](#) —— MCP 协议（agentgateway MCP gateway 的协议基础）
- gateway-api —— K8s Gateway API（agentgateway 控制面 API 基础）
- xds —— xDS 协议（控制面 → 数据面分发）
- hbone —— Istio HBONE 隧道（agentgateway 跟 mesh 互操作）
- cel —— Common Expression Language（agentgateway 策略 IR）
- istio —— Istio（agentgateway 控制面骨架来源）
- HiClaw —— 应用层 Agent 协作平台（互补关系）
- agent-sandbox —— K8s SIG 的 AI Agent 运行时隔离（互补关系）
- [ai-agent-plugin-patterns](#) —— Agent 外挂设计原则
- agent-credential-isolation —— Agent 凭据零暴露模式

agentmemory

Rohit Ghumare 出品的本地化跨 Agent 持久记忆服务（仓库 [rohitg00/agentmemory](#), v0.9.21, Apache-2.0）。一个 worker 同时给 [claude-code](#) / Codex / Cursor / Gemini CLI / Hermes / OpenClaw / pi / OpenCode 及任何 [mcp](#) 客户端供应记忆。建立在 iii-engine 的 Worker/Function/Trigger 三原语之上。

解决的问题

AI 编码 Agent 跨会话是白板——每个新对话都得重新解释代码库、决策、踩过的坑。[claude-mem](#) 解决了单一 Agent（Claude Code）的版本；agentmemory 把这件事做到跨 Agent：所有 IDE / CLI Agent 共用同一个 :3111 worker、同一份 SQLite，记忆在工具之间流通。

核心特征

- 跨 Agent 共享：12 个 Claude Code hooks + Codex 6 hooks + 各 Agent 原生插件 + 53 个 [mcp](#) tools，所有客户端共用记忆
- 零外部依赖：本地 Node ESM + iii-engine + SQLite，无需 Postgres / pgvector / Qdrant / Redis
- 零 LLM 默认：启发式压缩 (buildSyntheticCompression) 是默认路径，要 AGENTMEMORY_AUTO_COMPRESS=true 才走 LLM——保护用户的 API token
- Context injection 默认关 (issue #143)：Hook 只抓不注入，避免 Claude Pro session token 被记忆查询消耗
- 三流混合检索：BM25 + Vector + Knowledge Graph，RRF (K=60) 融合，可选 reranker (hybrid-search-rrf)
- 多层记忆：32+ KV scope (episodic / working / semantic / procedural / graph / orchestration / lessons / crystals / sketches / sentinels / ...) + 强度衰减 sweep ([ebbinghaus-forgetting-curve](#))
- 实时 viewer：默认 :3113 上跑 WebSocket UI，看记忆实时流入
- 950+ tests 通过 + 公开评测：LongMemEval / Coding-Life benchmark 跑分内置在仓库

架构骨架

- 总线：iii-engine WebSocket（钉版 v0.11.2，因为 v0.11.6 sandbox 模型不兼容当前 worker）
- 存储：iii-engine StateModule 抽象的 SQLite，路径 ./data/state_store.db
- 业务函数：src/functions/*.ts 约 60 个 mem::* 函数，扁平注册在 src/index.ts:204-303
- 触发层：124 REST endpoints /agentmemory/* (:3111) + 53 MCP tools（默认仅 8 暴露）+ event triggers
- 状态层：内存 BM25 倒排表 + 内存 VectorIndex (Float32Array cosine) + HybridSearch + 周期 IndexPersistence

详细参见 [src-agentmemory-architecture](#)。

与 claude-mem / powermem 的关系

三者解决同一问题（[agent-memory](#)），但走向了三种不同的实现：

维度	agentmemory	claude-mem	powermem
部署形态	Node + iii-engine + SQLite	Node + Express daemon + chroma + bullmq	OceanBase 数据库扩展
客户端范围	任何 MCP 客户端 + 8 个 Agent 原生插件	专一 Claude Code	任何后端通过 SQL/向量 API 接入
检索栈	BM25 + Vector + Graph (RRF)	Vector (chroma) + FTS5	向量 + 全文 + 稀疏 + 图 (四路)
LLM 用量	默认零 LLM	LLM 压缩必需	LLM 抽取必需
衰减机制	强度衰减 + 4 类定时 sweep	无（只压缩不衰减）	working/short/long 三层 Ebbinghaus
MCP tool 数	53（默认 8）	6	由后端决定

设计哲学层面：[claude-mem](#) 是 [ai-as-compressor](#) 的纯净实现（LLM 压噪声）；agentmemory 是对该哲学的成本反思（默认关 LLM，启发式优先）；powermem 是把同一套机制下沉到数据库层。

关键依赖

- iii-engine v0.11.2（外部 Rust 进程，Worker/Function/Trigger 总线）
- iii-sdk（TypeScript binding）
- @modelcontextprotocol/sdk — MCP 协议
- @xenova/transformers — 本地嵌入（零云依赖路径）
- @clack/prompts — CLI 交互

已知限制

- 内存 VectorIndex 暴力扫表，5 万 observation 量级以上需要替换为 ANN（HNSW / FAISS / pgvector）
- 默认无 AGENTMEMORY_SECRET，任何能访问 localhost 的本机进程都能读写记忆
- unhandledRejection 顶层 60s 节流吞掉异常，可能掩盖真正的 bug
- 添加一个 MCP tool 需同步 7 处文件（AGENTS.md 列出），PR 漏改风险高

相关页面

- [src-agentmemory-architecture](#) — 完整架构源摘要
- [agent-memory](#) — 领域综述
- [claude-mem](#) — 单 Agent 版同类
- powermem — 数据库后端同类
- [claude-context](#) — 互补项目（代码检索 MCP）
- [event-driven-memory-pipeline](#) — Hook → Compress → Index → Inject 范式
- hybrid-search-rrf — 三流 RRF 方法论
- [ebbinghaus-forgetting-curve](#) — 衰减理论
- [ai-as-compressor](#) — 哲学对照
- [claude-code](#) — 12 hooks 主要消费者
- [mcp](#) — 53 个 tool 的协议基础

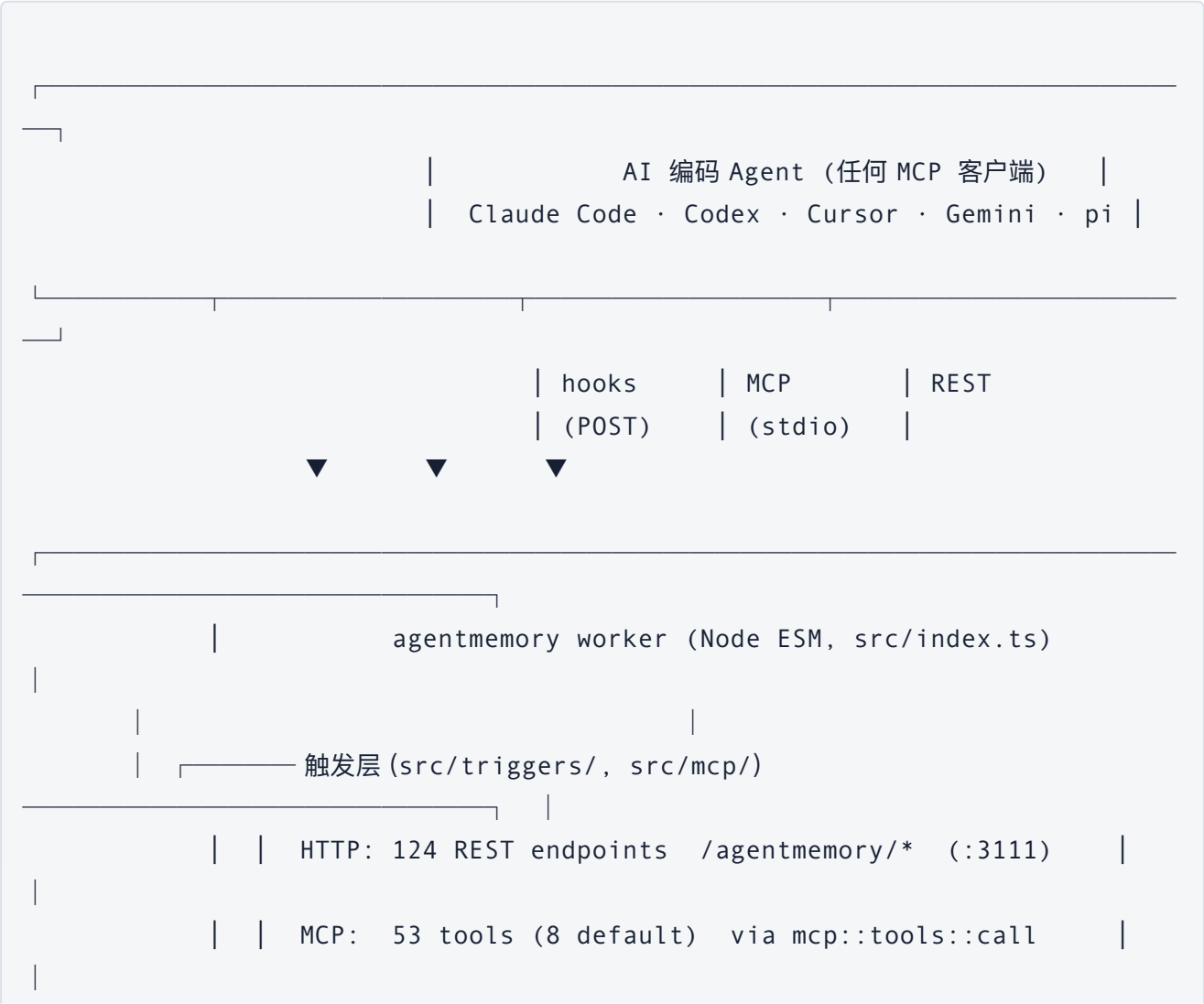
agentmemory 架构与设计思路分析

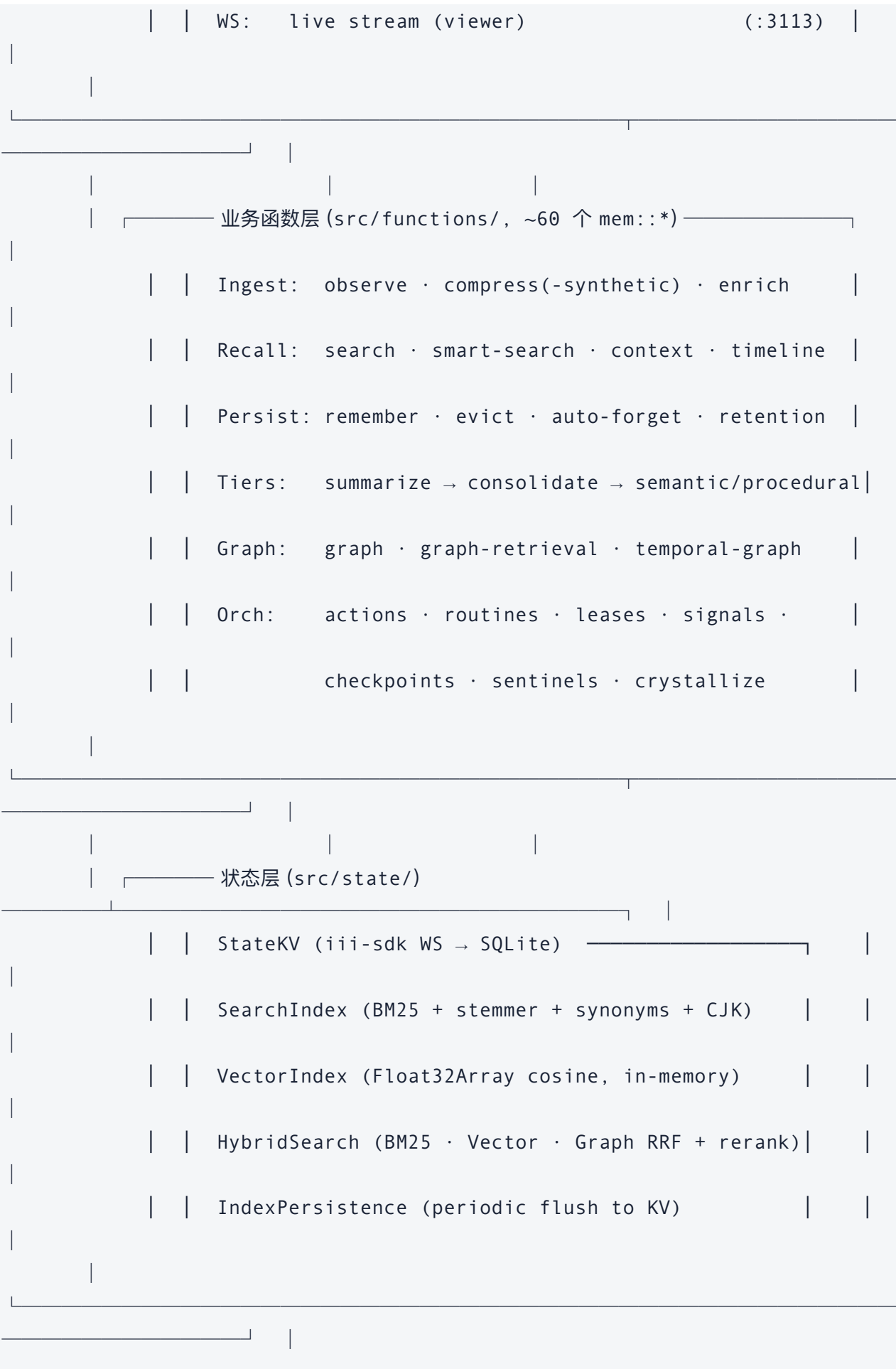
原文: raw/agentmemory-architecture-analysis.md · 仓库: <https://github.com/rohitg00/agentmemory> · 分析版本 v0.9.21

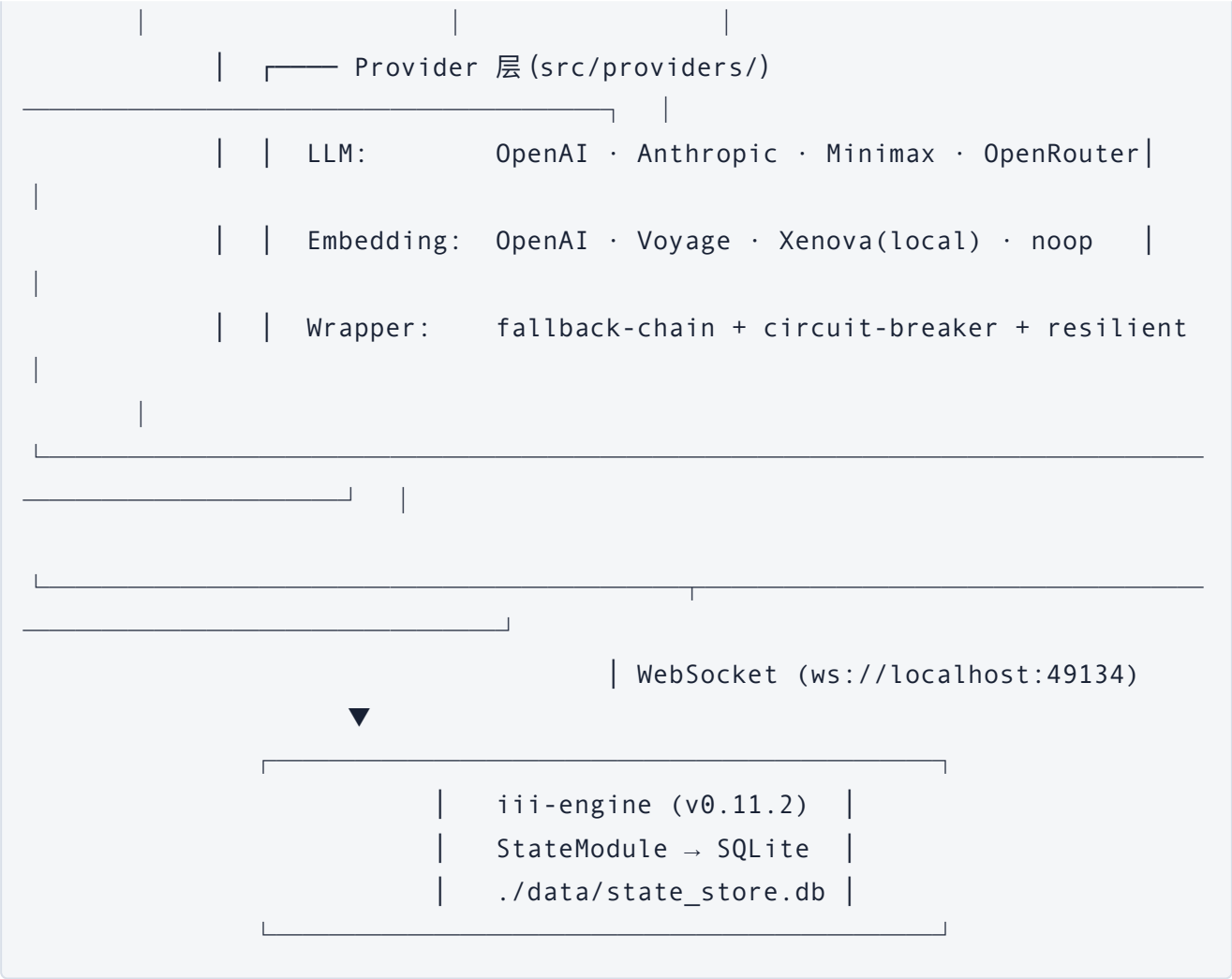
一句话定位

[agentmemory](#) 是为 AI 编码 Agent ([claude-code](#) / Codex / Cursor / Gemini CLI / Hermes / OpenClaw / pi / OpenCode 及任何 [mcp](#) 客户端) 提供持久跨会话记忆的本地服务: 通过 Hook 抓取工具调用 ([event-driven-memory-pipeline](#)) → 零 LLM 启发式压缩 ([ai-as-compressor](#) 的反向取舍) → 三流混合检索 (BM25 + Vector + Knowledge Graph, hybrid-search-rrf) → MCP / REST / 12 个 Hook 把上下文回注 Agent。所有客户端共用同一个 :3111 worker, 状态落在 iii-engine 托管的 SQLite。

核心架构图







模块分层

层 / 模块	职责
引导层	加载 config、连接 iii-engine、串起 50+ 函数注册、启动 viewer、安装定时器、绑定信号
触发层	124 REST endpoints + 53 MCP tools + 事件触发；所有都走 sdk.registerFunction + sdk.registerTrigger({type:"http"})
业务函数层	~60 个 mem:xxx 函数：observe / compress / search / remember / consolidate / graph / lessons / crystallize / ...
状态层	KV scope（32+ 类记忆命名空间） + BM25 倒排表 + Vector cosine + Hybrid RRF + 周期持久化 + 重排器
Provider 层	LLM 与嵌入抽象，带 fallback 链、断路器、resilient 包装
Hooks（独立脚本）	Claude Code 生命周期 Hook 脚本，不导入 iii-sdk，stdin 读 JSON → fetch POST → AbortSignal.timeout

层 / 模块	职责
Viewer	实时 WebSocket UI (mem-live stream) , 默认 :3113
CLI	agentmemory / agentmemory demo / agentmemory connect <agent>; 含 iii-engine 自动安装与版本钉定
Plugin	各 Agent 的原生插件清单 (hooks + skills + MCP 接入)
Evals	LongMemEval / Coding-Life 评测脚本 + 内部 metrics-store + self-correct + validator

关键约束（源自 AGENTS.md）：

- **iii-engine** 是强制总线：所有函数注册都走 `sdk.registerFunction / sdk.trigger`，禁止绕过 iii-sdk 用独立 SQLite 或进程内替代方案。
- **REST endpoint** 必须白名单字段：不要把 `req.body` 原样塞给 `sdk.trigger`，要显式挑字段。
- **MCP tool 增删要同步 7 处**：tools-registry / server switch / triggers/api / index 注册 + 计数 / 测试 / README / plugin.json。这是高维护成本耦合点。
- **Hook 脚本不依赖 iii-sdk**：保持轻量，`AbortSignal.timeout` 包裹 `fetch`，任何错误必须吞掉，不能阻塞 Agent。

关键数据流

Ingest（写路径）



```

OR : compressFunction()                                ← LLM 压缩
(AGENTMEMORY_AUTO_COMPRESS=true)
|
▼
CompressedObservation { id, title, facts[], narrative, concepts[], files[],
importance }
|
▼
SearchIndex.add() (BM25, 内存倒排表 + termFreq + IDF)
+
VectorIndex.add() (Float32Array, cosine, embedding 异步获取)
|
▼
WebSocket stream → Viewer (mem-live group)
|
▼
[周期] IndexPersistence 把 BM25 + Vector 序列化回 KV

```

Recall (读路径)

```

query (string)
|
▼
QueryExpansion          ← reformulations + temporalConcretizations +
entityExtractions
|
▼
HybridSearch.tripleStreamSearch() (src/state/hybrid-search.ts:77)
|
├─▶ BM25      SearchIndex.search(query, 2N)
|              stemmer + synonyms + CJK segmenter
|
├─▶ Vector    embeddingProvider.embed(query) → VectorIndex.search()
|              Float32Array 暴力扫表 + cosineSimilarity
|
└─▶ Graph     extractEntitiesFromQuery() →
GraphRetrieval.searchByEntities(depth=2)
              LLM 离线抽取 entity/edge, 在线只做 BFS
|
▼

```

```

RRF 融合 (K=60, w_bm25=0.4, w_vec=0.6, w_graph=0.3)
|
▼
[可选] Reranker (RERANK_ENABLED=true) → src/state/reranker.ts
|
▼
Token-budget trim
|
▼
返回给 Agent (format = full / compact / narrative)

```

Boot 启动序列

```

loadConfig + provider + embeddingProvider
|
▼
registerWorker(engineUrl)  —— WebSocket ——> iii-engine :49134
|                               |
|   invocationTimeoutMs: 180s (src/index.ts:168)   |
|   telemetry { project_name: "agentmemory", ... } |
▼
new StateKV(sdk) + new MetricsStore + new DedupMap + new VectorIndex
|
▼
register* (50+ 业务函数, src/index.ts:204-303)
|   privacy / observe / compress / search / context / summarize /
|   migrate / file-index / consolidate / patterns / remember / evict /
|   relations / timeline / profile / auto-forget / export-import /
enrich /
|   claude-bridge? / graph? / consolidation-pipeline / team? /
governance /
|   actions / frontier / leases / routines / signals / checkpoints /
mesh /
|   branch-aware / flow-compress / sentinels / sketches / crystallize /
|   diagnostics / facets / verify / lessons / obsidian-export /
reflect /
|   working-memory / skill-extract / cascade / sliding-window /
|   query-expansion / temporal-graph / retention / compress-file /
replay
▼

```

```

HybridSearch 装配 (bm25Index + vectorIndex + embeddingProvider + kv + 权重)
|
▼
registerApiTriggers + registerEventTriggers + registerMcpEndpoints
|
▼
indexPersistence.load()
├─ 恢复 BM25 索引 (如有) → bm25Index.restoreFrom()
├─ 恢复 Vector 索引
│   ├─ validateDimensions(activeDim)
│   ├─ mismatch != 0 ?
│   │   ├─ AGENTMEMORY_DROP_STALE_INDEX=true → 丢弃并 console.warn
│   │   └─ 否则 → throw Error (拒绝启动)
│   └─ vectorIndex.restoreFrom()
|
▼
needsRebuild ? void rebuildIndex(kv) ← fire-and-forget, 可能数小时
      : backfill BM25 from KV.memories (legacy gap before #257)
|
▼
bootLog "Ready. Triple-stream (BM25+Vector+Graph) search active."
|
▼
startViewerServer(restPort + 2, kv, sdk, secret, restPort)
|
▼
后台定时器 (全部 .unref()) :
  auto-forget          每 3,600,000ms (1h)
  lesson-decay-sweep   每 86,400,000ms (24h)
  insight-decay-sweep  每 86,400,000ms (24h)
  consolidate-pipeline 每 7,200,000ms (2h)
|
▼
process.on("SIGINT" | "SIGTERM", shutdown)
  healthMonitor.stop + dedupMap.stop + indexPersistence.stop
  viewerServer.close → indexPersistence.save → sdk.shutdown → exit(0)

```

补充说明：

- 超时：iii-sdk invocationTimeoutMs=180s (src/index.ts:168)。在写压力下 state::set 偶尔超过 30s 默认值 (issue #204)。
- 错误传递：顶层 process.on("unhandledRejection") 60s 节流后日志，不让一次 timeout 杀掉长生

命周期 worker。

- 回退路径：embedding provider 缺失时退化为 BM25-only；vector 索引 search 失败 fallback 到 BM25-only；graph 检索失败被 best-effort try/catch 吞掉。

设计决策与哲学

- **iii-engine 作为强制总线**：所有 50+ 业务函数都走 `sdk.registerFunction / sdk.trigger`，不允许绕过。代价是引入外部 iii-engine 进程依赖（钉到 v0.11.2，因为 v0.11.6 sandbox 模型不兼容当前 worker）；收益是统一审计、WebSocket 重放、远端代理可能性。这与 [claude-mem](#) 的“进程内 chroma + bullmq”形成鲜明对比——agentmemory 把“消息总线”外置，[claude-mem](#) 把它内置。
- **三流混合检索 + RRF**：BM25 抓字面（文件名/命令）、Vector 抓语义、Graph 抓实体关系；RRF (K=60) 比加权求和更鲁棒。设计上与 powermem 的“向量+全文+稀疏+图四路”接近，但 agentmemory 没有稀疏第四路、用 SQLite + 内存索引替代 OceanBase。参见 `hybrid-search-rrf`。
- **零 LLM 压缩为默认**（issue #138）：`buildSyntheticCompression()` 用 `toolName` 启发式推断 `ObservationType` + 正则抽 files。理由：默认开启 LLM 压缩 = 用户 API key 按工具调用频率持续烧 token。AGENTMEMORY_AUTO_COMPRESS=true 才进 LLM 路径。这是对 [ai-as-compressor](#) 的反向取舍——“AI 是好压缩器但成本不可见”。
- **Context injection 默认关**（issue #143）：Hook 只抓不注入。“自动塞 4000 字记忆进 Claude”对 Claude Pro 用户是 token 杀手——所以默认关，要 AGENTMEMORY_INJECT_CONTEXT=true 显式开启。这是项目对“成本可见性”的执着。
- **向量维度守卫**：持久化 vector index 维度与当前 embedding provider 不匹配时拒绝启动而不是 silently 让 cosineSimilarity 跨维度永远返回 0。可通过 AGENTMEMORY_DROP_STALE_INDEX=true 显式丢弃旧索引。“宁愿失败也不要静默错误”。
- **In-process BM25 + Vector**：不依赖 Postgres / pgvector / Qdrant；BM25 用 Map<term, Set> 倒排表 + termCount，Vector 用 Map 暴力扫表 + cosine。极简实现，5 万条以内体感无延迟；超过此规模需替换为 ANN（HNSW / FAISS / pgvector）。
- **多层记忆模型 + 强度衰减**：32+ KV scope（episodic / working / semantic / procedural / graph / orchestration / lessons / insights / crystals / sketches / sentinels / routines / leases / signals / checkpoints / facets / ...）。`applyDecay()` 用 `strength * 0.9^periods` 指数衰减，配合 auto-forget (1h)、lesson-decay-sweep (24h)、consolidation (2h) 形成 Ebbinghaus 风格的“主动遗忘”机制——参见 [ebbinghaus-forgetting-curve](#)，与 powermem working/short/long 三层定时调度同构。
- **Hooks 与 SDK 解耦**：Hook 脚本是 standalone Node，不导入 iii-sdk，只走 HTTP；`AbortSignal.timeout()` 包裹 fetch，全 try/catch 吞错。设计目标：Hook 失败绝不能阻塞 Agent 工具调用。这与 [claude-mem](#) 的 hook 方式如出一辙——都是 [event-driven-memory-pipeline](#) 的最佳实践沉淀。
- **MCP “默认 8 / 全量 53”双轨**：53 个 MCP tools 是完整能力面，但默认只暴露 8 个核心工具（recall / save / compress_file / 等）给 Agent，避免选项过多让模型迷失；AGENTMEMORY_TOOLS=all 全量暴露给高级用户。
- **fire-and-forget rebuild**：首次启动若索引为空，`rebuildIndex` 可能耗时数小时（每条 observation 一次 embedding 调用，受云端速率限制），所以绝不 await——不阻塞 viewer 端口绑定与服务可用。索引边用边补，搜索质量随时间收敛。

关键组件深入解读

src/state/hybrid-search.ts — 检索核心

HybridSearch.tripleStreamSearch(query, limit, entityHints?) 三流并行: BM25 同步取 $\text{limit} * 2$; Vector 异步 embed(query) 失败则降级 BM25-only; Graph 用 extractEntitiesFromQuery 或传入 entityHints, BFS 深度 2。顶 5 个 vector 结果做 "expansion search" (用 obsId 做相似度扩散, 找邻居)。searchWithExpansion(query, limit, expansion) 把 query reformulations $\times$ 三流并行做 Promise.all, 再用 combinedScore 去重最大值。

src/state/schema.ts — KV scope 总线

KV 是 const 对象, 既有固定 key (mem:sessions / mem:memories / mem:audit) 又有函数形式 (observations(sessionId) / teamShared(teamId) / embeddings(obsId))。这种"函数化 key"模式让一个 KV scope 能横向分片到 sessionId / teamId / userId, 避免单 scope 无限增长。
generateId(prefix) 用 ts.toString(36) + uuid.slice(0,12) 生成时间排序友好的 ID;
fingerprintId(prefix, content) 用 sha256 前 16 字符做内容寻址 (content-addressable dedup);
jaccardSimilarity() 给 dedup 提供回退方案。

相关页面

- [agentmemory](#) — 项目实体页 (这是该实体的源摘要)
- [claude-mem](#) — 同类项目 (专一 Claude Code, 进程内 chroma + bullmq)
- powermem — 同类项目 (OceanBase 后端, 四路检索 + Ebbinghaus)
- [claude-context](#) — 互补项目 (代码语义检索 MCP)
- [claude-code](#) — 12 个 Hook 集成的主要 Agent
- [mcp](#) — 53 个 MCP tool 的协议基础
- hybrid-search-rrf — 三流 RRF 检索方法论
- [event-driven-memory-pipeline](#) — Hook $\rightarrow$ Compress $\rightarrow$ Index $\rightarrow$ Inject 闭环
- [ebbinghaus-forgetting-curve](#) — 强度衰减 + 周期 sweep 的理论基础
- [ai-as-compressor](#) — 与 agentmemory 默认零 LLM 压缩的取舍对照
- [agent-memory](#) — Agent 长期记忆领域综述

AgentScope

AgentScope 2.0 是面向生产化多 Agent 应用的 Python 框架。它把 Agent 推理循环、模型 provider、工具 / [mcp](#) / skill、workspace、权限、人类确认、长任务 offload 和 FastAPI 服务层拆成可组合的异步组件。详见 [src-agentscope-architecture](#)。

架构边界

AgentScope 的核心协议是 `Agent.reply_stream()` 事件流，而不是只返回最终文本。模型输出、thinking、tool call、tool result、确认请求和外部执行结果都会进入 `AgentEvent / Msg`，这让 SDK、SSE、存储和前端可以共享同一套执行语义。

关键设计

- Agent 只依赖模型、toolkit、workspace、permission 和 middleware 抽象。
- ReAct loop 把“等待人类确认/外部执行”作为一等状态，可以暂停后恢复。
- Workspace 是工具资源和上下文外溢边界，可接 Local / Docker / E2B。
- 慢工具可以 offload 到后台，完成后再把结果注入后续 reasoning。
- 服务层用 FastAPI / SessionManager / storage 把 agent app 多租户化。

选型判断

需要写多 Agent 应用和服务化 agent app 时看 AgentScope。需要只做记忆组件看 [ReMe](#) 或 [mem0](#)；需要 Kubernetes 运行时部署看 [agentscope-runtime](#)；需要强 sandbox 隔离看 agent-sandbox / [openshell](#)。

AgentScope Runtime

AgentScope 生态的生产化 Agent-as-a-Service runtime，把 AgentApp、Runner、protocol adapters、sandbox manager 和 K8s/serverless deployers 组合起来。详见 [src-agentscope-runtime-architecture](#)。

架构边界

它处在 framework app 与生产 runtime 之间：比 agent-sandbox 更上层，比完整 managed teammate 产品更底层。仓库 README 已提示能力并入 AgentScope 2.0，所以更适合作为架构迁移参考。

AgentApp 执行流程

```
AgentApp / Agent definition
    ↓
Runner: session + event loop
    ↓
Protocol Adapter: HTTP / SSE / RPC
    ↓
Workspace / Tool / MCP
    ↓
Sandbox Manager 或 K8s/Serverless Deployer
    ↓
AgentEvent / stream / persistence
```

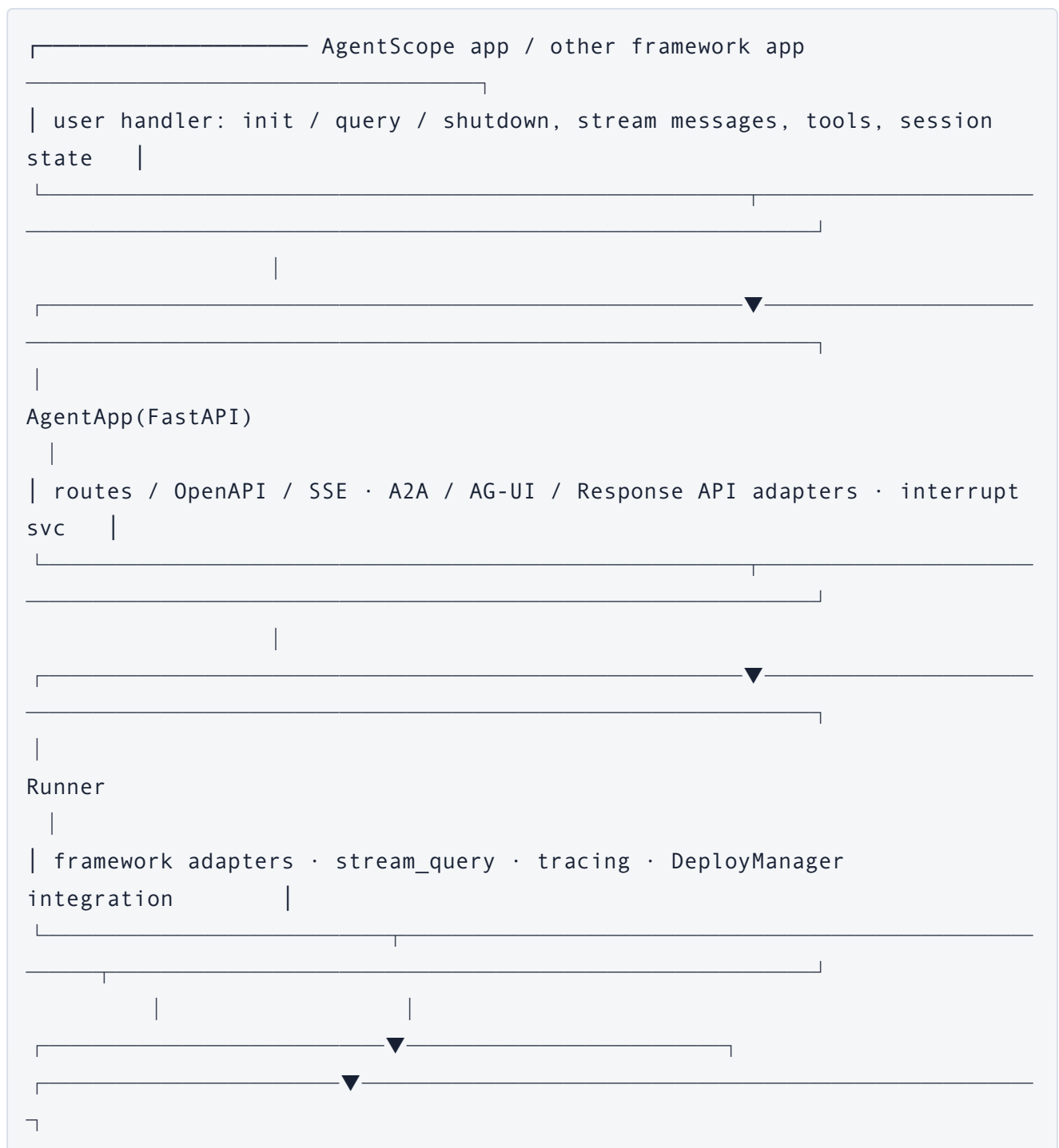
选型判断

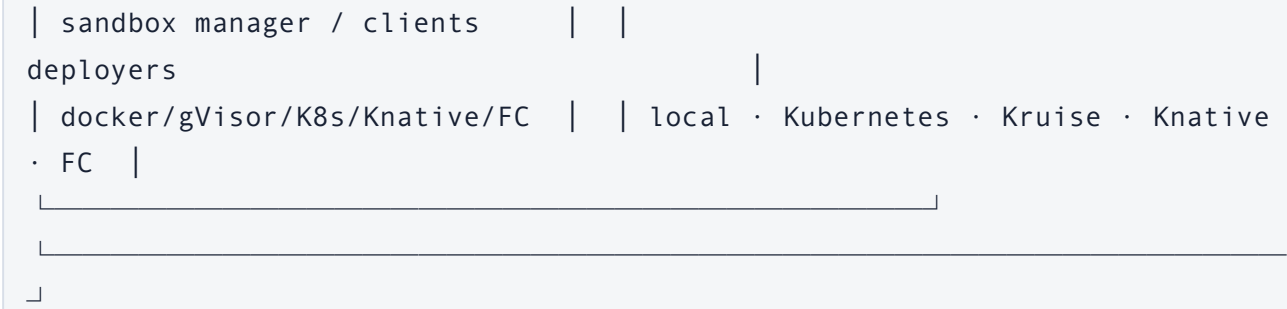
- 要理解 AgentScope app 如何服务化：看 AgentScope Runtime。
- 要理解底层容器/沙箱隔离：看 agent-sandbox 和 agent-runtime-substrate。
- 要理解多 Agent 协作产品层：看 [agentcube](#)、HiClaw、Multica 类项目。

AgentScope Runtime 架构与设计思路分析

agentscope-ai/agentscope-runtime 是 AgentScope 生态把 framework app 包装成生产 API/runtime 的过渡层: AgentApp(FastAPI) 暴露 streaming endpoint, Runner 统一 agent handler 生命周期, deployers 把本地、Kubernetes、Knative/FC 等环境抽象成可部署服务, sandbox manager 给工具调用提供隔离执行入口。README 明确提示这些能力已经并入 AgentScope 2.0, 仓库更像生产 runtime 设计参考。

核心架构图





模块分层

层/目录	责任
src/agentscope_runtime/engine/app/agent_app.py	AgentApp 直接继承 FastAPI，负责 endpoint、OpenAPI schema 注入、protocol adapters、分布式 interrupt service 和中间件。
src/agentscope_runtime/engine/runner.py	核心执行器，管理 query_handler/init_handler/shutdown_handler，把 handler 输出统一成 streaming event。
engine/deployers/**	把 runtime 投递到 local、Kubernetes、Knative、Kruise、FC、ModelStudio 等环境。
sandbox/** + common/container_clients/**	sandbox registry、manager server、workspace/storage mixin，以及 docker/gVisor/K8s/Knative/FC client。

关键数据流

- 请求进入 /process 或 A2A/Response API adapter，AgentApp 反序列化成 AgentRequest，交给 Runner.stream_query。
- Runner 检查 framework type 和 health，再调用用户注册的 query handler；同步、异步、generator、async generator 输出都被包成统一 stream。
- 工具调用通过 adapter 转成 runtime tool；需要隔离时走 sandbox client/manager，manager 再落到 Docker、gVisor、K8s、serverless 容器。

设计决策

- 把 Agent app 做成 FastAPI 子类，而不是另起 RPC 框架，换来 OpenAPI、中间件、生命周期和生态兼容。
- protocol adapter 是外壳，Runner 是内核；A2A/AG-UI/Response API 可以并存。
- sandbox 和 deployer 分离：前者处理工具调用隔离，后者处理整个 agent service 如何运行。

对比定位

和 [src-agentscope-architecture](#) 相比，它不是 agent 编程框架，而是服务化/部署/隔离壳；和 agent-sandbox 相比，它不把 workload 建模成 K8s CRD，而是在 Python runtime 内部暴露 Agent-as-a-Service；和 Agent Substrate 相比，它偏应用 runtime，不做高密度 actor/worker multiplexing。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

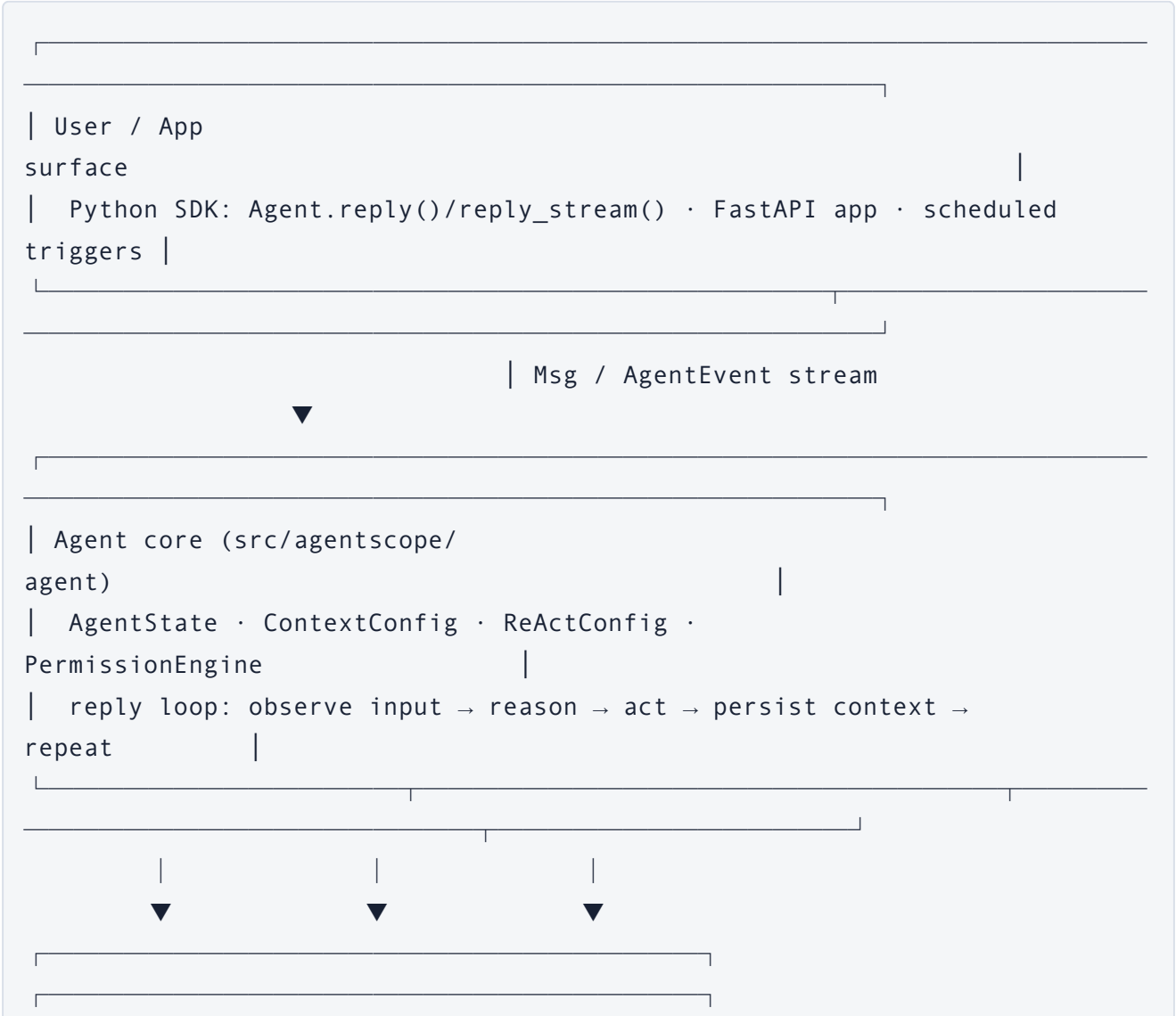
AgentScope 架构与设计思路分析

原文: raw/agentscope-architecture-analysis.md · 仓库: [agentscope-ai/agentscope](#) · 分析版本 main HEAD e129177 (2026-06-01)

一句话定位

AgentScope 2.0 是一个面向生产化多 Agent 应用的 Python 框架：它把 Agent 推理循环、模型 provider、工具 / [mcp](#) / skill、workspace、权限、人类确认、长任务 offload 和 FastAPI 服务层拆成可组合的异步组件。它的核心不是固定编排 DSL，而是以 `Agent.reply_stream()` 为中心的事件流，把 ReAct 过程中的模型输出、tool call、tool result、确认请求和外部执行结果统一建模成可持久化的 `AgentEvent / Msg`。

核心架构图





模块分层

层 / 模块	职责
公共 SDK / Agent 核心	管理 session state、上下文、ReAct 循环、事件流和最终消息聚合。
模型抽象与 provider	把统一 Msg / content blocks 转成各 provider API 请求，把流式 / 非流式响应转回 ChatResponse。

层 / 模块	职责
工具、MCP、skill	注册 Python tools、MCP tools、agent skills，生成 JSON schema，执行工具并归一成 ToolChunk / ToolResponse。
权限与确认	在工具执行前做 deny/ask/allow、模式检查、危险路径检查，并通过事件请求用户确认或外部执行。
Workspace / offload	为每个 session 提供本地 / Docker / E2B 执行环境，暴露 tools/MCP/skills，并持久化压缩上下文和超长工具结果。
FastAPI app/service	多租户、多 session 服务层；负责 agent 组装、SSE 事件输出、存储、调度、后台任务。
长任务与计划任务	把慢工具 offload 到后台，结果完成后重新注入上下文；用 APScheduler 驱动定时 agent run。

这个分层的关键约束是：Agent 只依赖抽象接口，不直接关心 provider SDK、Docker/E2B 细节、Redis 存储或 FastAPI 请求对象。服务层通过 `get_agent()` 每次从 `storage/session/workspace` 重新组装一个 Agent，Agent 内部再用统一事件流表达执行过程。

关键数据流



next reasoning iteration or final Msg

服务化路径是在这个本地流外面包一层持久化和 SSE:

HTTP POST /chat

FastAPI router

encode AgentEvent as "data: <json>"

ChatService.stream_chat

└ build ToolOffloadMiddleware

└ get_agent(storage, workspace_manager, user, agent, session)

└ load AgentRecord / SessionRecord / AgentState

└ instantiate model and optional fallback model

└ get per-session workspace

└ build Toolkit(tools + skills + MCPs)

└ SessionManager.run(session_id) serializes the run

└ persist input Msg

└ stream Agent.reply_stream() events

└ reconstruct AssistantMsg by append_event()

└ persist reply Msg + updated AgentState

设计决策与哲学

- 事件流是核心协议，而不是“最终文本”：模型调用、文本 / thinking / data / tool-call delta、tool result、reply end 都是事件；这让 SDK、SSE、存储和前端 UI 可以共享同一种流式语义，也适合做 [agent-memory](#) 或观测流水线。
- ReAct 循环把“等待人类/外部系统”作为一等状态：确认和外部执行通过 RequireUserConfirmEvent / RequireExternalExecutionEvent 暂停，再用 continuation event 恢复，比阻塞式输入更适合 Web 服务。
- 工具执行被拆成“验证/权限/上下文写入”和“原始 I/O”两层：permission 语义在 Agent 内部，工具 I/O 通过 Toolkit.call_tool() 和 middleware 拦截；这和 agent-credential-isolation、[ai-agent-plugin-patterns](#) 里强调的边界治理一致。

- **Workspace** 是工具资源和上下文外溢的边界：Local / Docker / E2B workspace 给同一个 Agent 暴露 tools、[mcp](#)、skills 与 offload，和 agent-sandbox 的“执行环境原语”在问题域上相邻，但 AgentScope 更偏应用框架。
- 慢工具不取消，而是 offload 后再注入：服务层把超时工具转为后台任务，先给模型 synthetic result，任务完成后再把结果作为 hint 注入下一次 reasoning，避免单个慢 I/O 卡死 SSE 回复。

核心组件

Agent

Agent 构造时接收模型、toolkit、middlewares、state、context/react config 和 offloader。核心 `_reply_impl()` 先区分新输入和 continuation event，再进入 `cur_iter < max_iters` 的 ReAct loop：需要 reasoning 时准备 system prompt、summary、context、tool schemas 并调用模型；模型产出 tool calls 后按 sequential/concurrent batch 执行；如果触发用户确认或外部执行，就返回等待消息，让后续事件恢复同一个 reply。

App Service

FastAPI chat router 只负责把 `ChatService.stream_chat()` 的 `AgentEvent` 编码成 SSE。
`ChatService` 才是服务端运行入口：它构造 `ToolOffloadMiddleware`，调用 `get_agent()` 从 storage、session、credential、workspace 组装 Agent，在 `SessionManager.run(session_id)` 中串行运行同一 session，并把输入消息、reply 消息、更新后的 `AgentState` 写回 storage。

相关页面

- [mcp](#)
- [agent-memory](#)
- [ai-agent-plugin-patterns](#)
- agent-credential-isolation
- [claude-code](#)
- agent-sandbox
- [nanobot](#)

AI Agent Frameworks Star 项目清单整理

原文: raw/ai-agent-frameworks-stars-list.md · 来源: [googs1025 的 AI Agent Frameworks Stars list](#) · 抓取日期: 2026-06-03 · 仓库数: 109

一句话定位

这个 Star list 不是单纯的“Agent 框架合集”，而是围绕 AI Agent 工程化的完整生态地图：上层是 OpenClaw / Hermes / [claude-code](#) / OpenCode 这类可直接使用的个人或编码 Agent；中层是 LangChain、LangGraph、Dify、AgentScope、Eino、ADK、Dapr 等 Agent / workflow 框架；底层则延伸到 [mcp](#)、Skills、[agent-memory](#)、观测、评测、网关、安全沙箱、kubernetes runtime 和多云算力。

分层地图





速读分组

分组	代表项目	价值
个人 Agent / Agent OS	OpenClaw, Hermes, OpenClaude, NemoClaw, QwenPaw, OpenHarness, IronClaw	把 Agent 做成可直接使用的个人助手、协作 OS 或多平台运行时。
Coding Agent / Claude Code 生态	claude-code , OpenCode, ECC, Claw Code, oh-my-openagent, Pi, Multica, claude-code-router	围绕 terminal coding agent 形成 router、template、memory、skills、trace、token、IM bridge 和 delegation 工具链。
Agent 框架 / Workflow 平台	LangChain, LangGraph, Dify, AgentScope, Eino, ADK, Dapr, YoMo	提供 Agent 编排、状态图、workflow、tool calling、部署和应用开发框架。
MCP / SDK / Gateway	FastMCP, MCP Python SDK, GitHub MCP, Playwright MCP, agentgateway , Plano	统一 tool/resource 接入、代理、治理和协议 SDK。

分组	代表项目	价值
Skills / Prompt Pack	Anthropic Skills, agent-skills, Matt Pocock skills, awesome-agent-skills, finance/code review skills	把 Agent 能力从硬编码插件迁移成 Markdown + scripts + workflow 的可复用能力包。
记忆 / 上下文 / 观测 / 评测	claude-mem , mem0, agentmemory , ReMe, powermem, claude-context , Langfuse, AgentOps	为 Agent 增加长期记忆、代码语义上下文、trace、cost、eval 和质量反馈。
Cloud-native runtime	agent-sandbox, agentcube , OpenShell, Docker Agent, SkyPilot, KAgent, kubewizard	解决 Agent 的隔离、调度、多云算力、K8s 排障和 runtime 管理。

项目清单

个人 Agent、Agent OS 与多 Agent Runtime

- [openclaw/openclaw](#): 个人 AI assistant, 多 OS / 多平台, 是这个 list 中最强的端侧 Agent 信号。
- [NousResearch/hermes-agent](#): 个人 Agent 方向, 强调“会随用户成长”。
- [Gitlab/openclaude](#)、[NVIDIA/NemoClaw](#)、[NVIDIA/OpenShell](#): OpenClaw / Hermes / OpenClaude 生态的安全 runtime、托管推理和跨环境运行路线。
- [agentscope-ai/QwenPaw](#)、[HKUDS/OpenHarness](#)、[nearai/ironclaw](#)、[agentscope-ai/HiClaw](#): 个人 assistant、Agent harness、Agent OS、多人协作 Agent runtime。
- [OpenCoworkAI/open-cowork](#): 桌面端把 [claude-code](#)、[mcp](#) tools、Skills、sandbox 和 IM integration 打包成 end-user app。
- [swarmclawai/swarmclaw](#)、[clawwork-ai/ClawWork](#)、[agent-substrate/substrate](#): 自托管 multi-agent runtime、OpenClaw client 和 Agent substrate。

Coding Agent、Claude Code 周边与工程化工作台

- [anthropics/claude-code](#): Anthropic 官方 terminal agentic coding tool。
- [anomalyco/opencode](#)、[ultraworkers/claw-code](#)、[Hmbown/CodeWhale](#)、[code-yeongyu/oh-my-openagent](#): 开源 coding agent / agent harness / terminal coding agent 方向。
- [affaan-m/ECC](#): 跨 [claude-code](#)、Codex、OpenCode、Cursor 的 agent harness 优化系统, 覆盖 skills、memory、security、research-first development。
- [garrytan/gstack](#)、[shanraishan/claude-code-best-practice](#)、[davila7/claude-code-templates](#): Claude Code 配置、工作流和最佳实践集合。
- [multica-ai/multica](#)、[earendil-works/pi](#)、[musistudio/claude-code-router](#)、[openai/codex-plugin-cc](#): coding agents 的托管平台、toolkit、模型路由和任务委托。
- [thedotmack/claude-mem](#)、[zilliztech/claude-context](#)、[tirth8205/code-review-graph](#): coding agent 长期记忆、代码语义搜索和本地代码图。

- [chenhg5/cc-connect](#)、[liaohch3/claude-tap](#)、[junhoyeo/tokscale](#): IM bridge、API trace viewer 和 token usage tracker。

Agent 框架、Workflow 平台与应用开发框架

- [langchain-ai/langchain](#)、[langchain-ai/langgraph](#): Agent engineering platform 和图式 resilient agents 框架。
- [langgenius/dify](#): 生产级 agentic workflow 平台。
- [agentscope-ai/agentscope](#)、[agentscope-ai/agentscope-runtime](#): 可观察、可理解、可信任的 Agent 构建框架与生产 runtime。
- [cloudwego/eino](#)、[google/adk-go](#)、[tmc/langchaingo](#)、[YaoApp/yao](#): Go 生态 Agent / LLM app framework。
- [dapr/dapr](#)、[yomorun/yomo](#): 事件驱动、workflow、edge / serverless 方向的 Agent-native cloud runtime。
- [TauricResearch/TradingAgents](#)、[datawhalechina/hello-agents](#)、[panaversity/learn-agentic-ai](#): 垂直多 Agent 框架与教程型资源。

MCP、SDK、Gateway 与工具协议层

- [PrefectHQ/fastmcp](#)、[modelcontextprotocol/python-sdk](#): Python MCP servers / clients 开发基础。
- [github/github-mcp-server](#)、[microsoft/playwright-mcp](#)、[rohitg00/kubectl-mcp-server](#): GitHub、浏览器自动化和 Kubernetes 的 MCP server。
- [agentgateway](#)、[katanemo/plano](#): Agent / MCP / LLM 流量的 proxy、data plane、observability、安全和路由治理。
- [anthropics/claude-agent-sdk-python](#)、[anthropics/anthropic-sdk-go](#)、[severity1/claude-agent-sdk-go](#): Anthropic / Claude Agent SDK 生态。

Skills、Prompt Pack 与可复用 Agent 能力包

- [anthropics/skills](#): Anthropic 官方 Agent Skills 公共仓库，是 Skills 机制的主参考。
- [addyosmani/agent-skills](#)、[mattpocock/skills](#)、[libukai/awesome-agent-skills](#)、[softaworks/agent-toolkit](#): 面向 AI coding agents 的工程 skills、个人 skills、指南和工具包。
- [Cocoon-AI/architecture-diagram-generator](#)、[RKiding/Awesome-finance-skills](#)、[Zhen-Bo/pragmatic-clean-code-reviewer](#): 架构图、金融分析、代码审查等垂直 skills。
- [googs1025/claude-code-explorer-skill](#)、[googs1025/clean-code-reviewer-skills](#)、[googs1025/code-review-skill](#): 已迁移/可迁移到 Codex 的源码解读与代码审查 skills。

记忆、上下文、观测与评测

- [claude-mem](#)、[mem0ai/mem0](#)、[agentmemory](#)、[agentscope-ai/ReMe](#)、[powermem](#): Agent 记忆的多条路线，分别偏 coding-agent session memory、通用 memory layer、持久 memory service、memory kit 和持久化记忆中间件。
- [claude-context](#)、[tirth8205/code-review-graph](#): 代码语义检索和代码图，解决大代码库上下文选择。

- langfuse/langfuse、AgentOps-AI/agentops、agentscope-ai/OpenJudge: Agent / LLM observability、cost tracking、benchmark、eval 和质量奖励。

Cloud-native / Kubernetes / Infra 边界项目

- agent-sandbox、[agentcube](#)、paperclipinc/openclaw-operator: Kubernetes 上的 Agent runtime 隔离、有状态会话编排和 OpenClaw operator。
- skypilot-org/skypilot: 跨 K8s / Slurm / 公有云 / on-prem 的 AI workload 控制平面。
- kagent-dev/kagent、qingwave/kubewizard、googs1025/kube-agent-helper: Kubernetes-native Agentic AI、自动排障 Agent 和 K8s diagnostic assistant。
- docker/docker-agent、armadaproject/armada、kubewall/kubewall、kubernetes/kube-state-metrics: Agent builder/runtime、batch queue、K8s dashboard 和 observability substrate。

观察

- 这个 list 的核心信号是“Agent 工程开始分层”: 可直接使用的 Agent / Agent OS 在上层爆发, 底层同时出现 Skills、[mcp](#)、memory、observability、gateway、sandbox 和 cloud-native runtime。
- [claude-code](#) 已形成一个事实生态: 官方工具、router、templates、skills、memory、trace viewer、token tracker、IM bridge、Codex plugin、code graph 等项目都围绕同一个 terminal coding agent 扩展。
- OpenClaw / Hermes / OpenClaude / Claw 系项目非常强, 说明“个人 Agent + 多平台入口 + sandbox + memory + messaging”正在变成独立产品形态, 而不只是框架 demo。
- Agent 框架路线分成三类: LangChain/LangGraph 这类 Python 图/链框架, Dify/AgentScope 这类平台化 runtime, 以及 Eino/ADK/langchaingo/Yao 这类 Go 生态 framework。
- [mcp](#) 已经从协议变成基础设施层: 官方 SDK、FastMCP、GitHub MCP、Playwright MCP、Kubernetes MCP、[agentgateway](#) / Plano 这类 proxy 都在同一个 tool/resource 接入面上竞争。
- Skills 是新的“可迁移能力包”: Anthropic 官方 skills、Addy Osmani / Matt Pocock / finance / architecture / code review skills 都把 Agent 能力从代码插件转成 Markdown + scripts + workflow 的组合。
- 记忆和上下文正在成为独立产品: [claude-mem](#)、mem0、[agentmemory](#)、ReMe、powermem、[claude-context](#)、code-review-graph 分别从 session history、universal memory、coding-agent memory、memory kit、RAG/search、code graph 角度切入。
- K8s / cloud-native 项目不是 list 的主体, 但很关键: agent-sandbox、[agentcube](#)、OpenShell、Docker Agent、SkyPilot、KAgent、kubewizard 等说明 Agent runtime 开始需要隔离、调度、观测和多云算力。

优先深挖候选

优先级	项目	原因
1	openclaw/openclaw	Star 数和生态扩展最强, 代表个人 Agent / Agent OS 产品化路线。

优先级	项目	原因
2	anthropics/skills	Skills 作为 Agent 能力包的官方样板，适合和 Codex skills 迁移实践对照。
3	affaan-m/ECC	同时覆盖 Claude Code / Codex / OpenCode / Cursor 的 harness 优化系统，适合研究跨 Agent 工程模式。
4	anomalyco/opencode	开源 coding agent 主线，适合与 Claude Code / Codex / OpenClaw 对比。
5	langchain-ai/ langgraph	图式 Agent runtime 标准参照，适合对比 AgentScope / Dify / Dapr workflow。
6	PrefectHQ/fastmcp	MCP server/client 开发的主流 Python 工具。
7	claude-mem	跨 Agent 持久上下文方向，与 agentmemory / powermem / mem0 互补。
8	agentscope-ai/ agentscope-runtime	生产级 Agent runtime，带安全 tool sandboxing 和 observability，适合连接 AgentScope / HiClaw / QwenPaw。
9	katanemo/plano	Agentic proxy / data plane，和 agentgateway 共同代表 Agent 网络与治理层。
10	OpenCoworkAI/open-cowork	把 claude-code 、 mcp 、Skills、sandbox 和 IM 集成到桌面端，代表 end-user packaging。

相关页面

- [claude-code](#)
- [mcp](#)
- [claude-mem](#)
- [claude-context](#)
- [agentmemory](#)
- powermem
- [agent-memory](#)
- [ai-agent-plugin-patterns](#)
- agent-sandbox
- [agentcube](#)
- [agentgateway](#)

AI Agent 外挂的 9 条设计原则

构建"AI Agent 外挂"工具（MCP 插件、Hook 插件、记忆系统、RAG 系统）的通用工程经验。从 [claude-context](#) 与 [claude-mem](#) 两个开源项目抽象而来。

适用对象

任何给 Agent 加外部能力的工具：

- MCP / A2A 协议插件
- Agent runtime 的 Lifecycle Hook（如 Claude Code）
- RAG / 向量检索系统
- 长期记忆系统
- 代码理解 / 文档检索工具

9 条原则

1. 三段式接口分层（抽象 / 协议 / 引擎）

客户端层 → 协议层 → 引擎层
(UI) (MCP) (纯领域逻辑)

- 引擎层不感知协议——明天换 A2A 只动外层
- 协议层只做转换 + 状态管理
- 客户端独立 UI

case: claude-context 把核心拆成 packages/core、packages/mcp、packages/vscode-extension，引擎层完全不知道 MCP 存在。

2. 所有外部依赖都做成 Strategy Pattern

```
abstract class Embedding {  
  abstract embed(text: string): Promise<Vector>  
}  
// OpenAI / Voyage / Gemini / Ollama 各自实现
```

LLM、向量库、记忆存储——凡是外部 SDK 一律包接口。本地 Ollama 还是云端 GPT-5 只是配置切换。

case: claude-context 的 4 种 embedding × 2 种 vectordb × 2 种 splitter 可任意组合。

3. 降级链 (Graceful Degradation)



永不让一个边缘 case 卡死整个 pipeline。Agent 调用工具失败时应有"次优工具"或"裸 LLM 兜底"。

4. Merkle DAG 做状态指纹

详见 merkle-dag-fingerprint。

不只是文件，对话历史去重、知识库增量、工具调用缓存都适用。用内容 hash 而非时间戳/版本号。

5. 协议通道纪律：stdout 只跑协议

```
// MCP 服务第一行代码
console.log = (...args) => process.stderr.write('[LOG] ' + args.join(' ') +
'\n');
```

任何与 Agent 通信的进程 (stdin/stdout 管道、Unix socket、MCP server) 都要第一行就收紧 stdout。一个 `console.log("debug")` 就让协议全炸。

6. 协作式取消 (AbortSignal) 穿透整个调用栈

```
async indexCodebase(..., signal?: AbortSignal)
  async processFileList(..., signal?: AbortSignal) {
    if (signal?.aborted) throw new IndexAbortError()
  }
```

长时任务（爬取、批量推理、链式工具调用）必须从入口透传 signal 到最底层。否则用户"取消"按钮就是骗人的。

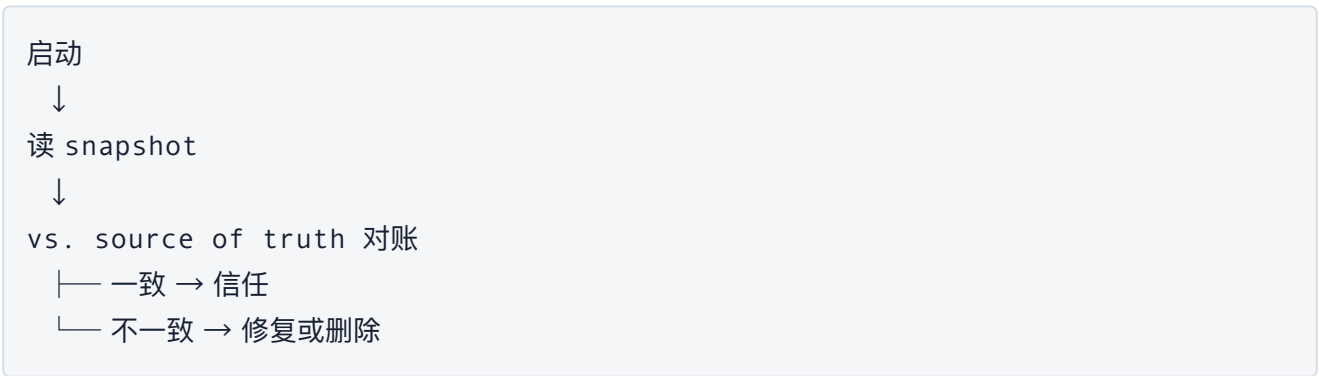
case: claude-context issue #199——`clear_index` 返回后后台仍在写"已清空"的 collection，加 signal 在文件边界自检才修好。

7. 流式批处理 (Streaming Batches)

读 → 处理 → 写
batch=100 滚动
内存常驻一个 batch, 而非整个数据集

RAG 索引、文档处理、批量评测——一律流式。配合协作式取消，可中断的进度也是用户体验。

8. 快照自愈 vs. 信任快照



永远假设你的持久化状态是脏的。

case: claude-context issue #295——旧版本的 {indexedFiles:0, totalChunks:0} 被客户端误判为未索引→触发 force reindex→无限循环。启动时用 Milvus 真实行数对账才修复。

[claude-mem](#) 的 outbox 重试机制也是同一思路：AI 生成非确定，状态可能脏，必须能恢复。

9. 混合检索而非单一检索

详见 hybrid-search-rrf。

信号	用途
向量相似度	语义匹配
关键词 / BM25	精确符号
时间衰减	最近优先（记忆场景）
metadata 过滤	业务约束

RRF 合并: $score = \sum 1/(k + rank_i)$ 。多路召回是企业级 RAG 标配。

迁移检查表

构建任何"AI Agent 外挂"工具时逐项检查：

- ☐ 分层：引擎层是否完全不感知协议层？换协议是否只动外层？
- ☐ 接口化：外部 SDK（LLM、DB、Embedding）是否都有抽象接口？
- ☐ 降级：每个工具是否有 fallback 路径？整个 pipeline 是否"永不崩"？
- ☐ 指纹：增量更新是否用内容 hash 而非时间戳？
- ☐ 通道：协议通道（stdout/socket）是否被无关日志污染？
- ☐ 取消：长时任务能否在任意点中断？signal 是否透传到最底层？
- ☐ 流式：是否一次性加载整个数据集？能否改成 batch 滚动？
- ☐ 自愈：启动时是否做状态对账？快照损坏是否能恢复？
- ☐ 多路召回：检索/记忆是否只用单一信号？能否多路并行 + RRF 合并？

与其他设计概念的联动

原则	相关概念页
#4 Merkle 指纹	merkle-dag-fingerprint
#9 混合检索	hybrid-search-rrf
#1-3, #7-8	event-driven-memory-pipeline 的边缘/后台分层、outbox
AI 调用边界	ai-as-compressor

参考

- [src-claude-context-architecture](#)（来源 + 9 条原则的原文）
- [src-claude-mem-architecture](#)（多个原则的另一组佐证）

AI Gateway

AI Gateway 是面向 LLM/MCP/Agent 流量的入口治理层，通常处理 provider 适配、认证、凭据托管、rate limit、routing、redaction、guardrails、telemetry 和 policy。

当前项目族

项目	定位
agentgateway	LLM/MCP/A2A 三协议 Rust 数据面 + Gateway API 控制面
envoy-ai-gateway	Envoy Gateway 上的 GenAI provider translator / extproc / policy
kgateway	Gateway API / Envoy xDS / API+AI Gateway 能力分支
Higress	AI Native API Gateway，和 HiClaw/凭据托管强相关
Plano	AI-native proxy/data plane，偏 model routing / guardrails / agent orchestration
gateway-api-inference-extension	Gateway API 的 inference endpoint picking 协议与 InferencePool API
semantic-router	按 intent、difficulty、risk、capability 和 system state 选择 model/recipe 的控制层
routellm	用 strong/weak model threshold 做质量-成本路由并提供离线评测
llm-d-router	面向 InferencePool endpoint 的 EPP，按 KV、load、priority、health 做 pod/worker 选择

和 inference routing 的关系

inference-routing 是 AI Gateway 的一个子问题：请求应该去哪个模型、哪个 endpoint、哪个 pod。AI Gateway 还要处理凭据、安全、协议转换和治理。

不要把“网关”理解成一个单一组件：Envoy AI Gateway 偏 edge/provider governance，Semantic Router/RouteLLM 偏 model or capability selection，Gateway API Inference Extension 偏标准协议，llm-d Router 偏 K8s inference endpoint picking。完整对比见 [src-k8s-gateway-routing-comparison](#)。

AI 作为压缩器

把 LLM 当"信息浓缩器"而不是"问答器"——压缩 token 成本换后续检索效率。

由 [claude-mem](#) 的架构验证的核心设计哲学。

反直觉之处

大多数 AI 应用把 LLM 当作问答接口——用户提问 → LLM 直接回答。

[claude-mem](#) 反过来用：LLM 不面向用户，而是后台默默地把噪声日志压成结构化事实。

噪声大的 tool log (数千 token)

↓ LLM 压缩 (一次性成本)

结构化 Observation (百级 token)

```
{
  facts: ["修复了 ingest workflow 中的 wikilink 漏抽问题"],
  concepts: ["wiki maintenance", "link integrity"],
  filesModified: ["wiki/concepts/xxx.md"]
}
```

↓

向量库 + FTS5 双索引

↓

后续 N 次会话检索 (持续收益)

经济学逻辑

维度	不压缩	压缩后
一次性成本	0	一次 LLM 调用 (~\$0.001 用 Haiku)
每次检索 token	数千 (全文)	数百 (结构化摘要)
检索精度	低 (关键词匹配噪声)	高 (concepts 字段)
跨会话复用	难 (全文太长)	易 (短小可拼)

核心洞察：压缩是一次性付费、永久收益的操作。每多一次检索，回报就翻一倍。

为什么便宜模型够用

压缩任务的特点：

- 输入是结构化的 tool log（不需要复杂推理）
- 输出是预定义 schema（不需要创造性）
- 错误可以通过 outbox 重试（不需要一次到位）

→ Claude Haiku / Gemini Flash 完全胜任，成本可压到主模型的 1/10。

[claude-agent-sdk](#) 的 multi-provider 抽象就是为此而生。

输出 Schema 设计

不要让 LLM 自由发挥，必须给定 schema：

```
{
  title: string,           // 一句话标题（用于 search 摘要层）
  narrative: string,       // 叙述（用于详情层）
  facts: string[],        // 离散事实（可索引）
  concepts: string[],     // 抽象标签（用于预检索）
  filesRead: string[],
  filesModified: string[]
}
```

每个字段都对应某一层检索使用——schema 设计直接决定后续检索能力。

可迁移性

任何"AI Agent + 长期存储"场景都该考虑：

- 不只是对话历史，而是工具调用、API 响应、错误日志
- 不只是聊天 Agent，运维 Agent（压缩告警）、代码 Agent（压缩 PR diff）同样适用
- 与 [event-driven-memory-pipeline](#) 配合形成完整闭环

参考

- [src-claude-mem-architecture](#)

Claude Agent SDK

@anthropic-ai/claude-agent-sdk —— Anthropic 提供的 Agent 编程 SDK，核心是 `query()` 方法，可在脚本/服务内驱动 Claude 完成多步工具调用任务。

在 claude-mem 中的角色

[claude-mem](#) 用它来做后台异步压缩——把噪声大的工具调用日志压成结构化 XML 观察，体现 [ai-as-compressor](#) 设计哲学。

调用链：

```
ProviderObservationGenerator.run()
  → claude-agent-sdk.query(prompt, tools=[])
  → 返回 XML 格式响应
  → sdk/parser.ts 解析为 MemoryItem
```

Multi-Provider 抽象

claude-mem 把 Agent SDK 调用抽象到 Provider 层，支持：

- ClaudeProvider — 官方 Anthropic
- GeminiProvider — Google Gemini
- OpenRouterProvider — 多模型路由

抽象的好处：压缩任务可以用便宜模型（Haiku / Gemini Flash），把成本压到最低。

关键文件

- `src/sdk/{Claude,Gemini,OpenRouter}Provider.ts` — 三个 provider 实现
- `src/sdk/parser.ts` — XML 响应解析
- `src/sdk/prompts.ts` — 压缩 prompt 模板（系统效果的核心）

参考

- [src-claude-mem-architecture](#)

Claude Code 插件机制

[claude-code](#) 的官方插件机制通过 `--plugin-dir`、`marketplace install` 或本地插件目录加载。一个插件通常由 `.claude-plugin/plugin.json`、`hooks/`、`skills/`、`.mcp.json`、`scripts/config` 等根级目录组成。`Hook + Skill + MCP server` 是最常见的三条扩展通道。

本页以 `powermem` 的 Claude Code 插件为参考实现；2026-06-12 重新核验 `PowerMem` 仓库时，最新 tag 为 `v1.1.3`，`apps/claude-code-plugin/` 仍包含 `.claude-plugin/`、`hooks/`、`skills/`、`config/`、`.mcp.json`。

插件结构

路径	作用
<code>.claude-plugin/plugin.json</code>	插件 manifest: name、description、version、author、repository 等
<code>hooks/</code>	Claude Code lifecycle hook 的配置、wrapper 和二进制/脚本
<code>skills/</code>	Slash command / skill 入口，每个目录提供一个能力
<code>.mcp.json</code>	MCP server 配置，允许插件暴露或连接 MCP 工具
<code>scripts/ / config/</code>	初始化、启动、停止、配置和诊断辅助

`PowerMem` 的 manifest 当前声明 `memory-powermem`，`description` 是给 Claude Code 增加 `add/search/update/delete memory` 能力，并指向 `OceanBase / PowerMem` 仓库。

Hook 事件模型

Claude Code hook 不是只覆盖 4 个事件。当前常见事件族包括：

事件	插件用途
<code>SessionStart</code>	初始化上下文、检查服务状态、准备本地环境
<code>UserPromptSubmit</code>	在用户 prompt 进入模型前检索记忆并注入 additional context
<code>PreToolUse</code>	工具调用前做授权、审计或上下文记录
<code>PostToolUse</code>	工具调用后采集结果、记录文件/命令行为
<code>Stop</code>	turn 结束后异步写记忆、flush 状态
<code>PreCompact / PostCompact</code>	上下文压缩前后保存/恢复重要信息

PowerMem 的关键路径是 UserPromptSubmit hook：自动搜索相关记忆，并把结果注入到 Claude Code 的上下文。它用 Go 原生二进制 powermem-hook 加 shell/PowerShell wrapper，避免要求用户的 Claude Code 进程直接依赖 Python 环境。

Skill 格式

Skill 是 Markdown + scripts 的能力包。对 Claude Code 来说，skill 既是用户可见的 slash command，也是给模型看的工作流说明。PowerMem 当前提供：

- init：创建插件本地 venv，安装 powermem 后端和默认本地 embedding 依赖；
- remember：主动写入记忆；
- recall：主动检索记忆；
- status：检查 server / hook / config 状态；
- stop：停止本地服务；
- reset：清理本地状态。

这和 Codex skills 的哲学一致：SKILL.md/Markdown 是能力说明，脚本负责可执行动作，宿主 Agent 决定何时调用。

.mcp.json 与双模式

- Go 原生二进制 powermem-hook（跨平台编译，无 Python 依赖）+ bash/PowerShell wrapper
- UserPromptSubmit hook 自动检索注入到对话上下文（POWERMEM_PROMPT_SEARCH=0 关闭）
- 默认 HTTP 模式 → POST /api/v1/memories；可选 MCP 模式 → 通过 [mcp](#) 工具
- marketplace 安装后需要 /reload-plugins，再运行 /memory-powermem:init

HTTP 模式适合本地 server 已经运行、hook 只需快速 REST 调用的场景；MCP 模式适合把 memory 能力作为工具暴露给 Agent，让 Agent 主动调用 add/search/update/delete。

与 [claude-mem](#) 的对照

维度	PowerMem Claude Code 插件	claude-mem
定位	PowerMem 后端的 Claude Code connector	独立 Claude Code memory 插件
后端	PowerMem server / SDK / OceanBase/SeekDB 等存储	自带 memory pipeline / worker / vector store
hook 重点	UserPromptSubmit 检索注入 + skill 主动管理	跨会话采集、压缩、检索、注入
安装形态	Claude Code plugin / marketplace / --plugin-dir	npm/本地插件式安装

维度	PowerMem Claude Code 插件	claude-mem
适合场景	已决定用 PowerMem 作为统一 memory backend	只想给 Claude Code 快速加长期记忆

选型判断

- 需要统一 memory backend、MCP/API/CLI/Dashboard 多入口：看 powermem 插件。
- 只关心 Claude Code 的跨会话记忆闭环：看 [claude-mem](#)。
- 需要把插件能力迁移到其他 Agent 宿主：关注 Skill + MCP + hook 三层是否解耦，避免把逻辑写死在某个生命周期事件里。

相关页面

- 平台：[claude-code](#)
- 参考实现：powermem、src-powermem-architecture
- 同类项目：[claude-mem](#)（独立 npm 包，与 powermem 插件功能重叠但定位不同—— claude-mem 是独立 agent 记忆，powermem 插件只是接入层）
- 工具协议：[mcp](#)

Claude Context

[zilliztech/claude-context](#), v0.1.13。Zilliz 出品的 [MCP](#) 插件，把整个代码库通过 语义检索"塞进" AI Agent 上下文。

解决什么

让 [claude-code](#) / Cursor / Gemini CLI / Codex 等 Agent 在大型 repo 中一次查询直达目标，避免多轮 grep/read 探索。

三段式架构

层	包	职责
引擎	packages/core	Splitter / Embedding / VectorDB / Sync —— 不感知协议
协议	packages/mcp	4 个 MCP 工具 + handlers + SyncManager + Snapshot
客户端	packages/vscode-extension、packages/chrome-extension	独立 UI

详见 [ai-agent-plugin-patterns](#) 的"三段式接口分层"原则。

4 个 MCP 工具

- index_codebase — 全量索引
- search_code — 自然语言查询
- clear_index — 清除（协作式取消）
- get_indexing_status — 进度查询

关键技术选型

- Splitter: AST (tree-sitter × 9 语言) → LangChain 字符切分兜底
- Embedding: OpenAI / Voyage / Gemini / Ollama 四选一
- VectorDB: Milvus / Milvus-RESTful
- 检索: Dense + Sparse 双向量 + RRF 重排（详见 hybrid-search-rrf）
- 增量同步: Merkle DAG, 5 分钟后台轮询

关键常量

参数	值	说明
EMBEDDING_BATCH_SIZE	100	流式批处理
CHUNK_LIMIT	450000	Milvus 单 collection 上限保护
chunkSize / chunkOverlap	2500 / 300	AST splitter 默认

与 claude-mem 的对照

[claude-mem](#) 解决跨会话记忆，[claude-context](#) 解决代码库理解——但两者共享大量设计原则：接口分层、协议通道纪律、流式批处理、状态自愈。汇总在 [ai-agent-plugin-patterns](#)。

演进方向

接口层稳定，实现层在扩展：

- 多 provider（Gemini Embedding 2、Solidity 语言支持）
- 后台同步可配置化（`CLAUDE_CONTEXT_BACKGROUND_SYNC` env）
- 请求级 splitter 切换

参考

- [src-claude-context-architecture](#)
- DeepWiki: <https://deepwiki.com/zilliztech/claude-context>

Claude Context 架构深度分析

原文: raw/claude-context-architecture-analysis.md · 仓库: [zilliztech/claude-context](#) · 分析版本 v0.1.13

一句话定位

[claude-context](#) 是 Zilliz 出品的 [MCP](#) 插件——把整个代码库通过 语义检索 "塞进" AI Agent 的上下文，让 [claude-code](#) / Cursor / Gemini CLI / Codex 等 Agent 不必多轮"瞎找" 就能直达相关代码。

解决的核心痛点

传统做法	问题	Claude Context 方案
把整个目录塞进 prompt	Token 爆炸	向量库存储，只取 top-K
Agent 多轮 grep/read	慢、消耗工具调用次数	一次语义查询直达
仅靠关键词匹配	找不到概念相关代码	dense + sparse 混合检索 + RRF 重排

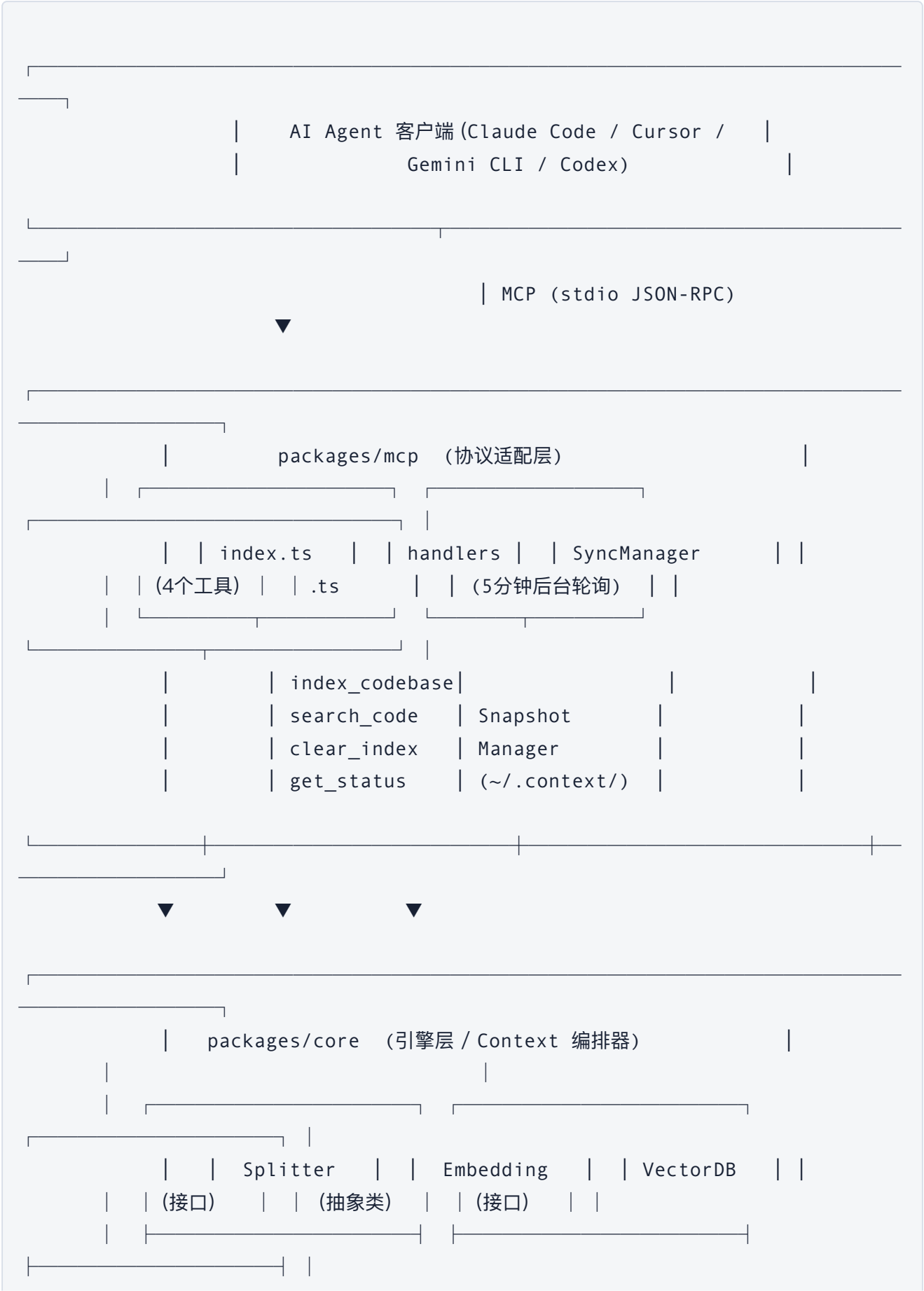
一句话：代码分块 → 向量化 → 存入 Milvus → 混合检索 → 通过 MCP 暴露给 AI Agent。

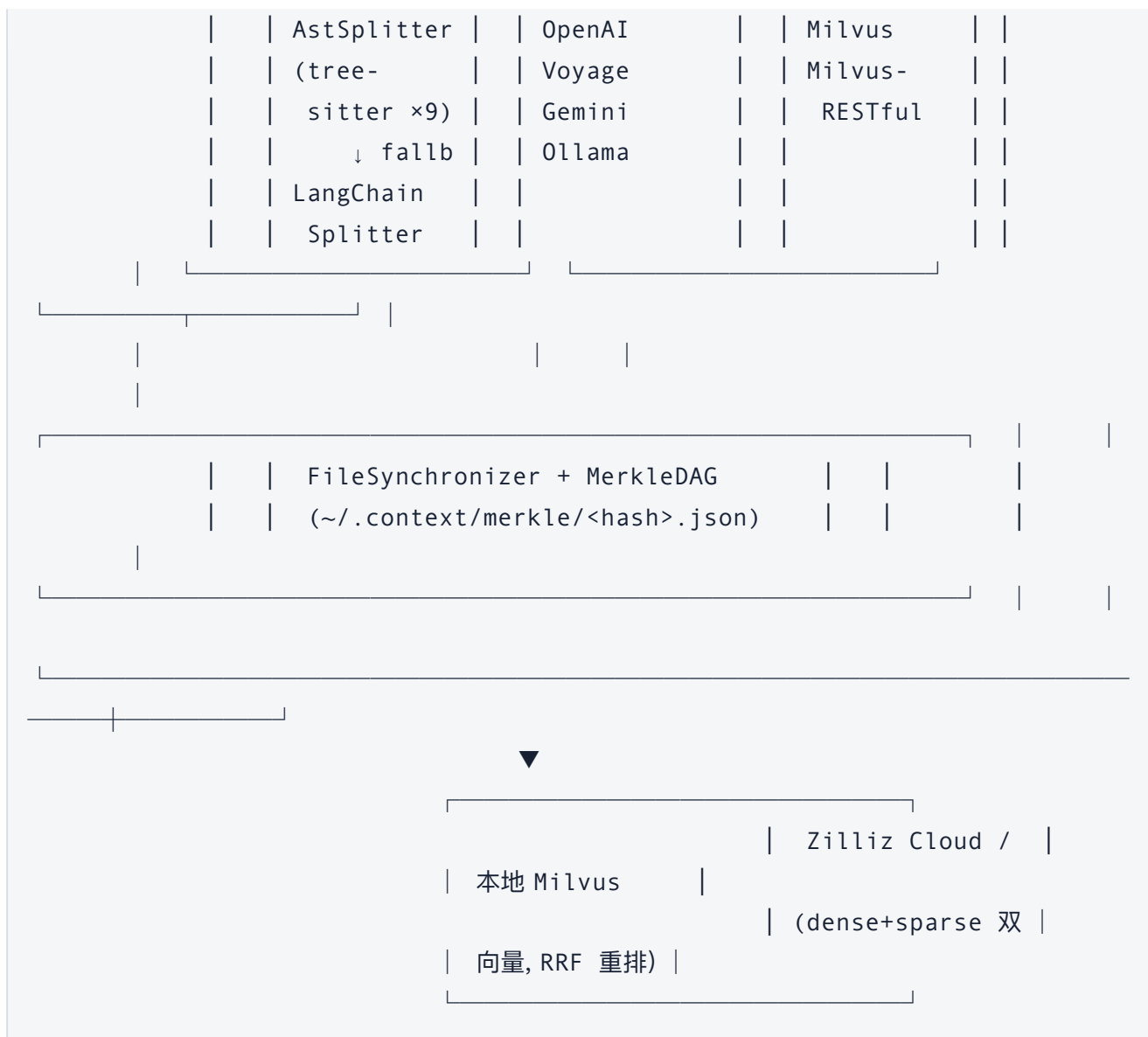
Monorepo 架构

```
claude-context/  
├─ packages/  
│   ├─ core/                # 引擎层（纯领域逻辑）  
│   │   ├─ splitter/        # AST / LangChain  
│   │   ├─ embedding/       # OpenAI/Voyage/Gemini/Ollama  
│   │   ├─ vectordb/        # Milvus / Milvus-REST  
│   │   ├─ sync/            # Merkle DAG 增量  
│   │   └─ context.ts       # 编排器  
│   ├─ mcp/                 # MCP 协议适配  
│   ├─ vscode-extension/  
│   └─ chrome-extension/
```

三段式分层：引擎层（core）不感知协议 / 协议层（mcp）只做转换 / 客户端独立 UI。

完整系统架构图





三大核心流程

索引 (indexCodebase)

1. loadIgnorePatterns() .gitignore / .contextignore
2. prepareCollection() 建 Milvus collection (dense + sparse)
3. getCodeFiles() 按扩展名/ignore 规则递归
4. processFileList() 逐文件流式:
 - └─ splitter.split() AST → 失败 fallback LangChain
 - └─ chunkBuffer 累积到 100
 - └─ embedding.embed() + vectorDB.insert()
 - └─ signal?.aborted → IndexAbortError
5. FileSynchronizer.initialize() Merkle DAG 快照

关键常量：EMBEDDING_BATCH_SIZE=100、CHUNK_LIMIT=450000、chunkSize=2500/
overlap=300

检索 (semanticSearch)

```
query
├─ embedding.embed()    → dense
└─ 原 query              → sparse
    ↓
vectorDB.hybridSearch(
  [dense:{nprobe:10}, sparse:{drop_ratio:0.2}],
  rerank:{strategy:'rrf', k:100}
)
  ↓
deduplicateResults() → {content, path, lines, score}
```

详见 hybrid-search-rrf。

增量同步 (reindexByChange)

旧 Merkle root ≠ 新 root → checkForChanges() → {added, removed, modified} → 删向量 / 重新
嵌入 → 新快照写 ~/.context/merkle/<md5>.json。

详见 merkle-dag-fingerprint。

MCP 工具集 (4 个)

工具	作用
index_codebase	全量索引
search_code	自然语言查询
clear_index	清除（带协作式取消）
get_indexing_status	进度查询

关键设计决策

设计点	理由
Splitter / Embedding / VectorDB 三层接口	多组合，避免锁死 OpenAI

设计点	理由
AST 优先, LangChain 兜底	函数/类边界比固定长度更语义化
Merkle DAG 文件指纹	千万行 repo 增量同步
Hybrid Search + RRF	语义 + 精确符号双覆盖
stdout 强制重定向 stderr	MCP 用 stdout 传 JSON-RPC
AbortSignal 协作式取消	issue #199 后台仍在写"已清空"
Snapshot 自愈	issue #295 客户端误判触发无限重建
CHUNK_LIMIT = 450000	Milvus 单 collection 上限保护

完整原则汇总: [ai-agent-plugin-patterns](#)。

与 claude-mem 的对照

	claude-mem	claude-context
目标	跨会话记忆	代码库理解
数据	工具调用日志 → 压缩观察	源代码 → AST 分块
检索	three-tier-search-protocol	hybrid-search-rrf
增量	内容哈希去重	merkle-dag-fingerprint
集成	Claude Code Lifecycle Hook	MCP 工具

两者都是 [AI Agent 外挂](#) 模式的优秀样本。

Git 演进信号

近 30 天高频改动: `core/src/context.ts` (12)、`mcp/src/handlers.ts` (10)、`config.ts` (7)、`sync.ts` (6)。

最近主线:

1. 后台同步可配置化 (#314) — `CLAUDE_CONTEXT_BACKGROUND_SYNC` env
2. 取消语义健壮性 (#199 / #369)
3. 多 embedding provider (#366 Gemini Embedding 2)
4. 多语言支持 (#367 Solidity)
5. 请求级 splitter (#363)

特征: 接口层稳定, 实现层在扩展。典型成熟期开源项目。

配置示例 (Claude Code)

```
claude mcp add claude-context \  
-e OPENAI_API_KEY=sk-xxx \  
-e MILVUS_ADDRESS=your-zilliz-endpoint \  
-e MILVUS_TOKEN=your-zilliz-key \  
-- npx @zilliz/claude-context-mcp@latest
```

claude-mem

给 [claude-code](#) 装上长期记忆的开源插件（v13.1.0, 仓库 [thedotmack/claude-mem](#)）。

解决的问题

宿主 Claude Code 本身没有跨会话记忆——每次开新会话都是白板。claude-mem 通过宿主提供的 6 个生命周期 hook 无侵入采集工具调用，异步压缩成结构化"观察"存进本地双索引，下次开会话时自动检索并注入相关历史。

核心特征

- 跨会话连续性：用户问"上次怎么解决这个 bug"时能真的检索到上次的步骤
- 本地优先：所有数据存 `~/.claude-mem/`，开源核心可审计
- 无侵入采集：通过宿主 hook 接口工作，不修改 Claude Code
- 多 profile 支持：两个环境变量切换工作账号 / 私人账号

架构骨架

- 边缘：`bun-runner.js` + 6 个 Lifecycle Hook handler
- 后台：Worker Service (Express daemon, 端口 `37700 + uid%100`) + BullMQ
- 存储：SQLite (FTS5) + Chroma 向量库

详细工作流参见 [event-driven-memory-pipeline](#)。

设计哲学

claude-mem 是 [ai-as-compressor](#) 设计哲学的典型实现——LLM 不是用来"回答用户"，而是用来把噪声大的工具调用日志压缩成结构化字段。压缩用的 token 成本，换来后续每次会话的检索效率。

关键依赖

- [claude-agent-sdk](#) — 异步压缩/总结
- BullMQ — 后台任务队列（待建实体页）
- Chroma — 向量库（待建实体页）
- better-sqlite3 + FTS5 — 主存储 + 全文检索
- [MCP](#) — Skill / 远程模式

部署模式

模式	存储	适用
单机	SQLite + Chroma	个人开发者
server-beta	Postgres + MCP + Docker	团队共享 + 审计

通过 `runtime-selector.ts` 在两种模式间切换，保留单机回退。

演进方向

从「单机插件」演化到「团队可共享的后台服务」（PR #2383）。

参考

- 完整分析：[src-claude-mem-architecture](#)

claude-mem 架构与设计思路分析

原文: raw/claude-mem-architecture-analysis.md · 仓库: [thedotmack/claude-mem](#) · 分析版本 v13.1.0

一句话定位

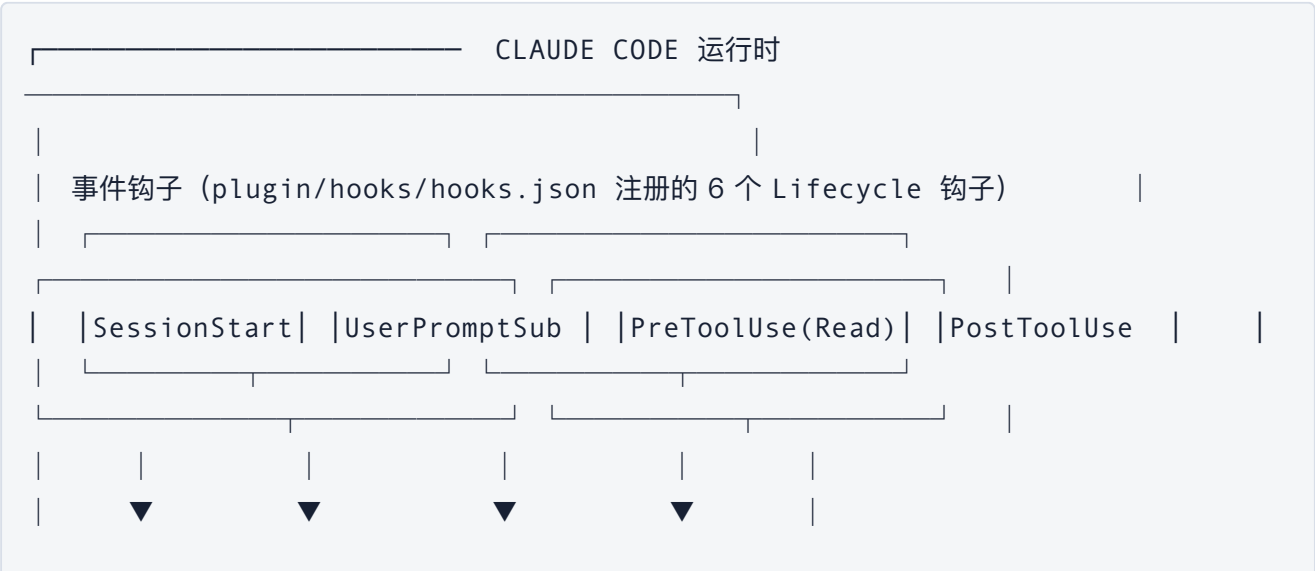
[claude-mem](#) 是给 [claude-code](#) 装上"长期记忆"的开源插件——通过宿主的 6 个 Lifecycle Hook 无侵入采集工具调用，用 [claude-agent-sdk](#) 异步压缩成结构化"观察 (observation)"，存进本地 SQLite + Chroma 双索引；下次开会话时自动检索相关历史并注入上下文。

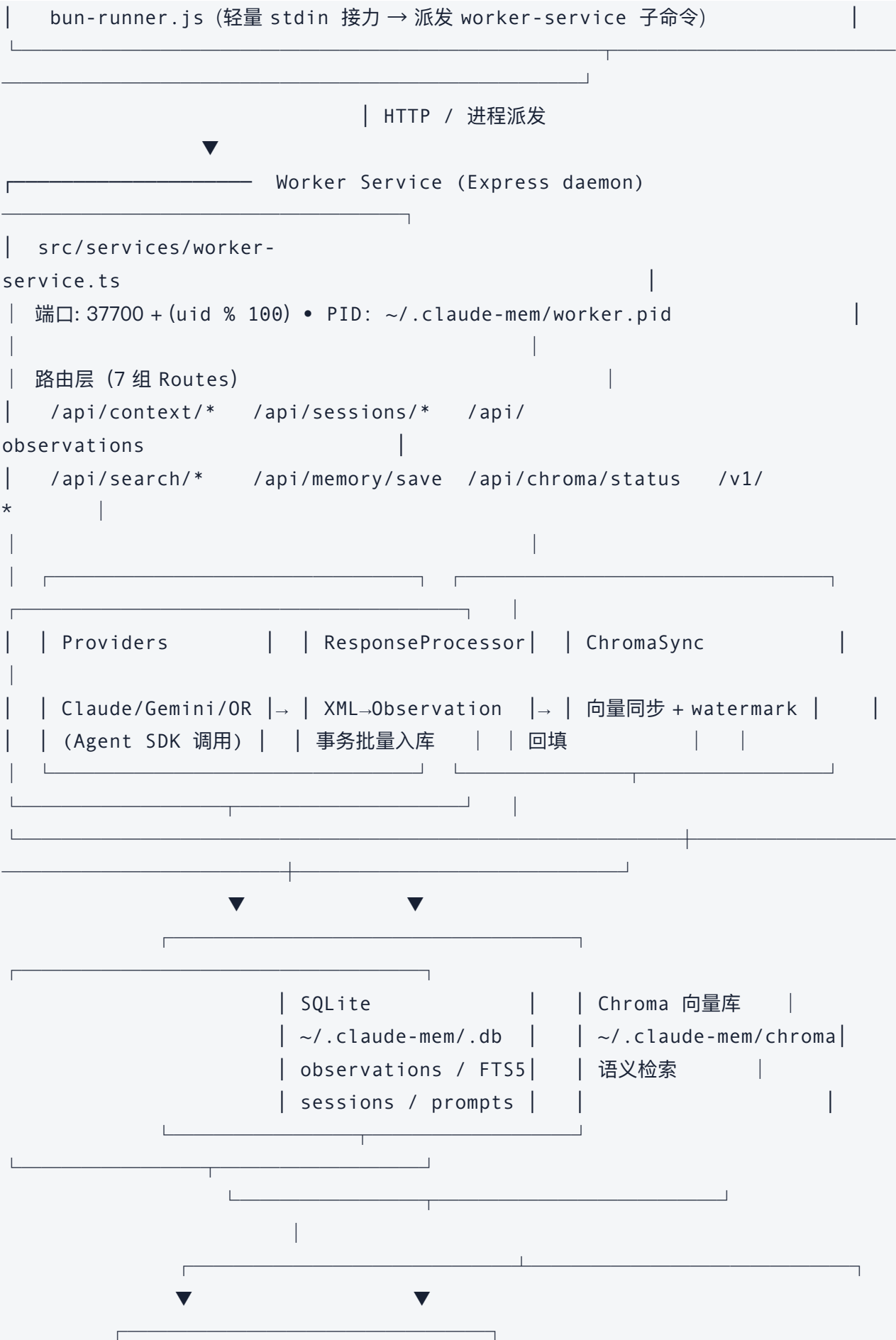
核心架构（三层）

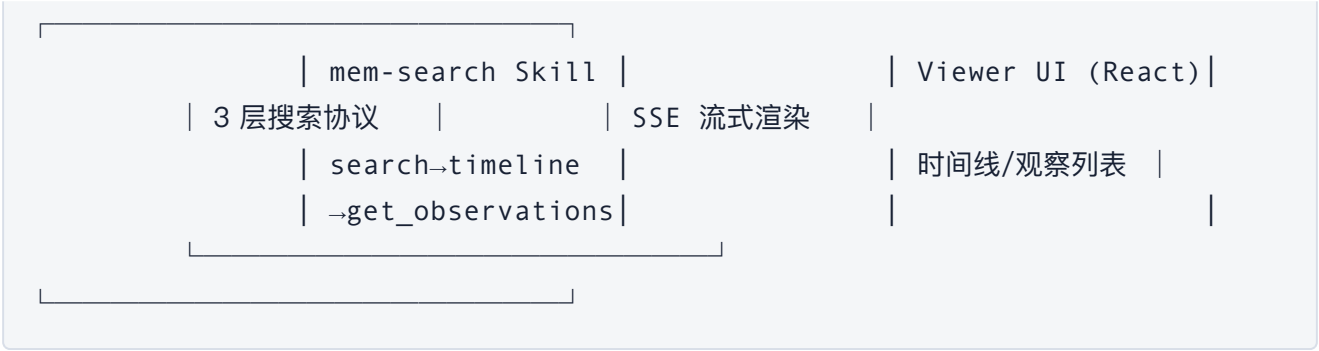
层	组件	职责
边缘层	bun-runner.js + 6 个 hook handler	轻量 stdin 接力，禁止做 AI 推理
后台层	Worker Service (Express daemon, 端口 37700 + uid%100) + BullMQ	异步压缩、入库、向量同步
存储层	SQLite (FTS5 全文) + Chroma (向量)	双索引

详见 [event-driven-memory-pipeline](#)。

完整架构图







六个生命周期钩子

阶段	触发	Handler	作用
Setup	插件初始化	version-check.js	版本兼容性
SessionStart	启动/clear/compact	context.ts	启 worker + 注入历史上下文
UserPromptSubmit	用户提交 prompt	session-init.ts	创建 session + 触发语义搜索
PreToolUse(Read)	读文件前	file-context.ts	查当前文件相关观察
PostToolUse	任何工具调用后	observation.ts	入队等待压缩
Stop	会话结束	summarize.ts	生成会话摘要

五步闭环

1. 捕获：Hook → bun-runner.js → worker handler → POST 给本地 worker
2. 入队：写 outbox (Postgres) 或 SQLite 暂存，BullMQ 调度
3. AI 压缩：ProviderObservationGenerator 调 Agent SDK，把 tool log 压成 XML 观察
4. 解析存储：sdk/parser.ts → MemoryItem schema → 事务写入 SQLite + 异步推 Chroma
5. 注入回流：SessionStart 预检索 → system 消息；或 mem-search Skill 按需查询

六大可移植设计模式

1. 生命周期钩子采集 — 任何有 before/after tool_use 钩子的 runtime 都能挂
2. 边缘轻量 + 后台 AI 重活 — 用户交互路径上绝不 AI 推理
3. 三层搜索协议 — search → timeline → get_observations, 详见 [three-tier-search-protocol](#)
4. AI 作为压缩器 — 详见 [ai-as-compressor](#)
5. session_id 双轨制 — contentSessionId (会话) vs memorySessionId (记忆线)
6. Outbox + 内容哈希去重 — AI 生成非确定，必须去重；outbox 让重试不丢消息

MemoryItem Schema

```
{
  kind: 'observation' | 'summary' | 'prompt' | 'manual',
  type: string,
  title: string,
  narrative: string,      // 自然语言叙述
  facts: string[],       // 离散事实点
  concepts: string[],    // 抽象概念标签
  filesRead: string[],
  filesModified: string[]
}
```

[note] 关键洞察
别只存"对话历史"，先压缩成"事实/概念/读写文件"等可索引字段；语义搜索叠在结构化层之上。

多 Profile 隔离

```
export CLAUDE_MEM_DATA_DIR="$HOME/.claude-mem-work"
export CLAUDE_MEM_WORKER_PORT=37800
```

所有路径（DB、Chroma、日志、PID）从 env 派生，shell 级隔离。

演进信号

近 30 天最活跃：worker-service.cjs（88 次）、mcp-server.cjs（75 次）。PR #2383 把 worker 重
写为 server-beta：事件管道 + Postgres + MCP + Docker + 团队审计。趋势是从「单机插件」→「团队后台服
务」，但 runtime-selector.ts 保留单机 SQLite 回退。

关键文件索引

关注点	路径
钩子注册	plugin/hooks/hooks.json
Hook 派发	plugin/scripts/bun-runner.js
Worker 入口	src/services/worker-service.ts

关注点	路径
Provider 抽象	src/sdk/{Claude,Gemini,OpenRouter}Provider.ts
XML 解析	src/sdk/parser.ts
上下文构建	src/services/context/ContextBuilder.ts
压缩 Prompt	src/sdk/prompts.ts
搜索 Skill	plugin/skills/mem-search/SKILL.md

最小可行落地（抄作业 5 步）

给任何 Agent 加记忆的最小套路：

1. 采集：tool callback 里 `record_event(event)` 塞 SQLite outbox
2. 压缩：cron/worker 跑 `compress_batch()`，喂给便宜模型（Haiku / Gemini Flash）输出 `{title, narrative, facts[], concepts[]} JSON`
3. 双索引：SQLite FTS5 + 向量库
4. 注册搜索工具：`search_memory(query)`，描述写"用户提到过去/上次时调用"
5. 会话启动预检索：基于"项目路径 + 最近活跃概念"拼 system prompt 前缀

Code Graph

Code graph 把仓库从文件/文本块提升为符号、调用、依赖、变更、风险和证据的图结构，服务于 review、RAG、影响面分析和 agent 工具调用。

和语义搜索的区别

code-semantic-search 关注“给定 query 找相似代码块”；code graph 关注“代码元素之间如何连接”。两者互补：语义搜索负责召回，图负责解释路径、边界和影响。

代表项目

- [code-review-graph](#): local-first code intelligence graph for MCP/CLI/VSCode。
- GitNexus: browser-side repo knowledge graph + Graph RAG + taint analysis。

选型提示

需要 line-level evidence、调用关系、变更影响面时，code graph 比纯 vector search 更有解释力；需要快速语义召回时，vector/hybrid search 更轻。

code-review-graph

local-first code intelligence graph，面向 MCP/CLI/VSCode/skills 提供代码结构、变更理解和 review 相关上下文。详见 [src-code-review-graph-architecture](#)。

架构边界

它代表 Code RAG 从“语义搜索”进入“代码图谱”的路线。与 [claude-context](#) / [memsearch](#) 不同，code-review-graph 更强调图结构、调用关系、review evidence 和本地工具集成。

选型判断

适合需要结构化代码理解、review evidence、MCP 工具暴露的本地工作流。若目标是自动生成 repo wiki，应看 deepwiki-open 和 repo-wiki-generation。

code-review-graph 架构与设计思路分析

tirth8205/code-review-graph 是本地优先 code intelligence graph，目标是给 MCP/CLI/coding agent 提供结构化代码上下文，尤其服务 code review、delta review、debug/refactor。仓库有 Python core、tools、VSCode extension、skills、docs/tests。

核心架构图



模块分层

层/目录	责任
code_review_graph/**	Python 核心图构建/查询。
code_review_graph/tools/**	CLI/MCP 工具。
skills/**	build graph、review changes、review PR、debug issue 等技能。
code-review-graph-vscode/**	VSCode extension。

关键数据流

- 本地仓库被解析成代码图。
- agent/CLI/MCP 根据任务查询相关节点、边、diff 或 review context。
- 技能把常见 review/refactor 流程固化为 agent instructions。

设计决策

- local-first 保护代码隐私，也减少服务端依赖。
- 从 vector search 升级到 code graph，适合补 code RAG 地图。
- skills 与 MCP tools 并存，说明它面向 coding agent workflow。

对比定位

和 Claude Context/memsearch 相比，它更图谱化；和 deepwiki-open 相比，它服务交互式 review，不是生成静态 wiki。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

Codex Plugin for Claude Code

Claude Code 插件形态的 Codex 接入层，把 `/codex:*` slash command、Codex app-server broker、后台 job 状态和 session hook 组合起来。详见 [src-codex-plugin-cc-architecture](#)。

架构边界

它不是新的 coding agent，而是 [claude-code](#) 与 [codex](#) 之间的 delegation bridge。核心价值在于跨 agent 审查、rescue task、review gate 和 job 状态管理。

选型判断

适合：已经用 Claude Code，但希望引入 Codex 做独立 review 或救援任务。

不适合：想替代 Codex CLI 本体、做通用插件框架、或远程消息平台入口；这些分别看 [codex](#)、[agent-skills-plugin-system-map](#)、cc-connect。

相关概念

- [agent-delegation](#)
- [ai-agent-plugin-patterns](#)
- [agent-skills-plugin-system-map](#)

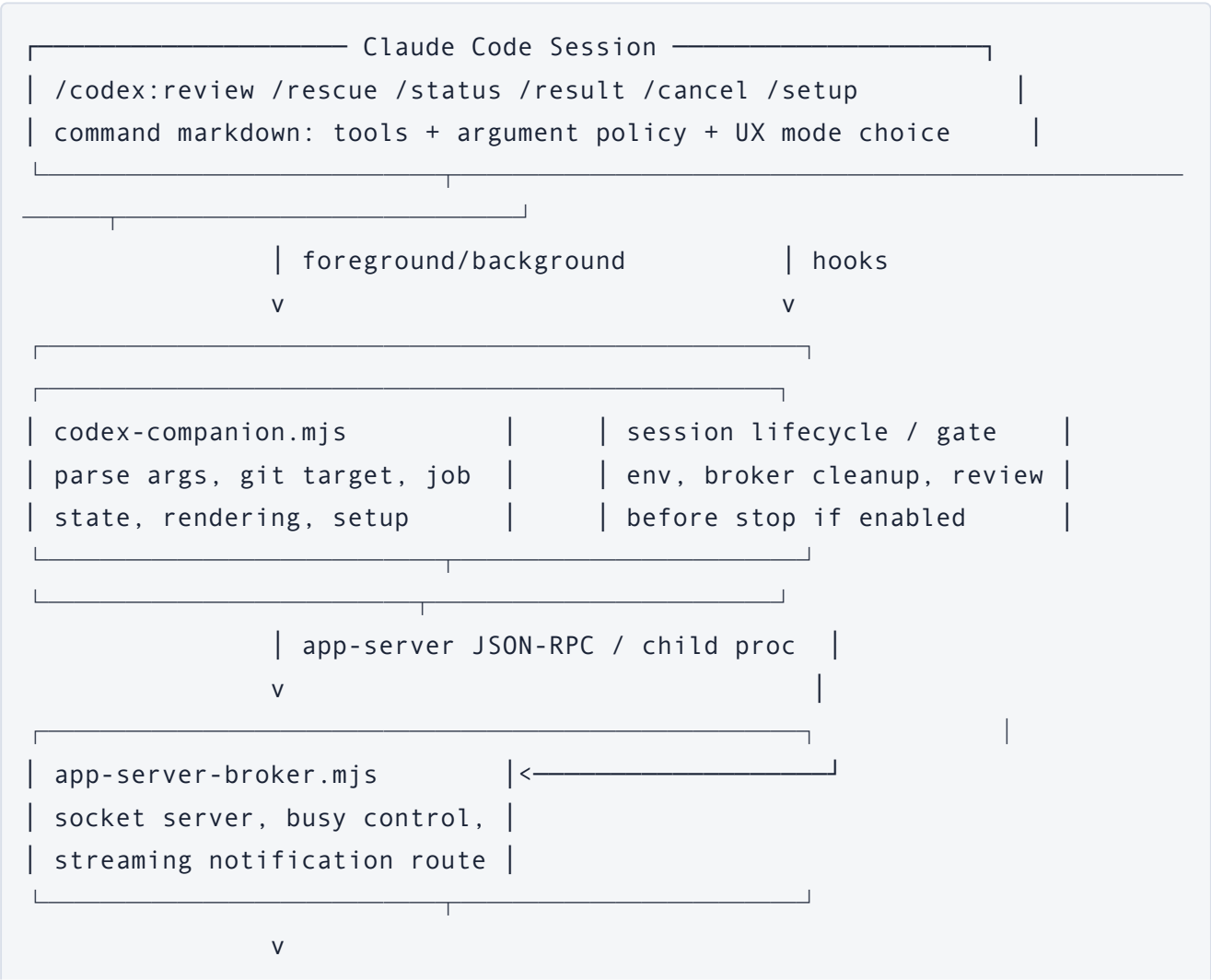
Codex Plugin for Claude Code 架构与设计思路分析

原文: raw/codex-plugin-cc-architecture-analysis.md · 仓库: <https://github.com/openai/codex-plugin-cc> · 分析版本 HEAD 807e03a (2026-04-18)

一句话定位

Claude Code 插件形态的 Codex 接入层，把 `/codex:* slash command`、Codex app-server broker、后台 job 状态和 session hook 组合起来，让 Claude Code 可以调用 Codex 做 review、adversarial review 和 rescue task。它不是新的 agent runtime，而是一个跨 agent 委托/审查胶水层。这页和 [claude-code codex ai-agent-plugin-patterns agent-skills-plugin-system-map](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图



模块分层

层 / 模块	职责
Claude 插件入口	把 Claude Code slash command 映射为 review/rescue/status/result/cancel/setup 等操作，并约束 allowed-tools 与交互策略。
Companion CLI	统一解析参数、检查 Codex 安装/登录、收集 git review context、创建/更新 job state、渲染结果。
App-server broker	复用 Codex app-server 连接，通过 JSON-RPC over socket 转发 turn/review 请求，并串行化 streaming ownership。
生命周期与审查 gate	在 Claude session start/end 写入环境、清理 broker/job，并可在 stop 前强制 Codex review gate。
测试与发布	Node test runner 覆盖命令渲染、broker endpoint、git target、state/runtime 行为。

关键数据流

```

User runs /codex:review --background
|
├─ command markdown estimates git diff size and chooses background UX
|
├─ codex-companion.mjs review parses --base/--scope and git context
|
├─ app-server broker starts or reuses Codex app-server connection
|
├─ Codex review stream writes tracked job log/state
|
└─ /codex:status and /codex:result read the persisted job snapshot

```

设计决策与哲学

- 插件是“薄入口”，状态在 companion 层集中：命令 markdown 只负责 Claude Code 侧工具权限和交互策略，真正的参数、job、输出渲染集中在 codex-companion.mjs，避免多个 slash command 各自实现状态机。

- broker 复用 app-server，但主动串行化 streaming 请求：app-server-broker.mjs 用 active request/stream socket 防止多个 Claude command 同时占用 Codex app-server，保留 interrupt 的例外路径。
- 把 review gate 做成可选 hook：stop-review-gate-hook.mjs 只有配置打开时才阻断 session stop，默认不改变 Claude Code 的结束语义。

与同类项目的架构差异

维度	Codex Plugin for Claude Code	同类 A	同类 B
扩展形态	Claude Code plugin + Codex CLI	Claude Code 自身 slash command/agent	OpenAI Codex CLI 原生命令
核心价值	跨 agent 委托与审查	单 agent 内部技能/子代理	独立 coding agent workflow
状态边界	插件 state + Codex app-server/job log	Claude session state	Codex session/rollout/app-server state

选型提示

如果你在快速理解或选型，先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”，还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题：比如 trace viewer 不能替代 remote control bridge，GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

相关页面

- [claude-code](#)
- [codex](#)
- [ai-agent-plugin-patterns](#)
- [agent-skills-plugin-system-map](#)
- [coding-agent-selection-map](#)

Ebbinghaus 遗忘曲线

德国心理学家 Hermann Ebbinghaus（1885）通过自身实验得出的记忆衰减经验曲线。在 LLM agent 记忆框架里被改造为 三层记忆 + 数学衰减 + 强化反馈 的算法基座，powermem 是当前最完整的工程实现。

数学模型

R = e^(-t/S)

- $R \in [0, 1]$ — 当前 retention 强度（记忆"残留度"）
- t — 距上次访问的时间（小时）
- S — strength 参数（越大衰减越慢；强化访问会增大 S ）

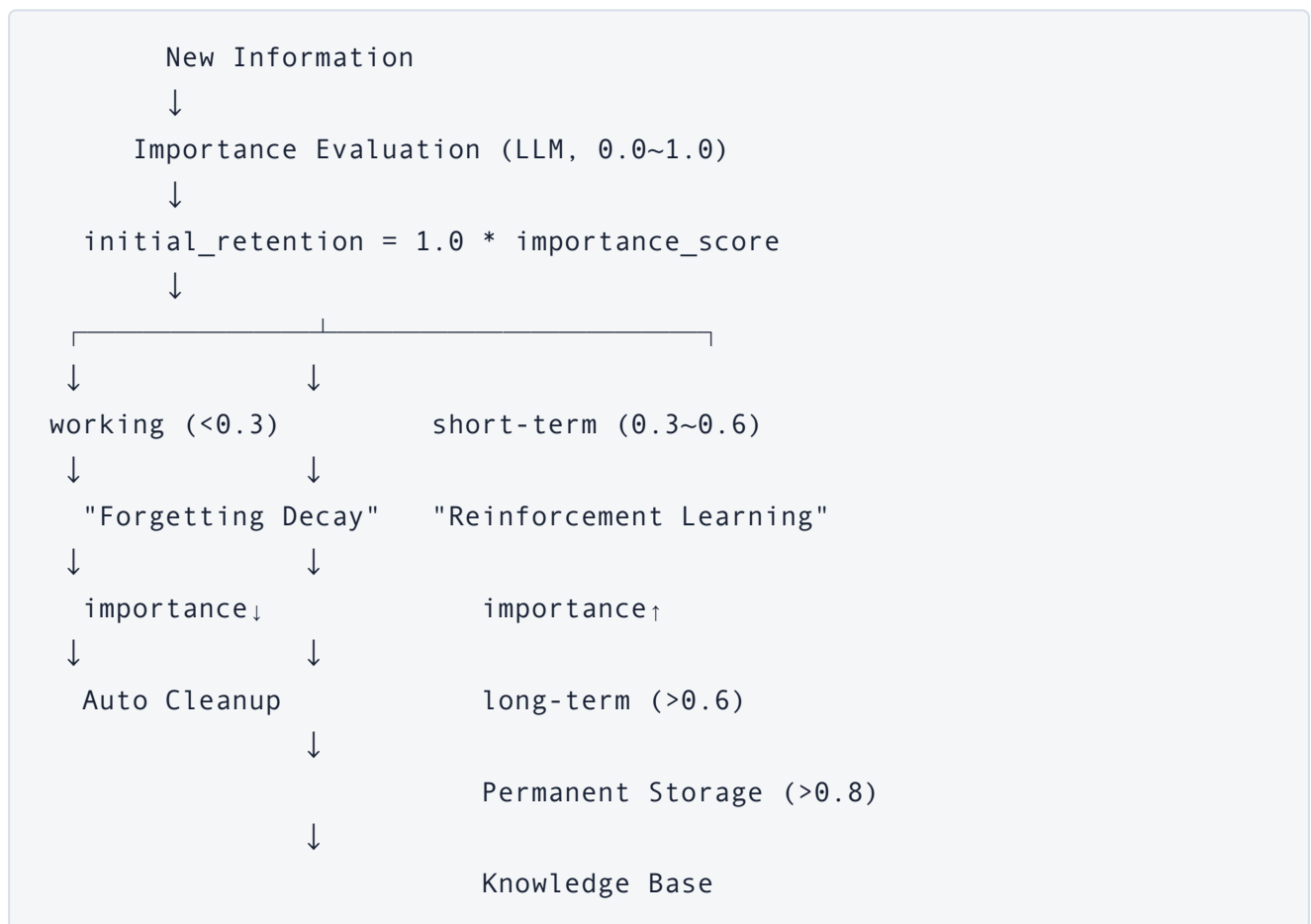
衰减是 指数式 的：刚记完忘得快，多次复习后衰减率降低 —— 这正是间隔重复（spaced repetition）的理论根基。

工程映射：三层记忆 + 三阈值

PowerMem 在 intelligence/ebbinghaus_algorithm.py 里把曲线参数化（默认值）：

参数	默认值	含义
initial_retention	1.0	新记忆的初始强度（按 LLM 评出的 importance_score 缩放）
decay_rate	0.1	基础衰减率
reinforcement_factor	0.3	命中检索时强度增量
working_threshold	0.3	< 该值 → 工作记忆
short_term_threshold	0.6	0.3 – 0.6 → 短期记忆
long_term_threshold	0.8	> 0.6 → 长期记忆；> 0.8 → 永久存储
review_intervals	[1, 6, 24, 72, 168] 小时	复习时间表（1h / 6h / 1d / 3d / 1w）

完整生命周期 (PowerMem 实现)



- 被检索命中 → $\text{retention} += \text{reinforcement_factor}$ → 可能晋升上一层;
- 超过 review_interval 仍未命中 → 按 $R = e^{(-t/S)}$ 衰减;
- 跌破 working_threshold → MemoryOptimizer 离线扫描后清理。

为什么这套机制重要

- 解耦存储与智能: retention / next_review / importance_score 都只是 metadata JSON 列; 不需要每次写入都跑昂贵 LLM。MemoryOptimizer 定时离线扫描即可完成晋升/清理 —— 对应 [ai-as-compressor](#) 的"边缘轻、后台重"哲学。
- 天然契合 token 经济性: 长期记忆只占总量的小比例, 搜索时优先返回 → token 用量降到原 baseline 的 ~3% (PowerMem LOCOMO: 0.9k vs 26k tokens)。
- 跨项目可复用: MemoryOS、Letta (前 MemGPT)、claude-mem 等都用了类似抽象, 但 PowerMem 是把它做成"可配置 4 参数 + 3 阈值"的工程化最彻底的版本。

同类实现对照

框架	记忆衰减机制	三层抽象	数学模型暴露
powermem	Ebbinghaus $R=e^{(-t/S)}$	working / short / long 	4 参数 + 3 阈值全可配
claude-mem	时间衰减 + 重要性评分	implicit	半透明
Letta (MemGPT)	上下文窗口管理 + archival	core / archival 二层	由 LLM agent 自驱
MemoryOS	类生物记忆系统	sensory / short / long	学术原型

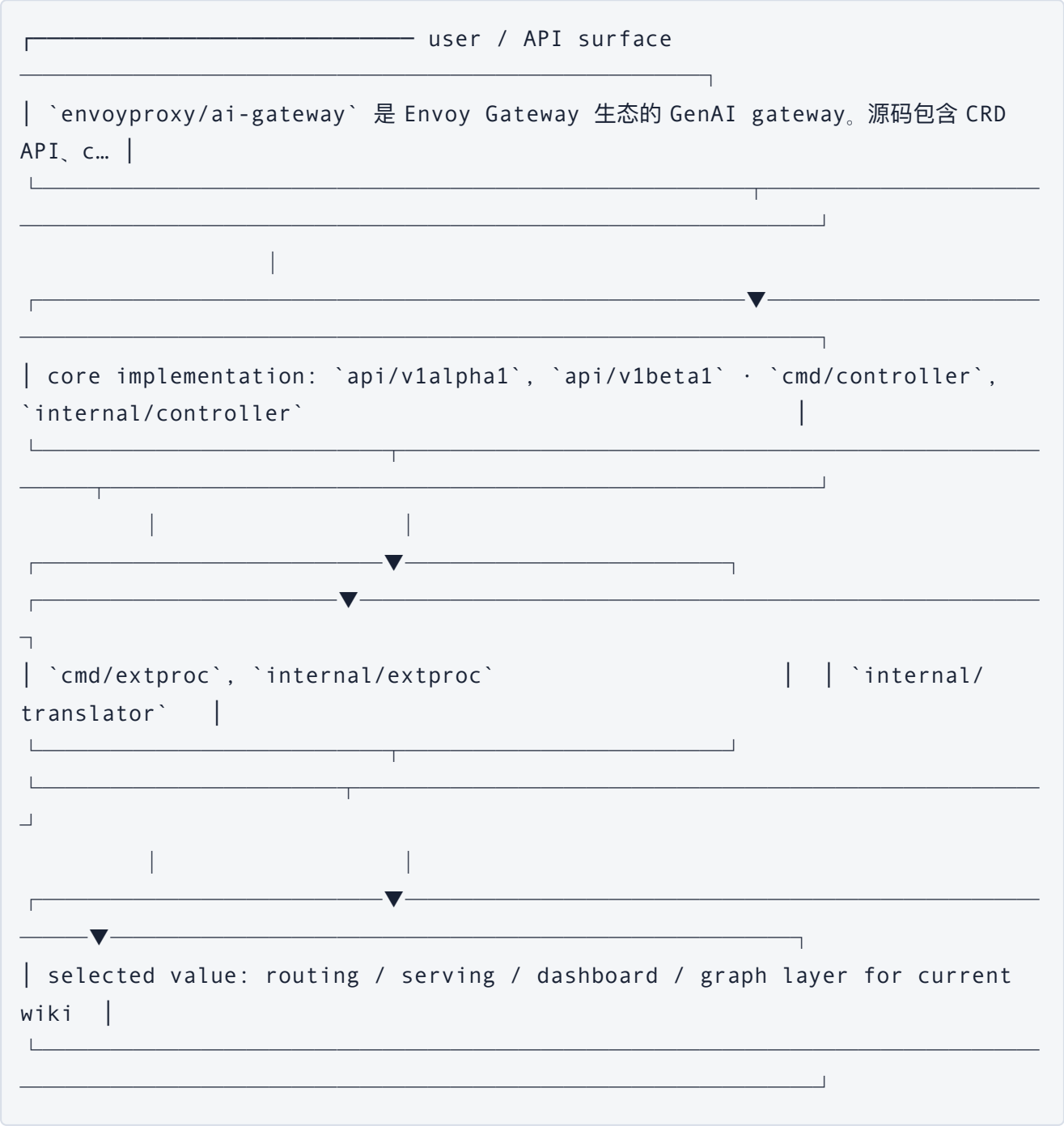
相关页面

- 工程实现：powermem → intelligence/ebbinghaus_algorithm.py
- 领域综述：[agent-memory](#)
- 设计哲学：[ai-as-compressor](#)
- 间隔重复（spaced repetition）的认知科学根基 —— Anki、SuperMemo 同源

Envoy AI Gateway 架构与设计思路分析

envoyproxy/ai-gateway 是 Envoy Gateway 生态的 GenAI gateway。源码包含 CRD API、controller、extproc、provider translators、backend auth、body/header mutator、rate limit、redaction、MCP proxy、metrics/tracing 和大量 data-plane/e2e tests。它解决的是 LLM/API 访问治理，不是模型 serving engine。

核心架构图



模块分层

层/目录	责任
api/v1alpha1, api/v1beta1	AI Gateway CRD。
cmd/controller, internal/controller	控制面。
cmd/extproc, internal/extproc	Envoy External Processing 数据面。
internal/translator	OpenAI/Bedrock 等 provider 翻译。

关键数据流

1. Gateway/HTTPRoute 流量进入 Envoy。
2. extproc 根据 AI Gateway 配置改写/校验/路由请求。
3. translator 处理不同 provider 协议差异，auth/ratelimit/redaction 处理治理。

设计决策

- 建立在 Envoy Gateway 上，复用成熟 L7 控制面。
- 把 provider protocol translation 当核心能力，最近 Bedrock translator 修复就是例子。
- MCP proxy 表明它不只管 LLM，也在扩展到 tool gateway。

对比定位

和 agentgateway 相比，Envoy AI Gateway 更 Envoy Gateway/CRD-first；和 Higress/kgateway 相比，它更专注 GenAI，而不是通用 API gateway。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

k8m

k8m 是轻量 Kubernetes AI dashboard，把集群浏览、操作权限、异常检测和 AI/MCP 能力放进一个小型控制台。详见 [src-k8m-architecture](#)。

架构边界

k8m 是 dashboard-first，不是 CLI assistant，也不是通用 Agent framework。它更适合把 AI 辅助嵌到集群管理 UI 里，而不是让 agent 直接接管复杂 workflow。

关键设计

- Go backend 的 `internal/dao`、`pkg/service`、`pkg/controller` 和 `pkg/plugins` 负责资源访问与插件能力。
- `ui/**` 提供管理控制台。
- AI/MCP 能力通过插件进入资源解释、排障和操作辅助。
- 项目价值主要在产品形态参考：小型控制台如何接入 AI，而不是 agent loop 本身。

选型判断

需要 UI-first K8s AI 管理入口时看 k8m。需要单二进制 dashboard 对照看 kubewall；需要 kubectl workflow 看 [kubectl-ai](#)；需要 agentic K8s 操作层看 [kagent](#)。

k8m 架构与设计思路分析

weibaohui/k8m 是轻量 K8s AI dashboard。目录显示 Go backend (internal, pkg/controller/service/plugins) + 前端 UI + deploy manifests + MCP/自托管 AI 文档，定位是把集群浏览、操作权限、异常检测和 AI/MCP 能力压进一个小型控制台。

核心架构图



模块分层

层/目录	责任
internal/dao, pkg/service	后端数据访问和业务服务。
pkg/controller, pkg/plugins	K8s 控制/插件扩展。
pkg/middleware, pkg/models	HTTP 中间件和模型定义。
ui/**	前端控制台。

关键数据流

1. 用户通过 UI 选择集群/资源。
2. 后端调用 K8s API 和 plugins 获取资源状态。
3. AI/MCP 功能用于解释、排障或辅助生成操作。

设计决策

- Dashboard-first，比 CLI assistant 更适合非命令行用户。
- 插件/MCP 让 AI 能力可插拔，而不是写死在 controller 中。
- 项目活跃度比 P0 项目弱，适合作为形态参考。

对比定位

和 kubewall 相比，k8m 更强调 AI/MCP；和 kagent 相比，它是管理界面，不是 agent workflow 平台。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

kagent

Cloud Native agentic AI 项目，把 Kubernetes / DevOps 操作、工具、Helm/UI 和 MCP 能力包装成 agentic workflow。详见 [src-kagent-architecture](#)。

架构边界

kagent 不应和通用 Agent framework 混放。它的主场是 ai-ops：围绕 K8s/DevOps 操作场景做 agentic automation，而不是提供通用个人 coding agent 执行 loop。

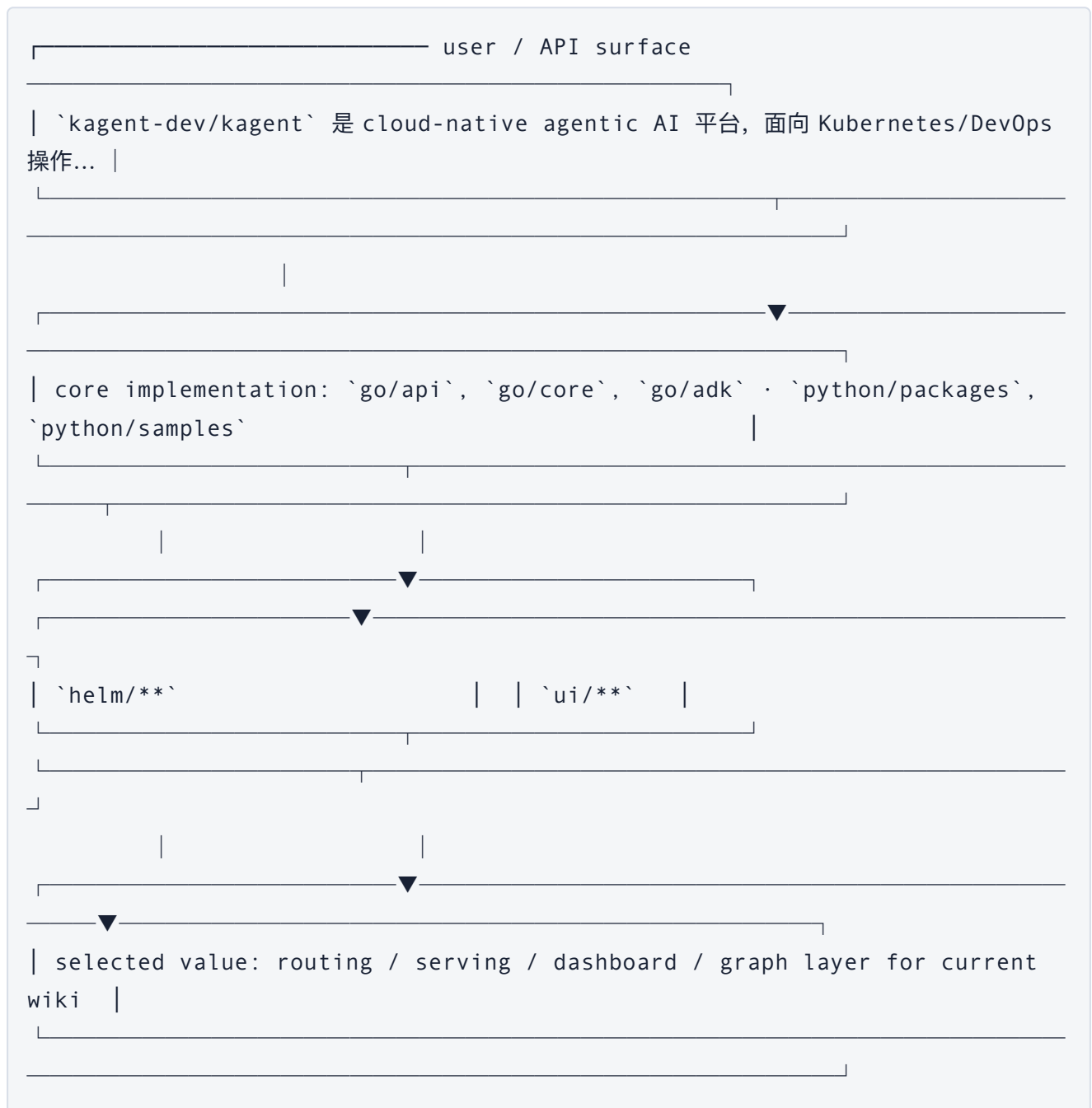
选型判断

- 想在 kubectl 里问答/执行：看 kubectl-ai。
- 想在 dashboard 管理 K8s：看 k8m / kubewall。
- 想把 K8s/DevOps 操作组织成 agentic workflow：看 kagent。

kagent 架构与设计思路分析

kagent-dev/kagent 是 cloud-native agentic AI 平台，面向 Kubernetes/DevOps 操作而不是通用 chat agent。仓库由 Go control plane、Python package/samples、Helm charts、UI、内置 tools/skills 组成，适合补 ai-ops 中“agentic workflow + MCP + K8s runtime”的路线。

核心架构图



模块分层

层/目录	责任
go/api, go/core, go/adk	Go 侧 API、核心对象和 ADK 集成。
python/packages, python/samples	Python agent/tool package 和示例。
helm/**	kagent、agents、tools 的安装分发。
ui/**	管理界面。

关键数据流

1. 用户从 UI/CLI/API 创建或调用 agent。
2. agent 通过工具/MCP/K8s API 执行 DevOps 操作。
3. Go control plane 负责状态、配置、流式事件和部署对象，Helm 安装到集群。

设计决策

- 按 cloud-native control plane 组织，而不是单机 CLI。
- 把 tools/skills 作为可扩展资产，适合和 MCP 网关路线结合。
- 最近 Bedrock streaming 修复说明其模型 provider 事件流是关键路径。

对比定位

和 kubectl-ai 相比，kagent 更像平台；和 k8m/kubewall 相比，它不是 dashboard，而是 agentic workflow 运行层。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

kubectl-ai

kubectl-ai 是 GoogleCloudPlatform 出品的 kubectl 入口 Kubernetes assistant。它以 Go CLI / agent loop 为核心，内置 bash、kubectl、journal、sessions、sandbox，并支持 MCP server mode。详见 [src-kubectl-ai-architecture](#)。

架构边界

kubectl-ai 是 CLI-first 的 ai-ops 工具，不是 dashboard，也不是完整 agent workflow 平台。它把自然语言问题转成 kubectl/bash 查询和解释，并能作为 MCP server 被其他 agent 调用。

关键设计

- cmd/** 提供 CLI 命令入口。
- pkg/agent 和 pkg/tools 管理 agent loop 与内置工具。
- pkg/mcp 让它从 standalone CLI 变成 K8s tool server。
- pkg/sessions / pkg/journal 保留会话与操作记录。

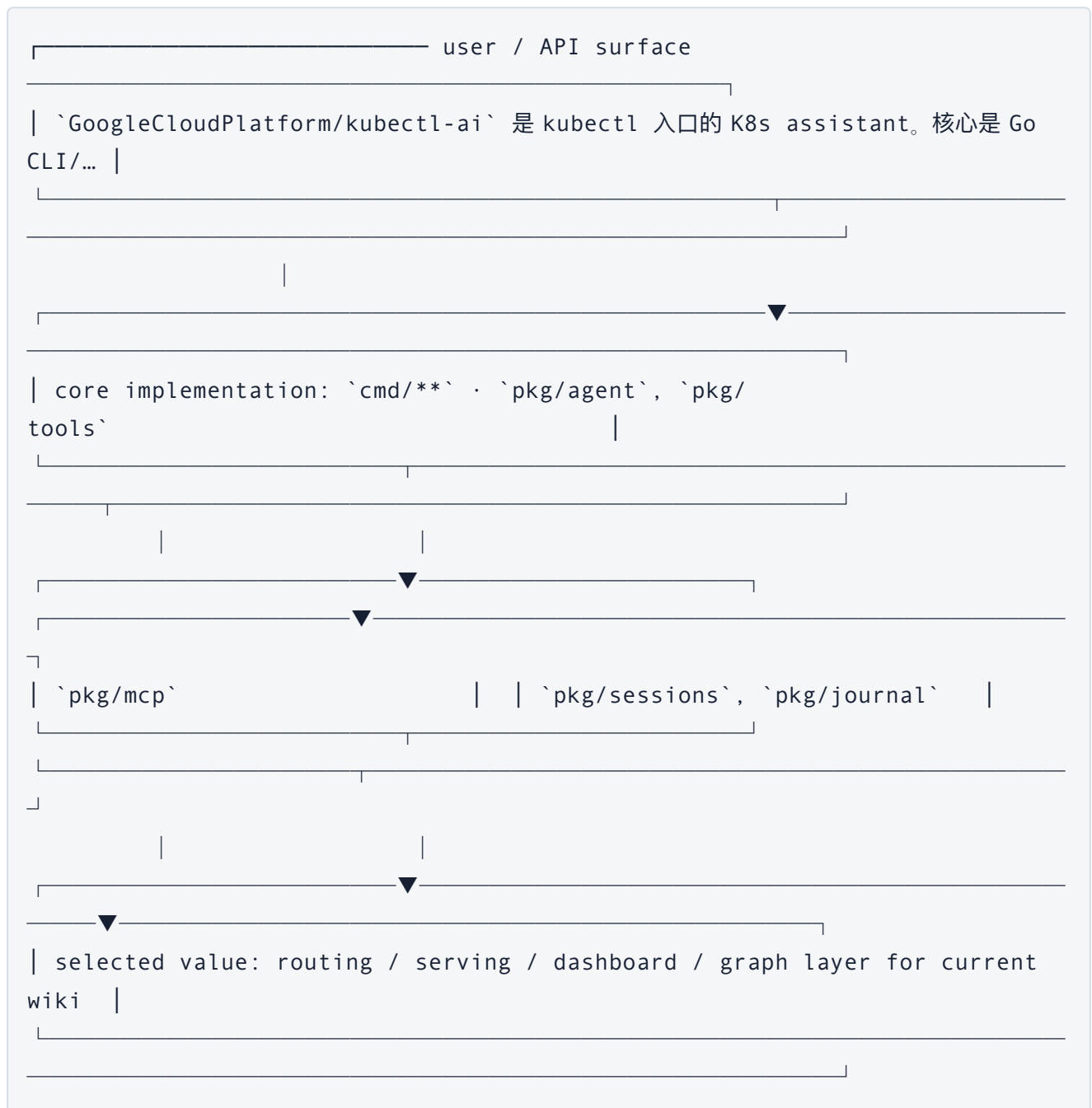
选型判断

想在终端里自然语言操作 Kubernetes，看 kubectl-ai；想把 K8s/DevOps 操作组织成 agentic workflow，看 [kagent](#)；想用 UI 管集群并接入 AI，看 [k8m](#) / kubewall。

kubectl-ai 架构与设计思路分析

GoogleCloudPlatform/kubectl-ai 是 kubectl 入口的 K8s assistant。核心是 Go CLI/agent loop，内置 bash、kubectl、journal、sessions、sandbox 和 MCP server mode；最近 commit 特别修复 MCP server mode 下 built-in tools 注册，说明它既是交互 CLI，也是可被其他 agent 调用的 K8s tool server。

核心架构图



模块分层

层/目录	责任
cmd/**	CLI 命令入口。
pkg/agent, pkg/tools	agent loop 和 bash/kubectl 等工具。
pkg/mcp	MCP server mode。
pkg/sessions, pkg/journal	会话与操作记录。

关键数据流

1. 用户在终端输入自然语言问题。
2. agent 选择 bash/kubectl 内置工具查询集群。
3. 结果写入 journal/session，并生成解释或下一步命令。

设计决策

- 把 kubectl 作为主要 UX，降低 K8s 用户迁移成本。
- MCP server mode 让它从 standalone CLI 变成其他 agent 的 K8s backend。
- 工具面相对克制，适合安全审计。

对比定位

和 kagent 相比更轻、更 CLI-first；和 K8s dashboard 类项目相比，它不提供完整 UI，而是把 kubectl workflow 自然语言化。

相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

MCP Lifecycle Operator

MCP Lifecycle Operator 用声明式 API 部署、管理和安全滚动 MCP Servers，把 Agent tool server 生命周期放进 Kubernetes control plane。详见 [src-mcp-lifecycle-operator-architecture](#)。

架构边界

MCP server framework 解决怎么写工具；MCP lifecycle operator 解决工具服务器如何在集群里安全运行和升级。

什么时候用

场景	判断
需要 MCP lifecycle 能力	适合，MCP Lifecycle Operator 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 mcp , declarative-agent-management, agentgateway 组合。

核心组件

- MCPServer-like APIs
- Controller: deploy/rollout/status
- Integration: gateway/auth/config/secrets
- Production automation: health, rollout, lifecycle

选型提示

把 MCP Lifecycle Operator 放在 MCP lifecycle 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

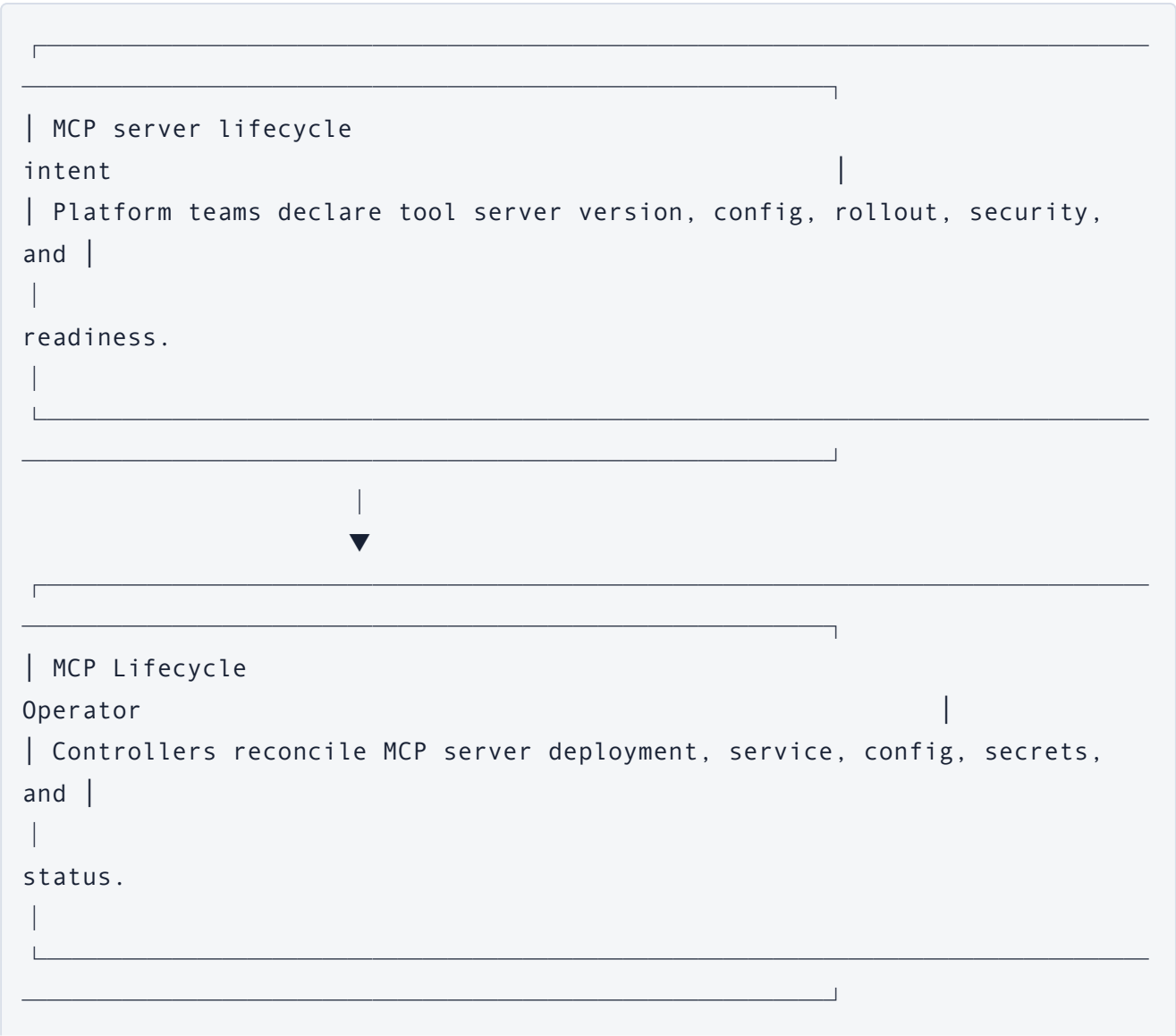
MCP Lifecycle Operator 架构与设计思路分析

原文: raw/mcp-lifecycle-operator-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/mcp-lifecycle-operator> · 优先级 P1

一句话定位

MCP Lifecycle Operator 用声明式 API 部署、管理和安全滚动 MCP Servers，把 Agent tool server 生命周期放进 Kubernetes control plane。

核心架构图





模块分层

层 / 模块	职责
MCPServer-like APIs	MCPServer-like APIs
Controller	deploy/rollout/status
Integration	gateway/auth/config/secrets
Production automation	health, rollout, lifecycle

关键数据流



设计决策与哲学

- 补齐 MCP lifecycle 维度: MCP Lifecycle Operator 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: MCP server framework 解决怎么写工具; MCP lifecycle operator 解决工具服务器如何在集群里安全运行和升级。
- 选型价值: 它应和 [mcp](#), declarative-agent-management, [agentgateway](#) 一起看, 而不是孤立评估。

相关页面

- [mcp-lifecycle-operator](#)
- [kubernetes](#)
- [mcp](#)
- [declarative-agent-management](#)
- [agentgateway](#)

mem0

通用 AI memory layer，面向应用和 Agent 提供长期记忆 API、SDK、OpenMemory、自托管服务和 MCP/CLI/agent plugins。详见 [src-mem0-architecture](#)。

架构边界

mem0 更像产品化 memory service / SDK，不是单个 coding agent 插件。它与 powermem、[agentmemory](#)、[memsearch](#) 的区别在于：mem0 更偏通用应用记忆和生态接入，PowerMem 偏数据库级混合检索与衰减模型，agentmemory 偏本地跨 agent memory bus，memsearch 偏 Markdown truth + Milvus hybrid search。

选型判断

需求	更适合
应用内用户记忆、SDK 接入	mem0
数据库/企业级混合检索记忆	powermem
本地 agent 间共享记忆	agentmemory
代码/对话 Markdown 语义记忆	memsearch

相关源码页

- [src-mem0-architecture](#)
- [agent-memory-selection-matrix](#)

mem0 架构与设计思路分析

mem0ai/mem0 是通用应用/agent 记忆层，而不是单一 coding-agent 插件。仓库同时包含 Python SDK、TypeScript SDK、自托管 server、OpenMemory dashboard/API、CLI、MCP server 和多 agent/editor 集成。2026 年 README 强调新算法从 UPDATE/DELETE 记忆维护转向 ADD-only fact accumulation，并通过 entity linking、BM25、semantic 和时间信号融合检索。

核心架构图



```
time |  
_____  
_____
```

模块分层

层/目录	责任
mem0/memory/ main.py	SDK 核心：参数校验、config 安全拷贝/脱敏、LLM 抽取、entity extraction、BM25 预处理、score/rank。
mem0/ vector_stores/**	多向量库适配层，是 mem0 保持通用性的关键。
openmemory/api/ **	自托管 API/dashboard 后端，包含 SQLite 模型、权限、配置、MCP server。
mem0-ts/**	TypeScript SDK，与 Python SDK 形成双语言应用接入面。

关键数据流

- 写入时，app 传入 text/messages 和 filters，SDK 拒绝顶层 user_id/agent_id/run_id，要求进入 filters，避免 scope 混乱。
- Memory.add 通过 LLM 做 additive extraction，提取 facts，不再默认修改旧事实；entity extractor 生成实体链接信号。
- 检索时 query 先被校验并 embedding；vector hit、BM25 keyword、entity matching、时间/metadata 信号进入 score_and_rank 融合。

设计决策

- ADD-only 降低记忆更新/删除误判风险，代价是长期需要检索排序和冗余控制更强。
- 把 cloud/self-host/library 分成三层：library 适合原型，自托管补 dashboard/auth，cloud 承接高级产品能力。
- 多后端 factory 让它比 powermem 更轻数据库绑定，但一致性和运维语义也更分散。

对比定位

与 powermem 相比，mem0 更像云产品 + SDK 生态，后端中立；PowerMem 更强调 OceanBase 四路混合检索和数据库内聚。与 [memsearch](#) 相比，mem0 没有 Markdown truth source，而是 API/DB/向量层。与 [agentmemory](#) 相比，mem0 更偏应用用户记忆，不是 coding-agent 本地工作流。

相关链接

- GitHub 当前状态底稿: [src-github-stars-backlog-current-state](#)
- 选型地图: [github-stars-backlog-implementation-map](#)

memsearch

memsearch 是 Zilliz 开源的跨平台 [agent-memory](#) 系统（仓库 [zilliztech/memsearch](#)，分析版本 v0.4.6 / HEAD 018a85f）。它面向 [claude-code](#)、Codex、OpenCode、OpenClaw 等 coding agents，把会话摘要保存成 `.memsearch/memory/*.md`，并用 Milvus 做可重建的 hybrid search index。

解决什么

coding agent 的长期记忆通常卡在两个问题：一是不同宿主的 hook/transcript 格式不一致，二是历史上下文不能每次全量注入。memsearch 的答案是：宿主插件负责 capture，Python core 负责 Markdown index，Agent 需要时显式调用 memory-recall 做渐进式检索。

核心特征

- **Markdown source-of-truth**：daily memory、project summary、user profile 都是 .md 文件，Milvus 只是 shadow index。
- **跨平台插件**：支持 Claude Code、Codex、OpenCode、OpenClaw，各平台有独立 hooks、skills、transcript parser。
- **Milvus hybrid retrieval**：dense embedding + BM25 sparse + RRF，详见 hybrid-search-rrf。
- **渐进式 recall**：L1 search snippet → L2 memsearch expand full section → L3 transcript anchor。
- **本地优先**：插件默认路径偏 memsearch[onnx] + Milvus Lite，也可切 OpenAI/Google/Voyage/Jina/Mistral/Ollama/local 等 embedding provider。
- **记忆治理**：maintenance task 可按 digest/interval 维护 .memsearch/PROJECT.md 和 .memsearch/USER.md。

架构骨架

层	组件	职责
插件层	plugins/{claude-code,codex,opencode,openclaw}	捕获对话、写 Markdown、注入 recall 提示
CLI/API	memsearch index/search/expand/watch/compact/config	用户和插件统一入口
Core	MemSearch	扫描、切块、增量索引、搜索、压缩
Storage	MilvusStore	schema、dense/BM25 hybrid search、metadata query
Config	config.py	

层	组件	职责
		defaults/global/project/CLI override 与 env:VAR

完整分析见 [src-memsearch-architecture](#)。

与同类关系

memsearch 与 [claude-context](#) 同属 Zilliz 生态，并共享 Milvus hybrid retrieval，但 claude-context 主要索引代码库，memsearch 主要索引会话记忆。

与 [agent-recall](#) 相比，memsearch 更偏 Markdown journal + semantic recall；agent-recall 更偏 [mcp](#) tools + SQLite scoped facts。与 [agentmemory](#) / powermem 相比，memsearch 更轻量、更贴近个人 coding agent hook 工作流。

相关页面

- [src-memsearch-architecture](#)
- [agent-memory](#)
- milvus
- hybrid-search-rrf
- [three-tier-search-protocol](#)
- [claude-code](#)
- [claude-context](#)
- [agent-recall](#)
- [agentmemory](#)
- powermem

memsearch 架构与设计思路分析

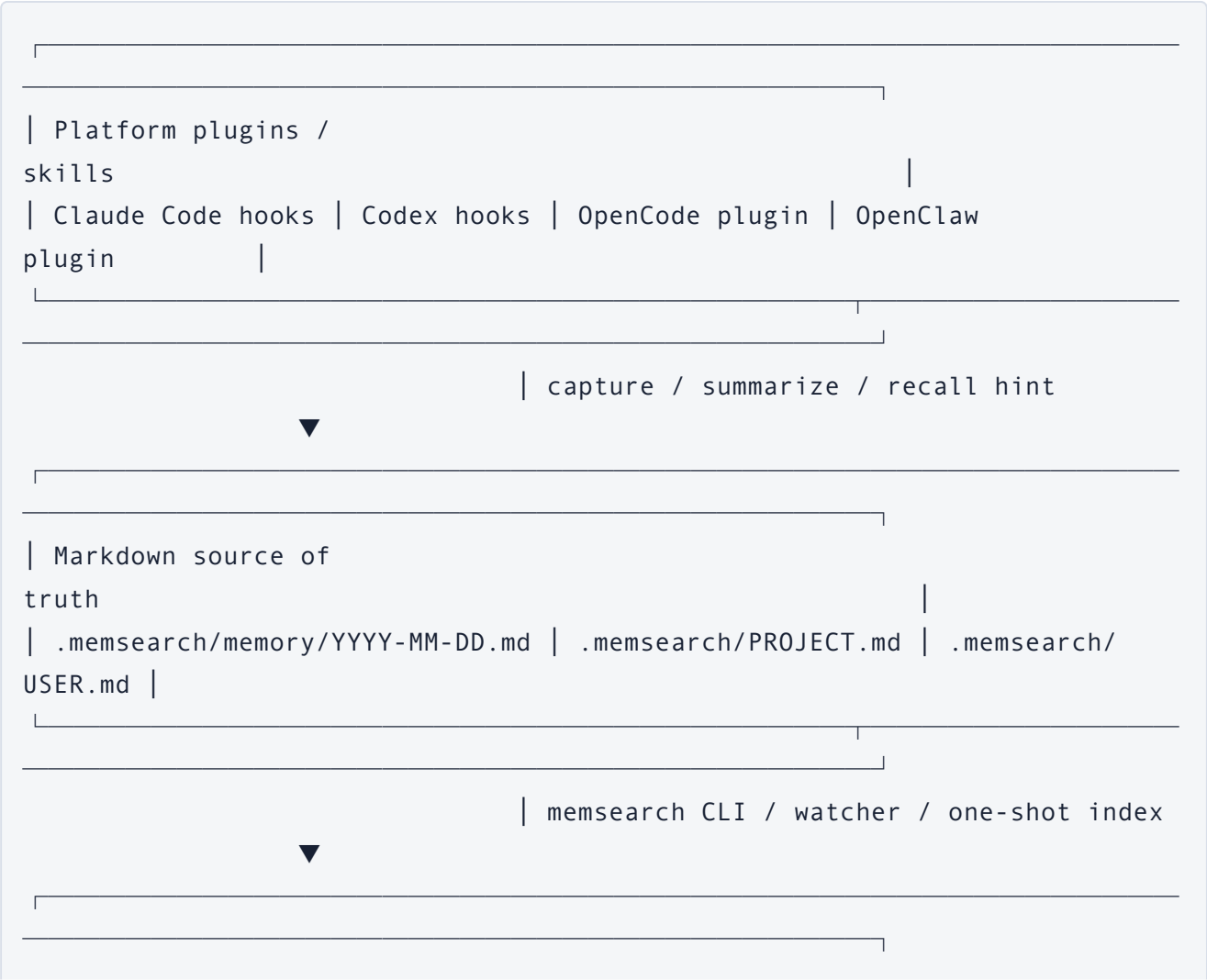
原文: raw/memsearch-architecture-analysis.md · 仓库: <https://github.com/zilliztech/memsearch> · 分析版本 v0.4.6 / HEAD 018a85f

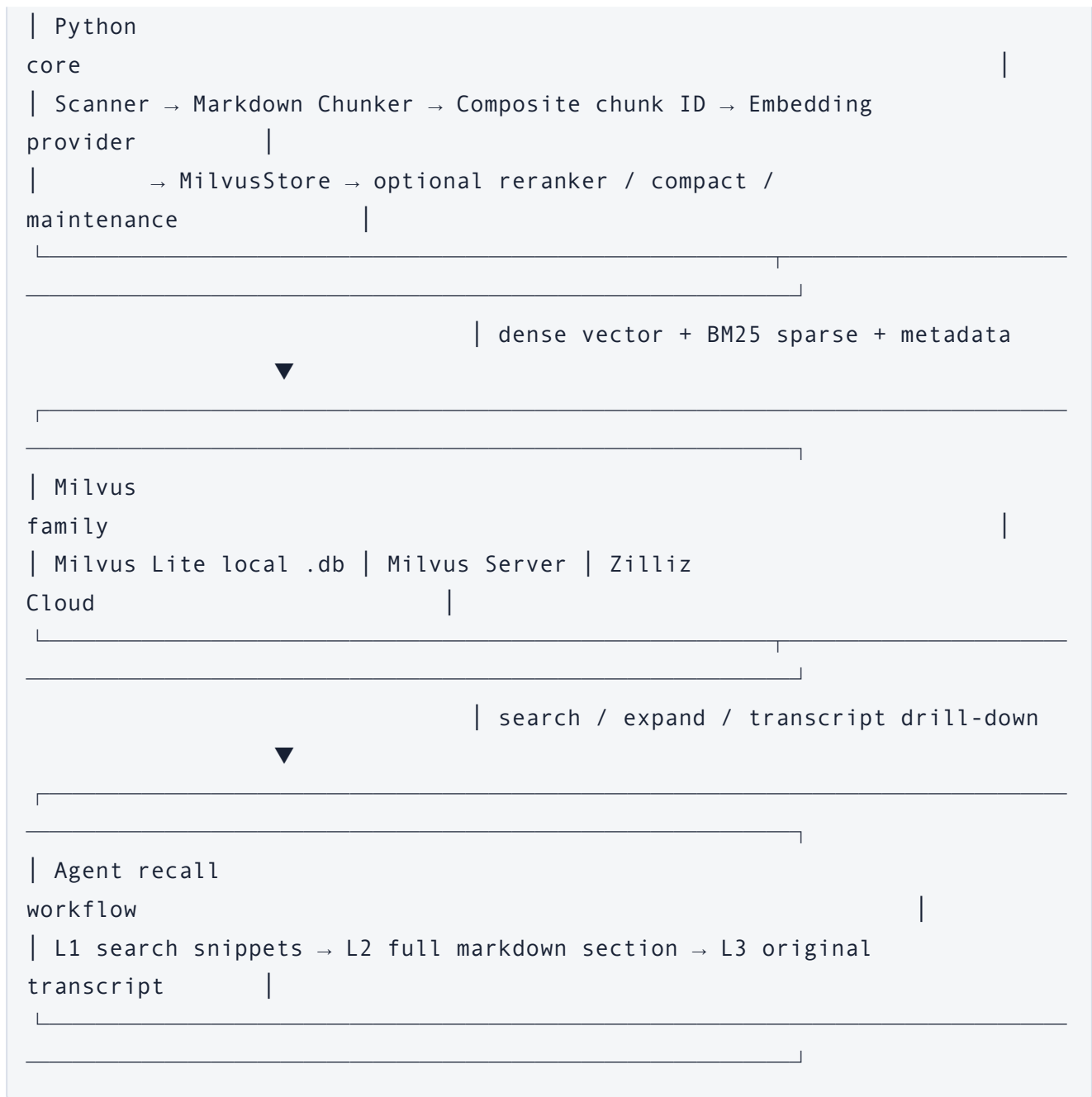
一句话定位

[memsearch](#) 是 Zilliz 出品的跨平台 [agent-memory](#) 系统: [claude-code](#)、Codex、OpenCode、OpenClaw 等平台通过 hook/skill 捕获会话摘要, 写入 `.memsearch/memory/*.md`, 再把 Markdown chunk 索引到 Milvus / Milvus Lite / Zilliz Cloud。

它的关键设计是 Markdown source-of-truth + 可重建 shadow index。查询时走 dense embedding + BM25 sparse + RRF, 再用 expand 和 transcript anchor 做 search → full section → original transcript 的渐进式回溯。

核心架构图





模块分层

层 / 模块	职责
用户入口	memsearch index/search/expand/watch/compact/config 等 CLI 命令
平台插件	Claude Code / Codex / OpenCode / OpenClaw hook、skill、transcript parser
编排核心	MemSearch 统一扫描、切块、增量索引、搜索、压缩、watch
Markdown 处理	发现 Markdown 文件，按 heading/paragraph/line/sentence 切 chunk
Embedding 抽象	OpenAI/Google/Voyage/Jina/Mistral/Ollama/local/ONNX provider protocol

层 / 模块	职责
存储检索	Milvus collection schema、BM25 Function、dense/sparse hybrid search
记忆压缩	daily memory compact、PROJECT/USER maintenance、prompt templates
配置	defaults → global config → project config → CLI flags, 支持 env:VAR

分层边界是：平台插件负责把宿主会话变成 Markdown 和 anchor；Python core 不理解某个宿主的 transcript 格式，只处理 Markdown → chunks → Milvus → search/expand。

关键数据流

1. 捕获与索引

```

Agent session ends / prompt submitted
|
▼
Platform hook parses transcript and summarizes turn
|
▼
Append markdown block to .memsearch/memory/YYYY-MM-DD.md
|
▼
Scanner finds markdown files
|
▼
Chunker splits by heading, paragraph, line, sentence
|
▼
chunk_id = sha256(markdown:source:start:end:content_hash:model)[0:16]
|
├─ if chunk_id already indexed: skip embedding
|
├─ if old chunk_id missing from file: delete stale record
|
▼
Embed cleaned content, store original content + metadata in Milvus

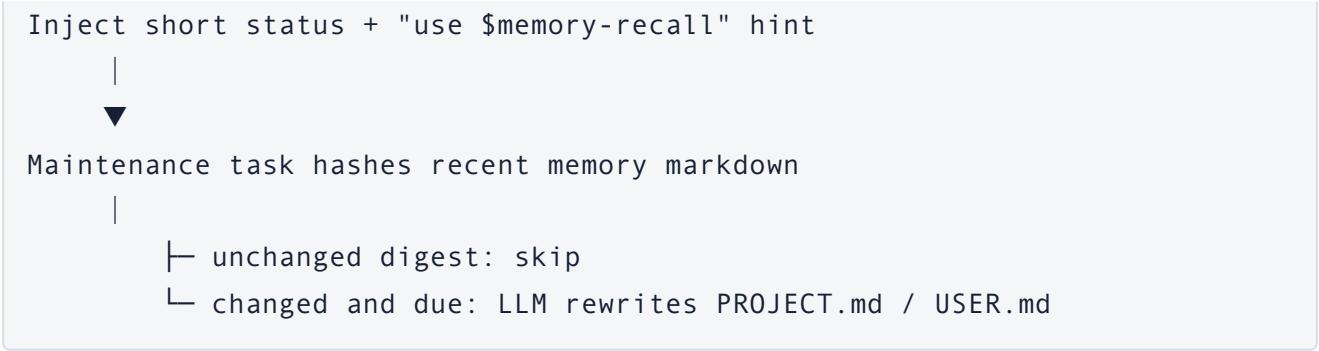
```

2. 语义检索与渐进式回溯

```
User asks a question that needs memory
|
▼
$memory-recall skill runs memsearch search --json-output
|
▼
Query embedding + raw query text
|
|   ├── dense AnnSearchRequest over embedding
|   └── BM25 AnnSearchRequest over sparse_vector
|
▼
Milvus hybrid_search + RRFRanker(k=60)
|
▼
L1: snippets with chunk_hash/source/heading/line range
|
▼
memsearch expand <chunk_hash>
|
▼
L2: full heading section from source markdown
|
▼
Optional L3: parse original transcript anchor
```

3. 启动注入与长期维护

```
Agent starts in project
|
▼
Plugin resolves project root and collection name
|
|   ├── first run: install/warm uvx memsearch[onnx]
|   ├── first config: set embedding.provider = onnx
|   ├── Milvus Lite: run one-shot background index
|   └── Milvus Server/Zilliz: start watch singleton
|
▼
```



设计决策与哲学

- **Markdown 是 source of truth**: Milvus 只是 shadow index, `compact()` 也把摘要写回 daily Markdown 后再索引, 和 [agent-recall](#) 的 SQLite truth model 不同。
- **chunk 主键绑定 model**: 主键包含 source、line range、content hash、embedding model; 这同时支撑增量索引和模型切换后的索引隔离。
- **检索由 Milvus 同时承担 dense 和 BM25**: content 通过 analyzer + BM25 Function 生成 sparse vector, 再与 dense vector 做 RRF。
- **平台插件和核心解耦**: Codex/Claude/OpenCode/OpenClaw 处理 hook 生命周期和 transcript; core 只处理 Markdown, 可跨宿主复用。
- **渐进式上下文控制**: search 返回 L1 片段, expand 返回 L2 完整 heading section, 必要时再沿 anchor 回到 L3 transcript, 和 [three-tier-search-protocol](#) 的目标一致。
- **本地优先但可远端**: 插件首次体验偏 ONNX + Milvus Lite; 配置同时支持 OpenAI 等 provider、Milvus Server 和 Zilliz Cloud。
- **维护任务偏治理而非事实存储**: PROJECT/USER 摘要按输入 digest 和 interval 更新, 事实仍来自 `.memsearch/memory/*.md`。

关键组件

MemSearch 是核心编排器: 扫描 Markdown、切 chunk、计算新旧 chunk 差集、只 embedding 新 chunk、删除 stale chunk。embedding 前会剥掉 HTML comments, 避免 session/turn/transcript path 污染向量; 但 Milvus 里保留原始 content, 方便 expand 和 transcript drill-down。

MilvusStore 是检索关键: collection 里同时有 embedding dense vector、content analyzer field、BM25 生成的 sparse_vector 和 source/heading/line metadata。搜索时 dense 和 BM25 两路并发请求, 再用 RRF 融合。

与同类关系

维度	memsearch	claude-context	agent-recall	agentmemory / powermem
主要对象	coding agent 会话记忆 Markdown	代码库语义检索	MCP-native 实体/ 关系/观察	跨 Agent 记忆服务 / 中间件

维度	memsearch	claude-context	agent-recall	agentmemory / powermem
source of truth	.memsearch/*.md	indexed code files	SQLite	SQLite / OceanBase / service storage
检索	Milvus dense + BM25 + RRF	Milvus dense/sparse + RRF	SQLite/FTS + scope	BM25/vector/graph 或多路混合
上下文控制	search → expand → transcript	code search chunks	briefing cache + open/search	viewer/context injection/tools

[memsearch](#) 和 [claude-context](#) 都来自 Zilliz，并共享 Milvus hybrid retrieval 的技术路径；区别是 claude-context 解决“当前代码库理解”，memsearch 解决“过去会话记忆”。

相关页面

- [memsearch](#)
- [agent-memory](#)
- milvus
- hybrid-search-rrf
- [three-tier-search-protocol](#)
- [claude-code](#)
- [claude-context](#)
- [agent-recall](#)
- [agentmemory](#)
- powermem

Model Context Protocol (MCP)

Anthropic 推出的 AI Agent 工具/资源/Skill 接入协议，基于 JSON-RPC over stdio（或 SSE）。让任意 Agent 客户端（Claude Code / Cursor / Gemini CLI / Codex）通过统一协议加载第三方工具。

角色

端	谁	干啥
Server	插件 / 工具进程	暴露工具（tools/list、tools/call）和资源
Client	AI Agent runtime	启动 server 子进程，转发 LLM 的工具调用请求

在本知识库中的应用

- [claude-context](#) —— 通过 MCP 暴露 4 个代码检索工具（index_codebase / search_code / clear_index / get_indexing_status）
- [claude-mem](#) —— server-beta 模式下用 MCP 暴露记忆查询（替代 hook-only 模式）

协议通道纪律

[!warning] stdout 只跑协议

MCP 用 stdout 传 JSON-RPC，任何 console.log 都会污染协议导致客户端解析失败。

第一行代码就应该把 stdout 重定向到 stderr：

ts

console.log = (...args) => process.stderr.write('[LOG] ' + args.join('') + '\n');

这是 [ai-agent-plugin-patterns](#) 中"协议通道纪律"原则的典型案例。

配置示例（Claude Code）

```
claude mcp add <name> -e KEY=VALUE -- <command> <args>
```

启动后 Claude Code 会把 server 注册的工具加入自己的 tool list，LLM 可像调用内置工具一样调用。

参考

- [src-claude-context-architecture](#)

- [src-claude-mem-architecture](#)

nanobot

HKUDS 出品的极简个人 AI Agent 框架。Claude Code / Codex / OpenClaw 风格的轻量级长跑 Agent。

核心数据

字段	值
项目主仓	https://github.com/HKUDS/nanobot
本次 ingest	https://github.com/googs1025/nanobot (fork)
协议	MIT
版本	v0.2.0 (截至 2026-05-20)
语言 / 运行时	Python ≥3.11, 单 asyncio event loop
代码规模	~16k 行 Python, 无 C 扩展
安装	<code>pip install nanobot → nanobot onboard</code>
程序化用法	<code>from nanobot import Nanobot; Nanobot.from_config().run(msg)</code>

它解决什么问题

把"个人聊天机器人 + 长跑 AI Agent"这件事压成一个能看懂、能改的小内核：

- 接入多 IM 渠道很烦 → 17 个内置 channel (Telegram / Discord / Slack / Feishu / WeChat / QQ / Email / Matrix / DingTalk / WhatsApp / WeCom / MSTEams / WebSocket / CLI / ...)，加新渠道写一个 module + entry_points 即可。
- 多家 LLM SDK 差异大 → 7 个内置 backend (anthropic / azure_openai / bedrock / github_copilot / openai_codex / openai_compat / openai_responses)，自家 openai_compat_provider 走 OpenAI 协议方言覆盖 DeepSeek / Kimi / Qwen / vLLM / Ollama / LongCat / MiMo / StepFun / ...
- LLM 失败要切模型 → FallbackProvider 在 provider 层做透明 failover，Agent 不感知；has_streamed 防拼接错乱 + 3×60s 熔断。
- 用户中途想打断 / 改主意 → 状态机 + checkpoint：每次工具执行后写 runtime_checkpoint，/ stop 取消后下次 inbound 用 _state_restore 续接；或不打断，用 mid-turn 注入直接塞进 messages 末尾。
- 长期记忆 / 主动行为 → Skills / Memory / Dream / Heartbeat 全部以上下文 (Markdown 文件) 形式注入 system prompt，而非编排层；HeartbeatService 周期触发 LLM 自己决定 skip/run。

6. MCP 工具生态 → 通过 `mcp>=1.26` 接 [mcp](#) server stdio，工具自动归入 `ToolRegistry` 暴露给 LLM。

核心架构（一句话版）

Channels → MessageBus(2 × `asyncio.Queue`) → AgentLoop(8 态状态机) → Providers(可 fallback)

详情见源摘要：[src-nanobot-architecture](#)。

关键技术决策

- bus 解耦 channel ↔ agent：channel 不 import agent，agent 不 import channel。
- 8 态状态机替代单巨函数：RESTORE → COMPACT → COMMAND → BUILD → RUN → SAVE → RESPOND → DONE，每态一个可单测 handler，跳转走 `_TRANSITIONS` 显式表。
- pkgutil + entry_points 自动发现：built-in shadows external，新增 channel 不需要改主代码。
- provider 级 failover：FallbackProvider 实现 LLMProvider，Agent 透明感知不到；`has_streamed` 信号防止已吐字后跨模型拼接错乱。
- per-session 串行 + 跨 session 并行：`asyncio.Lock` × `Semaphore(3)`。
- 能力即上下文：skills / memory / heartbeat / dream 都是注入 system prompt 的 Markdown 文件，而非编排层 — 体现 [ai-agent-plugin-patterns](#) 的「Markdown is the universal interface」。

重要依赖

依赖	用途
<code>openai >= 2.8.0</code>	OpenAI 原生 SDK（取代 litellm）
<code>anthropic >= 0.45.0</code>	Anthropic 原生 SDK（含 <code>thinking_blocks</code> ）
<code>mcp >= 1.26</code>	mcp 协议接入第三方工具 server
<code>typer</code>	CLI（onboard / agent / gateway / login / ...）
<code>pydantic v2</code>	配置 schema
<code>filelock</code>	cron 任务跨进程持久化
<code>croniter + zoneinfo</code>	cron 表达式解析 + 时区
<code>oauth-cli-kit</code>	交互式 OAuth 登录（GitHub Copilot / ...）

内置 Skills

clawhub · cron · github · image-generation · long-goal · memory · my · skill-creator · summarize · tmux · weather

每个 skill 是 workspace 下一个 markdown 目录，包含 description + 操作 prompt + 可选工具，按需被 LLM 通过 skill 列表选择激活。

14 个内置斜杠命令

/new /model /history /goal /dream /dream-status /dream-stop /pairing /help /status /restart /stop /memory /skills

三档路由优先级：priority（/stop 类，主循环立即拦截）→ exact → prefix。

部署形态

- CLI 单进程：nanobot agent 启动事件循环，所有渠道在同一进程内并发。
- OpenAI 兼容 API：nanobot.api.server 把 nanobot 当成上游 LLM 暴露 SSE 流，供外部工具调用。
- Gateway 模式：多 agent 实例的反代（用 nanobot gateway）。
- WebUI：编译后的 SPA 随 wheel 发布，通过 WebSocket channel 服务。

与同类对比

维度	nanobot	claude-code	autogen / langgraph
定位	个人长跑 IM Agent	CLI 内开发助手	编排框架
用户交互	多 IM 渠道 / WebUI	终端 stdin/stdout	API / 自托管 UI
状态机	显式 8 态 + checkpoint	Hook lifecycle	图驱动
Provider 解耦	FallbackProvider 装饰	原生 Anthropic	由用户实现
记忆	Markdown 注入 system prompt	CLAUDE.md / auto-memory	由用户实现
插件机制	pkgutil + entry_points	Hook + slash command + MCP	代码继承

设计灵感

README 明确说"inspired by Claude Code / Codex / OpenClaw"。最直接的影子：
- Skills + Memory + AGENTS.md / SOUL.md / USER.md / TOOLS.md 的工作区约定 ← [claude-code](#) 的 CLAUDE.md 系

- 斜杠命令系 ← [claude-code](#) 的 slash command
- MCP 集成 ← Claude Code 同期推进
- 8 态状态机 + checkpoint 续接 ← Codex 长跑 agent 的中断恢复需求

相关页面

- [src-nanobot-architecture](#) — 详细架构与设计哲学
- [claude-code](#) — 灵感来源 (Anthropic CLI Agent)
- [mcp](#) — 工具协议
- [claude-agent-sdk](#) — 对照：编程式 Agent SDK 范式
- [ai-agent-plugin-patterns](#) — 9 条设计原则中"pkgutil 自动发现"和"Markdown 即接口"在此体现

nanobot 架构与设计思路分析

原文: raw/nanobot-architecture-analysis.md · 仓库: <https://github.com/googs1025/nanobot> (fork 自 HKUDS/nanobot) · 分析版本 v0.2.0

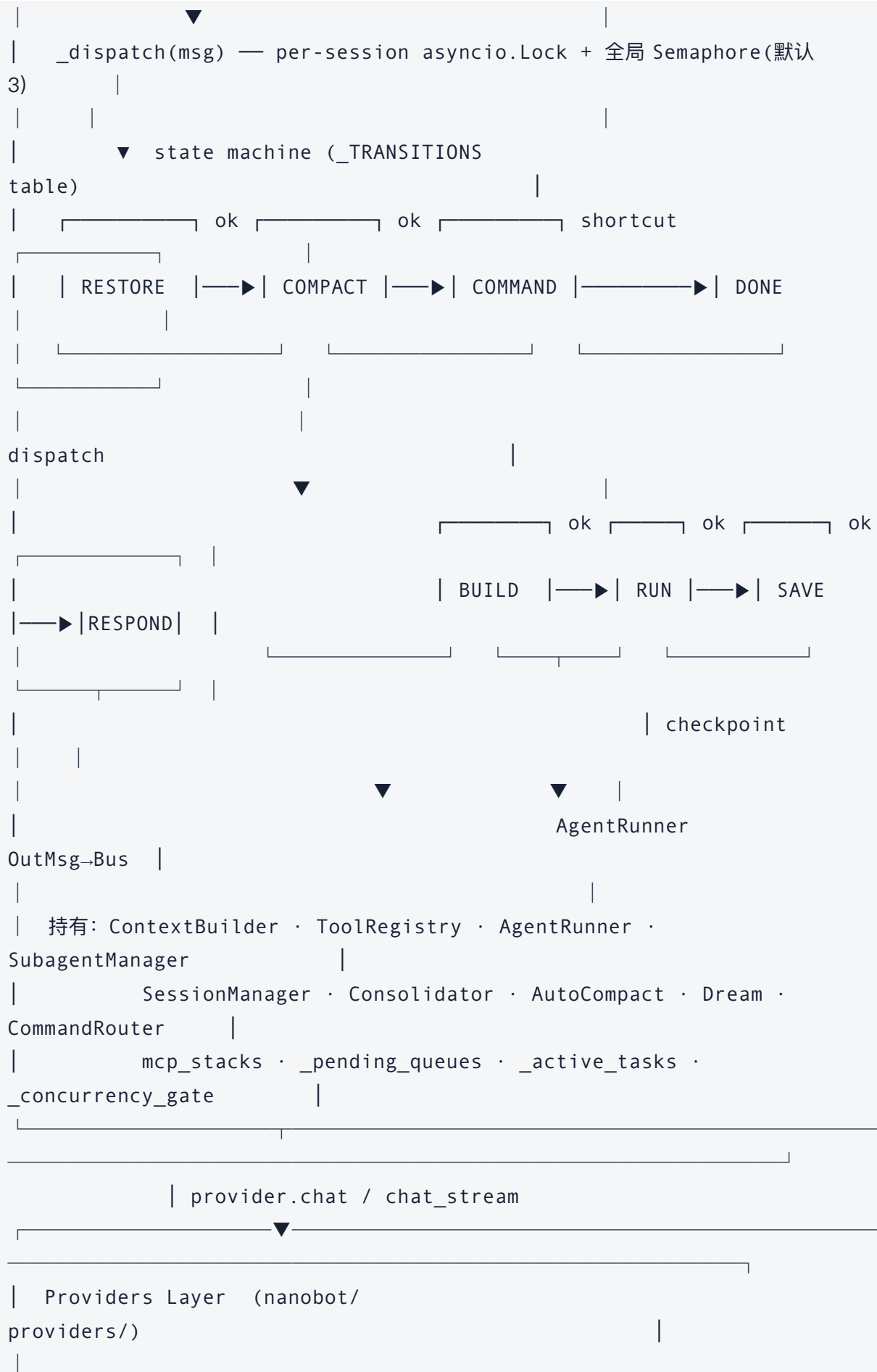
一句话定位

[nanobot](#) 是 HKUDS 的极简个人 AI Agent 框架 (Python ≥ 3.11 , MIT, v0.2.0) , 定位为「[claude-code](#) / Codex / OpenClaw 风格的轻量级长跑 Agent」。它通过一个事件驱动的 8 态 Agent 状态机, 把 17 个聊天渠道、7+ 家 LLM 厂商、[mcp](#)、技能 / 记忆 / Cron / Heartbeat 编织成一个 ~16k 行的可读小内核——核心路径让开发者「能看懂、能改」, 而周边能力 (channel / provider / skill) 通过 pkgutil 自动发现 + entry_points 插件机制接入, 主代码不需要动。

核心架构图







```

factory.make_provider(config)
|
|   └─► _make_provider_core() – backend
switch:
|
|   anthropic | azure_openai | bedrock |
github_copilot
|
|   openai_codex | openai_compat (默认) |
openai_responses
|
|   └─► FallbackProvider(primary, [fallbacks],
factory)
|
|   - 请求级 failover; circuit breaker (3 fail × 60s cooldown)
|
|   - has_streamed → 已吐字就放弃失败转移, 避免重复输出
|
|   - _NON_FALLBACK 错误 (auth/quota/content_filter) 直接返回
|
|   LLMProvider(ABC): chat / chat_stream / 内置重试政策 + 结构化错误码
|

```

支撑设施 (与 AgentLoop 平级):

cron/service.py	CronService	: at / every / cron(croniter+tz),
FileLock 持久化		
heartbeat/service.py	HeartbeatService	: 2 phase—LLM 虚拟工具调用决定 skip/
run		
api/server.py	OpenAI-Compatible API	+ SSE 流
cli/commands.py	Typer CLI	: nanobot onboard / agent / gateway /
login ...		
nanobot.py	Programmatic facade	: Nanobot.from_config().run(msg)
agent/tools/	read_file/exec/grep/web_*/notebook/spawn/mcp/	
message/...		
skills/	clawhub · cron · github · image-generation · long-	
goal ·		
	memory · my · skill-creator · summarize · tmux ·	
weather		

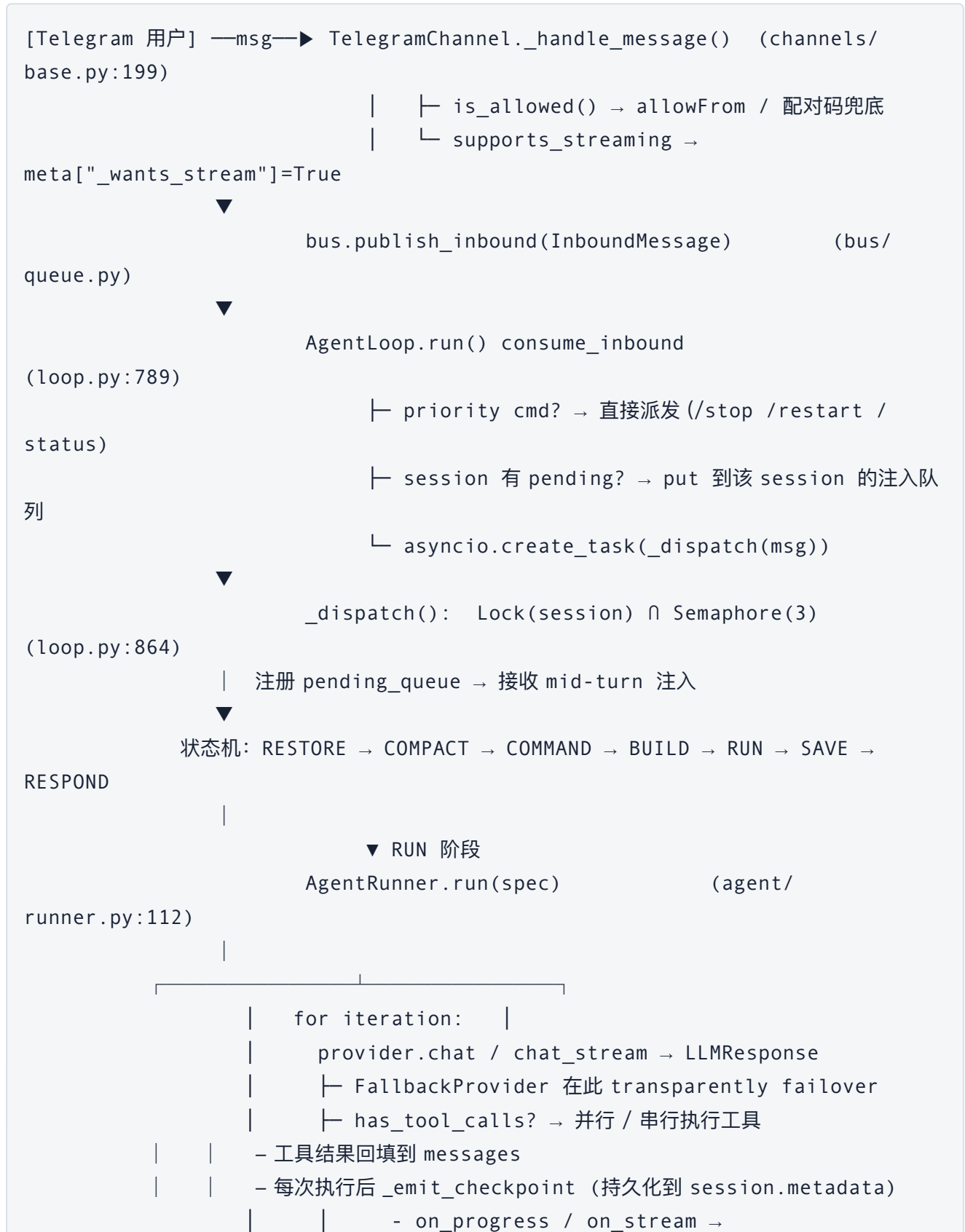
模块分层

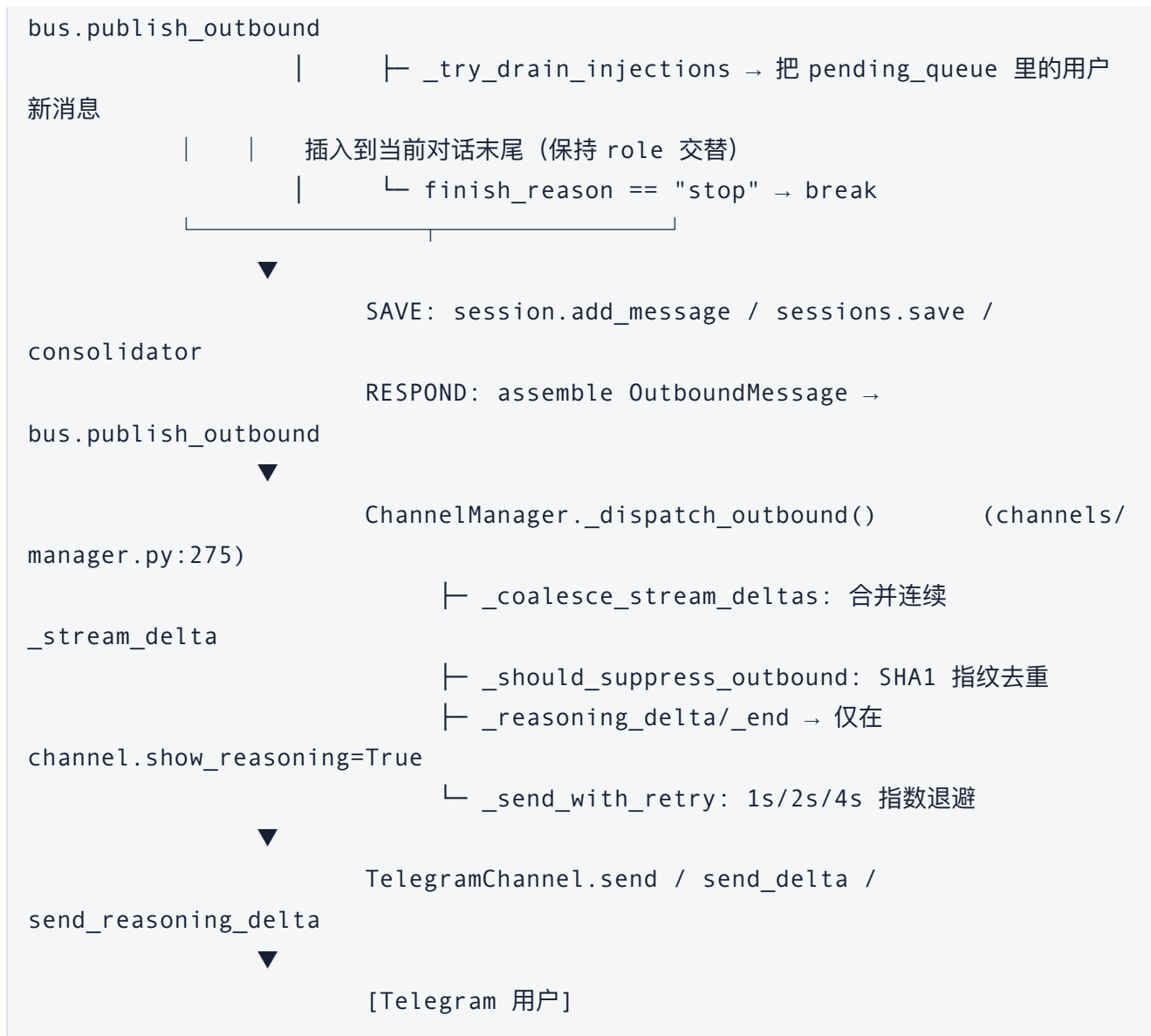
层 / 模块	职责
CLI / SDK 入口	Typer CLI (onboard/agent/gateway/...) + 程序化外观 Nanobot.from_config().run()
渠道层	BaseChannel 抽象; registry.discover_all 用 pkgutil + entry_points 自动发现; ChannelManager 启动 / 路由 / 重试 / 流式合并 / 去重
消息总线	两条 asyncio.Queue; InboundMessage / OutboundMessage dataclass, channel 与 agent 解耦的唯一桥梁
Agent 内核	AgentLoop 8 态状态机; AgentRunner provider-agnostic tool-using 循环; checkpoint / mid-turn 注入
上下文构建	ContextBuilder 拼 system prompt; SkillsLoader workspace + builtin 合并; MemoryStore + Consolidator + Dream 两阶段记忆
工具集	ToolRegistry 注册 filesystem / shell / web / search / mcp / notebook / spawn / message / ..., 按 OpenAI tool-call schema 暴露
Provider 层	LLMProvider ABC + 内置重试; make_provider 工厂; FallbackProvider 请求级 failover + 熔断; spec 注册表标记 OAuth/local/direct
会话	Session 持久化到 ~/.nanobot/sessions/<key>.json; goal_state 支撑 /goal 长目标
命令路由	三档优先级: priority(/stop)、exact、prefix; 14 个内置斜杠命令 (/new /model /history /goal /dream* /pairing /help /status /restart)
调度 & 主动唤起	CronService (at/every/cron + ZoneInfo + FileLock) ; HeartbeatService 周期 LLM 决定 skip/run
OpenAI 兼容 API	把 nanobot 当成上游 LLM 暴露给外部工具, 支持 SSE 流
WebUI	WebUI 编译产物随 wheel 发布; WebSocket 渠道托管 + 静态文件

- 分层关键约束:
- bus 是唯一的渠道↔Agent 通道: channel 不 import agent, agent 不 import channel, 两边都只看 bus/events.py。
 - provider 不感知 fallback: FallbackProvider 本身实现 LLMProvider, 对 Agent 透明; 工厂方法 _make_provider_core 创建的 plain provider 不再包 fallback, 防递归。
 - command router 优先于状态机: /stop 类 priority 命令在 AgentLoop.run() 主循环里就被拦截, 不进入 _dispatch 任务, 从而能取消正在执行的任务。

- per-session 串行 + 跨 session 并行：每个 session_key 一把 `asyncio.Lock`；全局 `Semaphore(NANOBOT_MAX_CONCURRENT_REQUESTS, 默认 3)` 控制总并发。

关键数据流





中断与恢复路径:



tool_pending)

复)

```
|      · iteration
|      · assistant_message (已生成的部分回
|
|      · completed_tool_results
|      · pending_tool_calls
├─ _clear_pending_user_turn(session)
└─ sessions.save(session)
```



finally: 把 pending_queue 里残留的 InboundMessage
重新 publish_inbound 回总线 (不丢消息)

下一次 inbound 时 —► _state_restore (loop.py:1220) 读 runtime_checkpoint
+ pending_user_turn, 把上次中断的上下文物化进 history,
新消息接着这段历史继续推理。

设计决策与哲学

- 小内核 + 可插拔层 (核心 DNA) : bus 解耦 channel 与 agent, channels/registry.py 用 pkgutil + entry_points 自动发现 channel, providers/factory.py 走工厂 + Fallback 装饰——三处机制让新增 channel / provider 几乎不动主代码。这种风格和 [claude-code](#) 的 hook plugin 思路一脉相承。
- 事件驱动状态机替代单巨函数: TurnState 8 个枚举 + _TRANSITIONS 跳转表把一次会话切成可单测的 handler, "/stop → checkpoint → 下次 RESTORE 续接"成为状态机自然的一环, 不是散弹式 if-else。
- Provider 级 Failover 而非 Agent 级: FallbackProvider 实现 LLMProvider 接口对 Agent 透明; has_streamed 信号防止已吐字后跨模型拼接错乱; 3 次失败 × 60s 冷却的熔断器; _NON_FALLBACK_ERROR_KINDS={auth, permission, content_filter, refusal, context_length, invalid_request} 区分"换模型救不了"的错误提前短路。
- 告别 litellm, 回归原生 SDK (2026-03-21 commit) : 现在直接用 openai + anthropic 原生 SDK + 自家 openai_compat_provider 走 OpenAI 协议方言适配 DeepSeek / Kimi / Qwen / vLLM / Ollama / ... 以代码量换控制力, 可以精细处理 reasoning_content、Anthropic thinking_blocks、各家结构化错误码、prompt cache header。
- Mid-turn 注入而非"排队下一轮": 每个活跃 session 持有 asyncio.Queue(maxsize=20), 用户在 Agent 工作时再发消息会塞进队列, 由 runner 在工具调用之间插入到 messages 末尾。_MAX_INJECTIONS_PER_TURN=3 防失控; task 取消时残留消息重新 publish_inbound 回总线。

- **Outbound 合并 + 去重**：流式 `_stream_delta` 在 dispatcher 循环里贪心合并同 (channel, chat_id) 的连续片段，遇到边界塞回本地 buffer；普通消息用 SHA1 指纹 + `origin_message_id` 去重，防止 Hook 误重复发送。
- **DM 配对码代替"静默拒绝"**：未在 allowFrom 名单的 sender 私聊机器人会收到一次性配对码而不是被无视，根治了"我加好友怎么没反应"的体验问题。群聊则静默拒绝防被骚扰。
- **Skills / Memory / Dream / Heartbeat 是上下文层而非编排层**：ContextBuilder 把 identity / AGENTS.md+SOUL.md+USER.md+TOOLS.md / MEMORY.md / always skills / skills 列表 / Dream 处理后的历史 / 归档 summary 拼成 system prompt；HEARTBEAT.md 由 HeartbeatService 周期读取，再让 LLM 通过虚拟 heartbeat tool 决定 skip/run——"该不该跑后台任务"也交给模型。这种「能力即上下文」的设计与 [ai-agent-plugin-patterns](#) 中的「Markdown is the universal interface」一致。
- **Per-session 串行 + Cross-session 并行**：_session_locks 保证同一会话内消息按到达顺序处理；Semaphore(3) 限制全局总并发。多群 / 多人场景下既不会"一个会话死等另一个"也不会无限并发打爆 provider 配额。

关键组件深入解读

AgentLoop 状态机 (nanobot/agent/loop.py)

AgentLoop 是 ~1600 行的核心类，构造时一次性装配整个 agent 运行所需的所有协作者。run() 是无限循环：从 bus 拿 InboundMessage → priority 命令短路 → 检查 session 是否已有 task（有则路由到 pending_queue 做 mid-turn 注入）→ 否则 `asyncio.create_task(_dispatch(msg))`。_dispatch 在锁 + 信号量保护下进入状态机，状态机由 _TRANSITIONS 表驱动。

最有意思的细节是 checkpoint：runner 在每次工具执行后把当前轮 phase / iteration / assistant_message / completed_tool_results / pending_tool_calls 写进 `session.metadata["runtime_checkpoint"]`。一旦 task 被 /stop 取消，_dispatch 的 except 分支会调 _restore_runtime_checkpoint 把"半成品"物化回 session 历史；下次 inbound 时 _state_restore 读出来继续。这把「中断恢复」从异常处理变成了状态机一等公民。

相关页面

- [nanobot](#) — 项目实体页
- [claude-code](#) — 设计灵感来源 (Anthropic CLI Agent)
- [mcp](#) — Model Context Protocol, nanobot 通过 `mcp>=1.26` 接入第三方工具
- [claude-agent-sdk](#) — 对照：另一种 Agent 编程 SDK 范式
- [ai-agent-plugin-patterns](#) — Agent 外挂的 9 条设计原则 (pkgutil 自动发现、Markdown 即接口在此体现)

NemoClaw

NemoClaw 是 NVIDIA 为 [OpenShell](#) sandbox 内 always-on AI Agent 提供的 TypeScript CLI 控制面。它把 guided onboarding、gateway、provider inference、sandbox 生命周期、agent setup、network policy、凭证迁移和 e2e 验证收束成一个命令面。详见 [src-nemoclave-architecture](#)。

架构边界

NemoClaw 不是新的推理引擎，也不是新的多 Agent framework。它更像 [OpenShell](#) 的 host-side 编排层：在宿主机上规划 gateway、provider、sandbox、OpenClaw/Hermes runtime、policy preset 和 messaging channel，然后把结果应用到 sandbox。

关键设计

- 命令层保持很薄，`src/commands/**` 主要负责参数解析。
- `onboard` 是可恢复状态机，串起 `preflight`、`gateway`、`provider inference`、`sandbox`、`agent setup` 和 `policy`。
- Provider 凭证迁移到 `gateway`，agent 通过 `inference.local` 调模型，符合 `agent-credential-isolation`。
- Policy 默认 `deny-by-default`，再叠加 `selected presets` 和 `messaging channel` 所需网络权限。
- Messaging channel 通过 `manifest` 编译成 `credential`、`network policy`、`agent render`、`health check` 等计划。

选型判断

需要把 OpenClaw/Hermes 这类个人 agent 放进可审计 sandbox 并接入多 provider 推理时看 NemoClaw。需要底层 runtime enforcement 时看 [openshell](#)；需要 Kubernetes 原生 sandbox 时看 `agent-sandbox`；需要统一治理 LLM/MCP/A2A 出口时看 [agentgateway](#)。

NemoClaw 架构与设计思路分析

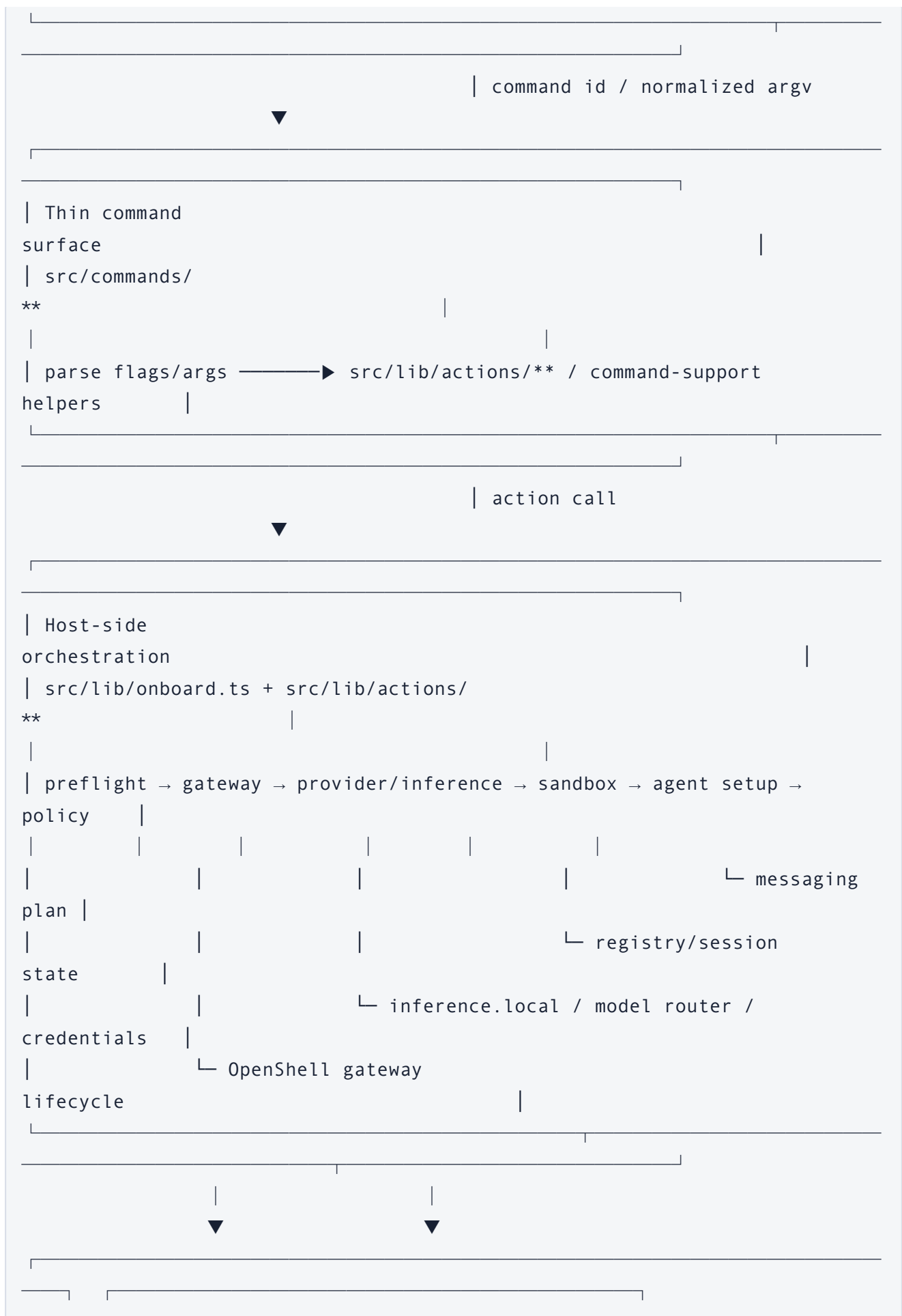
原文: raw/nemoclave-architecture-analysis.md · 仓库: https://github.com/NVIDIA/NemoClaw · 分析版本 HEAD 3c0340a

一句话定位

[NemoClaw](#) 是 NVIDIA 为 [OpenShell](#) sandbox 内运行 always-on AI Agent 提供的 TypeScript CLI 控制面：它把 guided onboarding、gateway、OpenClaw / Hermes agent 配置、routed inference、网络策略、凭证迁移、sandbox 生命周期和 e2e 验证收束到一个命令面。它更接近 agent-sandbox 与 [agentgateway](#) 之间的 host-side 编排层，而不是新的推理引擎；推理侧通过 inference.local 接入 NVIDIA/OpenAI/Anthropic/Gemini/vllm/Ollama 等 provider。

核心架构图







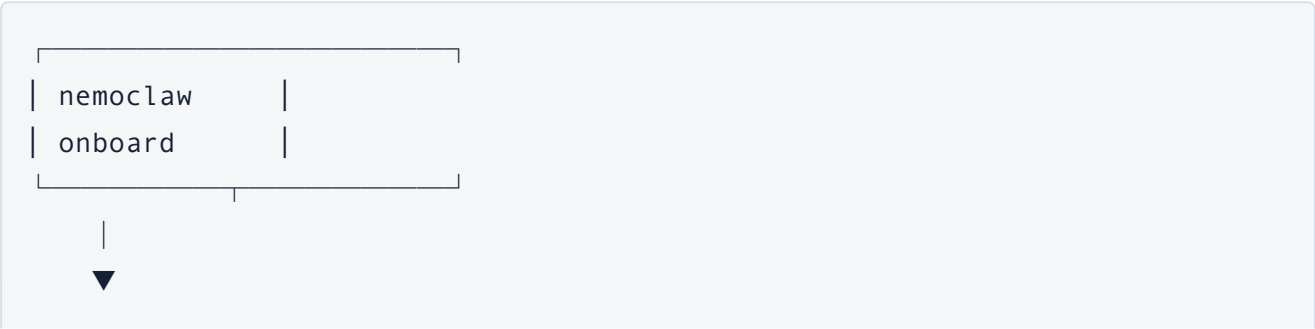
模块分层

层 / 模块	职责
CLI 入口与公共语法	兼容 nemoclaw <sandbox> <action>, 做 argv normalization、route translation、sandbox 名检查, 再转给 oclif。
命令层	src/commands/** 只解析 flags/args 并调用 action, 不承载产品行为。
Action / onboard 编排层	把命令变成 preflight、gateway、provider/inference、sandbox、agent setup、policy 的 host-side workflow。
领域规划层	provider 映射、policy/lifecycle 分类、messaging manifest 编译等尽量保持纯函数或可测试规划逻辑。
外部边界适配	封装 OpenShell、Docker、HTTP、shell 子进程和 privileged sandbox exec。
状态与凭证	~/ .nemoclaw JSON registry/session/shields 状态; 凭证只在进程 env staging, gateway 是 system of record。
安全与策略	baseline deny-by-default policy、policy presets、shields up/down/status、redaction/hash。
Messaging 扩展	channel manifest → serializable plan → OpenShell provider/policy/agent config applier。
推理与模型路由	inference.local 统一 sandbox 内出口, host/gateway 侧映射到 cloud 或 local provider。
Blueprint 与测试	OpenShell sandbox 资产、policy preset、runtime scripts、unit/e2e/e2e-scenario 验证矩阵。

核心分层约束是"薄命令 + 厚编排 + 纯规划 + 外部边界适配"。这与 [ai-agent-plugin-patterns](#) 里的接口化、边界清晰、可迁移能力包方向一致, 也和 agent-credential-isolation 的凭证边界设计相互呼应。

关键数据流

Onboard 主流程



```
| src/lib/onboard.ts:onboard |
```

- ```
| 1. branding/env/options: agent, non-interactive, resume, fresh |
```
- ```
| 2. validate sandbox name + usage notice + provider hint |
```
- ```
| 3. acquire onboard lock + load/create resumable session |
```
- ```
| 4. stage legacy credentials into process.env for gateway migration |
```
- ```
| 5. select agent: OpenClaw / Hermes |
```
- 



---

```
| state handlers |
```

```
| handlePreflightState |
```

```
| └ Docker/OpenShell/GPU/DNS/platform checks |
```

```
| handleGatewayState |
```

```
| └ start/reuse/recover named OpenShell gateway |
```

```
| handleProviderInferenceState |
```

```
| └ provider/model/credential route; model router/local proxy |
```

```
| handleSandboxState |
```

```
| └ create/reuse/recreate sandbox, persist registry/session |
```

```
| handleAgentSetupState |
```

```
| └ install/sync OpenClaw or Hermes runtime config |
```

```
| handlePoliciesState |
```

```
| └ apply baseline + selected policy presets |
```

---



---

```
| OpenShell gateway + sandbox side effects |
```

```
| openshell gateway start/reuse |
```

```
| openshell provider create/update --credential <ENV_KEY> |
```

```
| openshell sandbox create/connect/exec |
```

```
| openshell policy set |
```

---

```
| ~/.nemoclave/sandboxes.json + session files |
```

Onboard 是可恢复的 host-side 状态机，不是一次性安装脚本。它支持 resume/fresh/non-interactive、全局 lock、step 记录和失败恢复；近期提交也集中在把 provider inference、sandbox branch、OpenClaw setup 等分支抽成 FSM result。

## Messaging channel 编译与应用

```
| channel manifests |
| telegram/slack/... |
```

```
| registry.list()
```



```
| MessagingWorkflowPlanner |
| - de-duplicate configured channels |
| - reject unsupported channels for selected agent |
| - build ManifestCompilerContext |
```



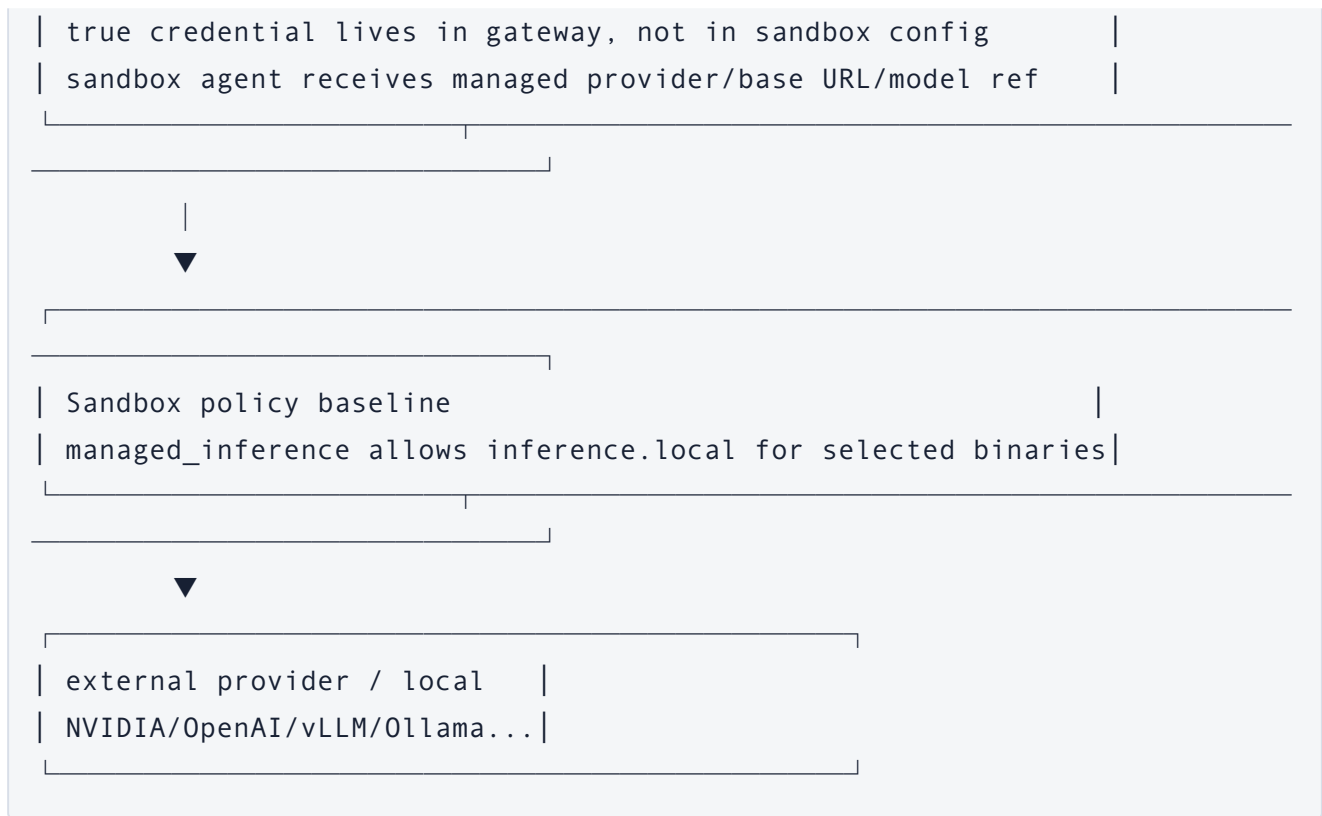
```
| ManifestCompiler.compile |
| |
| resolve manifests |
| compile per-channel inputs + enrollment/reachability hooks |
| fan out into: |
| credentialBindings |
| networkPolicy |
| agentRender |
| buildSteps |
| stateUpdates |
| healthChecks |
```



Messaging 子系统的核心是 declarative manifest compiler: channel 不直接侵入 onboard 主流程，而是声明 secret/config input、policy preset、agent render、hook phase 和 health check，再编译成 serializable plan。

## 推理路由与凭证边界





这里的设计重点是 agent-credential-isolation: sandbox 看到的是 `inference.local` 和 `managed provider/model ref`, 真正 provider credential 由 [OpenShell](#) gateway 托管。local Ollama/vLLM 还有专用 token env, 避免误用 host OpenAI key。

## 设计决策与哲学

- 命令面兼容产品语法, 但不复制业务层: public dispatcher 负责 `nemoclave <sandbox> <action>` 到 oclif command 的转换, 产品行为仍在 actions/onboard。
- Onboard 是可恢复状态机: resume/fresh/non-interactive、session、lock、step result 和 failure marking 都是一等流程, 而不是 shell script 串联。
- Declarative assets + host orchestrator: `nemoclave-blueprint/**` pin 住 sandbox image digest、OpenShell/OpenClaw 版本、inference profile、router 和 policy; TypeScript 负责消费和编排。
- 凭证由 gateway 托管: NemoClaw 只做当前进程 env staging 与 legacy migration, 新凭证不写本地明文文件。
- 网络默认最小化: baseline policy 只开放核心 agent、managed inference 和必要服务; GitHub、messaging、package index 等走用户显式 policy preset。
- Shields 只能 host-side 控制: sandbox 不能自己降级或升级 shields; host 通过 privileged path 做 config lock、policy 切换和 hash seal。
- Messaging 走 manifest compiler: 新增 channel 的主要扩展面是 manifest + hook, 而不是修改 onboard 主流程。
- 本地 JSON 状态优先: `~/.nemoclave registry/session` 降低部署复杂度, 但用 advisory lock、resume 和 recovery 补齐一致性。

## 关键组件深入解读

### Onboard 编排器

`src/lib/onboard.ts` 是复杂度中心，但它更像 coordinator：初始化运行模式、校验 sandbox 名、获取 consent、抢 lock、加载/创建 session、迁移 legacy credentials，再顺序调用 `preflight/gateway/provider/sandbox/agent/policy` 状态处理器。近期 git 历史显示维护方向是在保留一个端到端入口的同时，把巨型分支拆成 FSM result 和可测试状态模块。

### Messaging manifest compiler

Messaging compiler 对每个 channel manifest 做 input resolution、credential availability、enrollment/reachability hook，然后 fan out 成 credential binding、network policy、agent render、build step、state update 和 health check。Applier 接受 serializable `SandboxMessagingPlan`，说明 plan 是跨 host/OpenShell/sandbox 边界传递的 contract，而不是运行时对象。

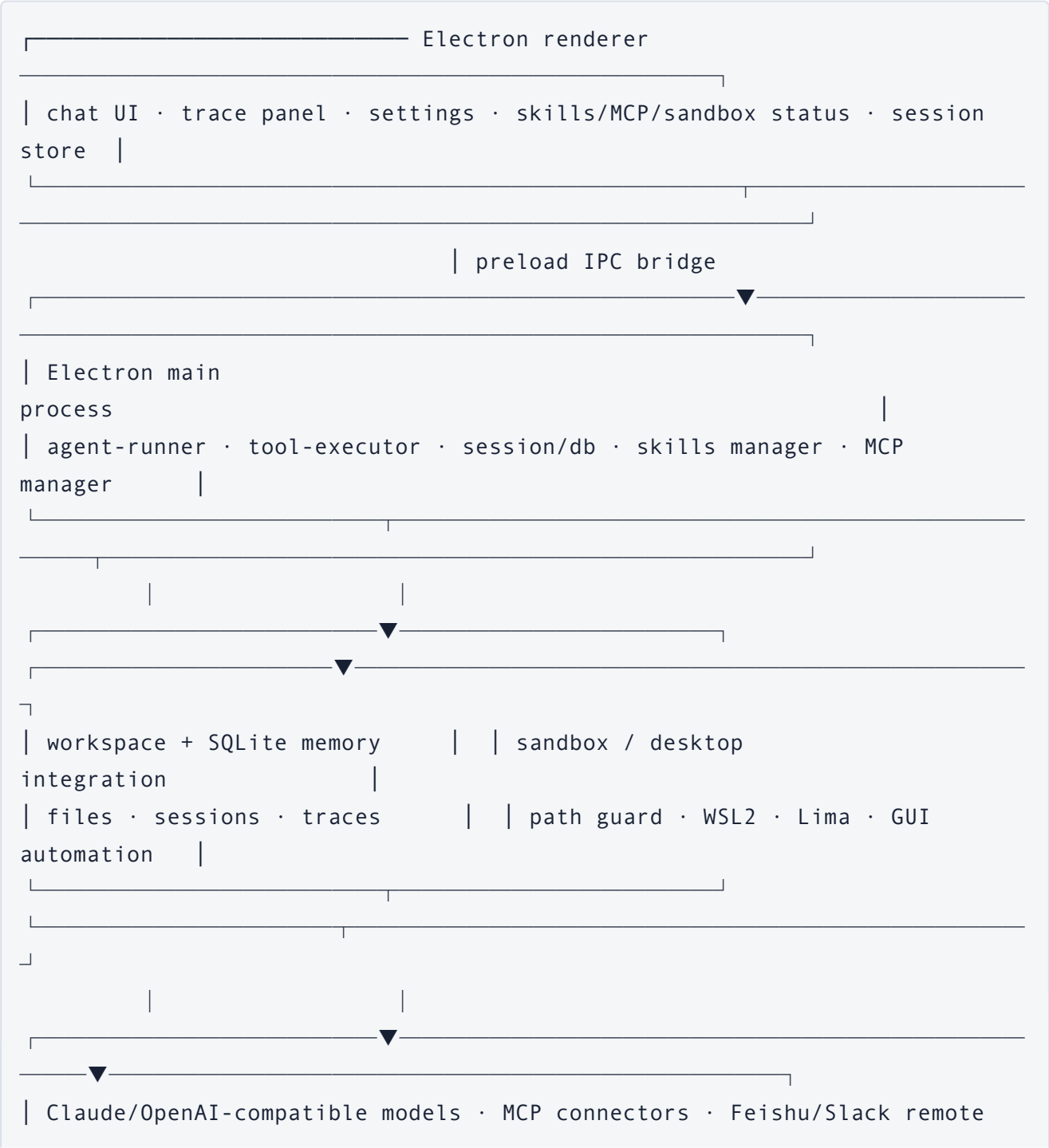
## 相关页面

- [agent-sandbox](#)
- [agentgateway](#)
- [agent-credential-isolation](#)
- [cloud-native-security](#)
- [vllm](#)
- [ai-agent-plugin-patterns](#)
- [NVIDIA](#)
- [OpenShell](#)
- [OpenClaw](#)
- [Hermes](#)

# Open Cowork 架构与设计思路分析

OpenCoworkAI/open-cowork 是桌面端 agent host，而不是 terminal coding agent。它用 Electron main/preload/renderer 包装模型 API、Claude/Pi-compatible agent runner、SQLite/session、Skills、MCP、workspace 文件管理、WSL2/Lima 沙盒和远程控制，让非开发者通过 GUI 使用 AI 自动化文件、文档和桌面应用。

## 核心架构图



input |

## 模块分层

| 层/目录                                                    | 责任                                                |
|---------------------------------------------------------|---------------------------------------------------|
| src/main/claude/agent-runner.ts                         | agent 执行核心，测试覆盖 history rebuild 和 SDK session 恢复。 |
| src/main/sandbox/**                                     | path guard 与 WSL/Lima sandbox 相关逻辑。               |
| src/main/skills/**, .claude/skills                      | 技能加载、安装、校验、启停。                                    |
| src/main/memory/**, src/main/session/**, src/main/db/** | SQLite/session/memory 管理。                         |

## 关键数据流

- 用户在 GUI 选择 workspace 并输入任务，renderer 通过 preload IPC 调用 main process。
- agent-runner 组装模型、历史、技能、MCP 工具和 workspace 上下文，产生 trace/tool events。
- 文件/命令工具先过 workspace path guard；若 WSL2/Lima 可用，bash 路由到 VM，否则回退本机受限执行。

## 设计决策

- 桌面 app 优先降低使用门槛，代价是 Electron/IPC/OS sandbox/installer 复杂度。
- 安全模型是多级：path guard 是基础，WSL2/Lima 是增强；README 也提醒仍需审查高风险操作。
- Skills 处理文档/PPT/XLS/PDF 这类桌面生产力任务，区别于 coding-agent 纯代码 workflow。

## 对比定位

和 Multica 相比，Open Cowork 是单用户桌面入口；和 Codex/Pi 相比，它不是终端执行 loop，而是 GUI host；和 OpenShell/agent-sandbox 相比，它有 VM 级隔离但不是 K8s/runtime primitive。

## 相关链接

- GitHub 当前状态底稿：src-github-stars-backlog-current-state
- 选型地图：github-stars-backlog-implementation-map

# OpenAI Codex CLI

OpenAI 官方 terminal coding agent，重点是把仓库上下文、工具调用、审批/沙箱、patch 和会话状态组织成一个本地开发循环。详见 [src-codex-architecture](#)。

## 架构边界

Codex CLI 是执行 loop 本体：它负责读写仓库、调用工具、维护 session/app-server/MCP 等运行状态。它不是消息平台、trace viewer 或 token 报表工具；这些能力分别由 cc-connect、claude-tap、tokscale 这类生态配套补齐。

## 选型判断

| 场景                                     | 判断                                                                   |
|----------------------------------------|----------------------------------------------------------------------|
| 官方 OpenAI coding agent                 | 优先看 Codex CLI                                                        |
| 想在 Claude Code 内调用 Codex review/rescue | 看 <a href="#">codex-plugin-cc</a> 和 <a href="#">agent-delegation</a> |
| 想观察请求上下文                               | 看 claude-tap                                                         |
| 想统计用量成本                                | 看 tokscale                                                           |

## 相关源码页

- [src-codex-architecture](#)
- [src-codex-plugin-cc-architecture](#)

# OpenKruise Agents

OpenKruise Agents 是 OpenKruise 面向 AI Agent sandbox lifecycle management 的 Kubernetes 原生平台。它把 Sandbox / SandboxSet / SandboxClaim / SandboxTemplate CRD、E2B-compatible API、Envoy 路由扩展、agent-runtime sidecar、identity token 和动态 CSI mount 组合成一套“在 K8s 上运营 Agent sandbox”的控制面。

详细架构见 [src-openkruise-agents-architecture](#)。

## 关键事实

- 仓库: [openkruise/agents](#)
- 当前核验: GitHub API 显示默认分支 master, 最近 push 2026-06-12, 约 202 stars / 78 forks, Apache-2.0, 主语言 Go
- 本 wiki 分析版本: HEAD 0e58df8 (2026-06-12)
- 核心入口: agent-sandbox-controller、sandbox-manager、traffic-extension、sandbox-gateway
- 关键 API: Sandbox、SandboxSet、SandboxClaim、SandboxTemplate
- 产品入口: E2B-compatible / sandboxes、/ snapshots、pause/resume/connect/timeout/debug 等 API
- 典型场景: code interpreter、cloud desktop、Claude Code / OpenClaw sandbox、RL/HITL/open-world training、长寿命 workspace

## 架构定位

### E2B 请求到 Sandbox 的流程

```
E2B API / CRD request → sandbox-manager
├─ create / claim / clone / checkpoint
└─ route + identity + key storage
 ↓
 SandboxClaim / SandboxSet / Sandbox
 ↓
 agent-sandbox controller
 ↓
 Pod + PVC + agent-runtime / CSI
 ↓
 Envoy ext_proc → Sandbox Pod IP:port
```

OpenKruise Agents 处在 agent-sandbox 和 [agentcube](#) 中间偏平台化的一层：

- 比 agent-sandbox 更完整：不仅有 sandbox lifecycle primitive，还把 warm pool、claim、E2B API、路由、identity、CSI 和 runtime sidecar 接起来。
- 比 [agentcube](#) 更底层：不主打 AgentRuntime / CodeInterpreter invocation 产品抽象，而是提供更通用的 sandbox 管理、clone/checkpoint、route 和 E2B 兼容面。
- 比 substrate 更 K8s CRD 化：Substrate 聚焦 actor/worker multiplexing 和 wake routing；OpenKruise Agents 聚焦可声明、可预热、可暂停/恢复、可路由的 sandbox。
- 可作为 Agent framework 的远端 workspace backend：AgentScope、coding agent、browser/desktop agent 可以把它当作 E2B-like 执行环境。

## 核心组件

| 组件                             | 作用                                                                                                                      |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| agent-sandbox-controller       | K8s controller-runtime 控制面，管理 Sandbox、SandboxSet、SandboxClaim、SandboxUpdateOps、SecurityTokenRefresh。                    |
| sandbox-manager                | E2B-compatible HTTP API 控制面，负责 claim/clone/pause/resume/delete checkpoint、API key storage、team permission 和 route sync。 |
| traffic-extension              | Envoy external processing gRPC 服务，把 sandbox 请求改写为 original-dst 路由。                                                      |
| sandbox-gateway                | Envoy Go HTTP filter 版本的数据面扩展，解析 sandboxID/port 并写 dynamic metadata。                                                    |
| sandboxcr infra                | Infrastructure / Sandbox 抽象的 K8s CRD 实现，把 API 请求落到 SandboxClaim/Sandbox/PVC/Pod。                                        |
| agent-runtime / identity / CSI | sandbox 内 runtime 初始化、短期 token 传播、动态 volume mount 的扩展点。                                                                 |

## 关键判断

- Warm pool 是控制面协议：SandboxSet 保持可用库存，SandboxClaim 显式领取；这比简单 Pod pool 更适合并发、超时、失败保留和滚动更新。
- E2B API 降低接入成本：上层 SDK 可以用 familiar API 创建、暂停、恢复、连接 sandbox；平台侧仍保留 K8s CRD/GitOps/RBAC 能力。
- 路由不走 Service per sandbox：Envoy ext\_proc / Go filter 用 sandboxID 查 route registry，把流量导到 Pod IP:port，适合大量短会话/长会话混合的 sandbox。
- 安全边界要继续补齐：identity provider、API key storage、token propagation、runtimeClass、NetworkPolicy 和 audit 是生产采用前必须验证的部分。

## 什么时候优先看它

- 需要在 K8s 上搭建 E2B-like sandbox service;
- 希望同时拥有 CRD 管理面和 SDK/API 产品面;
- 需要 warm pool 降低 Agent / Code Interpreter 冷启动;
- 需要 pause/resume、checkpoint/clone、动态存储挂载、路由和多租户 namespace/team;
- 想把 AI Agent workspace 从本地/Docker/E2B SaaS 迁到自有 Kubernetes。

## 什么时候不适合

- 只需要一个本地 Agent 框架或 Python workflow runtime;
- 只想学习最小 sandbox CRD，不想同时理解 manager/gateway/identity/CSI;
- 对强 runtime enforcement 的要求高于生命周期管理，此时还要叠加 cloud-native-security、OPA/egress gateway、gVisor/Kata/seccomp 等机制;
- 当前业务不能接受快速演进 API 和 roadmap 中尚未完成的 storage/network/runtime/scheduling/observability 能力。

# OpenKruise Agents 架构与设计思路分析

原文: raw/openkruise-agents-architecture-analysis.md · 仓库: <https://github.com/openkruise/agents> · 分析版本 HEAD 0e58df8 (2026-06-12)

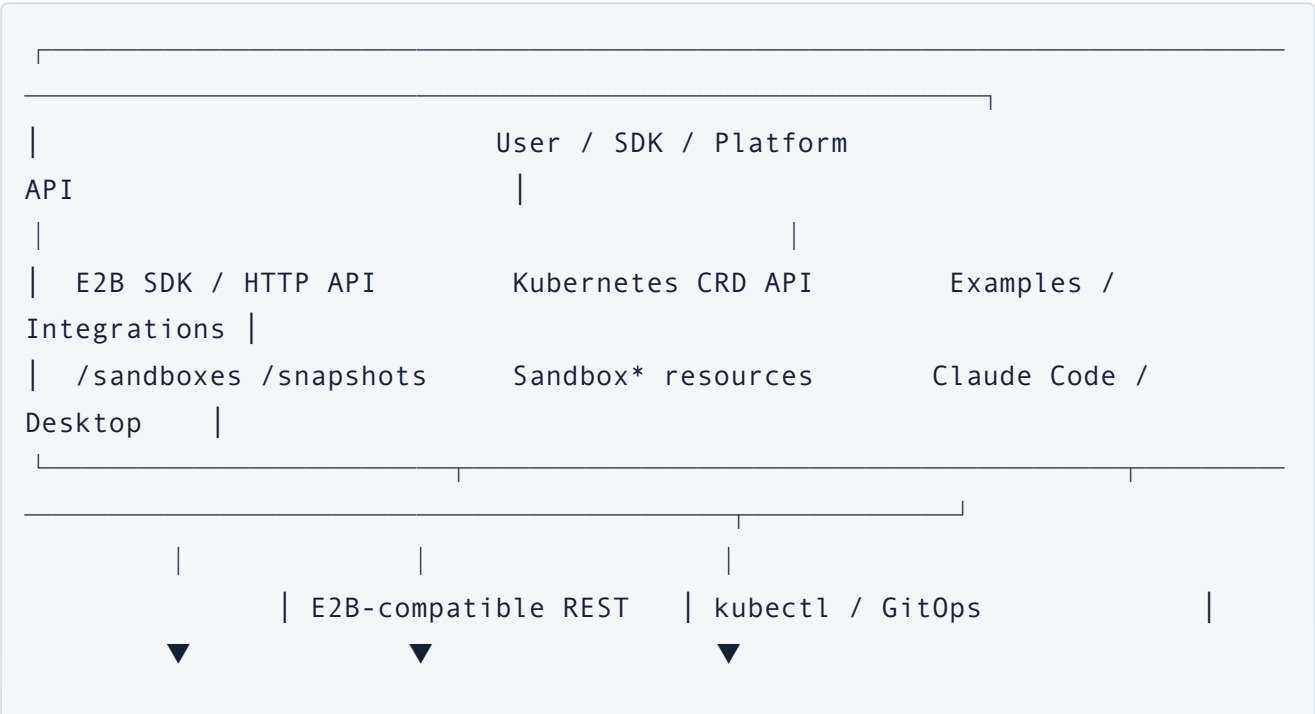
## 一句话定位

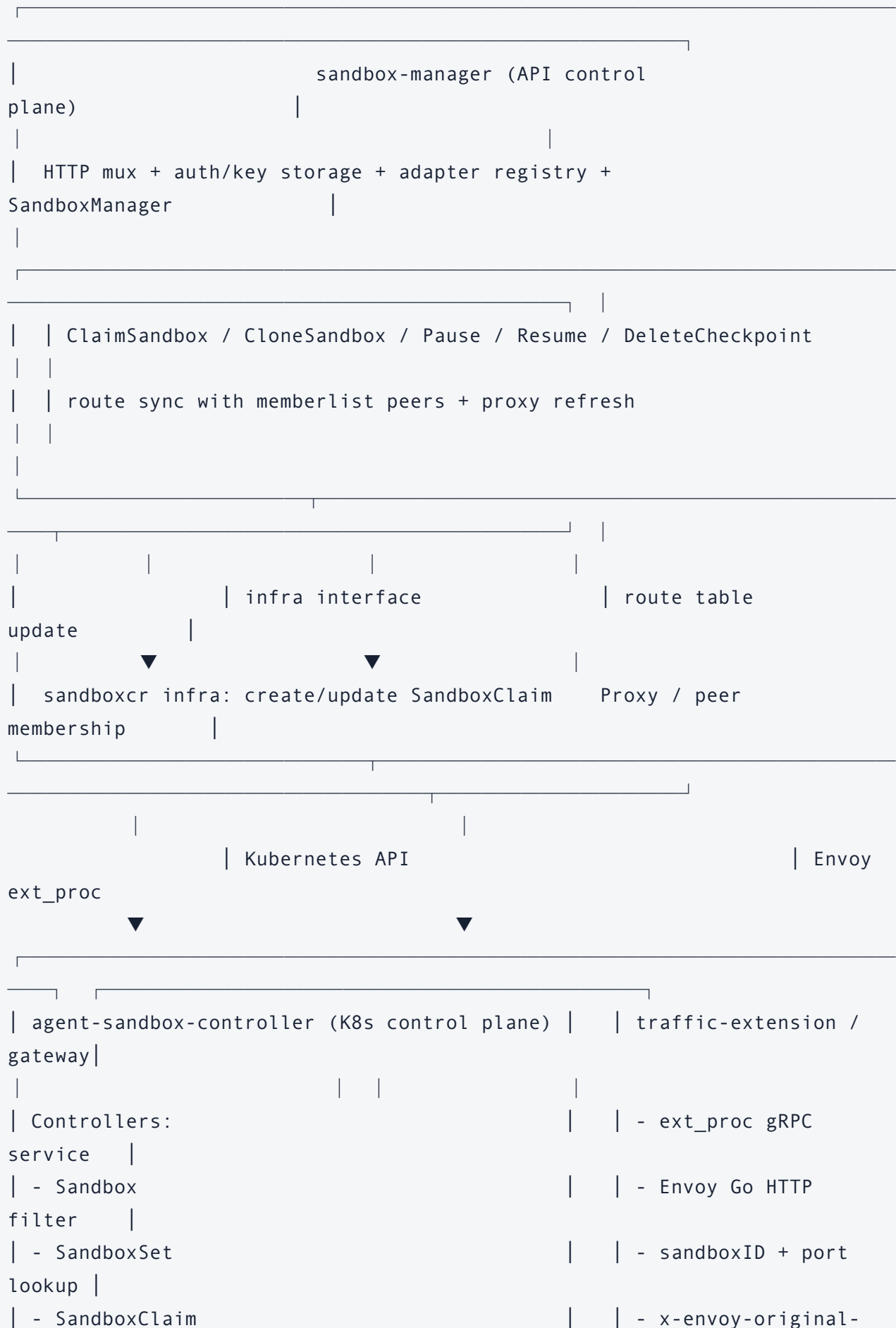
openkruise/agents 是 OpenKruise 面向 AI Agent sandbox lifecycle management 的 Kubernetes 原生平台层。它不只是一个 Sandbox CRD controller，而是把四类能力放在同一个工程里：

- 用 Sandbox / SandboxSet / SandboxClaim / SandboxTemplate 表达可暂停、可预热、可领取、可升级的 Agent sandbox。
- 用 agent-sandbox-controller 管 Pod/PVC/template/hash/status/rolling update/warm pool 等生命周期。
- 用 sandbox-manager 暴露 E2B-compatible API，把“创建 sandbox / clone checkpoint / pause / resume / route”变成产品 API。
- 用 traffic-extension / sandbox-gateway 接入 Envoy，把外部请求按 sandbox identity 路由到具体 Pod IP 和端口。

它和 agent-sandbox 的关系更像“上层发行版 + 兼容实现 + 产品化控制面”：README 明确说明会保持与 SIG agent-sandbox API 的兼容，同时在 SIG API 可用前内置 sandbox API 和实现。

## 核心架构图







# 模块分层

| 层 / 模块        | 职责                                                                                                                                                  |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| CRD API       | 定义 Sandbox、SandboxSet、SandboxClaim、SandboxTemplate，覆盖 pause、persistent contents、runtimes、claim、template、upgrade、dynamic volume mount。               |
| K8s 控制面       | controller-runtime manager；注册 Sandbox/SandboxSet/SandboxClaim/SandboxUpdateOps/SecurityTokenRefresh controller、webhook、field index、metrics cleanup。 |
| API 控制面       | 暴露 E2B-compatible API；把 request adapter、auth/key storage、infra backend、memberlist peer 和 route proxy 组合成 sandbox 管理服务。                              |
| Sandbox infra | 定义抽象 Infrastructure / Sandbox 接口；sandboxcr 实现用 K8s CRD 完成 claim、clone、pause/resume、checkpoint、CSI mount、identity token。                             |
| 路由数据面         | Envoy ext_proc 或 Go HTTP filter；从请求解析 sandboxID/port，查 route registry，写入 original-dst 元数据并做 peer 同步。                                                |
| Runtime 扩展    | agent-runtime sidecar、CSI volume mount provider、identity provider 和 token propagation 扩展点。                                                          |
| 示例与集成         | code interpreter、desktop、OpenClaw、Claude Code 等场景，展示如何把热池、E2B SDK、sandbox image 和 session 复用组合起来。                                                   |

## 关键对象

### Sandbox

Sandbox 是单个运行环境的期望状态。重要字段包括 paused、persistentContents、shutdownTime / pauseTime、runtimes、lifecycle、upgradePolicy、template / templateRef / volumeClaimTemplates。它把暂停、恢复、持久文件系统、runtime sidecar、动态存储和升级策略纳入同一个 K8s 对象。

### SandboxSet

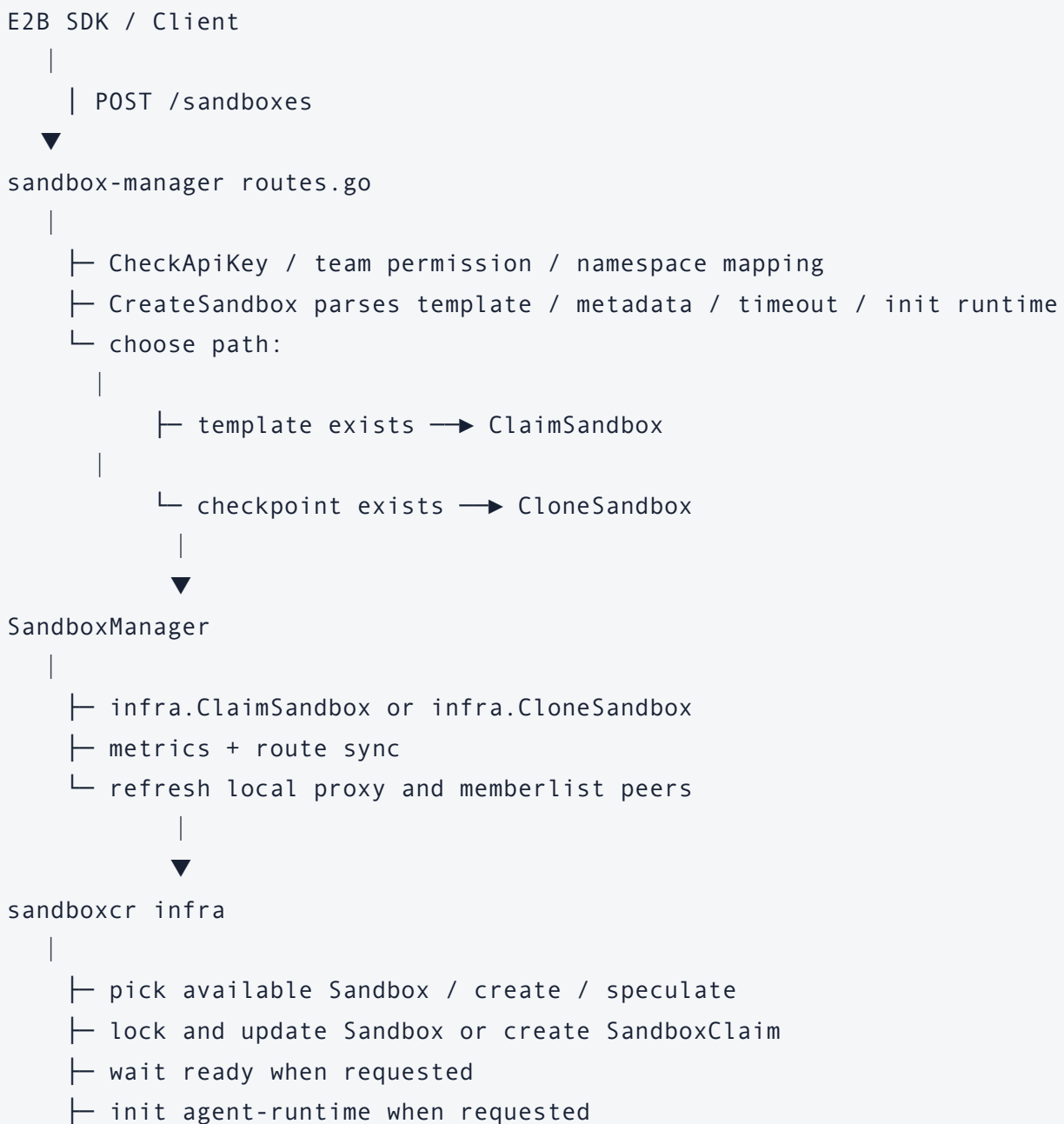
SandboxSet 维护一组“未被 claim 的可用 sandbox”，本质是 warm pool 和滚动更新控制器。replicas 表示需要保持的 unused sandboxes 数量，status 跟踪 available、updated、current/update revision，labels/ annotations 则承载 pool、template、claim、owner、hash、lock、restore-from、init-runtime request、memberlist url 等协议状态。

## SandboxClaim

SandboxClaim 是把“用户请求一个 sandbox”变成控制面协议的关键对象。它通过 `templateName` 选择 `pool/template`，用 `createOnNoStock` 控制无库存时是否冷创建，用 `inplaceUpdate` 支持 `image/CPU` 更新，用 `dynamicVolumesMount` 支持请求时 CSI 挂载，并用 `reserveFailedSandbox` 支持失败现场保留。

## 关键数据流

### E2B API 创建 sandbox

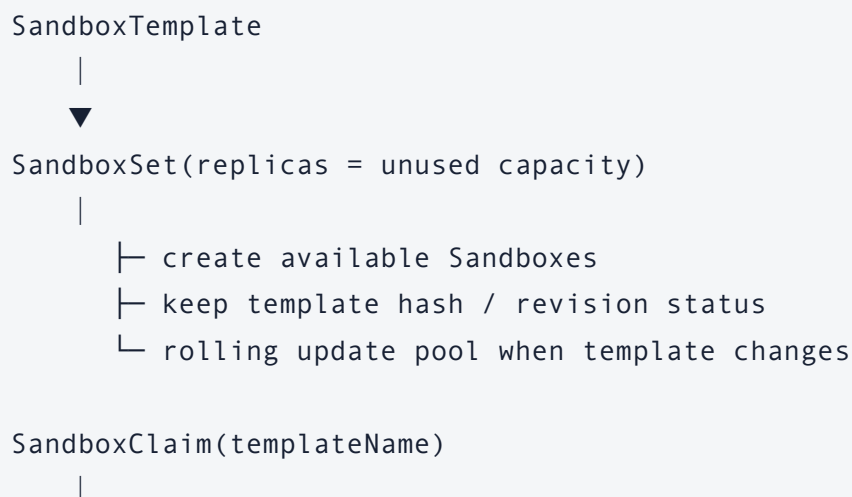


- └─ issue and propagate identity token when feature enabled
- └─ mount CSI dynamic volumes when requested

## 请求路由到 sandbox



## Warm pool / claim 生命周期



```

└─ select available Sandbox from matching SandboxSet
└─ lock and mark claimed / owner / timestamp
└─ apply labels, annotations, env vars and inplace update
└─ optionally initialize runtime and dynamic CSI mount
└─ wait ready -> Completed

```

## 设计决策

- 把 warm pool 做成声明式资源，而不是进程内队列：SandboxSet 让“保持 N 个可用 sandbox”由 K8s controller 持续收敛；SandboxClaim 则把领取动作显式化，便于处理并发、超时、失败保留和 owner/label 不变量。
- E2B API 是产品入口，CRD 是平台入口：开发者可以通过 E2B SDK 创建/暂停/恢复 sandbox，平台团队也可以通过 Sandbox\* CRD 做 GitOps、RBAC、namespace、多租户和控制面调试。
- 数据面和控制面解耦：sandbox-manager 负责创建、clone、pause、resume 和 route sync；Envoy ext\_proc / Go filter 只负责把请求映射到已知 sandbox route。高频请求不直接打 Kubernetes API。
- runtime 扩展是可插拔而不是硬编码：runtimes 字段和 agent-runtime、csi、traffic-proxy config 让 sidecar、动态存储、流量代理成为可声明能力；identity provider 接口也允许社区默认 token 与企业级 provider 分开演进。
- 身份安全还在工程化演进中：最近提交把 secret-backed key storage 的 ticker 刷新改成 informer-driven refresh，说明 E2B API key / token 生命周期已经成为真实控制面问题。

## 与同类项目的区别

| 项目                         | 主要抽象                                                 | 与 OpenKruise Agents 的区别                                                                                                                              |
|----------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| agent-sandbox              | Sandbox CRD 原语                                       | 更偏 SIG 级基础 API；OpenKruise Agents 把 warm pool、E2B API、route proxy、identity/CSI/runtime 扩展也产品化进同一发行版。                                                  |
| <a href="#">agentcube</a>  | AgentRuntime / CodeInterpreter session orchestration | AgentCube 用 Router + WorkloadManager 把 agent-sandbox 包成 invocation API；OpenKruise Agents 直接提供 E2B-compatible API 和更完整的 sandbox lifecycle/runtime 扩展。 |
| substrate                  | WorkerPool / ActorTemplate 高密度 actor substrate       | Substrate 更强调 actor multiplexing、wake routing、snapshot；OpenKruise Agents 更贴近 K8s CRD + E2B sandbox 管理。                                               |
| <a href="#">agentscope</a> | AgentApp / Runner / deployer                         | AgentScope Runtime 是 Python app 服务化壳；OpenKruise Agents 是 Kubernetes sandbox lifecycle substrate，可作为 workspace/E2B 后端。                                |
| <a href="#">OpenShells</a> | Gateway + supervisor + policy proxy                  |                                                                                                                                                      |

| 项目 | 主要抽象 | 与 OpenKruise Agents 的区别                                                                                      |
|----|------|--------------------------------------------------------------------------------------------------------------|
|    |      | OpenShell 的强项是 sandbox 内 runtime enforcement 和 policy;<br>OpenKruise Agents 的强项是 K8s 生命周期、热池、E2B API 和路由控制面。 |

## 采用建议

- 需要“在 Kubernetes 上提供 E2B-like sandbox 服务”时，优先看 [openkruise-agents](#)。
- 只想学习最小 CRD 原语时，先看 agent-sandbox；OpenKruise Agents 需要同时理解 controller、manager、gateway、runtime sidecar 和 identity/CSI。
- 已经有 Agent framework、想接远端 workspace/sandbox backend 时，重点验证 E2B API 兼容范围、template 管理、timeout、文件/命令 API 和路由行为。
- 高风险不可信代码执行仍要额外验证 runtimeClass、NetworkPolicy、seccomp/AppArmor、egress gateway、token revoke 和 audit；OpenKruise Agents 提供扩展点，但不是自动替代强 runtime enforcement。

## 风险与待验证点

- API 仍处在快速演进期，README/roadmap 显示 pool、storage、network、runtime、scheduling、observability、SDK 都有大量后续事项。
- SandboxClaim 并发领取、create-on-no-stock、speculate、lock 和 expectations 是正确性关键，需要用压力测试验证。
- 路由表同步依赖 manager/gateway peer 状态，生产要验证实例重启、网络分区和 route stale 时的 fail-close 行为。
- checkpoint/hibernation 的 memory/filesystem/GPU memory 能力要按具体 runtime 和存储后端验证，不能只看字段设计。

# OpenShell

OpenShell 是 NVIDIA 面向 autonomous AI Agent 的安全私有运行时。它把 agent 放进可审计、可连接、可恢复的 sandbox 中运行，并把 Gateway 控制面、sandbox supervisor enforcement、OPA/Z3 policy pipeline、provider credential 管理和 `inference.local` 推理路由拆成清晰边界。详见 [src-openshell-architecture](#)。

## 架构边界

OpenShell 不是 Agent framework。它不负责定义 ReAct loop 或多 Agent 工作流，而是负责让这些 agent 在受控环境中执行：Gateway owns desired state，Supervisor owns runtime enforcement。

## 核心组件

- Gateway control plane：管理 sandbox、provider、policy、settings、inference、session 和 compute orchestration。
- Runtime driver：把 Docker、Podman、Kubernetes、VM/libkrun 等后端收敛成统一 compute driver。
- Sandbox supervisor：在 sandbox 内准备 filesystem、process limits、network namespace、TLS、proxy、SSH 和 agent child。
- Policy proxy：用 OPA 做 L4/L7 网络裁决，绑定进程身份并做 SSRF 防护。
- `inference.local`：由 sandbox proxy 拦截并路由到 provider，避免 agent 直接持有真实模型凭证。

## 选型判断

适合把本地或托管 coding agent 放到强安全边界里运行。和 agent-sandbox 相比，OpenShell 更偏完整安全 runtime；和 substrate 相比，它不追求大规模 actor snapshot multiplex；和 [agentgateway](#) 相比，它治理运行时与网络 enforcement，而不是集群入口网关。

# OpenShell 架构与设计思路分析

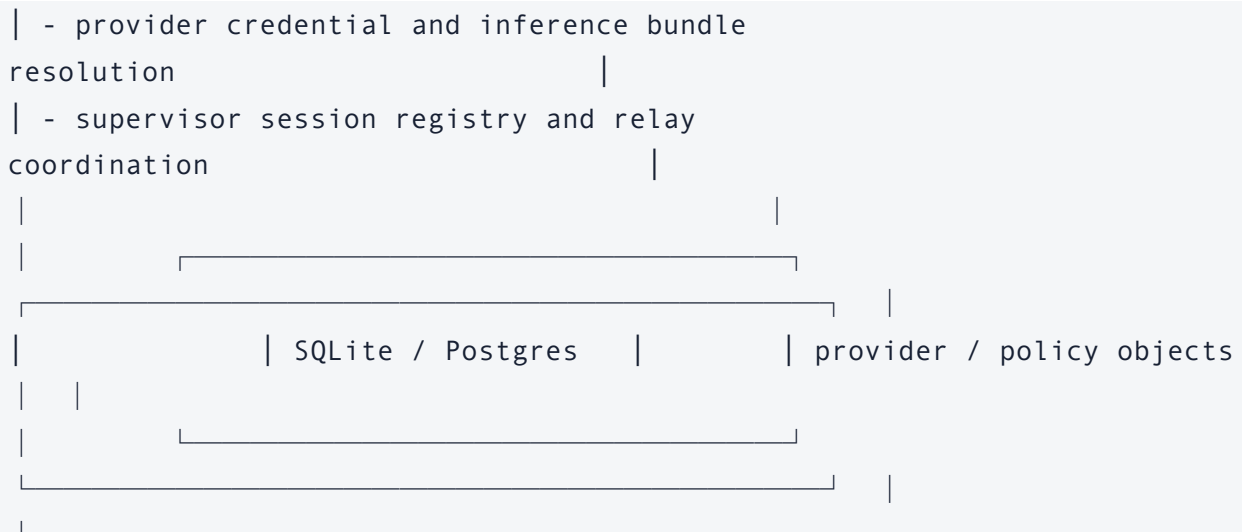
原文: raw/openshell-architecture-analysis.md · 仓库: <https://github.com/NVIDIA/OpenShell> · 分析版本 HEAD 97986d9

## 一句话定位

[OpenShell](#) 是 NVIDIA 面向 autonomous AI Agent 的安全私有运行时：它不是新的 Agent framework，而是把 Agent 放进可审计、可配置、可连接、可恢复的 sandbox 里运行。它的关键手段是把 Gateway 控制面、sandbox 内 Supervisor enforcement、OPA/Z3 policy pipeline、provider credential 管理和 `inference.local` 推理路由拆成清晰边界；这也解释了它为什么能和 [NemoClaw](#)、agent-sandbox、[agentgateway](#) 互补。

## 核心架构图







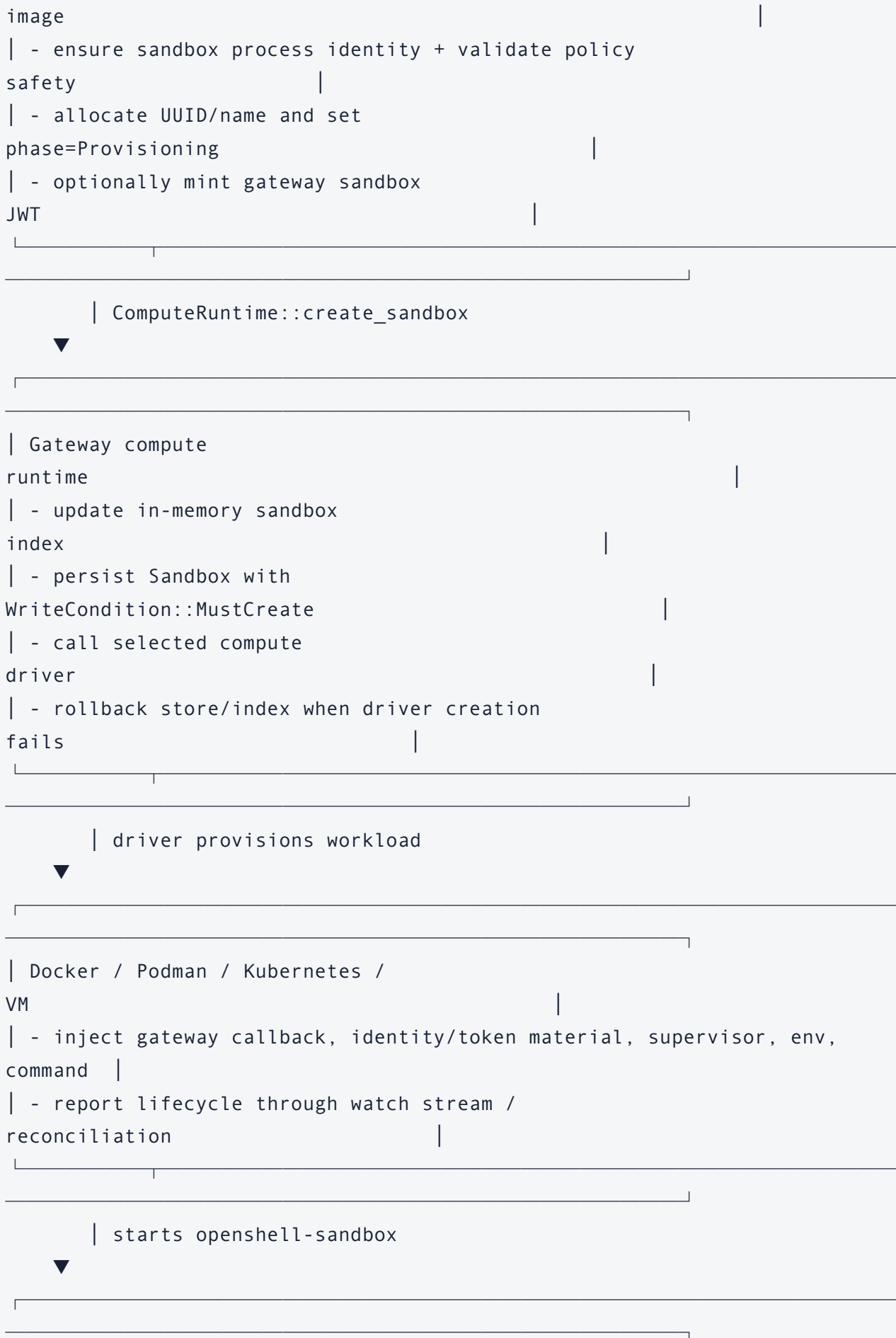
## 模块分层

| 层 / 模块               | 职责                                                                                                                       |
|----------------------|--------------------------------------------------------------------------------------------------------------------------|
| 用户入口                 | CLI/SDK/TUI 解析 gateway context、sandbox name、auth token，把用户意图变成 Gateway API 调用。                                           |
| Gateway 控制面          | API、认证、持久状态、policy/settings/provider/inference 配置、compute orchestration、relay 协调。                                        |
| 持久化                  | protobuf object store，SQLite/Postgres 后端，resource_version CAS；生产写路径必须显式使用 WriteCondition 或 update_message_cas。           |
| Compute runtime      | 抽象 Docker/Podman/Kubernetes/VM driver；负责 create/delete/watch/reconcile/startup resume。                                   |
| Sandbox supervisor   | sandbox 内安全边界：加载 policy/settings、准备 filesystem/process/netns/TLS、启动 proxy/SSH、拉取 provider env、启动 agent child、热更新 policy。 |
| Policy proxy         | 用 OPA 做 L4/L7 网络决策，用 /proc/net/tcp 绑定调用进程身份，做 SSRF 防护、TLS/L7 inspection、凭证 rewrite、inference.local 路由。                   |
| Provider / inference | provider discovery、credential 注入、OpenAI/Anthropic/NVIDIA 等模型路由；sandbox 内 inference.local 不走普通 OPA network policy。        |

OpenShell 的分层约束很强：Gateway owns desired state，Supervisor owns runtime enforcement。Gateway 不做 agent egress 的 per-request 裁决，真正的 安全 enforcement 放在 sandbox 内，那里能看到进程身份、文件系统、网络命名空间、TLS 流和 runtime credentials。

## 关键数据流







创建路径体现了 OpenShell 的一致性优先：Gateway 先验证 spec/policy、生成 sandbox identity 和 phase，再由 compute runtime 以 MustCreate 写入 store，随后才调用具体 driver；driver 失败时会回滚 store/index，避免留下“半创建”的 sandbox。Supervisor 启动则是另一条安全 pipeline：加载 policy/OPA、拉 provider env、准备 filesystem/netns/seccomp/TLS、启动 proxy/SSH、建立 outbound session，最后才 spawn restricted agent child。

## 设计决策与哲学

- 控制面和 enforcement 分离：Gateway 存储和分发策略，但请求时裁决放在 sandbox 内 Supervisor/proxy/kernel controls，因为那里能绑定 binary identity、TLS/L7 和 runtime credentials。
- Supervisor 主动连 Gateway：sandbox 通过 outbound ConnectSupervisor 持有 live session，Gateway 的 connect/exec/file sync 通过 relay 协调，不要求每种 runtime 暴露可直连地址。
- driver 薄、语义厚：Docker/Podman/Kubernetes/VM driver 只翻译平台操作；sandbox lifecycle、phase、policy/settings/provider/inference 是 OpenShell 自己的语义层。
- 持久状态按对象存储 + CAS：状态对象统一进 protobuf object store，生产写路径通过 CAS 或 MustCreate，降低并发 Gateway handler/reconciler 互相覆盖的风险。
- 网络策略绑定进程身份：proxy 通过 /proc/net/tcp 解析 socket owner，把 binary path/hash/ancestor/cmdline 放进 OPA input，避免一个 sandbox 内所有进程共享同一网络权限。
- 推理路由作为特殊本地能力：https://inference.local 由 sandbox proxy TLS terminate 并转给 openshell-router，避免 agent 直接持有 provider credentials 访问模型端点；这和 agent-credential-isolation 的思想一致。
- policy proposal 默认人工审批：denial 可以生成 draft policy chunk，但 Gateway 会用 Z3 prover 检查 policy delta；只有显式 auto 模式且 prover delta 为空才自动批准。

## 关键组件深入解读

### Gateway Compute Runtime

ComputeRuntime 是 Gateway 对 sandbox lifecycle 的核心封装。它把 Docker、Kubernetes、Podman、remote VM 都收敛成同一个 driver trait，Gateway 层只关心 create/delete/list/watch/validate。创建路径先持久化再调用 driver，失败回滚；删除路径先 CAS 设置 Deleting，清理 sandbox-owned records，再调用 driver delete。watch/reconcile 是长期后台任务：watch loop 应用 driver event，reconcile loop 定期比对 store 与 backend，发现 backend 缺失就走 orphan pruning。

近期 HEAD 97986d9 正好改了 startup resume：legacy 或未初始化 sandbox row 会 decode 成 proto 默认 Unspecified phase，之前 resume sweep 会跳过它，导致 Gateway 重启后既不恢复也不报错；最新修复把 Unspecified 视为应当运行，backend 存在则 resume，不存在则 mark Error。

### Supervisor Relay 与 Sandbox Proxy

OpenShell 的 connect/exec/file sync 不走 Gateway 直接拨 sandbox，而是由 Supervisor 先向 Gateway 建立 gRPC session。Gateway 想打开 relay 时，先等待 session，登记 pending channel，再向 Supervisor 发 RelayOpen；Supervisor 用 RelayStream 首帧 claim channel。这个顺序避免反向连接先到而 Gateway 还没有等待者的竞态。

Proxy 是数据面的核心：先解析 CONNECT，inference.local:443 进入 inference interception；其他目标先做进程身份绑定和 OPA policy evaluation，再做 SSRF 防护、TLS/L7 inspection、credential rewrite 或 raw relay。L7 和 inference 都在 L4 allow 之后发生，tls: skip 则显式退回 raw tunnel。

## 相关页面

- [NemoClaw](#)
- agent-sandbox
- [agentgateway](#)
- cloud-native-security
- agent-credential-isolation
- [mcp](#)
- vllm

# ReMe

ReMe 是 [agentscope](#) 生态的 memory management toolkit。它同时提供文件优先的 ReMeLight 和更完整的 vector/service pipeline，把 personal memory、task memory、tool memory、working memory 拆成独立 summary / retrieve op。详见 [src-reme-architecture](#)。

## 架构边界

ReMe 关注的是 AgentScope 应用里的 context management 和跨 session 记忆，不是独立的通用记忆服务。ReMeLight 偏个人可读、可迁移、可手改；vector/service pipeline 偏规模化 agent 应用。

## 关键设计

- MEMORY.md、daily journal、dialog JSONL 和 tool result cache 保留人工可读 source-of-truth。
- personal / task / tool / working memory 分别 summary 和 retrieve，避免一个大 memory 表统管所有语义。
- pre-reasoning hook 检查 token，必要时压缩旧消息并 offload 长工具输出。
- vector pipeline 可做 semantic retrieval、rerank/fusion 和频率/效用更新。

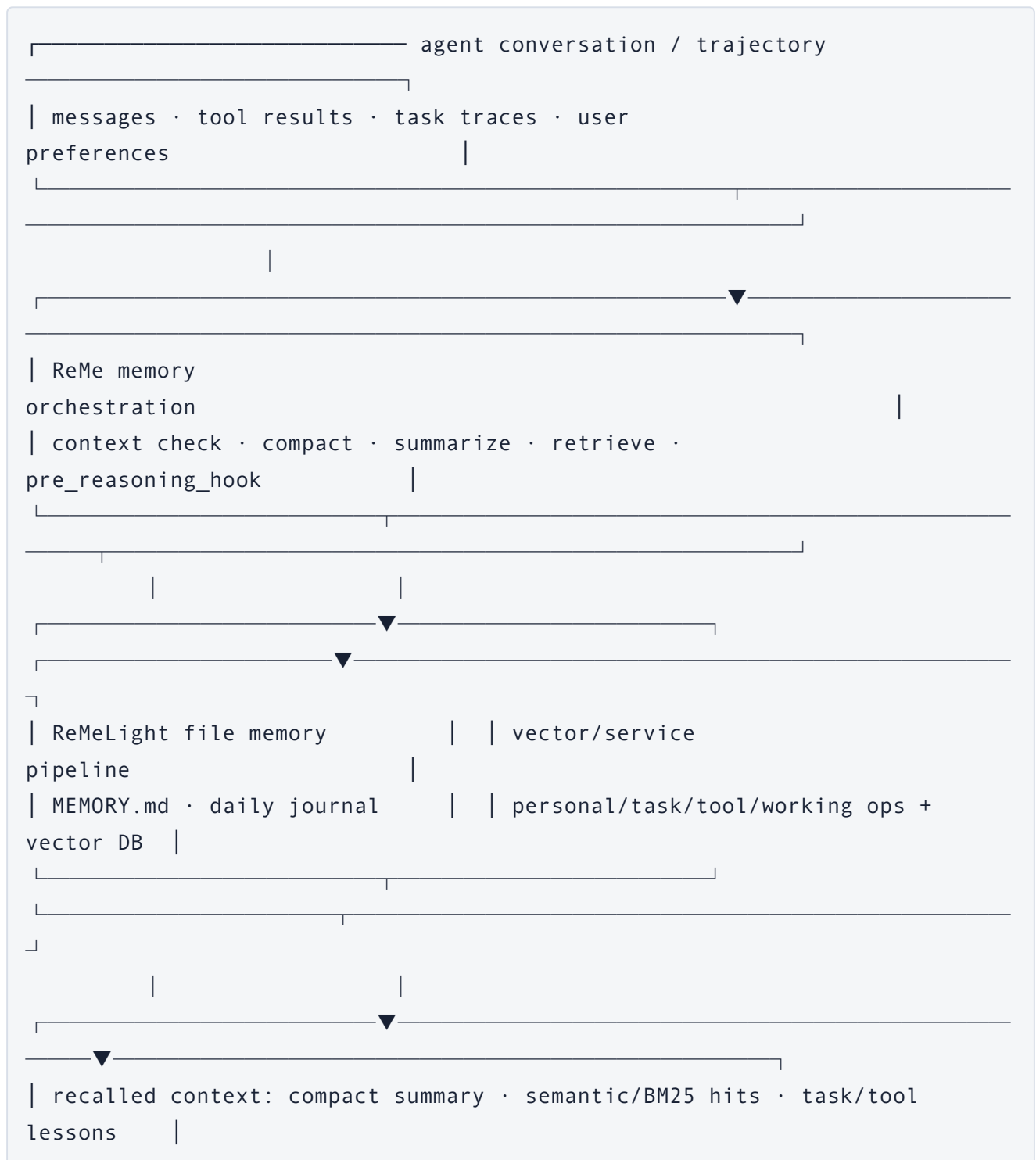
## 选型判断

AgentScope 内部应用优先看 ReMe；需要跨 framework、server、MCP 和插件生态的通用记忆层看 [mem0](#)；需要 Markdown source-of-truth + Milvus hybrid search 看 [memsearch](#)；需要 OpenClaw/Hermes 深度集成看 [tencentdb-agent-memory](#)。

# ReMe 架构与设计思路分析

agentscope-ai/ReMe 是 AgentScope 生态的 memory management toolkit。它同时保留“memory as files”的 ReMeLight 和更完整的 vector/service pipeline：personal memory、task memory、tool memory、working memory 都有独立 summary/retrieve op，目标是解决长对话 context window 和跨 session stateless 两个问题。

## 核心架构图



## 模块分层

| 层/目录                                                  | 责任                                                                                                           |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| reme/reme_light.py +<br>reme/memory/file_based/<br>** | 文件型记忆系统：MEMORY.md、daily journal、dialog JSONL、tool_result cache。                                              |
| reme_ai/summary/**                                    | personal/task/tool/working 四类总结 pipeline，包含 observation、reflection、dedup、validation、trajectory segmentation。 |
| reme_ai/retrieve/**                                   | personal/task/tool/working 检索 pipeline，包含 query rewrite、semantic rank、rerank/fusion、文件 grep/read/write。      |
| reme_ai/vector_store/**                               | 向量记忆更新、频率/效用更新、recall op。                                                                                    |

## 关键数据流

1. ReMeLight 在 pre-reasoning 前检查 token，过阈值则压缩老消息、保留近期上下文，并把长 tool output offload 到文件。
2. 长期记忆写入 MEMORY.md 和 memory/YYYY-MM-DD.md，原始对话进入 dialog/YYYY-MM-DD.jsonl，便于人工迁移/修改。
3. vector pipeline 将 personal/task/tool/working memory 拆成不同 summary 和 retrieve op，按场景选择信息，而不是用一个大表统管所有记忆。

## 设计决策

- 文件优先版本适合个人 agent：可读、可迁移、可手改；vector/service 版本适合规模化 AgentScope 应用。
- 按 memory type 拆 pipeline 比 mem0 的通用 memory layer 更重，但更贴合 agent 工作流。
- 把 tool result 单独 offload，说明 ReMe 主要关心 context 爆炸，不只是长期偏好。

## 对比定位

和 [mem0](#) 相比，ReMe 更框架内生，记忆类型更细；和 [memsearch](#) 相比，它不是 Markdown journal + Milvus shadow index，而是 context management + file/vector memory；和 [tencentdb-agent-memory](#) 相比，ReMe 更轻、更贴 AgentScope hooks。

## 相关链接

- GitHub 当前状态底稿: [src-github-stars-backlog-current-state](#)
- 选型地图: [github-stars-backlog-implementation-map](#)

# TencentDB-Agent-Memory

TencentDB-Agent-Memory 是腾讯云开源的 OpenClaw / Hermes Agent 记忆插件（仓库 TencentCloud/TencentDB-Agent-Memory，npm 包 @tencentdb-agent-memory/memory-tencentdb，分析版本 v0.3.6 / HEAD f92b102）。它同时处理长期个性化记忆和短期上下文卸载。

## 解决什么

长程 Agent 有两类不同的“记忆”压力：

- 跨会话长期记忆：用户偏好、SOP、历史事件和项目背景不能每次重讲。
- 单次长任务上下文压力：工具输出、报错、搜索结果会快速挤满 context window。

TencentDB-Agent-Memory 用 L0→L1→L2→L3 处理长期记忆，用 context offload + Mermaid MMD 处理短期任务日志。

## 核心特征

- 四层长期记忆：L0 Conversation、L1 Atom、L2 Scenario、L3 Persona。
- OpenClaw hook 集成：before\_prompt\_build 做 auto-recall，agent\_end 做 auto-capture。
- Agent 工具：注册 tdai\_memory\_search 和 tdai\_conversation\_search 让 Agent 主动下钻。
- 双存储后端：默认 SQLite + sqlite-vec + FTS5，也支持 Tencent Cloud VectorDB。
- 混合检索：keyword / embedding / hybrid RRF；TCVDB 路径支持 native hybridSearch。
- Context offload：工具日志写入 refs/\*.md，上下文注入 Mermaid 任务画布和 node\_id。
- Hermes sidecar：Node Gateway 暴露 /recall、/capture、/search/\*、/session/end、/seed，Python provider 管理进程。

## 架构骨架

| 层        | 组件                                          | 职责                                  |
|----------|---------------------------------------------|-------------------------------------|
| Host     | OpenClaw plugin / Hermes provider / Gateway | 捕获宿主事件，转成 core 调用                   |
| Core     | TdaiCore                                    | recall、capture、search、session flush |
| Pipeline | MemoryPipelineManager                       | L1/L2/L3 调度和恢复                      |
| Store    | SQLite / TCVDB                              | L0/L1 检索和 profile sync              |
| Offload  | src/offload/*                               | 长任务日志卸载、MMD 生成、L3 压缩                |

完整分析见 [src-tencentdb-agent-memory-architecture](#)。

## 与同类关系

和 [agentmemory](#) 相比，它更深度绑定 OpenClaw/Hermes 的 hook/context engine 体验；和 [memsearch](#) 相比，它不是 Markdown journal + Milvus shadow index，而是 JSONL/store/scene Markdown/persona Markdown 的分层组合；和 powermem 相比，它更偏 Agent 插件和上下文工程，而不是通用记忆中间件。

## 相关页面

- [src-tencentdb-agent-memory-architecture](#)
- [agent-memory](#)
- [event-driven-memory-pipeline](#)
- [three-tier-search-protocol](#)
- hybrid-search-rrf
- [agentmemory](#)
- [memsearch](#)
- powermem

# TencentDB-Agent-Memory 架构与设计思路分析

原文: raw/tencentdb-agent-memory-architecture-analysis.md · 仓库: <https://github.com/TencentCloud/TencentDB-Agent-Memory> · 分析版本 v0.3.6 / HEAD f92b102

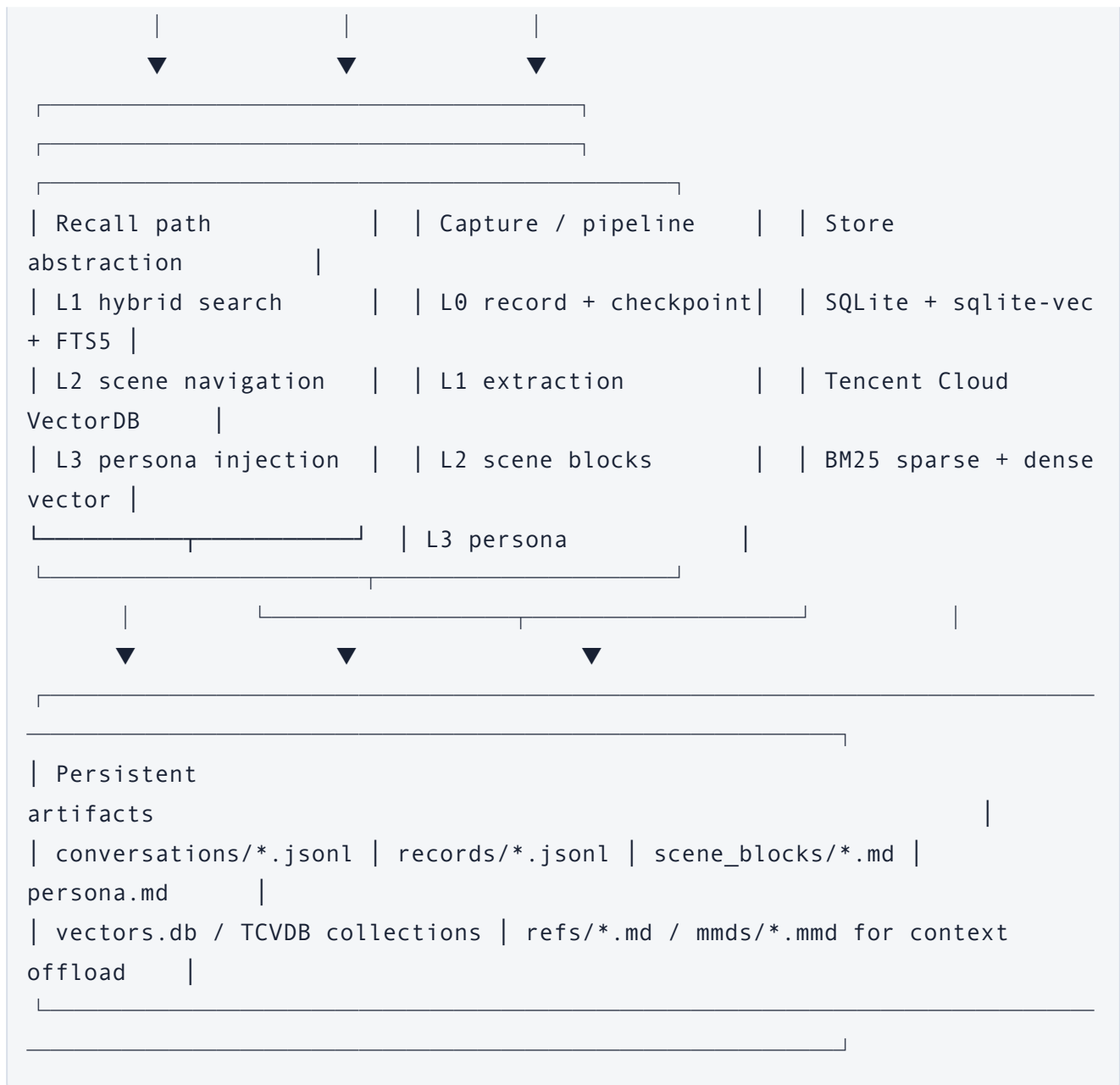
## 一句话定位

[tencentdb-agent-memory](#) 是腾讯云出品的 OpenClaw / Hermes Agent 记忆插件，npm 包名 @tencentdb-agent-memory/memory-tencentdb。它把长期记忆做成 L0 Conversation → L1 Atom → L2 Scenario → L3 Persona 的分层语义金字塔，同时把短期长任务日志做 context offload，压缩成可回溯的 Mermaid 符号画布。

它和普通“向量库里塞历史”的差别在于：底层保留证据，中层建立结构，高层注入可读画像/场景导航；召回时既能自动注入 L1/L2/L3，也能让 Agent 调 tdai\_memory\_search / tdai\_conversation\_search 主动下钻。

## 核心架构图





## 模块分层

| 层 / 模块            | 职责                                                                  |
|-------------------|---------------------------------------------------------------------|
| OpenClaw 插件入口     | 解析配置、注册工具、挂载 auto-recall / auto-capture / shutdown hook             |
| Host-neutral core | 用 HostAdapter / LLMRunnerFactory 抹平 OpenClaw、Hermes、Gateway 差异      |
| L0 捕获             | 原子游标捕获新消息，写 conversations JSONL，并可写 L0 向量索引                         |
| L1 结构化记忆          | LLM 从 L0 提取 persona/episodic/instruction 记忆，去重/合并后写 records 和 store |

| 层 / 模块           | 职责                                                              |
|------------------|-----------------------------------------------------------------|
| L2 场景块           | 工具型 LLM 在 scene_blocks/ 沙箱内维护场景 Markdown、索引和导航                  |
| L3 用户画像          | 根据变更场景增量生成 persona.md，并附加 scene navigation                      |
| Pipeline 调度      | L1 阈值/idle/warmup, L2 downward-only timer, L3 全局串行队列            |
| 检索与存储            | SQLite / TCVDB 后端抽象, FTS/BM25/vector/hybrid search              |
| Context offload  | 工具日志卸载、Mermaid 画布、L3 压缩、MMD 注入和 token 预算控制                      |
| Hermes / Gateway | Node HTTP sidecar 暴露 TDAI Core; Python provider 管理 sidecar 生命周期 |

## 关键数据流

### 1. OpenClaw 自动召回与捕获

```

OpenClaw before_prompt_build
|
▼
index.ts caches original prompt and calls TdaiCore.handleBeforeRecall()
|
▼
auto-recall sanitizes user text
|
├─ L1: keyword / embedding / hybrid search
├─ L2: read scene index and build scene navigation
└─ L3: read persona.md
|
▼
Return prependContext + appendSystemContext
|
▼
OpenClaw agent runs with injected memory
|
▼
OpenClaw agent_end
|
▼
TdaiCore.handleTurnCommitted()
|
├─ checkpoint.captureAtomically() records only new messages

```

- └ write L0 JSONL and optional L0 vector rows
- └ notify MemoryPipelineManager

## 2. L0→L1→L2→L3 长期记忆管线

L0 raw messages captured



MemoryPipelineManager.notifyConversation(sessionKey)



- └ threshold path: conversation\_count >= warmup/everyN
- └ idle path: l1Idle timer fires
- └ flush path: shutdown / session end



L1 Runner



- └ read L0 from VectorStore or JSONL fallback
- └ LLM scene segmentation + memory extraction
- └ batch dedup / update / merge / skip
- └ write records/YYYY-MM-DD.jsonl + store rows



L2 Runner



- └ read changed L1 records
- └ LLM edits scene\_blocks/\*.md inside sandbox
- └ normalize filenames / sync scene\_index
- └ optional persona update signal



L3 Runner



- └ read changed scene blocks
- └ LLM writes persona.md
- └ append fresh scene navigation

### 3. 存储与检索策略

```
Config storeBackend
|
| ├── sqlite
| | ├── vectors.db
| | ├── L1 records + vec0 + FTS5
| | ├── L0 conversations + vec0 + FTS5
| | └── client-side hybrid = FTS5 + embedding + RRF
|
| └── tcvdb
| ├── l1_memories collection
| ├── l0_conversations collection
| ├── profiles collection
| ├── server-side dense embedding
| ├── client-side BM25 sparse vector
| └── native hybridSearch + RRFrerank
```

### 4. Context offload 短期记忆

```
after_tool_call captures heavy tool result
|
▼
Write raw refs/*.md and offload entries
|
▼
L1 summarizes tool pairs
|
▼
L1.5 judges task boundary / active MMD
|
▼
L2 updates Mermaid task canvas with node_id mapping
|
▼
before_prompt_build / contextEngine assemble
|
| ├── mild compression: replace old tool results with summaries
| └── aggressive compression: delete/compress until below threshold
```

- └ emergency fallback: truncate oversized messages
- └ inject active/history MMD within token budget

## 设计决策与哲学

- 宿主中立 core: OpenClaw in-process 插件和 Hermes sidecar 都走同一个 TdaiCore, 这和 [event-driven-memory-pipeline](#) 的“采集层/处理层分离”一致。
- 长期记忆是分层金字塔: L0 保留原始对话, L1 提取 atom, L2 归纳 scenario, L3 生成 persona, 避免把历史平铺成无结构向量堆。
- 动态/稳定上下文分离: L1 recall 走 prependContext, L2/L3/工具指南走 appendSystemContext, 降低 prompt cache 被动态片段击穿的概率。
- JSONL 和 store 各司其职: records JSONL 是 append-only 备份/恢复材料, SQLite/TCVDB 是检索引擎。
- 混合检索可本地可云端: SQLite 路径用 FTS5 + embedding + RRF; TCVDB 路径用 server-side embedding + BM25 sparse + native hybridSearch, 属于 hybrid-search-rrf 的工程化变体。
- offload 保留可追溯性: 原始工具日志放 refs/\*.md, 上下文里只保留 Mermaid MMD 和 node\_id, 符合 [three-tier-search-protocol](#) 的“先高层、再下钻”思想。

## 关键组件

TdaiCore 是项目核心 facade: handleBeforeRecall()、handleTurnCommitted()、searchMemories()、searchConversations() 给所有宿主复用。它内部用 promise gate 防 scheduler start 并发竞态, 并在 shutdown 前 drain 后台 L0 embedding 任务。

MemoryPipelineManager 是长期记忆节奏控制器: L1 有 threshold / idle / flush 三条触发路径, warmup 从 1→2→4 逐步放缓; L2 用 downward-only timer; L3 是全局串行队列, 避免画像生成并发改写。

## 与同类关系

| 维度     | TencentDB-Agent-Memory        | <a href="#">agentmemory</a> | <a href="#">memsearch</a>             | powermem                |
|--------|-------------------------------|-----------------------------|---------------------------------------|-------------------------|
| 主形态    | OpenClaw 插件 + Hermes sidecar  | 跨 Agent worker / MCP / REST | 多平台 coding agent memory CLI/plugin    | 记忆中间件 / SDK / API / MCP |
| 长期记忆层次 | L0→L1→L2→L3                   | 多层记忆 + graph/vector/BM25    | Markdown memory + Milvus shadow index | working/short/long + 衰减 |
| 短期上下文  | Context offload + Mermaid MMD | context injection 可配置       | search/expand/transcript              | 偏长期记忆和应用层               |
| 存储     |                               | SQLite                      | Markdown + Milvus                     |                         |

|    |                                      |                             |                           |                       |
|----|--------------------------------------|-----------------------------|---------------------------|-----------------------|
| 维度 | TencentDB-Agent-Memory               | <a href="#">agentmemory</a> | <a href="#">memsearch</a> | powermem              |
|    | SQLite + sqlite-vec/<br>FTS5 或 TCVDB |                             |                           | OceanBase /<br>SeekDB |

这个项目最有辨识度的地方是把“长期个性化记忆”和“短期任务上下文卸载”合并到同一个插件，而不是只做 recall。

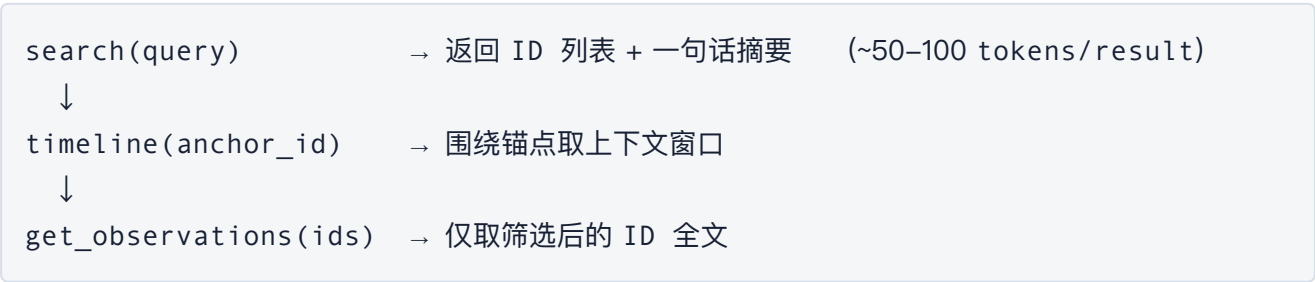
## 相关页面

- [tencentdb-agent-memory](#)
- [agent-memory](#)
- [event-driven-memory-pipeline](#)
- [three-tier-search-protocol](#)
- hybrid-search-rrf
- [ai-as-compressor](#)
- [agentmemory](#)
- [memsearch](#)
- powermem

# 三层搜索协议

让 Agent 在记忆库中检索时避免上下文爆炸的设计模式。由 [claude-mem](#) 的 mem-search Skill 实现。

## 协议三层



## 核心思想

永远先返低 token 摘要，让模型筛选后再取详情。

朴素做法：一次返回所有匹配结果的全文 → 单次查询可能吃掉数千 token，命中 10 条无关结果就废了一半上下文。

三层协议做法：

| 阶段               | 返回              | Token 量级 | 决策方        |
|------------------|-----------------|----------|------------|
| search           | 20 个 ID + 一句话摘要 | ~1-2k    | 模型决定哪些值得深看 |
| timeline         | 锚点附近的事件序列       | ~500     | 模型决定要不要全文  |
| get_observations | 筛选后 3-5 条全文     | ~3-5k    | 真正读细节      |

相比一次性全文返回，省 10x token。

## 为什么 Agent 能自主调用

claude-mem 把这套协议注册成 Skill (plugin/skills/mem-search/SKILL.md)，描述里写明触发时机。Claude 在判断"用户问到过去"时会自主决定调用——SKILL.md 的描述文案艺术决定了触发精度。

## 可迁移性

任何"AI Agent + 长期存储"场景都适用：

- 给 RAG 加分层检索（先返 chunk ID，再取选中 chunk 全文）

- 给企业知识库 Agent 加分层调用（先返文档元信息，再取段落）
- 工具调用结果分级（先返摘要，模型按需 expand）

## 与朴素 RAG 的对比

| 维度       | 朴素 RAG         | 三层协议          |
|----------|----------------|---------------|
| 一次返多少    | top-K 全文 chunk | top-N ID + 摘要 |
| 谁决定要哪些   | 检索算法（固定）       | LLM（动态）       |
| Token 效率 | 低              | 高             |
| 实现复杂度    | 低              | 中             |

## 参考

- [src-claude-mem-architecture](#)

# 事件驱动的记忆管道

给 AI Agent 加长期记忆的核心架构模式：「事件采集 → AI 压缩 → 双索引存储 → 反向注入」的闭环。

由 [claude-mem](#) 实践验证，可移植到任何有生命周期 hook 的 Agent runtime。

## 五步闭环

```
Hook 事件
 ↓ (1) 捕获（边缘层）
原始 event (JSON)
 ↓ (2) 入队（持久化层）
Outbox 表
 ↓ (3) AI 压缩（后台层）
结构化 Observation (XML)
 ↓ (4) 解析双索引（存储层）
SQLite (FTS5) + 向量库
 ↓ (5) 注入回流（消费层）
新会话 system prompt / Skill 检索
```

### Step 1: 捕获（边缘层）

Hook 触发 → 轻量 runner（如 bun-runner.js）收集 stdin → 派发到 handler → POST 给本地 worker。

[warning] 边缘层禁止做 AI 推理

用户交互路径上必须是同步、毫秒级、零网络依赖。AI 调用全部异步化。

### Step 2: 入队（持久化层）

worker 把原始 event 写入 outbox 表（Postgres 的 LISTEN/NOTIFY 或 SQLite 暂存），BullMQ 等队列调度后台任务异步处理。

为什么必须 outbox：AI 生成是非确定的，失败必须能重试；同步处理会阻塞用户。

### Step 3: AI 压缩（后台层）

调用 [压缩器](#) 把噪声大的 tool log 压成结构化 {title, narrative, facts[], concepts[], filesRead[], filesModified[]}。

claude-mem 用 XML 输出格式 + Zod schema 校验，便宜模型（Haiku / Gemini Flash）足够。

### Step 4：解析存储（双索引层）

事务化写入：

- SQLite + FTS5：主存储 + 全文检索
- Chroma 向量库：语义检索（异步推送 + watermark 回填）

去重策略：内容哈希 + UNIQUE generation\_key + ON CONFLICT。

### Step 5：注入回流（消费层）

- 自动注入：SessionStart 时按"项目路径 + 最近活跃概念"预检索，渲染成 system 消息塞回上下文窗口
- 按需查询：用户问"上次怎么做的"时，[三层搜索协议](#) 通过注册的 Skill 自动触发

## 关键设计原则

| 原则              | 含义                              |
|-----------------|---------------------------------|
| 边缘轻量 + 后台 AI 重活 | hook 同步只走 stdin → HTTP，AI 压缩异步  |
| Outbox 模式       | 失败可重试，不丢消息                      |
| 内容哈希去重          | AI 输出非确定，避免重复污染索引               |
| 三层数据            | 原始 event → 结构化观察 → 向量 embedding |
| session_id 双轨   | 记忆生命周期 ≠ 会话生命周期                 |
| 边缘剥离敏感数据        | <private> 标签在 hook 层剥离，零信任      |

## 可迁移到的 Runtime

任何有 before/after tool\_use 钩子的 Agent runtime：

- Claude Code Lifecycle Hooks
- OpenAI Assistants 的 step events
- AutoGen 的 agent\_event 回调
- LangChain 的 callback handlers

## 参考

- [src-claude-mem-architecture](#)